The iOS Interview Handbook - Rapid Drill Sheet

Cover the answer block with your hand or a card. Read the question, articulate your answer, then reveal.

Swift Fundamentals

Value Types vs Reference Types

Q: What's the difference between value types and reference types in Swift? When would you use each?

Short answer: A value type is copied on assignment and when passed, so every variable owns an independent value; a reference type is shared, so several variables point at one instance with an identity. Structs, enums and tuples are value types (and so are `Array`, `Dictionary` and `String`, which use copy-on-write to make copies cheap).

Key points:

- Value types: `struct`, `enum`, tuples; `let` makes the whole value immutable, `var` allows `mutating` methods
- Reference types: `class` (identity via ===, inheritance, `deinit`, ARC), `actor` (reference + isolation), closures
- A `let` reference only freezes the reference, not the object's properties
- Copy-on-write: standard collections copy their buffer only when a shared buffer is mutated (`isKnownUniquelyReferenced`)
- Concurrency: non-public structs/enums with `Sendable` members are inferred `Sendable`; classes need `final` + immutable `Sendable` state, a `Mutex`, or actor isolation
- Noncopyable types (`~Copyable`): structs/enums that are *moved*, not copied, for unique ownership (file handles, tokens) with `consuming`/`borrowing`
- `InlineArray<N, Element>` (Swift 6.2, iOS 26+): fixed-size, inline storage value type with no heap allocation
- A struct containing a class reference copies the reference, not the object (shared state leaks through)

Related to check: Copy-on-write, mutating functions, weak vs unowned, `Sendable`, actors, noncopyable types, `InlineArray`/`Span`

Noncopyable Types (~Copyable)

Q: What are noncopyable types in Swift? What do `consuming` and `borrowing` mean, and when would you use them?

Short answer: Every Swift type is implicitly `Copyable`; writing `~Copyable` removes that, so a struct or enum has exactly one owner and is *moved* rather than copied. The compiler then tracks ownership statically: you can borrow the value, mutate it in place, or consume it, and using it after it's been consumed is a compile error.

Key points:

- Parameters of a noncopyable type must state ownership: `borrowing` (read-only access, caller keeps it), `inout` (mutate in place), `consuming` (callee takes ownership)
- The same words on methods describe `self`: `borrowing func read()`, `consuming func close()`
- `consume x` ends a binding's lifetime explicitly; later use is "used after consume"
- Structs and enums can have a `deinit`; inside a `consuming` method, `discard self` skips it (for example after a close that already released the resource)

- Noncopyable generics (Swift 6.0): <T: ~Copyable> accepts both kinds; extension Box: Copyable where Wrapped: Copyable {} restores copying conditionally. Protocols opt in with protocol Resource: ~Copyable
- Optional holds noncopyable values: optional chaining, switch, and take() to move the value out and leave nil
- Swift 6.4: noncopyable types can conform to Equatable and Hashable (synthesis included); this builds at an iOS 18 target

Related to check: Value vs reference types, copy-on-write, generics, InlineArray / Span, Mutex, Equatable and Hashable, optionals

---

InlineArray and Span

Q: What are InlineArray and Span in Swift 6.2, and why were they added?

Short answer: They're Swift 6.2's answer to "safe code that's as fast as unsafe pointers". InlineArray<N, Element> is a fixed-size array whose count is part of the type and whose elements are stored inline in the enclosing value: no heap buffer, no reference count, no copy-on-write check.

Key points:

- InlineArray<3, Float> uses an *integer generic parameter* (SE-0452); create it from a literal, init(repeating:), or a closure InlineArray<4, Int> { i in i * i }
- It's not a Sequence: index it (for i in a.indices) or read it through a.span. for x in inlineArray needs iOS 27, because it goes through Swift 6.4's Iterable protocol
- Every copy is a full element-by-element copy, so keep it small
- Span<T> (read-only), MutableSpan<T> (in-place writes), RawSpan (untyped bytes, span.bytes), UTF8Span (validated UTF-8, string.utf8Span)
- Array, ContiguousArray, InlineArray and String.UTF8View expose .span; mutable collections expose .mutableSpan (iOS 26+)
- Bounds-checked like Array, but no retain/release and no ownership transfer
- Context: Swift 6.2 also adds opt-in **strict memory safety** (-strict-memory-safety), which warns on every unsafe construct until it's marked with the unsafe keyword; Span is how you remove those sites rather than annotate them

Related to check: Heap vs stack, copy-on-write, collections, object allocation, noncopyable types, generics (integer generic parameters), availability checking

---

Optionals and Optional Binding

Q: What are optionals in Swift? How do you safely unwrap them?

Short answer: An optional says in the type system that a value might be absent: String? is shorthand for Optional<String>, an enum with .some(value) and .none. The compiler won't let you use a String? as a String, so "forgot to check for null" becomes a compile error instead of a crash.

Key points:

- T? == Optional<T>; nil is .none. Only optionals can be nil
- Shorthand binding: if let name { ... } / guard let name else { return } rebinds the same name
- guard let unwraps for the rest of the scope and forces an early exit; if let unwraps only inside its block
- Combine conditions with commas: if let a, let b, a > b { ... }
- ?? provides a default and short-circuits (the right side is an autoclosure)

- `?.` returns an optional; the whole chain is `nil` if any link is `nil`
- `!` traps on `nil`; implicitly unwrapped optionals (`T!`) are a legacy of Objective-C interop and outlets
- Optionals are pattern-matchable: `case let x?`, `case nil`, `switch` over `.some`/`.none`

Related to check: guard vs if let, optional chaining, nil coalescing, optional map/flatMap, implicitly unwrapped optionals, pattern matching

---

guard vs if let

Q: When would you use `guard let` instead of `if let`?

Short answer: Both unwrap optionals and test conditions; the difference is scope and intent. `guard` states a precondition: if it fails, the `else` block *must* leave the scope, and the bound values stay available for the rest of the function, so the happy path reads top to bottom without nesting.

Key points:

- `guard ... else { }` must exit with `return`, `throw`, `break`, `continue`, or a `Never`-returning call such as `fatalError`
- Values bound by `guard let` are in scope after the guard; values bound by `if let` only inside the `if` block
- Shorthand binding: `guard let user else { return }` instead of `guard let user = user`
- Combine conditions with commas: `guard let id, id > 0, let url = URL(string: path) else { ... }`
- `guard let self` inside `[weak self]` closures (Swift 5.7 shorthand)
- `if`/`switch` expressions: `let title = if let user { user.name } else { "Guest" }`
- `guard` also works with plain Booleans and patterns: `guard case .loaded(let items) = state else { return }`

Related to check: Optionals, optional chaining, nil coalescing, error handling, pattern matching, `Never` (`fatalError` in `guard` blocks), capture lists (`guard let self`)

Closures and Escaping

Q: What's the difference between escaping and non-escaping closures? When do you need @escaping?

Short answer: A non-escaping closure is guaranteed to run (or be discarded) before the function it was passed to returns, so it can't outlive the call. An escaping closure can be stored or called later, after the function returns, and the compiler requires you to mark that parameter `@escaping`.

Key points:

- Closure parameters are non-escaping by default; the compiler can optimize them and you don't need `self.` for implicit captures
- Need `@escaping` when the closure is stored in a property, called after an asynchronous operation, or passed to another escaping parameter
- Optional closure parameters (`(() -> Void)?`) are implicitly escaping
- Escaping closures strongly capture `self` and other references; use `[weak self]` when the closure is stored by something `self` owns
- Escaping closures called from another thread or isolation domain should be `@Sendable` (Swift 6 checks their captures)
- Modern code often replaces completion handlers with `async` functions, which removes most escaping closures

Related to check: Retain cycles, capture lists, weak vs unowned, `@Sendable` closures, async/await

---

Protocols and Protocol-Oriented Programming

Q: Explain protocol-oriented programming in Swift. How does it differ from object-oriented programming?

Short answer: Protocol-oriented programming means designing around small protocols that describe capabilities, giving them default behavior in protocol extensions, and conforming value types to them, instead of building class hierarchies. In OOP a type gets behavior by *being* a subclass, which ties it to one parent, its stored state and reference semantics.

Key points:

- Protocols declare requirements; extensions add default implementations and helper methods
- Conformance works for value types and can be added retroactively (`extension Int: Drawable`)
- Associated types (`associatedtype Item`) make protocols generic; primary associated types allow `some Collection<Int>` / `any Collection<Int>`
- Write `any P` for existential types; it's required for protocols with associated types or `Self` requirements, and `some P` is preferred when one concrete type is used
- **Dispatch trap:** methods declared only in an extension (not as requirements) are statically dispatched, so a conformer's "override" is ignored when called through the protocol type
- Constrained extensions (`extension Collection where Element: Numeric`) add behavior only where it makes sense
- Swift 6: protocols that cross isolation are often `Sendable`; a `@MainActor` type can satisfy a non-isolated protocol with an isolated conformance, `extension VC: @MainActor Delegate` (Swift 6.2)

Related to check: Protocol composition, associated types, opaque types, type erasure, generics, extensions, `Sendable` and isolated conformances

---

Generics

Q: How do generics work in Swift? When would you use them?

Short answer: Generics let you write one function or type that works over many types while keeping full compile-time type safety: `Array<Element>`, `Optional<Wrapped>` and `Result<Success, Failure>` are all generic. You declare type parameters, constrain them with protocols (`T: Equatable`) or `where` clauses, and the compiler checks every use.

Key points:

- Type parameters: `func swapValues<T>(_ a: inout T, _ b: inout T)`; constraints: `<T: Hashable>`, `where C.Element == Int`
- Conditional conformance: `extension Stack: Sendable where Element: Sendable {}`
- `some Protocol` in parameter position is shorthand for an anonymous generic parameter: `func total(_ v: some Collection<Int>)`
- Primary associated types (`Collection<Int>`, `Sequence<String>`) constrain associated types inline and remove most hand-written type erasers (`AnySequence` etc.)
- Generics (static, specialized, caller chooses the type) vs `any Protocol` existentials (a box, dynamic dispatch, value can change type)
- **Parameter packs** (`each T` / `repeat each T`): any number of differently typed arguments, e.g. a `zip` over N sequences
- **Noncopyable generics:** every generic parameter implicitly requires `Copyable`; `<T: ~Copyable>` lifts that so the code also accepts move-only types
- **Integer generic parameters** (Swift 6.2, iOS 26+ runtime): `struct Vector<let count: Int>`, the basis of `InlineArray<count, Element>`

Related to check: Opaque types (`some`) vs existentials (`any`), protocols with associated types, noncopyable types, `InlineArray` / `Span`, `Sendable`, `inout` parameters

---

Parameter Packs (Variadic Generics)

Q: What are parameter packs in Swift? What problem do they solve?

Short answer: Parameter packs (Swift 5.9, SE-0393) let a generic function or type take *any number* of type parameters, each of which can be a different type, while keeping every one of them statically typed. Before packs, APIs like SwiftUI's `ViewBuilder`, `zip` or Combine's `combineLatest` were written as a stack of overloads for 2, 3, 4 … arguments, which is why a `VStack` used to stop at ten children.

Key points:

- `each T` declares a *type pack*; `repeat each T` is a *pack expansion* (a pattern repeated for each element)
- Parameters: `_ value: repeat each T`; results: `-> (repeat each T)` returns a tuple of the same shape
- Constraints apply per element: `<each T: Equatable>`, or `where repeat each T: Sendable`
- Two packs used in one `repeat` must have the same length (*same-shape* requirement), which the compiler checks
- Swift 6.0 (SE-0408): iterate with `for item in repeat each value`; zip two packs with `for (l, r) in repeat (each lhs, each rhs)`
- Generic *types* can be variadic too (`struct Query<each Column>`); this needs the iOS 17 runtime, so it's fine at an iOS 18 baseline
- Real APIs built on packs: SwiftUI `ViewBuilder.buildBlock`, Foundation `#Predicate<each Input>` and `#Expression`

Related to check: Generics, tuples, SwiftUI `@ViewBuilder`, macros (`#Predicate`), `Sendable`, structured concurrency (`async let`, task groups)

Error Handling

Q: How does error handling work in Swift? Explain throw, try, try?, and try!.

Short answer: A function that can fail is marked `throws`, it signals failure with `throw` and any value conforming to `Error`, and every call site must acknowledge it with `try`. Errors propagate up the call stack until a `do`/`catch` handles them, and `async throws` works the same way across suspension points.

Key points:

- Model errors as enums (or structs with context) conforming to `Error`; add `LocalizedError` for user-facing messages
- `try` inside `do`/`catch` or a throwing function; `catch` clauses can pattern-match specific cases
- `try?` converts to an optional and discards the error; `try!` crashes on failure
- `defer` runs cleanup on every exit path, including throws
- Typed throws (Swift 6, SE-0413): `throws(ParseError)`; a `catch` over code that only throws `ParseError` gets `error` typed as `ParseError`
- `throws(any Error)` is the same as `throws`; `throws(Never)` is the same as non-throwing
- Typed throws let generic code propagate the caller's error type (`throws(E)`), replacing many `rethrows` uses

Related to check: Result type, async throws, `LocalizedError`, `defer`, task cancellation, NSError bridging

---

Typed Throws

Q: What are typed throws in Swift 6? When should you use them, and when shouldn't you?

Short answer: Typed throws (SE-0413, Swift 6.0) let a function declare exactly which error type it can throw: `func parse() throws(ParseError)`. Callers that catch it get `error` statically typed as `ParseError` and can `switch` over it exhaustively, and no existential box is allocated.

Key points:

- `throws` is shorthand for `throws(any Error)`; `throws(Never)` is a non-throwing function
- Inside, `throw .empty` infers the declared type; throwing anything else is a compile error
- `do`/`catch` infers the caught type from the body: if everything inside throws `ParseError`, `error` is `ParseError`. You can also spell it: `do throws(ParseError) { ... }`
- Subtyping: a `throws(E)` function can be called from an untyped `throws` function (the error widens to `any Error`), but not from one that throws a different concrete type; wrap it: `catch { throw .parse(error) }`
- Generic pass-through: `func measure<T, E: Error>(_ body: () throws(E) -> T) throws(E) -> T`. With a non-throwing closure, `E` is `Never` and the call needs no `try`
- The standard library adopted it: `Sequence.map` and `filter`, `Optional.map`, `Result(catching:)` and `withUnsafeBufferPointer` propagate the closure's error type

Related to check: Error handling (`throw`/`try`/`do-catch`), Result type, generics, `rethrows`, async throws and `Task`, testing error paths

---

Property Wrappers

Q: What are property wrappers? Give examples of when you'd use them.

Short answer: A property wrapper is a type marked `@propertyWrapper` that owns the storage for a property and adds reusable behaviour around reads and writes, like clamping, validation or persistence. The compiler rewrites `@Clamped(0...100) var volume = 50` into a hidden stored wrapper plus a computed property that forwards to its `wrappedValue`.

Key points:

- Requires `wrappedValue`; `init(wrappedValue:)` lets you write `= initialValue`
- Extra init parameters configure it: `@Clamped(0...100)`
- Optional `projectedValue` is exposed with `$`: `$isOn` on a `@Binding` is a `Binding<Bool>`
- Good for repeated logic: validation, clamping, `UserDefaults` persistence (`@AppStorage("theme")`)
- Works on local variables, stored properties and function parameters

Related to check: @State and @Binding, @AppStorage, @Observable macro, property observers, key paths

---

Macros

Q: How do Swift macros work? How are they different from property wrappers?

Short answer: A Swift macro is a compile-time code generator: the compiler hands the macro the syntax tree of the code it's attached to, the macro returns new Swift syntax, and that expansion is type-checked and compiled like code you wrote yourself. Macros come in two families: *freestanding* macros, spelled

`#name`, which produce an expression or declarations where they appear (`#Preview`, `#Predicate`, `#expect`), and *attached* macros, spelled `@Name`, which add accessors, members, peers or conformances to a declaration (`@Observable`, `@Model`, `@Entry`).

Key points:

- Freestanding roles: `expression` (`#Predicate`, `#expect`) and `declaration` (`#Preview`)
- Attached roles: `peer`, `accessor`, `member`, `memberAttribute`, `extension` (adds conformances), and `body`
- `@Observable` uses `member` + `memberAttribute` + `extension`; `@Entry` uses `accessor` + `peer` (it generates the private `EnvironmentKey` you used to write by hand)
- Building with Xcode 27, SwiftUI's `@State` is a macro too (`macro State()`, back-deploying to iOS 13): the default value is created only the first time SwiftUI instantiates the view. With Xcode 26 and earlier it's the `State` property wrapper
- Implementation: a macro is declared with `#externalMacro(module:type:)` and implemented in a separate compiler-plugin target using **swift-syntax**; the plugin runs in a sandbox with no file-system or network access
- Macros must be deterministic and only see the syntax they're given, not other files or full type information

Related to check: Property wrappers, Observable macro and Observation, SwiftData `@Model` and `#Predicate`, SwiftUI `@Entry` and `#Preview`, Swift Testing `#expect`, parameter packs

---

weak vs unowned

Q: What's the difference between weak and unowned references? When do you use each?

Short answer: Both are non-owning references that don't increase the retain count, so both break strong reference cycles under ARC. A `weak` reference is always optional and is automatically set to `nil` when the object is deallocated.

Key points:

- `weak`: always optional; becomes `nil` when the referenced object deallocates
- Swift 6.2 allows `weak let` (SE-0481) for a weak reference you never reassign; before that `weak` had to be `var`. A `final class` with a `weak let` of a Sendable type can be `Sendable`
- `unowned`: non-optional, no `nil` check; traps if the object is already gone
- Use `weak` for delegates, parent back-references that can outlive the parent, and escaping closures with uncertain timing
- Use `unowned` when the referenced object always outlives the reference (e.g. a credit card that always belongs to a customer)
- `unowned(unsafe)` skips the runtime check entirely; avoid it outside performance-critical code

Related to check: Retain cycles, ARC, capture lists, closures, `Sendable`

Access Control

Q: Explain Swift's access control levels: open, public, package, internal, fileprivate, and private.

Short answer: Swift has six access levels, and they're scoped to modules and source files rather than to classes. `open` and `public` expose an API outside the module, and only `open` lets other modules subclass or override it; `package` (Swift 5.9) exposes it to other modules in the same Swift package; `internal` is the default and means the whole module; `fileprivate` is the file; `private` is the enclosing declaration plus its extensions in the same file.

Key points:

- `open`: visible outside the module and subclassable/overridable there (classes and class members only)
- `public`: visible outside the module; can't be subclassed or overridden from outside
- `package`: visible to every module in the same package (the compiler gets a `-package-name`), invisible to clients
- `internal` (default), `fileprivate`, `private`; `private` members are visible to same-file extensions of the type
- Setter access can be narrower: `public private(set) var balance`
- A declaration can't be more visible than the types it uses (a `public` function can't take an `internal` type)
- Members of a `public` type default to `internal`, and the synthesized memberwise init is `internal`
- Access-level imports (Swift 6, SE-0409): `internal import Analytics` keeps a dependency out of your public interface; `public import Foundation` re-exports types you use in public API

Related to check: Modules and SPM packages, `@testable import`, `final` and dynamic dispatch, API design, extensions

---

Computed vs Stored vs Lazy Properties

Q: What's the difference between computed, stored, and lazy properties?

Short answer: A stored property is part of the instance's memory layout and holds a value (`let` or `var`). A computed property has no storage: its getter (and optional setter) runs on every access, so it's for values derived from other state.

Key points:

- Stored: `let` or `var`; can have `willSet`/`didSet` observers; extensions can't add stored properties
- Computed: `get` (and optionally `set` with `newValue`); always `var`; runs every access, no caching
- Read-only computed properties can be effectful: `var data: Data { get async throws { ... } }`
- Lazy: must be `var`; the initializer can reference `self`; evaluated at most once (per instance) on first read
- Lazy is **not thread-safe**: two threads reading it at the same time can both run the initializer
- Global variables and `static let` properties are lazy too, but their initialization *is* thread-safe (run once)

Related to check: Property observers, property wrappers, `@Observable`, `mutating` and value semantics, thread safety, `Sendable`

Extensions

Q: What are extensions in Swift? What can and can't they do?

Short answer: An extension adds functionality to an existing type (including types you don't own, like `String` or `UIView`) without subclassing or touching the original source. Extensions can add methods, computed properties, initializers, subscripts, nested types and protocol conformances, but never stored properties, because they can't change a type's memory layout.

Key points:

- Can add: instance and static methods, computed properties, initializers, subscripts, nested types, protocol conformances

- Cannot add: stored properties, property observers on existing properties, or overrides of existing members (only `@objc` members can be overridden from an extension)
- A struct initializer added in an extension keeps the compiler's memberwise initializer; a class extension can add only `convenience` initializers
- Constrained extensions: `extension Array where Element: Hashable`, and conditional conformances such as `extension Box: Equatable where Value: Equatable`
- Protocol extension methods that are **not** declared as requirements are statically dispatched (see the pitfall below)
- Conforming a type you don't own to a protocol you don't own needs `@retroactive` (Swift 6), which acknowledges the risk that the owner later adds the same conformance
- Swift 6.2 isolated conformances: `extension SettingsViewController: @MainActor UITableViewDataSource` lets a main-actor type satisfy a nonisolated protocol

Related to check: Protocols and protocol-oriented programming, static vs dynamic dispatch, access control, generics and conditional conformance, isolated conformances (concurrency), nested types

---

Enums and Associated Values

Q: How do Swift enums with associated values work? Give an example.

Short answer: A Swift enum with associated values is a sum type (a tagged union): a value is exactly one of the cases, and each case can carry its own payload of different types. You get the payload back out with pattern matching in `switch`, `if case` or `guard case`, and `switch` must be exhaustive, so adding a case makes the compiler point at every place that needs updating.

Key points:

- Each case can have its own payload, optionally labeled: `case loading(progress: Double)`
- Raw values (`enum Tab: String`) are compile-time constants, the same type for every case; an enum can't have both raw values and associated values
- Extract with `switch`, `if case .loaded(let items) = state`, `guard case`, and `where` clauses on patterns
- `switch` and `if` are expressions (Swift 5.9), so a `switch` can directly produce a value
- `indirect` enables recursive cases (trees, expression ASTs)
- `Equatable`/`Hashable` are synthesized when all payloads conform; `CaseIterable` only for enums without payloads; `Codable` synthesis works with associated values (Swift 5.5)
- An enum is `Sendable` when all its payloads are

Related to check: Pattern matching, `Optional`, `Result`, state machines, `Equatable`/`Hashable`/`Codable` synthesis, `Sendable`

Codable

Q: What is Codable? How do you use it for JSON encoding and decoding?

Short answer: `Codable` is a typealias for `Encodable & Decodable`. If every stored property is itself `Codable`, the compiler synthesizes the conformance, including a `CodingKeys` enum, so a model type needs nothing more than the declaration.

Key points:

- Synthesized for structs, classes and enums (including enums with associated values, Swift 5.5) when all members are `Codable`
- `CodingKeys` renames or omits keys; omitted properties need default values

- Decoder strategies: `keyDecodingStrategy = .convertFromSnakeCase`, `dateDecodingStrategy = .iso8601`, `dataDecodingStrategy`
- Optional properties decode missing keys and `null` as `nil`; non-optional properties throw `keyNotFound`
- Custom `init(from:)` for defaults (`decodeIfPresent(...) ?? default`), nested containers, or polymorphic payloads
- `DecodingError` cases (`keyNotFound`, `typeMismatch`, `valueNotFound`, `dataCorrupted`) carry a `codingPath` for debugging

Related to check: `JSONDecoder`/`JSONEncoder`, `CodingKey`, DTO vs domain models, networking layer, `Sendable`, default actor isolation

---

Result Type

Q: What is the Result type? When would you use it instead of throws?

Short answer: `Result<Success, Failure: Error>` is an enum with two cases, `.success(value)` and `.failure(error)`, that represents the outcome of an operation as a value. That makes it the right tool when you need to *store* or *pass around* an outcome: completion-handler APIs, caching the last load result, collecting per-item results from a batch.

Key points:

- `Result<Success, Failure>`: `Failure` is a concrete error type or `any Error`
- Convert: `try result.get()` (throws `Failure`) and `Result { try work() }` (catching initializer, synchronous only; use `do`/`catch` for async work)
- Transform without unwrapping: `map`, `mapError`, `flatMap`, `flatMapError`
- Still useful for completion handlers in older APIs, stored state, and collecting outcomes in task groups
- `async throws` replaced most uses of `Result` in callbacks
- Typed throws (Swift 6) bring `Result`'s typed `Failure` to ordinary functions: `throws(ParseError)` is the throwing counterpart of `Result<T, ParseError>`, and `get()` throws `Failure` typed

Related to check: Error handling, typed throws, async/await, continuations, task groups

---

defer

Q: What does the defer keyword do? When would you use it?

Short answer: `defer` registers a block that runs when the current *scope* exits, however it exits: a normal return, a thrown error, `break`, `continue` or the end of a loop iteration. Multiple `defer`s run in reverse order, like a stack, so cleanup unwinds in the opposite order of setup.

Key points:

- Runs at the end of the enclosing scope (a function, `if` block, `do` block or loop iteration), not necessarily the function
- LIFO order when there are several
- The block reads variables at exit time, so it sees their final values
- It can't transfer control out: no `return`, `break` or uncaught `throw` inside; use `try?` or handle errors in a `do`/`catch`
- Only `defer`s already reached are registered; one after an early `return` never runs
- Swift 6.4 (SE-0493): `defer { await connection.close() }` in an `async` function

Related to check: Error handling, scope and control flow, `deinit` and RAII, task cancellation, `withTaskCancellationShield`

---

inout Parameters

Q: What does inout do? How does it differ from passing by reference?

Short answer: `inout` lets a function modify a caller's variable. Semantically it's *copy-in, copy-out* (call by value-result): the function works on the value and the result is written back when the function returns.

Key points:

- The caller passes a `var` with `&`: `clamp(&volume, to: 0...10)`; `let` constants and literals aren't allowed
- Write-back happens once at the end, so `didSet` on the argument fires once, even if the value didn't change
- Exclusivity: overlapping accesses (`swap(&a[0], &a[1])`) are compile-time errors when visible, run-time traps otherwise; use `swapAt`
- Escaping closures can't capture `inout` parameters; non-escaping closures can
- For a class instance, `inout` lets the function replace the *reference*; mutating the object's properties doesn't need `inout`
- Methods marked `mutating` receive `self` as an implicit `inout` parameter
- Swift 6: actor-isolated or main-actor state can't be passed `inout` to an `async` call, since the access would span a suspension point where other code could run on the actor

Related to check: Mutating functions, value types and copy-on-write, memory safety and exclusivity, closures and escaping, actors and isolation, `borrowing`/`consuming` (noncopyable types)

Optional Chaining

Q: What is optional chaining? When would you use it?

Short answer: Optional chaining lets you access properties, call methods, use subscripts, or assign through an optional with `?.`. If any link in the chain is `nil`, the whole expression stops right there and evaluates to `nil`, so nothing to the right runs, including argument expressions.

Key points:

- Works for properties, methods, subscripts (`dict["key"]?.first`) and optional closures (`onTap?()`)
- A method returning `Void` becomes `Void?` through a chain, so you can test whether the call happened
- Assignment works too: `user?.address?.street = "Main St"`; the right-hand side isn't evaluated if the chain is `nil`
- Combine with `??` for a default and with `if let` / `guard let` to bind the result
- `self?.update()` inside `[weak self]` closures is optional chaining on a weak reference
- Key paths can include it: `\User.address?.street` (read-only)
- With async code: `await model?.load()` and `try await client?.fetch()`

Related to check: Optionals, nil coalescing, `guard let` / `if let`, `Optional.map` / `flatMap`, weak references and capture lists, key paths

---

Equatable and Hashable

Q: What are Equatable and Hashable? When do you need to conform to them?

Short answer: `Equatable` means a type defines `==`, which unlocks `contains`, `firstIndex(of:)`, `removeAll` comparisons and SwiftUI's change detection. `Hashable` refines `Equatable` with `hash(into:)`, which is required for `Set` elements, `Dictionary` keys and `ForEach`/`List` identity by value.

Key points:

- Synthesis: structs (all stored properties conform), enums (all associated values conform; payload-free enums get it automatically); never classes
- Custom equality -> hash exactly the properties == compares (or a subset), never more
- Implement `hash(into:)` with `hasher.combine(...)`; `hashValue` is the legacy requirement
- `Hasher` is randomly seeded per process: never persist hash values or rely on `Set` order
- `Comparable` is synthesized for enums without payloads (by declaration order) and with `Comparable` payloads
- Swift 6.4: `Equatable` and `Hashable` no longer require `Copyable`, so a `~Copyable` type can conform (synthesis included); it compiled back to an iOS 18 target, but `Set` and `Dictionary` still need copyable elements

Related to check: `Set` and `Dictionary`, `Comparable`, `Identifiable`, conditional conformance, non-copyable types, SwiftUI diffing

---

Collections: Array, Set, Dictionary

Q: What are the main collection types in Swift? When would you use Array vs Set vs Dictionary?

Short answer: Swift's three core collections are generic value types backed by copy-on-write buffers. `Array` is ordered and allows duplicates, with O(1) access by index; `Set` stores unique `Hashable` elements with O(1) average membership tests and no defined order; `Dictionary` maps unique `Hashable` keys to values with O(1) average lookup.

Key points:

- `Array.contains` is O(n); `Set.contains` and dictionary lookup are O(1) on average (hashing)
- `Set` and `Dictionary` iteration order is unspecified and changes between runs (per-process hash seeding)
- Dictionary subscript returns an optional; `dict[key, default: 0] += 1` updates in place
- Useful APIs: `Dictionary(grouping:by:)`, `Dictionary(uniqueKeysWithValues:)`, `mapValues`, `removeAll(where:)`, set algebra (`union`, `intersection`, `subtracting`), and `count(where:)` (Swift 6.0)
- All three are `Sendable` when their elements are, so they cross actors freely
- `InlineArray<N, Element>` (Swift 6.2, iOS 26+): fixed count in the type, stored inline, no heap allocation and no copy-on-write; it is not a `Sequence`, so you index it or read it through its `span` (with the Swift 6.4 toolchain, `for-in` works on it from iOS 27 via the new `Iterable` protocol)
- For ordered-and-unique or a deque, use `OrderedSet`/`OrderedDictionary`/`Deque` from the swift-collections package

Related to check: Copy-on-write, `Hashable`, `Collection`/`Sequence` protocols, lazy collections, `Span`, swift-collections

---

Associated Types in Protocols

Q: What are associated types in protocols? How do they differ from generic type parameters?

Short answer: An associated type is a placeholder type inside a protocol that each conforming type fills in, usually by inference from its method signatures. `Sequence.Element` and `Collection.Index` are the classic examples.

Key points:

- Declared with `associatedtype Item`, optionally constrained (`associatedtype Key: Hashable`) or defaulted
- Usually inferred from the conformer's members; `typealias Item = Int` makes it explicit
- Constrain them at use sites with `where` clauses: `where C.Item: Equatable`
- Primary associated types go in angle brackets on the protocol and enable `some P<Int>` / `any P<Int>`
- Protocols with associated types *can* be used as existentials (`any Collection`) since Swift 5.7; members that return the associated type come back type-erased, and members that take it as a parameter can't be called unless it's constrained
- Generic parameters: many specializations per type (`Array<Int>`, `Array<String>`); associated types: one per conformance

Related to check: Generics, opaque types (`some`), existentials (`any`), type erasure, protocol-oriented programming, `Self` requirements

---

static vs class

Q: What's the difference between static and class for methods and properties?

Short answer: Both declare type-level members that you call on the type rather than an instance. `static` works in structs, enums, classes, actors and protocols, and can't be overridden; in a class, `static func` is equivalent to `final class func`.

Key points:

- `static`: any type, not overridable, statically dispatched; works for stored and computed properties
- `class`: classes only, overridable, dynamically dispatched; methods and computed properties only
- `static` stored properties are lazily initialized on first access and thread-safe to initialize
- Protocol requirements use `static`; a class can satisfy them with `class` members so subclasses can override
- Swift 6: `static var` is an error unless the type or property is `@MainActor` (or another global actor), the value is a `static let` of a `Sendable` type, it's wrapped in a `Mutex`, or (last resort) `nonisolated(unsafe)`
- Generic types can't have static stored properties (computed ones are fine)

Related to check: Self vs self, access control, singletons and `Sendable` (concurrency), `Mutex`, mainactor basics, class inheritance

---

Implicitly Unwrapped Optionals

Q: What are implicitly unwrapped optionals (IUOs)? When would you use them?

Short answer: An implicitly unwrapped optional, written `String!`, is an ordinary `Optional<String>` with permission for the compiler to force-unwrap it wherever a non-optional is needed. It exists for values that are `nil` briefly during setup but are guaranteed to be set before use: `@IBOutlet`s, which are connected after `init` when the view loads, and two-phase initialization, where a property needs `self` to build it.

Key points:

- Since Swift 4.2 (SE-0054) `T!` is not a separate type; it's `T?` with an implicit-unwrap attribute on the declaration
- The compiler first type-checks it as an optional and force-unwraps only when the code wouldn't compile otherwise; `let copy = label` gives `String?`, not `String!`
- Optional chaining and binding still work: `label?.text`, `if let label`
- Main uses: `@IBOutlet weak var titleLabel: UILabel!`, two-phase init, and Objective-C APIs without nullability annotations (imported as `T!`)

- In SwiftUI and programmatic UIKit there's rarely a reason to declare one

Related to check: Optionals, force unwrapping, `@IBOutlet` and view life cycle, two-phase initialization, `unowned`, dependency injection, `lazy` properties

---

Type Erasure

Q: What is type erasure? When and why would you use it?

Short answer: Type erasure hides a value's concrete type behind a less specific one, so code can store, return or mix values without exposing (or even knowing) what they really are. The language does it for you with existentials: `any Shape` is a box that holds any conforming value.

Key points:

- Existentials (`any P`, `any Collection<Int>`) are compiler-provided type erasure; generics and `some P` keep the type
- Primary associated types (`protocol Store<Item>`) enable constrained existentials and opaque types, replacing most `AnyFoo<T>` structs
- Manual eraser = generic struct storing closures (or a boxed base class) that forward to the wrapped value
- Still common: `AnyPublisher` (`eraseToAnyPublisher()`) to hide long Combine operator chains at API boundaries; `AnyHashable` for mixed-type dictionary keys; `AnyView` when branches return different view types and `@ViewBuilder` or `some View` can't help
- Costs: heap boxing for large values, indirect calls, lost specialization, and lost type information (no == across erased values without extra work)
- In Swift 6, an eraser that stores closures must think about `Sendable` (`@Sendable` closures, `Sendable` constraint) if it crosses actors

Related to check: Associated types, opaque types, protocols and POP, protocol composition, generics, Combine `AnyPublisher`, SwiftUI `AnyView`

String vs NSString

Q: What's the difference between String and NSString? When would you use each?

Short answer: `String` is Swift's native text type: a value type (copy-on-write), `Sendable`, stored as UTF-8 since Swift 5, and modeled as a collection of `Characters`, where each `Character` is an extended grapheme cluster, what a user sees as one character. `NSString` is Foundation's immutable reference type (with `NSMutableString` as a mutable subclass), indexed by UTF-16 code units and compared literally by code unit.

Key points:

- `String` counts graphemes: `"👨‍👩‍👧 ".count == 1`, but its `utf16.count` (and `NSString.length`) is 4 and its `utf8.count` is 8
- `String ==` uses canonical equivalence: `"e\u{301}" == "é"` is `true`; `NSString.isEqual(to:)` compares code units and returns `false`
- `String` is indexed by `String.Index`, not `Int`, because characters vary in size; `count` is O(n)
- `NSString` uses `NSRange` (UTF-16 offsets); convert with `Range(nsRange, in: string)` and `NSRange(range, in: string)`
- Bridging is cheap but not free: a bridged `NSString` is wrapped lazily and may be converted to native UTF-8 on first heavy use
- `NSString` isn't `Sendable` (`NSMutableString` could hide behind it); `String` is
- `Substring` shares storage with its base string; convert to `String` before storing

- UTF8Span (Swift 6.2, iOS 26+): `string.utf8Span` gives safe, validated, non-copying access to the UTF-8 bytes for parsers

Related to check: String interpolation, value vs reference types, copy-on-write, ranges, Unicode and grapheme clusters, Swift Regex, `Sendable`

Opaque Types (some)

Q: What are opaque return types (some)? How do they differ from returning a protocol?

Short answer: `some Protocol` as a return type means "one specific concrete type that conforms to the protocol, chosen by the function, hidden from the caller". It's a reverse generic: with a generic parameter the *caller* picks the type, with an opaque result the *callee* does.

Key points:

- `func makeLabel() -> some View`: every `return` must produce the same concrete type
- `@ViewBuilder` makes `if/else` branches one type (`_ConditionalContent<A, B>`), which is why SwiftUI branches work
- `any View` doesn't conform to `View`, so it can't be used as a view in `body`; wrap in `AnyView` or restructure
- `some P` in **parameter** position is sugar for a generic parameter: `func show(_ v: some View)` ≈ `func show<V: View>(_ v: V)`
- Primary associated types work on both: `some Collection<Int>`, `any Sequence<String>`
- `any P` gives heterogeneous storage (`[any Shape]`); passing an `any P` to a `some P` parameter opens the existential implicitly (Swift 5.7)
- Swift 6.4 (SE-0521): optional opaque and existential types without extra parentheses, `some View?` instead of `(some View)?`
- The `ExistentialAny` upcoming feature flags a bare protocol type (`let s: Shape`); it's a warning in Swift 6 mode and planned as an error in a future language mode, so write `any Shape` everywhere today

Related to check: Generics, existentials (`any`), protocols with associated types, SwiftUI `View` and `@ViewBuilder`, type erasure, lazy collections

---

`some` vs `any`

Q: What's the difference between `some` and `any`? How do you decide which one to use?

Short answer: `some Shape` means "one specific concrete type that conforms to `Shape`, fixed at compile time", and `any Shape` means "a box that can hold a value of *any* conforming type, decided at run time". With `some`, the compiler keeps the real type, so calls are specialized and associated types stay precise; with `any`, you get flexibility (different types in one array, a property that can change type) and pay with an existential container, dynamic dispatch and erased type information.

Key points:

- `some P` in a parameter is an anonymous generic parameter (the *caller* picks the type); in a result or property it's an opaque type (the *implementation* picks it)
- `any P` is a value of a different type: the existential. It doesn't conform to `P` itself, so `Wrapper<any Shape>` with `T: Shape` fails ("type 'any Shape' cannot conform to 'Shape' "); `any Error` is a rare self-conforming exception
- Passing an `any P` to a `some P` / generic parameter *opens* the existential implicitly (SE-0352), so heterogeneous storage and generic algorithms mix well
- Primary associated types work with both: `some Collection<Int>`, `any Repository<User>`

- Existential layout on 64-bit: a 3-word inline buffer plus type metadata plus one witness table per protocol (40 bytes for `any Shape`); values that don't fit the buffer are boxed on the heap. A class-constrained existential is just a reference plus witness tables (16 bytes)
- Write `any` explicitly on every existential; the `ExistentialAny` upcoming feature warns on bare `Shape` in Swift 6 and is planned as an error in a future language mode

Related to check: Opaque types (`some` return types and SwiftUI), generics, type erasure, protocols with associated types, `Sendable`, dependency injection

---

map, flatMap, compactMap

Q: Explain map, flatMap, and compactMap. When would you use each?

Short answer: All three transform the contents of a container. `map` applies a function to each element and keeps the shape: n elements in, n out.

Key points:

- `map`: `[T] + (T) -> U -> [U]`; on `Optional`, transforms the wrapped value if present
- `compactMap`: `[T] + (T) -> U? -> [U]`; `compactMap { $0 }` strips `nil`s from `[U?]`
- `flatMap` on sequences: `(T) -> some Sequence` -> flattened array (`[[1, 2], [3]]` -> `[1, 2, 3]`)
- `flatMap` on `Optional` / `Result`: chain a step that itself returns an optional or result
- The Optional-returning overload of `Sequence.flatMap` was renamed `compactMap` in Swift 4.1; using `flatMap` that way no longer compiles cleanly
- Swift 6: these methods use typed throws, so `try strings.map(parse)` propagates `parse`'s exact error type
- On `AsyncSequence`: `for await value in stream.compactMap(decode)` transforms lazily as values arrive

Related to check: Optionals, `Result`, lazy collections, key paths, `AsyncSequence` operators, typed throws, higher-order functions (`filter`, `reduce`)

Nil Coalescing

Q: What is the nil coalescing operator? When would you use it?

Short answer: `a ?? b` unwraps `a` if it has a value and otherwise evaluates and returns `b`. The right-hand side is an `@autoclosure`, so it runs only when needed, and the operator is right-associative, so `a ?? b ?? c` tries each in turn.

Key points:

- Two overloads: `T? ?? T -> T` (non-optional result) and `T? ?? T? -> T?` (still optional)
- Short-circuits: the default is evaluated only when the left side is `nil`
- The right side can be a throwing call, marked with `try` before the whole expression: `try cached() ?? load()`
- The right side is a non-`async` autoclosure, so `cached ?? await fetch()` doesn't compile
- Precedence: `??` binds looser than arithmetic, so `bonus ?? 0 + 10` means `bonus ?? (0 + 10)`
- Pairs naturally with optional chaining: `user?.address?.city ?? "Unknown"`
- For dictionaries you mutate, `counts[key, default: 0] += 1` beats `counts[key] = (counts[key] ?? 0) + 1`

Related to check: Optionals, optional chaining, `guard let` / `if let`, `@autoclosure`, operator precedence, error handling

---

Mutating Functions

Q: What does mutating mean for struct methods? Why don't classes need it?

Short answer: Methods on structs and enums receive `self` as an immutable value by default. Marking a method `mutating` makes `self` an implicit `inout` parameter, so the method can change stored properties or even replace `self` entirely, and the new value is written back to the caller's variable.

Key points:

- `mutating` = `self` is `inout`; you can assign `self = ...` (common for enum state machines)
- Mutating calls require a `var`; on a `let`, or on a temporary returned from a function, they're compile errors
- Protocol requirements that value types may implement by mutating must say `mutating func`; class conformers implement them without the keyword
- `nonmutating set` declares a setter that doesn't modify `self` (it writes through a reference); SwiftUI's `State` and `Binding` use this so you can assign in `body`
- Escaping closures can't capture a mutating `self` (`Task { count += 1 }` inside a `mutating func` is an error); capture a copy or move the state into a class or actor
- Mutating triggers copy-on-write: `Array`, `Dictionary` and `String` copy their buffer only if it's shared
- For noncopyable types, `consuming func` takes ownership of `self` and `borrowing` (the default for methods) reads it

Related to check: `inout` parameters, value vs reference types, copy-on-write, protocols, property wrappers (`nonmutating set`), noncopyable types (`consuming`/`borrowing`), actors

---

Pattern Matching

Q: What pattern matching capabilities does Swift have? Give examples.

Short answer: Pattern matching in Swift is a general mechanism, not just `switch`: the same patterns work in `switch`, `if case`, `guard case`, `for case ... in`, `while case`, and `catch` clauses. You can destructure enums with associated values, tuples, optionals, and types (`is`/`as`), bind values with `let`, and refine any case with `where`.

Key points:

- Enum case patterns with bindings: `case .failure(let error)`, `case let .loaded(items)`
- Tuple patterns, wildcard _, and range/expression patterns (`case 0..<10`)
- Optional pattern `case let x?` and `case nil`
- Type-casting patterns: `case let image as UIImage`, `case is String`
- `where` adds a boolean guard to any case; multiple patterns per case with commas
- `if case` / `guard case` for a single pattern; `for case` to filter while iterating
- Expression pattern uses ~=, which you can overload for custom matching
- `switch` expressions: `let label = switch state { case .idle: "Idle" ... }`
- `@unknown default` for non-frozen enums from other modules (SDK enums)

Related to check: Enums and associated values, optionals, ranges, error handling (`catch` patterns), type casting, `if`/`switch` expressions

---

Subscripts

Q: What are subscripts in Swift? When would you use them?

Short answer: A subscript is a special member that gives a type bracket syntax, `value[key]`, the way `Array`, `Dictionary` and `String` do. It's declared with `subscript(...) -> T`, can be get-only or get/set, can take any number of labeled or unlabeled parameters, can be generic, overloaded by parameter type, `static`, `throws`/`async` (get-only), and can have default arguments.

Key points:

- Parameters are unlabeled by default (unlike functions); add an argument label explicitly: `subscript(safe index: Int)`
- Get-only subscripts can omit `get { }`; setters receive `newValue`
- Overload by parameter type or label: `grid[row, col]`, `dict[key, default: 0]`
- Generic subscripts: `subscript<T: Decodable>(key: String, as type: T.Type) -> T?`
- Get-only subscripts can be `throws`/`async` (`try await cache[url]`)
- Static/class subscripts work on the type itself (`Config["apiURL"]`)
- Key-path subscripts `value[keyPath: \.name]` and `@dynamicMemberLookup` are built on subscripts
- In a struct, the setter is implicitly `mutating`

Related to check: Collections, key paths, dynamic member lookup, ranges, copy-on-write

---

Key Paths

Q: What are key paths in Swift? How do you use them?

Short answer: A key path is a type-safe, first-class reference to a property: `\Person.name` is a value of type `KeyPath<Person, String>` that you can store, pass around, and apply later with `person[keyPath: path]`. Because the root and value types are part of the type, the compiler checks every use, unlike Objective-C's string-based KVC.

Key points:

- Class hierarchy: `AnyKeyPath` -> `PartialKeyPath<Root>` -> `KeyPath<Root, Value>` (read-only) -> `WritableKeyPath` (mutable value-type paths) -> `ReferenceWritableKeyPath` (writes through a class reference)
- Components chain properties, optional chaining and subscripts: `\User.address?.city`, `\[String].first`, `\.self`
- Key paths can be used as functions (SE-0249): `users.map(\.name)`, `items.filter(\.isDone)`
- `@dynamicMemberLookup` with key-path subscripts gives type-safe proxies (`Binding`, `@Bindable`)
- Swift 6: a key path *literal* is inferred `Sendable` when all its components are (SE-0418); the stored `KeyPath` type isn't, so write `KeyPath<Root, Value> & Sendable` when a stored or passed key path must cross isolation

Related to check: Higher-order functions (`map`, `filter`), `@dynamicMemberLookup`, SwiftUI bindings and `@Bindable`, SwiftData sort descriptors and predicates, `Sendable`, KVO

Protocol Composition

Q: What is protocol composition? How does it differ from inheritance?

Short answer: Protocol composition combines several requirements into one type with `&`, such as `Named & Aged`, meaning "any type that conforms to both" without defining a new protocol for the pair. You can also mix in one class (`UIView & Themeable`) or `AnyObject`.

Key points:

- `A & B` is a type, usable anywhere a protocol type is: parameters, properties, `typealias`

- Write `any A & B` for existential values and `some A & B` (or `<T: A & B>`) for generics
- Compositions can include at most one class type, or `AnyObject` for class-only conformers
- `Codable` is itself `typealias Codable = Decodable & Encodable`
- Common Swift 6 composition: `any Service & Sendable`, or constraining with `Sendable` so values can cross isolation
- Inheritance shares implementation and stored state; composition shares contracts (plus defaults from protocol extensions)
- A protocol can also *inherit* others (`protocol Person: Named, Aged {}`); prefer that only when the combined concept has a real name and extra requirements

Related to check: Protocols and POP, generics, opaque types, type erasure, `Sendable`, class inheritance

---

Self vs self

Q: What's the difference between Self and self in Swift?

Short answer: Lowercase `self` is a value: the current instance inside instance members, and the type object itself inside `static` members. Uppercase `Self` is a type: the type of the current context.

Key points:

- `self`: the instance (or, in a `static` method, the metatype value); required in escaping closures unless captured, and to disambiguate shadowed names
- `Self`: the current type; `Self.defaultValue` reads a static member, `Self(...)` calls an initializer
- In protocols, `Self` refers to the conforming type (`static func == (lhs: Self, rhs: Self)` in `Equatable`)
- Protocols with `Self` requirements in parameter positions must be used as `some P`/generics or `any P` (with members that use `Self` unavailable on the existential)
- In classes, `–> Self` means "the dynamic type": you must return `self` or create it through a `required init`
- `type(of: self)` gives the dynamic metatype at runtime; `Self.self` is the metatype value of `Self`
- `Self` is also usable inside instance methods of structs/enums to refer to static members without repeating the name

Related to check: Static vs class, protocols and POP, associated types, capture lists, equatable and hashable, metatypes

---

Copy-on-Write

Q: What is copy-on-write? How does Swift use it?

Short answer: The language guarantees *value semantics* for structs and enums: after `var b = a`, changing `b` never changes `a`. Copy-on-write is how the standard library keeps that guarantee cheap for large values: `Array`, `Dictionary`, `Set` and `String` share one heap buffer between copies and only duplicate it when someone mutates while it's shared.

Key points:

- Value semantics is the contract; COW is one implementation of it
- Assignment, passing as an argument and capturing share the buffer: O(1)
- A `mutating` method checks uniqueness first: unique -> mutate in place; shared -> copy, then mutate
- Custom COW: a struct wrapping a `final class` storage, checked with `isKnownUniquelyReferenced(&storage)` before every write
- `consume x` (Swift 5.9) ends a local's lifetime, so the remaining value's buffer is unique and the next mutation doesn't copy

- Noncopyable types (~Copyable) can't be copied implicitly at all, so there's nothing to copy-on-write

Related to check: Value vs reference types, mutating and inout, ownership (consume, borrowing, ~Copyable), memory-performance/copy-on-write

---

Ranges

Q: How do ranges work in Swift? Explain closed, half-open, and one-sided ranges.

Short answer: A range is an interval between two bounds of a Comparable type. a...b is a ClosedRange that includes both ends, a..<b is a half-open Range that excludes the upper bound, and one-sided ranges (a..., ..<b, ...b) leave one end open so a collection fills it in.

Key points:

- ClosedRange (1...5) can't be empty; Range (1..<5) can (3..<3)
- lowerBound > upperBound traps at runtime when the range is formed
- One-sided: PartialRangeFrom (2...), PartialRangeUpTo (..<3), PartialRangeThrough (...3)
- Integer ranges are collections (count, map, reversed()); 0.0...1.0 is only for contains/~= and clamping
- Collection slicing: array[2...], array[..<3]; slices (ArraySlice, Substring) keep the original indices and share storage
- RangeExpression lets APIs accept any range kind; RangeSet (iOS 18) represents non-contiguous ranges
- stride(from:to:by:) / stride(from:through:by:) for steps other than 1 or descending loops

Related to check: Pattern matching, subscripts, collections, string vs NSString (String indices), Strideable, RangeSet

---

Sequence and IteratorProtocol

Q: What are Sequence and IteratorProtocol? How do they relate?

Short answer: IteratorProtocol is the engine: a type with a mutating func next() -> Element? that hands out one element at a time and returns nil when it's done. Sequence is the thing you can iterate: its one requirement is makeIterator(), and in exchange you get for-in plus the whole standard library toolbox (map, filter, reduce, contains, first(where:), lazy, ...) for free.

Key points:

- Sequence has primary associated type Element (some Sequence<Int>) and an associated Iterator
- A type can be its own iterator: conform to both and makeIterator() is provided
- Sequence makes no promise of multiple passes or non-destructive iteration; Collection adds indices, count, and repeatable traversal
- AnyIterator { ... } and sequence(first:next:) / sequence(state:next:) build sequences from closures without new types
- .lazy defers map/filter work until elements are pulled
- Swift 6 iterators are usually value types; an iterator is not shared between tasks
- AsyncSequence (AsyncStream, URL.lines, NotificationCenter.notifications) is the async counterpart; AsyncSequence<Element, Failure> has typed failure since Swift 6

Related to check: Lazy collections, collections, map/flatMap/compactMap, AsyncSequence and AsyncStream (concurrency), generics

---

Optional map and flatMap

Q: How do map and flatMap work on Optional?

Short answer: `Optional` is an enum (`.some(Wrapped)` / `.none`), and `map` and `flatMap` let you transform the wrapped value without unwrapping it first. `map` applies a `(Wrapped) -> U` closure when there's a value and gives back `U?`; if the optional is `nil`, the closure never runs.

Key points:

- `map`: `(Wrapped) throws -> U` gives `U?`; `nil` in, `nil` out, closure skipped
- `flatMap`: `(Wrapped) throws -> U?` gives `U?`; use it when the transform can fail (`Int.init`, `URL(string:)`, a dictionary lookup)
- Using `map` with an optional-returning closure gives a nested `U??`, which is almost never what you want
- Both are rethrowing (typed throws in Swift 6), so `try` passes through
- Optional chaining (`user?.address?.city`) is `flatMap` sugar for member access
- Don't confuse with `Sequence.flatMap` (flattens arrays) or `compactMap` (drops `nil`s from a sequence); the old `Sequence.flatMap` that dropped `nil`s was renamed to `compactMap`

Related to check: Optionals and optional binding, optional chaining, map/flatMap/compactMap, nil coalescing, `Result.map`/`flatMap`

Autoclosure

Q: What is @autoclosure? When would you use it?

Short answer: `@autoclosure` makes the compiler wrap an argument expression in a closure automatically, so the caller writes a plain expression but the function decides whether, and when, to evaluate it. That's how the standard library gets short-circuiting: the right side of `??`, `&&` and `||`, and the condition and message of `assert` and `precondition`, are autoclosures.

Key points:

- The parameter type is a closure with no arguments: `_ message: @autoclosure () -> String`
- The argument is evaluated zero, one or many times, depending on how often the function calls it
- Add `@escaping` if the closure is stored and called later
- It can throw (`@autoclosure () throws -> T` plus `rethrows`, as `??` does) or be async (`@autoclosure () async -> T`, so callers write `await` inside the argument)
- You can't pass an existing closure variable directly; you pass a call expression

Related to check: Closures and escaping, `??` and short-circuit evaluation, `assert`/`precondition`, `rethrows`, lazy properties

---

String Interpolation

Q: How does string interpolation work in Swift? Can you customize it?

Short answer: `"Hello, \(name)"` isn't concatenation: the compiler turns the literal into a series of calls on an interpolation builder, `appendLiteral("Hello, ")` then `appendInterpolation(name)`, and builds the result from that. For `String` the builder is `DefaultStringInterpolation`, which prints values through `CustomStringConvertible` (or `TextOutputStreamable`), falling back to `String(describing:)`.

Key points:

- Any expression works inside `\( )`; interpolated values must be printable (everything is, via `String(describing:)`)
- `CustomStringConvertible.description` controls `\(value)`; `CustomDebugStringConvertible` controls `debugPrint` and `\(String(reflecting:))`
- Extend `DefaultStringInterpolation` (aka `String.StringInterpolation`) with `mutating func appendInterpolation(...)`; extra labels become interpolation arguments
- Custom types adopt `ExpressibleByStringInterpolation` with a nested `StringInterpolation: StringInterpolationProtocol`
- Interpolating an optional prints `Optional("x")` and the compiler warns; unwrap or use `?? ""`
- Raw strings `#"\d+ \#(value)"#` and multi-line `"""` literals support interpolation too
- `Logger` interpolation is formatted lazily and redacts dynamic strings and objects by default; set it per value, e.g. `\(token, privacy: .private)`

Related to check: String vs NSString, `CustomStringConvertible`, `FormatStyle`, `os.Logger` privacy, String Catalogs / localization, custom operators

---

Custom Operators

Q: How do you define custom operators in Swift?

Short answer: There are two different things: *overloading* an existing operator for your type (+ for a `Vector`), and declaring a *new* operator. A new operator is declared once at file scope as `prefix`, `infix` or `postfix`, an infix operator gets a `precedencegroup` that defines how it binds relative to others, and the implementation is a `static func` on a type (or a global function).

Key points:

- Declaration: `infix operator •: DotProductPrecedence` at file scope; then implement `static func •` in the type
- `precedencegroup` sets `higherThan`/`lowerThan` (relative to groups like `AdditionPrecedence`), `associativity` (`left`, `right`, `none`) and `assignment`
- Infix operators without a group get `DefaultPrecedence`
- Compound assignment takes `inout`: `static func += (lhs: inout Vector, rhs: Vector)`
- Operators are allowed as protocol requirements (== in `Equatable`, < in `Comparable`)
- Overload ~= to make your own types work as `switch` patterns
- Some tokens are reserved: `=`, `->`, `.`, `//`, `/*`, prefix `&`, postfix `?` and `!`; the ternary `?:` can't be overloaded

Related to check: `Equatable`/`Comparable` operator requirements, pattern matching (~=), `inout`, result builders, type-checker performance

---

Lazy Collections

Q: What are lazy collections? When would you use them?

Short answer: Calling `.lazy` on a sequence or collection returns a wrapper (`LazySequence`, `LazyCollection`) whose `map`, `filter`, `compactMap` and friends don't run immediately. Instead they build a lightweight pipeline that computes each element only when someone iterates or subscripts it.

Key points:

- Eager `map`/`filter` allocate a full array at each step; lazy versions allocate nothing until consumed
- Work is driven by the consumer: `first`, `prefix`, `for-in`, or `Array(...)` to materialize
- Closures are stored and re-run on every access; side effects run again too

- `LazyFilterCollection.count`, `startIndex` and `isEmpty` may scan the base; subscripting a `LazyMapCollection` recomputes the element, so index-based iteration can call the closures more than once per element
- Lazy views store their closures, which usually makes them non-`Sendable`; materialize with `Array(...)` before sending results across actors
- Not the same as a `lazy var` property (computed once, then stored)
- `AsyncSequence` operators (`map`, `filter`, `compactMap` on an `AsyncStream`) are lazy by nature

Related to check: `map` / `filter` / `compactMap`, `Sequence` and `IteratorProtocol`, opaque types, `AsyncSequence`, `lazy var` properties, `Sendable`

@MainActor Basics

Q: What is @MainActor? When do you need it?

Short answer: `@MainActor` is an attribute that says "this code runs on the main thread", and the compiler enforces it. UI must only be touched from the main thread, so views, view controllers and the models that feed them are `@MainActor`.

Key points:

- Put it on a type (every member runs on main), a single function, or a closure
- SwiftUI `View`, `UIView` and `UIViewController` are already `@MainActor`
- From main-actor code, calling other main-actor code needs no `await`
- From elsewhere, calling main-actor code needs `await` (the compiler inserts the hop)
- A `Task { }` started in main-actor code also runs on the main actor
- It replaces `DispatchQueue.main.async` in new code

Related to check: Actors, `async`/`await`, Task, `nonisolated`, MainActor (deep dive in Concurrency)

Capture Lists

Q: What are capture lists in closures? When do you need them?

Short answer: By default a closure captures *variables*, not values: it keeps a strong reference to each captured object and sees later changes to captured `var`s. A capture list, `[weak self, count]`, changes that at the moment the closure is created: `weak`/`unowned` capture an object without retaining it, and a plain name captures the current value as an immutable copy.

Key points:

- `[weak self]` -> `self` is optional inside; `guard let self else { return }` then implicit `self` works (Swift 5.8)
- `[unowned self]` -> non-optional, crashes if `self` is gone; only for guaranteed lifetimes
- `[count]` or `[total = items.count]` -> value captured at creation, immutable inside
- Cycles need *storage*: self -> closure -> self. Non-escaping closures (`map`, `forEach`) and short-lived `Task`s don't create cycles
- Swift 6: closures that run concurrently (`@Sendable` or `sending`, such as task-group children and `Task.detached`) can't capture a local `var` the caller can still modify; capture a copy with `[value]`
- Values captured by a closure that crosses isolation must be `Sendable` or provably not used elsewhere (region-based isolation); in practice `[weak self]` in a `Task` is used from `@MainActor` or actor-isolated types

Related to check: weak vs unowned, retain cycles, closures and escaping, `Sendable` closures, `Task` lifetime and cancellation

---

Never Type

Q: What is the Never type? When is it used?

Short answer: `Never` is an *uninhabited* type, declared as an enum with no cases, so no value of it can ever exist. A function that returns `Never`, like `fatalError`, tells the compiler control never comes back, which is why it satisfies `guard`'s "must exit" rule and needs no dummy return value.

Key points:

- No values: a `switch` over `Never` needs no cases; a `Result<T, Never>` switch needs only `.success`
- Never-returning functions: `fatalError`, `preconditionFailure`, `exit`, `dispatchMain`, infinite run loops
- `Never` conforms to `Error`, `Equatable`, `Hashable`, `Comparable`, `Codable`, `Sendable` and others, so it can fill any slot those constraints require
- Typed throws (Swift 6): `throws(Never)` ≈ non-throwing; `throws(any Error)` ≈ `throws`
- `Task<Value, Never>.value` and `Result<T, Never>.get()` need no `try`
- Combine's `Publisher<Output, Never>` (e.g. `@Published`) can be assigned to properties because it can't fail

Related to check: Error handling and typed throws, `Result`, `guard`, `Task` and `AsyncStream` failure types, Combine publishers, assertions and preconditions

---

Dynamic Member Lookup

Q: What is @dynamicMemberLookup? When would you use it?

Short answer: `@dynamicMemberLookup` lets a type answer property-style access, `value.name`, for members it doesn't declare, by routing the access through a `subscript(dynamicMember:)`. There are two flavors.

Key points:

- Put the attribute on the type's primary declaration and implement `subscript(dynamicMember:)`
- `String` form: no compile-time checking; a typo becomes a runtime `nil` or wrong value
- Key-path form (Swift 5.1): `subscript<T>(dynamicMember kp: KeyPath<Wrapped, T>) -> T`, fully type-checked
- Add a `WritableKeyPath` overload with a setter to support assignment through the wrapper
- Can be combined with `@dynamicCallable` for dynamic-language interop
- Real uses: `Binding` / `Bindable` projections, lenses and wrappers, scripting interop, JSON trees

Related to check: Key paths, subscripts, SwiftUI `Binding`/`Bindable`, `@dynamicCallable`, property wrappers

---

Property Observers

Q: What are willSet and didSet? How do they differ from computed properties?

Short answer: `willSet` and `didSet` are observers attached to a stored property: `willSet` runs just before the new value is stored (with `newValue`), `didSet` runs just after (with `oldValue`). They're for reacting to a change, like clamping, logging or syncing a view.

Key points:

- Observers attach to stored properties (`var`, not `let`), including inherited stored *or computed* properties via `override`
- Not called during the type's own `init` (when setting its own properties), only after initialization
- `didSet` can reassign the property (e.g. clamping) without re-triggering itself
- Passing an observed property `inout` triggers the observers on write-back, even if unchanged
- Changing one element of an observed array or a field of an observed struct triggers `didSet` for the whole property (value semantics)
- Computed properties: no storage, can't have observers; use the setter instead
- Work alongside property wrappers and on `lazy` properties, but never on `let` constants

Related to check: Computed vs stored vs lazy, property wrappers, Observation (`@Observable`, `withObservationTracking`, `Observations`), inout parameters, KVO, Combine `@Published` (legacy)

Nested Types

Q: When would you use nested types in Swift?

Short answer: A nested type is a type declared inside another type, like `APIClient.Endpoint` or a `CodingKeys` enum inside a `Codable` struct. I use them when a type only makes sense in the context of its outer type: it scopes the name (no global `Endpoint`, `Error` or `Style` collisions), groups related code, and lets access control hide implementation details with `private` nested types.

Key points:

- Refer to it as `Outer.Inner` from outside; inside the outer type, just `Inner`
- Common uses: `CodingKeys`, per-type error enums, configuration and style structs, view-model `State` enums, `private` helper types
- Any type kind can nest (struct, enum, class, actor), and extensions can add nested types later
- Nested types inside a generic type can use the outer type's parameters (`Tree<Value>.Node` uses `Value`)
- Protocols can be nested in non-generic types since Swift 5.10 (SE-0404)
- A caseless `enum` is the idiomatic pure namespace (it can't be instantiated)
- Each nested type has its own conformances: nesting doesn't make it `Sendable` or `Codable`

Related to check: Access control, `Codable` and `CodingKeys`, enums, generics, extensions, module selectors (Swift 6.3)

Availability Checking

Q: How do you check API availability in Swift?

Short answer: Every SDK symbol carries an `@available` annotation, and the compiler compares it with your deployment target: if you target iOS 18 and call an iOS 26 API, the build fails until you guard the call. `if #available(iOS 26, *)` (or `guard`, or `#unavailable` for the inverse) is a runtime check that also tells the compiler the guarded code is safe.

Key points:

- `#available(iOS 26, *)`: `*` means "every other platform, at its minimum deployment target"
- `#unavailable(iOS 26)` (Swift 5.6) reads better than an empty `if #available {} else {}`
- `@available(iOS 26, *)` on a function, type or extension; `@available(*, deprecated, renamed: "new()")` and `@available(*, unavailable)` for API evolution
- Version numbers jumped from **iOS 18 to iOS 26** (Apple moved to year-based versions in 2025); there was no iOS 19 - 25

- Swift 6.4 accepts `anyAppleOS` (`@available(anyAppleOS 26, *)`) now that all Apple platforms share one version number; the iOS 27 SDK uses it itself
- Language features (typed throws, `if`/`switch` expressions, `package`, noncopyable types) need a newer *compiler*, not a newer OS. Runtime features (`InlineArray`, `Span`, integer generic parameters: iOS 26+; `UniqueArray`: iOS 27+) need the OS
- Some new APIs back-deploy: `@backDeployed(before:)` or inlinable wrappers ship the implementation in your app (e.g. Xcode 27's item-bound `alert(_:item:...)` compiles at an iOS 18 target)

Related to check: Deployment target vs base SDK, backward compatibility, `#if` conditional compilation, `@backDeployed`, API deprecation

---

UIKit & Views

View Lifecycle

Q: Explain the UIViewController and UIView lifecycle. What methods are called and when?

Short answer: A view controller is created, loads its view lazily (`loadView` -> `viewDidLoad`), then cycles through appearance callbacks every time it's shown or hidden: `viewWillAppear` -> `viewIsAppearing` -> layout -> `viewDidAppear`, and `viewWillDisappear` -> `viewDidDisappear`. Layout callbacks (`viewWillLayoutSubviews` / `viewDidLayoutSubviews`) run many times, whenever the size or content changes.

Key points:

- `loadView`: create the root view; override only when building it in code without calling `super`
- `viewDidLoad`: once per load. One-time setup: subviews, constraints, data sources. Bounds and traits aren't final yet
- `viewWillAppear`: before the transition starts. The view is not in the window yet, so traits and size may be stale
- `viewIsAppearing` (back-deployed to iOS 13): the view is in the hierarchy with accurate traits and size. The best place for appearance-time UI updates and scrolling to an initial position
- `viewWillLayoutSubviews` / `viewDidLayoutSubviews`: every layout pass, so keep them cheap and idempotent
- `viewDidAppear` / `viewWillDisappear` / `viewDidDisappear`: start or stop things that should only run on screen (analytics, timers, camera)
- `updateProperties()` (iOS 26+): runs before `layoutSubviews`; request it with `setNeedsUpdateProperties()`. Exists on both `UIView` and `UIViewController`
- Automatic observation tracking (on by default in iOS 26; opt in on iOS 18 with the `UIObservationTrackingEnabled` Info.plist key): reading `@Observable` properties in `updateProperties()`, `layoutSubviews`, `viewWillLayoutSubviews` and similar methods makes UIKit invalidate and re-run them when those properties change
- UIView side: `init` -> `willMove(toSuperview:)` / `didMoveToSuperview` -> `didMoveToWindow` -> `layoutSubviews` -> `draw(_:)` (only if needed)

Related to check: Layout cycle, View controller containment, Auto Layout, UITraitCollection (automatic trait tracking), Observation (`@Observable`), Memory warnings

---

Automatic Observation Tracking

Q: How does automatic observation tracking work in UIKit, and how does it change the way you bind a model to a view?

Short answer: UIKit can now track `@Observable` models the way SwiftUI does: when one of its update methods runs (`updateProperties()`, `layoutSubviews()`, `viewWillLayoutSubviews()`, a configuration update handler, and similar), UIKit records which observable properties the method *read*, and when any of them changes it invalidates that same pass, so the method runs again on the next update cycle. There's no `didSet`, KVO, Combine sink or delegate callback to write.

Key points:

- **Per property, per method:** only properties actually read during that call are tracked, and a change re-runs only the pass that read it (properties -> `updateProperties`, geometry -> layout)
- **Tracked methods** include `updateProperties()` (iOS 26+) on views and view controllers, `layoutSubviews()`, `viewWillLayoutSubviews()` / `viewDidLayoutSubviews()`, `updateConstraints()`, `draw(_:)`, and configuration updates such as a button's or cell's `configurationUpdateHandler` and `updateConfiguration(using:)`
- **Coalesced:** many changes in one run-loop turn produce one update; nothing runs synchronously inside the setter
- **Manual requests** still exist: `setNeedsUpdateProperties()` and `updatePropertiesIfNeeded()` (iOS 26+), alongside `setNeedsLayout()` / `layoutIfNeeded()`
- **Same model in both frameworks:** one `@MainActor @Observable` class can drive a SwiftUI view and a UIKit view controller during a migration

Related to check: Layout cycle (`updateProperties`, `setNeedsLayout`), View lifecycle, Observable macro, ObservableObject and `@Published`, UITraitCollection (automatic trait tracking), Cell reuse (`configurationUpdateHandler`), MVVM

Auto Layout

Q: How does Auto Layout work? What are constraints and how do you create them?

Short answer: Auto Layout describes layout as a set of linear relationships, `item1.attribute = multiplier x item2.attribute + constant`, each with a priority, and a constraint solver (based on Cassowary) computes frames that satisfy them. Each view needs enough constraints to fix its position and size, either explicitly or through its intrinsic content size.

Key points:

- Constraints relate two items (views or `UILayoutGuides`) or set a constant size; relations ==, <=, >=
- Anchors are type-safe: `NSLayoutXAxisAnchor`, `NSLayoutYAxisAnchor`, `NSLayoutDimension` can't be mixed
- `translatesAutoresizingMaskIntoConstraints = false` for views you constrain in code, or the autoresizing mask adds conflicting constraints
- Priorities (1 - 1000) let optional constraints break; intrinsic size adds hugging and compression-resistance constraints
- The engine runs as part of the layout pass: constraint changes mark layout dirty, and frames are set in `layoutSubviews`
- Guides: `safeAreaLayoutGuide`, `layoutMarginsGuide`, `readableContentGuide`, `keyboardLayoutGuide`, and in iOS 26 corner-adaptive guides from `layoutGuide(for:)`

Related to check: Layout cycle, Intrinsic content size, Constraint priority, Stack views, Safe area, Size classes, Keyboard handling

---

UITableView vs UICollectionView

Q: When would you use UITableView vs UICollectionView?

Short answer: For new UIKit screens, the default is `UICollectionView` for both lists and grids: a list is just a collection view with a list layout (`UICollectionLayoutListConfiguration`), and it gets the same table-style cells, swipe actions, accessories and sidebar styles. Collection views also handle multi-column grids, carousels and mixed sections that adapt to the available width, which matters now that iPhone apps resize (iPhone Mirroring, iPad, iPhone Duo).

Key points:

- **Collection view + list configuration:** `.plain`, `.grouped`, `.insetGrouped`, `.sidebar`, `.sidebarPlain` appearances; `UICollectionViewListCell` with accessories; swipe actions via `trailingSwipeActionsConfigurationProvider`.
- **Collection view extras:** compositional layout (grids, orthogonal carousels, per-section layouts that react to container width), `CellRegistration`, outline/expandable sections via section snapshots.
- **Table view:** single column only, built-in `.plain` / `.grouped` / `.insetGrouped` styles, simple API, mature editing/reordering support.
- **Shared:** cell reuse, diffable data sources, prefetching, content configurations (`UIListContentConfiguration`), `UIHostingConfiguration`.
- SwiftUI's `List` is the third option; you can also host SwiftUI content in collection view cells.

Related to check: UICollectionView list configuration, Compositional layout, Cell reuse, Diffable data source, Prefetching, SwiftUI List

---

Cell Reuse

Q: How does cell reuse work in UITableView/UICollectionView? Why is it important?

Short answer: Table and collection views keep only the visible cells (plus a small buffer) alive; when a cell scrolls off screen it goes into a reuse pool, and the next cell that's needed is dequeued from that pool and reconfigured. That keeps memory flat and scrolling smooth no matter how many items you have.

Key points:

- `CellRegistration` + `dequeueConfiguredReusableCell(using:for:item:)` replaces string identifiers, `register(_:forCellWithReuseIdentifier:)` and force casts; the handler is typed to the cell and item.
- Content configurations (`defaultContentConfiguration()` / `UIListContentConfiguration`) are value types: you build a fresh one and assign it, so stale state is replaced wholesale.
- `configurationUpdateHandler` runs whenever the cell's `UICellConfigurationState` changes (selected, highlighted, editing, disabled), the modern alternative to overriding `isSelected`.
- `UIHostingConfiguration` lets a cell's content be a SwiftUI view while UIKit still owns reuse, sizing and selection.
- Classic API still asked: `dequeueReusableCell(withIdentifier:for:)`, `prepareForReuse()`; `UITableView` has no `CellRegistration`, but its cells support content configurations too.

Related to check: Diffable data source, UICollectionView list configuration, UITableView/UICollectionView prefetching, UIHostingConfiguration, Compositional layout

---

Empty, Loading and Error States

Q: How do you show empty, loading and error states in a UIKit screen? And in SwiftUI?

Short answer: Both frameworks have a system answer, so you don't hand-build a centered label and spinner any more. In UIKit, a view controller has a `contentUnavailableConfiguration`: assign a `UIContentUnavailableConfiguration` (`.empty()`, `.loading()` or `.search()`, then customize image, text,

secondary text and buttons) and UIKit overlays a standard, Dynamic Type - aware view; set it to `nil` to show the content again.

Key points:

- **Model the phase explicitly:** loading, loaded (possibly empty), failed. Most bugs come from inferring state from `items.isEmpty` alone (an empty list while loading looks like "no data")
- **UIKit presets:** `.empty()` (image + text + optional buttons), `.loading()` (spinner + text), `.search()` (standard no-results look); properties include `image`, `text`, `secondaryText`, `button` / `secondaryButton` (`UIButton.Configuration`) with `buttonProperties.primaryAction`, and `background`
- **State:** `UIContentUnavailableConfigurationState` carries the trait collection and `searchText`, which defaults to the text of the view controller's search controller, so a search screen can switch between "no results" and "nothing here yet"
- **Update cycle:** `setNeedsUpdateContentUnavailableConfiguration()` coalesces requests, and UIKit also calls the update method itself when the state may have changed (for example search text); never call `updateContentUnavailableConfiguration(using:)` directly
- **SwiftUI:** `ContentUnavailableView("Title", systemImage:description:)` or the `label:description:actions:` form; `.search` uses the current `.searchable` text automatically, `.search(text:)` takes it explicitly

Related to check: UIRefreshControl, Diffable data source, SwiftUI List and `searchable`, Task modifier (loading), Error handling in networking, Accessibility in UIKit, UIHostingConfiguration

---

Safe Area

Q: What is the safe area? Why is it important?

Short answer: The safe area is the part of a view that isn't covered by system UI or hardware: the status bar, the Dynamic Island, the home indicator, rounded display corners, and the navigation, tab and toolbars your own containers add. Interactive content and text should sit inside it, while backgrounds and scrolling content extend to the edges and flow under the bars, which matters more with Liquid Glass bars in iOS 26.

Key points:

- `safeAreaLayoutGuide` for constraints; `safeAreaInsets` to read the values; `safeAreaInsetsDidChange()` / `viewSafeAreaInsetsDidChange()` to react
- `additionalSafeAreaInsets` on a view controller adds your own chrome (e.g. a custom bottom player bar) so children lay out around it
- Layout margins sit inside the safe area by default (`insetsLayoutMarginsFromSafeArea`); `directionalLayoutMargins` and `readableContentGuide` for text
- Scroll views apply the safe area as `adjustedContentInset` (`contentInsetAdjustmentBehavior`), so content starts below the bars but scrolls under them
- iOS 26+: `layoutGuide(for: .safeArea(cornerAdaptation:))` and `.margins(cornerAdaptation:)` return guides that also avoid window corners (rounded window corners and window controls on iPadOS 26), adapting along the axis you choose. `edgeInsets(for:)` reads the same region as insets
- Liquid Glass (iOS 26): standard bars are translucent glass; content should scroll beneath them. `UIBackgroundExtensionView` (iOS 26+) extends content under a sidebar or inspector

Related to check: Auto Layout, Layout margins, Scroll view content insets, Keyboard handling (`keyboardLayoutGuide`), Liquid Glass bars, Rotation and orientation

---

UIStackView

Q: What is UIStackView? When would you use it?

Short answer: `UIStackView` lays out its arranged subviews in a row or column and generates the Auto Layout constraints for you, based on `axis`, `distribution`, `alignment` and `spacing`. It's the quickest way to build linear layouts that adapt: hiding an arranged view collapses its space (and animates), and flipping `axis` turns a row into a column when the window gets narrow or the text gets large.

Key points:

- `axis`: `.horizontal` / `.vertical`; `spacing` plus `setCustomSpacing(_:after:)` for individual gaps
- `distribution` (along the axis): `.fill`, `.fillEqually`, `.fillProportionally`, `.equalSpacing`, `.equalCentering`
- `alignment` (across the axis): `.fill`, `.leading`/`.top`, `.center`, `.trailing`/`.bottom`, `.firstBaseline`, `.lastBaseline`
- Sizes come from arranged views' intrinsic content size plus hugging and compression-resistance priorities
- `isHidden` on an arranged subview removes it from layout; animate it inside `UIView.animate`
- `isLayoutMarginsRelativeArrangement` + `directionalLayoutMargins` for padding; `backgroundColor` and corner radius work since iOS 14
- `removeArrangedSubview(_:)` only stops managing the view; call `removeFromSuperview()` to remove it

Related to check: Auto Layout, Intrinsic content size, Constraint priority, Dynamic Type, Size classes, SwiftUI `ViewThatFits`

Responder Chain

Q: What is the responder chain? How does it work?

Short answer: The responder chain is the linked path of `UIResponder` objects, each pointing to its `next` responder, along which UIKit routes events that aren't tied to a specific view. Touches go to the view that hit testing found; key presses, edit menu commands, shake and remote-control events go to the first responder; and an action sent with a `nil` target walks up from the first responder until something implements it.

Key points:

- First responder: the object that receives key and editing events (a focused text field, or a view that returns `canBecomeFirstResponder == true` and called `becomeFirstResponder()`)
- Hit testing chooses the view for a touch; the responder chain is what happens when that view doesn't handle it
- Nil-targeted actions: `UIApplication.shared.sendAction(_:to: nil, from:for:)`, or `UIControl`/`UIAction` with no target
- `canPerformAction(_:withSender:)` and `target(forAction:withSender:)` decide who answers; `validate(_ command:)` updates menu state
- `keyCommands` is a **property** you override, not a method

Related to check: Hit testing, UIControl target-action (nil targets), UIKeyCommand / UIMenuBuilder, UIScene life cycle, Gesture recognizers

---

Hit Testing

Q: How does hit testing work? Explain hitTest(_:with:) and point(inside:with:).

Short answer: Hit testing finds the deepest view under a touch, and that view becomes the first responder for the touch. The window calls `hitTest(_:with:)` on itself.

Key points:

- `point(inside:with:)`: is this point (in my coordinates) inside me? Default is `bounds.contains(point)`
- `hitTest(_:with:)`: returns the frontmost, deepest view that contains the point, or `nil`
- Order: subviews from last to first (front to back), depth-first
- Skipped: `isHidden`, `isUserInteractionEnabled == false`, `alpha < 0.01`
- The hit view's gesture recognizers, and those of its ancestors, see the touch too, so recognizers on a superview can win over a control's touches

Related to check: Responder chain, Gesture recognizers, UIGestureRecognizerDelegate (`shouldReceive`), Accessibility (target sizes), View hierarchy

---

Storyboards vs Programmatic UI

Q: What are the tradeoffs between Storyboards and programmatic UI?

Short answer: Storyboards and XIBs give you visual layout, segues and quick iteration, but they're XML that merges badly, fails at runtime instead of compile time (missing outlets, wrong identifiers) and makes dependency injection awkward. Programmatic UIKit is plain Swift: it diffs and reviews cleanly, composes into reusable components and takes dependencies through `init`.

Key points:

- Storyboards: visual, segues, `@IBOutlet`/`@IBAction`, Interface Builder previews; merge conflicts, stringly typed identifiers, runtime crashes, slow to open when large
- XIBs: one view or cell per file, a middle ground that's still common for cells
- Programmatic: compile-time checked, testable, `init`-based dependencies, `#Preview` works for UIKit view controllers and views too
- SwiftUI: default for new UI; hosts inside UIKit via `UIHostingController` (screens) and `UIHostingConfiguration` (cells)
- Launch screen: a `LaunchScreen.storyboard` or the `UILaunchScreen` Info.plist dictionary; a launch screen is required for App Store submission starting with iOS 27

Related to check: Auto Layout, Segues (`@IBSegueAction`), IBOutlet weak vs strong, `UIHostingController` / `UIHostingConfiguration`, UIScene life cycle, Adaptive layout / iPhone Duo

---

Size Classes

Q: What are size classes? How do you use them for adaptive layout?

Short answer: Size classes are coarse traits, **compact** or **regular**, for the horizontal and vertical space available to a window or view controller. They describe the space, not the device: an iPhone in portrait is compact width, a full-screen iPad is regular in both directions, but a narrow iPad window is compact, and in iOS 27 iPhone apps resize (iPhone Mirroring, iPad, iPhone Duo), so an iPhone app can see regular width too.

Key points:

- Two traits: `horizontalSizeClass` and `verticalSizeClass`, each `.compact`, `.regular` or `.unspecified`

- They change at runtime: rotation, iPad windowing (iPadOS 26 windows resize freely), Slide Over, iPhone Duo folding and unfolding, iPhone Mirroring resizing
- React with automatic trait tracking (read them in `viewWillLayoutSubviews` / `layoutSubviews` / `updateProperties()` in iOS 26+) or `registerForTraitChanges([UITraitHorizontalSizeClass.self])`
- `traitOverrides.horizontalSizeClass` forces a size class for a subtree (e.g. a sidebar column that should lay out compact)
- System containers already adapt: `UISplitViewController` collapses in compact width; `UITabBarController` with `.sidebarAdaptable` shows a sidebar in regular width
- SwiftUI: `@Environment(\.horizontalSizeClass)`

Related to check: UITraitCollection, UISplitViewController, Rotation and orientation, Auto Layout, Compositional layout, SwiftUI `ArrangementView` (iOS 27.1+)

Resizable iPhone Apps

Q: iPhone apps are resizable in iOS 27. What do you have to change in a UIKit app?

Short answer: The assumption that breaks is "my app is the screen". In iOS 27 an iPhone app's window can be resized in iPhone Mirroring and on iPad, and on iPhone Duo it changes size when the user folds or unfolds the device, so every layout has to come from the view's own bounds, traits and layout environment.

Key points:

- **Size:** `view.bounds`, safe-area and layout-margin guides, or `windowScene.effectiveGeometry.coordinateSpace` (iOS 26+) for the scene. Never `UIScreen.main.bounds`
- **Scale:** `traitCollection.displayScale` for pixel math (thumbnails, Metal drawables, hairlines), not `UIScreen.main.scale`
- **The screen, if you really need it:** `view.window?.windowScene?.screen`
- **Decisions:** size classes for structure, measured width for detail; in compositional layout, `layoutEnvironment.container.effectiveContentSize`
- **Changes:** `viewWillTransition(to:with:)` fires for every size change, not only rotation; trait changes arrive through automatic trait tracking (`viewWillLayoutSubviews`, `updateProperties()` in iOS 26+) or `registerForTraitChanges`
- **SDK tiers on iPhone Duo:** an app built with the iOS 26 SDK runs centered with empty space around it, the iOS 27 SDK resizes to fill most of the inner display, and the iOS 27.1 SDK runs edge to edge with vertical toolbars and tab bars

Related to check: Size classes, Rotation and orientation, UITraitCollection, Safe area, Compositional layout, iPhone Duo poses and vertical bars, Discrete resizing and `UIRequiresFullScreen`, Scene lifecycle migration, SwiftUI adaptive layout for iPhone Duo

---

iPhone Duo Poses and Vertical Bars

Q: How does iPhone Duo change layout, and what happens to bars when you build with the iOS 27.1 SDK?

Short answer: iPhone Duo has a 5.4" outer display and a 7.6" inner display, and folding or unfolding changes the size of the app's window, so to the app a pose change is a size change plus, sometimes, a size-class change. What the user sees depends on the SDK you link against: built with the iOS 26 SDK, the app runs centered with empty space around it; with the iOS 27 SDK it resizes to fill most of the inner display; with the **iOS 27.1 SDK** it runs edge to edge, and toolbars and tab bars lay out in a **vertical bar** below the status bar, with the status bar itself changing axis.

Key points:

- **Folded vs unfolded:** folded is a phone-sized, compact-width canvas; unfolded is a large canvas that can be regular width. Treat both as sizes (`viewWillTransition`, traits, `windowScene(_:didUpdateEffectiveGeometry:)`), never as device states
- **Safe area:** the vertical bar adds a leading or trailing safe-area inset, so content pinned to `safeAreaLayoutGuide` moves out of the way; content pinned to the view's edges ends up under the bar
- **Item placement (iOS 27.1+):** `UIBarButtonItem.axisBehavior` is `.automatic`, `.verticalPreferred` (prefer the vertical bar when both kinds of bar exist) or `.horizontalOnly` (hidden when no horizontal bar exists)
- **Crowding (iOS 27.1+):** `navigationItem.verticalBarCompressionBehavior` decides what compresses first when the tab bar and bar items share the vertical bar: `.prefersBarItems` or `.prefersTabBar` (`.automatic` prefers the tab bar)
- **Opting out (iOS 27.1+):** override `preferredVerticalBarBehavior` to return `.disabled`, forward the choice from custom containers with `childForPreferredVerticalBarBehavior`, and call `setNeedsUpdateOfVerticalBarConfiguration()` when the answer changes
- **Which side:** `traitCollection.verticalBarEdge` (`.leading`, `.trailing`, `.unspecified`), observed with `registerForTraitChanges(UITraitCollection.systemTraitsAffectingVerticalBarEdge)`
- **Hinge (iOS 27.1+):** `UIHingeInteraction` reports a `UIHinge` with `status` (closed, partially open, fully open) and `angle`; SwiftUI has `onHingeChange`

Related to check: UINavigationBar customization (Liquid Glass bars, `visibilityPriority`), Safe area, Resizable iPhone apps, Size classes, UISplitViewController (collapse on fold), SwiftUI Toolbar, ArrangementView, Testing resizable layouts

---

Constraint Priority

Q: What is constraint priority? When would you change it?

Short answer: Every constraint has a priority from 1 to 1000. At 1000 (`.required`) it must be satisfied; anything lower is optional, and the solver satisfies it as closely as it can, giving up lower priorities first when constraints conflict.

Key points:

- Named values: `.required` (1000), `.defaultHigh` (750), `.defaultLow` (250), `.fittingSizeLevel` (50); arithmetic like `.defaultHigh + 1` works
- Optional constraints are satisfied *as closely as possible*, not simply dropped; an inequality at required priority plus an equality at lower priority is the classic "preferred but capped" pattern
- Content hugging and compression resistance are priorities on the implicit intrinsic-size constraints
- You **can't switch a constraint between required and optional while it's active** (it throws); start at 999 if you plan to change it, or activate and deactivate constraints instead
- Equal priorities on competing constraints produce ambiguous layout

Related to check: Auto Layout, Intrinsic content size, Content hugging and compression resistance, Stack views, Layout cycle

---

Layout Cycle

Q: Explain the layout cycle. When does layout happen?

Short answer: UIKit doesn't lay out immediately when something changes. You mark work as needed, and UIKit performs it once, at the end of the current run-loop turn, before the frame is rendered.

Key points:

- Request: `setNeedsUpdateConstraints()`, `setNeedsLayout()`, `setNeedsDisplay()`, and in iOS 26 `setNeedsUpdateProperties()`. Cheap, coalesced, deferred
- Force now: `updateConstraintsIfNeeded()`, `layoutIfNeeded()`, `updatePropertiesIfNeeded()` (iOS 26+). Use when you need current frames immediately, e.g. inside an animation block
- Override points: `updateProperties()` (iOS 26+) for content and styling; `layoutSubviews` / `viewWillLayoutSubviews` for geometry; `draw(_:)` for custom drawing only
- What triggers layout: bounds or size changes (rotation, window resize), adding or removing subviews, constraint changes, intrinsic size invalidation, trait changes
- Automatic tracking: traits read in layout methods (iOS 18) and `@Observable` properties read in `updateProperties()` / layout methods (iOS 26; opt in on iOS 18 with `UIObservationTrackingEnabled`) re-invalidate the right pass automatically

Related to check: Auto Layout, View lifecycle, Draw cycle (`setNeedsDisplay`), Intrinsic content size, UIView animations, Observation in UIKit

---

Diffable Data Source

Q: What is UICollectionViewDiffableDataSource / UITableViewDiffableDataSource? How does it differ from the traditional data source?

Short answer: A diffable data source replaces the index-path bookkeeping of `UITableViewDataSource` / `UICollectionViewDataSource` with **snapshots**: you build an `NSDiffableDataSourceSnapshot` describing the complete current state (sections and item identifiers), call `apply`, and UIKit diffs it against the previous snapshot and animates the inserts, deletes and moves. With the traditional data source you implement `numberOfSections`/`numberOfItems`/`cellForItem` and must keep `performBatchUpdates` perfectly in sync with your model, or the app crashes with "invalid number of items".

Key points:

- Generic over `SectionIdentifierType` and `ItemIdentifierType`, both `Hashable & Sendable`.
- You provide only a **cell provider** closure; counts come from the snapshot.
- `apply(_:animatingDifferences:)` computes the diff; `animatingDifferences: false` for the initial load.
- Edit an existing state with `dataSource.snapshot()`, mutate (append/insert/delete/move), and apply again.
- `NSDiffableDataSourceSectionSnapshot` models one section on its own, including **hierarchical** items (expand/collapse outlines in sidebars).
- Works for both `UITableView` and `UICollectionView`; pairs with `CellRegistration` and compositional/list layouts.

Related to check: Diffable data source identifiers and reconfiguring, Cell reuse, UICollectionView list configuration, Compositional layout, Hashable

---

UIScrollView Delegate

Q: What does UIScrollViewDelegate do? What are the key methods?

Short answer: `UIScrollViewDelegate` reports a scroll view's life: every offset change (`scrollViewDidScroll`), the drag (`willBeginDragging`, `willEndDragging(withVelocity:targetContentOffset:)`, `didEndDragging(willDecelerate:)`), deceleration, programmatic animations, zooming and scroll-to-top. Table and collection view delegates inherit it, so the same methods drive collapsing headers, snapping, "load more" and pausing work while the user scrolls.

Key points:

- `scrollViewDidScroll`: every offset change, including programmatic ones; keep it cheap.
- `scrollViewWillEndDragging(_:withVelocity:targetContentOffset:)`: change `targetContentOffset.pointee` to implement custom snapping.
- "Scrolling stopped" is `scrollViewDidEndDecelerating` **or** `scrollViewDidEndDragging(willDecelerate: false)`; programmatic animated scrolls end in `scrollViewDidEndScrollingAnimation`.
- Zooming: return the content view from `viewForZooming(in:)`, and set different `minimumZoomScale` / `maximumZoomScale`.
- `scrollViewShouldScrollToTop` / `scrollViewDidScrollToTop` for the status-bar tap.
- `scrollViewDidChangeAdjustedContentInset`: safe area and bar insets changed.

Related to check: ScrollView content insets, Safe area, UIRefreshControl, UINavigationBar customization (Liquid Glass), UITableView prefetching, SwiftUI scroll geometry

---

Present vs Push

Q: What's the difference between presenting and pushing a view controller?

Short answer: Pushing adds a view controller to a `UINavigationController` stack. It's for hierarchical drill-down: you get a back button, the back swipe, and a shared navigation bar, and you leave with `pop`.

Key points:

- Push: `navigationController?.pushViewController(_:animated:)`; back with `popViewController`, `popToRootViewController`
- Present: `present(_:animated:)`; back with `dismiss(animated:)`; the presented controller can embed its own `UINavigationController`
- `modalPresentationStyle`: `.automatic` (page sheet), `.pageSheet`, `.formSheet`, `.fullScreen`, `.overFullScreen`, `.popover`
- Sheets: `sheetPresentationController?.detents = [.medium(), .large()]`, `.custom { context in ... }`, `prefersGrabberVisible`, `largestUndimmedDetentIdentifier`
- `isModalInPresentation = true` blocks swipe-to-dismiss (e.g. unsaved changes)
- `preferredTransition = .zoom { _ in sourceView }` works for both push and present

Related to check: UINavigationController, UIViewController transitions (zoom, custom animators), UISplitViewController, Segues, Adaptive layout / iPhone Duo, Liquid Glass

---

View Controller Containment

Q: What is view controller containment? How do you add a child view controller?

Short answer: Containment lets one view controller embed another's view while keeping the parent - child relationship UIKit needs to forward appearance callbacks, traits, rotation and status-bar decisions, safe area and memory warnings. The sequence matters: `addChild(_:)`, add and constrain the child's view, then `didMove(toParent: self)`; and to remove, `willMove(toParent: nil)`, remove the view, then `removeFromParent()`.

Key points:

- `addChild` calls `willMove(toParent:)` for you; you call `didMove(toParent:)`. Removal is the mirror image
- The child's view must be sized from the **container view's bounds** (constraints to its edges), never the screen
- Appearance callbacks (`viewWillAppear`, `viewIsAppearing`, ...) are forwarded automatically; for custom transitions set `shouldAutomaticallyForwardAppearanceMethods = false` and call `beginAppearanceTransition` / `endAppearanceTransition`

- Swap children with `transition(from:to:duration:options:animations:completion:)`
- Traits for a child: `child.traitOverrides` (replaces the deprecated `setOverrideTraitCollection(_:forChild:)`)
- Delegation of system decisions: `childForStatusBarStyle`, `childForHomeIndicatorAutoHidden`, and in iOS 26 `childForInterfaceOrientationLock`
- SwiftUI: `UIHostingController` as a child; `sizingOptions = .intrinsicContentSize` when the parent should size around it

Related to check: View lifecycle, UISplitViewController, UIHostingController interop, UITraitCollection, Rotation and orientation, SwiftUI `ArrangementView`

Hosting SwiftUI in UIKit

Q: How do you embed SwiftUI in an existing UIKit app, and how would you migrate screen by screen?

Short answer: There are three entry points. A whole screen is a `UIHostingController(rootView:)` that UIKit pushes or presents like any view controller.

Key points:

- **Whole screen:** push `UIHostingController`; the hosted view's `.navigationTitle` and `.toolbar` fill the UIKit navigation item, so don't wrap it in another `NavigationStack`
- **Part of a screen:** `addChild` -> add and constrain `host.view` -> `didMove(toParent:)`; `sizingOptions .intrinsicContentSize` (self-sizing in Auto Layout) or `.preferredContentSize` (popovers, sheets)
- **Cells:** `cell.contentConfiguration = UIHostingConfiguration { Row(item) }` with `.margins(_:_:)`, `.background { }`, `.minSize(width:height:)`; UIKit keeps reuse, selection and swipe actions
- **Data:** pass an `@Observable` model as a plain property; the same model can update UIKit through automatic observation tracking (iOS 26, or iOS 18 with `UIObservationTrackingEnabled`)
- **Safe areas:** the hosting controller applies safe-area insets itself; set `safeAreaRegions = []` when the host is already inside safe content and you see double insets
- **iOS 26+:** a UIKit app can open SwiftUI scenes through a `UIHostingSceneDelegate` and `UISceneSessionActivationRequest(hostingDelegateClass:)`

Related to check: View controller containment, Cell reuse (`UIHostingConfiguration`), UIViewRepresentable (the other direction), Automatic observation tracking in UIKit, Navigation architecture / coordinators, Migration strategy, Safe area

---

Gesture Recognizers

Q: How do gesture recognizers work? What types are available?

Short answer: A gesture recognizer is attached to a view and watches the touches that hit-test to that view or its subviews. It runs a small state machine and fires its target-action when the touches match a gesture.

Key points:

- Built in: `UITapGestureRecognizer`, `UILongPressGestureRecognizer`, `UIPanGestureRecognizer`, `UIScreenEdgePanGestureRecognizer`, `UISwipeGestureRecognizer`, `UIPinchGestureRecognizer`, `UIRotationGestureRecognizer`, `UIHoverGestureRecognizer` (pointer and Apple Pencil hover)
- States: `.possible` -> `.began` -> `.changed` -> `.ended` / `.cancelled` / `.failed` (discrete: `.recognized` = `.ended`)

- `require(toFail:)` to sequence (single tap waits for double tap to fail)
- `cancelsTouchesInView` (default `true`), `delaysTouchesBegan`, `delaysTouchesEnded`
- Delegate for simultaneous recognition and for filtering touches

Related to check: UIGestureRecognizerDelegate, Hit testing, Responder chain, UIControl target-action, Animations (`UIViewPropertyAnimator`)

Animations

Q: How do you animate in UIKit? What options are available?

Short answer: UIKit gives you a ladder of tools. Block-based `UIView.animate` covers most state changes, and the modern spring form, `animate(springDuration:bounce:)`, is the default I reach for.

Key points:

- `UIView.animate(springDuration:bounce:)`: springs with a duration and bounce instead of damping and velocity math; `UIView.animate(withDuration:)` for simple curves
- `UIViewPropertyAnimator`: `pauseAnimation()`, `fractionComplete`, `isReversed`, `continueAnimation(withTimingParameters:durationFactor:)` for gesture-driven UI
- Animatable view properties: `frame`, `bounds`, `center`, `transform`, `alpha`, `backgroundColor`; constraints animate by changing `constant` then calling `layoutIfNeeded()` in the block
- Core Animation (`CABasicAnimation`, `CASpringAnimation`, `CAKeyframeAnimation`) for layer properties such as `shadowPath`, `cornerRadius`, `strokeEnd`
- `UIView.animate(_: Animation, changes:)` (SwiftUI `Animation` in UIKit): same springs as SwiftUI, and velocity-preserving retargeting
- Transitions: `UIView.transition(with:)` for content swaps; `preferredTransition = .zoom { }` for view controller navigation
- Symbol effects: `imageView.addSymbolEffect(.bounce)`

Related to check: UIView Animations (API detail), Core Animation / CALayer, UIViewController Transitions (zoom), Gesture Recognizers, Accessibility (Reduce Motion), SwiftUI animations

---

Intrinsic Content Size

Q: What is intrinsic content size? How does it affect Auto Layout?

Short answer: Intrinsic content size is the natural size a view wants from its content: a label from its text and font, a button from its title and insets, an image view from its image. Auto Layout turns it into implicit constraints per axis, a **content-hugging** constraint (don't grow beyond it) and a **compression-resistance** constraint (don't shrink below it), each with its own priority.

Key points:

- `intrinsicContentSize` returns a `CGSize`; use `UIView.noIntrinsicMetric` for an axis with no natural size (a plain `UIView` has none in either axis)
- Default priorities: hugging `.defaultLow` (250) for most views (labels 251 when created in Interface Builder), compression resistance `.defaultHigh` (750)
- Custom views override `intrinsicContentSize` and call `invalidateIntrinsicContentSize()` when the content that drives it changes
- Multi-line labels: the height depends on the width, which Auto Layout resolves during layout. `preferredMaxLayoutWidth` is rarely needed when the width is constrained
- Dynamic Type changes the intrinsic size of text views automatically (with `adjustsFontForContentSizeCategory`)

- Related sizing APIs: `systemLayoutSizeFitting(_:withHorizontalFittingPriority:verticalFittingPrio` for self-sizing cells and headers; `sizeThatFits(_:)` for manual layout

Related to check: Auto Layout, Constraint priority, Stack views, Dynamic Type, Self-sizing cells, Layout cycle

---

TableView Prefetching

Q: What is prefetching in UITableView? How does it improve performance?

Short answer: `UITableViewDataSourcePrefetching` tells you which rows are about to scroll into view, so you can start loading their data (images, records, the next page of results) before `cellForRowAt` needs it. The cell then appears with content instead of a placeholder, and the main thread does less work at display time.

Key points:

- `tableView.prefetchDataSource = self`; implement `tableView(_:prefetchRowsAt:)` and optionally `tableView(_:cancelPrefetchingForRowsAt:)`.
- Prefetch calls run on the main actor; only *start* work there, never do it synchronously.
- Index paths in a prefetch call may be out of order and may include rows past the data you've loaded, a signal to fetch the next page.
- Not every row is prefetched (first screen, fast flings), so `cellForRowAt` must load on its own too.
- Cancel with `Task.cancel()`; `URLSession`'s async APIs stop the request when their task is cancelled.

Related to check: UICollectionView prefetching, Cell reuse, Diffable data source, Infinite scroll feed, Task cancellation

---

IBOutlet weak vs strong

Q: Should IBOutlets be weak or strong? Why?

Short answer: For subviews of a view controller's view, either works. The superview already holds the subview strongly, so `weak` just avoids a redundant reference, and `strong` doesn't create a retain cycle, because a subview doesn't point back to the controller.

Key points:

- Subview outlets: `strong` (current recommendation) or `weak` (traditional); no cycle either way: VC -> view -> subview
- Must be `strong`: top-level nib objects other than the controller's `view`, gesture recognizers or views that you add and remove, and `NSLayoutConstraint` outlets you deactivate and reactivate
- `weak` outlets become `nil` when the object leaves the hierarchy, and an implicitly unwrapped `!` then crashes on access
- Prefer `@IBOutlet private var` so other types can't reach into your view hierarchy

Related to check: Retain cycles / ARC, View hierarchy, Storyboards vs programmatic, Segues, Auto Layout (constraint activation)

CALayer vs UIView

Q: What's the difference between CALayer and UIView? When would you use CALayer directly?

Short answer: Every `UIView` is backed by a `CALayer` and acts as that layer's delegate. The layer does the rendering, geometry and animation, and the view adds what an app needs on top: the responder chain and touches, Auto Layout, trait collections, accessibility and safe areas.

Key points:

- UIView: `UIResponder`, Auto Layout, `traitCollection`, accessibility, gesture recognizers
- CALayer: `position`/`bounds`/`anchorPoint`, `transform` (3D), `contents`, `cornerRadius`, shadows, borders, masks
- Specialized layers: `CAShapeLayer`, `CAGradientLayer`, `CAEmitterLayer`, `CATextLayer`, `CAReplicatorLayer`
- `override class var layerClass` to back a view with, for example, a `CAGradientLayer`
- Standalone layers animate **implicitly** (0.25 s); a view's backing layer doesn't, because the view returns no action outside an animation block

Related to check: Draw cycle, Core Animation (implicit/explicit animations, `CATransaction`), UIView animations, Trait collections, Liquid Glass (`UIGlassEffect`)

Keyboard Handling

Q: How do you handle the keyboard in UIKit? What are the key considerations?

Short answer: The modern answer is `keyboardLayoutGuide`: every view has one, its top edge tracks the keyboard (or the bottom safe area when the keyboard is hidden), and constraints to it animate with the keyboard automatically. So I pin the bottom of the scroll view or input bar to `view.keyboardLayoutGuide.topAnchor` and the content stays visible with no notification code.

Key points:

- `view.keyboardLayoutGuide.topAnchor`: the main anchor. With no keyboard, it sits at the bottom of the safe area (`usesBottomSafeArea`, settable since iOS 17)
- `followsUndockedKeyboard = true` to track floating and split keyboards on iPad; then use the guide's tracking constraints (`setConstraints(_:activeWhenNearEdge:)`) for edge-specific layouts
- `keyboardDismissPadding` (iOS 17) adds space above the keyboard for interactive dismissal
- Dismissal: `scrollView.keyboardDismissMode = .interactive` or `.interactiveWithAccessory`; `view.endEditing(true)` or `resignFirstResponder()`
- `inputAccessoryView` for a toolbar that rides on the keyboard; `inputView` to replace the keyboard
- Legacy notifications: `keyboardFrameEndUserInfoKey` is in **screen coordinates**; convert with `view.convert(frame, from: screen.coordinateSpace)` and use the overlap, not the raw height
- Selector-based observers don't need removing in `deinit` (since iOS 9). Block-based observers return a token you must keep and remove

Related to check: Safe area, Scroll view content insets, First responder and the responder chain, Auto Layout, UITextField / UITextView

---

Rotation and Orientation

Q: How do you handle device rotation and orientation changes?

Short answer: In iOS 27 I treat rotation as just one kind of size change. iPhone apps now resize in iPhone Mirroring, on iPad and on iPhone Duo, and iPad windows resize freely, so the layout should respond to the **available width and height** (size classes, view bounds, `viewWillTransition(to:with:)`), never to device orientation. Orientation itself is a *preference*: `supportedInterfaceOrientations` says what

you allow, `setNeedsUpdateOfSupportedInterfaceOrientations()` tells the system it changed, and `prefersInterfaceOrientationLocked` (iOS 26) asks to lock it temporarily, for example during a game.

Key points:

- Allowed orientations: `UISupportedInterfaceOrientations` in the Info.plist, narrowed by `supportedInterfaceOrientations` on the root or presented view controller
- Changing what's allowed at runtime: update the override, call `setNeedsUpdateOfSupportedInterfaceOrientations()`, and optionally `windowScene.requestGeometryUpdate(.iOS(interfaceOrientatio`
- `viewWillTransition(to:with:)` fires for **any** size change (rotation, window resize, split view), with a coordinator for animating alongside
- Decide landscape-like layout with `size.width > size.height`, not `UIDevice.current.orientation` (physical device, includes face up/down, and wrong for a windowed app)
- `prefersInterfaceOrientationLocked` (iOS 26+) plus `setNeedsUpdateOfPrefersInterfaceOrientationLocked()`; the system only locks when the scene is full screen, centered and unobstructed. Check `effectiveGeometry.isInterfaceOrientationLocked`
- `windowScene(_:didUpdateEffectiveGeometry:)` (iOS 26+) reports scene size and orientation changes; `effectiveGeometry.isInteractivelyResizing` tells you a live resize is in progress
- **`UIRequiresFullScreen` no longer opts out of resizing** with the iOS 27 SDK, on iPhone or iPad (TN3192). The system resizes the scene *discretely* instead: no live resize during the drag, then a single jump to the new size

Related to check: Size classes, UITraitCollection, Safe area, UIScene multi-window, View controller containment (`childForInterfaceOrientationLock`), SwiftUI `ArrangementView` (iOS 27.1+)

Discrete Resizing and UIRequiresFullScreen

Q: What does UIRequiresFullScreen do in iOS 27?

Short answer: It no longer does what its name says. Before iOS 27 the key only mattered on iPad, where it opted an app out of multitasking; it was deprecated in iPadOS 26.

Key points:

- **What changed:** the key used to be an iPad multitasking opt-out; with the iOS 27 SDK it means "resize me in steps", on iPhone (iPhone Mirroring, iPhone Duo, iPhone apps on iPad) and iPad
- **How you see it:** `windowScene(_:didUpdateEffectiveGeometry:)` (iOS 26+) with the new bounds; `effectiveGeometry.isInteractivelyResizing` (iOS 26+) tells you whether a drag is still in progress
- **Old behavior on older systems:** `UIRequiresFullScreenIgnoredStartingWithVersion` in the Info.plist keeps the previous full-screen behavior on earlier OS versions while you adopt resizing on new ones
- **The real replacements:** support resizing; set `windowScene.sizeRestrictions?.minimumSize` (`UISceneSizeRestrictions`, a preference the system may clamp) when content has a true minimum; override `prefersInterfaceOrientationLocked` (iOS 26+) for a temporary orientation lock, which the system honors only when the scene is full screen
- **Related iOS 27 change:** a launch screen is required for App Store submission

Related to check: Rotation and orientation (`prefersInterfaceOrientationLocked`, `viewWillTransition`), Resizable iPhone apps, UIScene multi-window (`effectiveGeometry`), Scene life-cycle migration, iPhone Duo poses and vertical bars, Metal and game loops

---

UICollectionViewCompositionalLayout

Q: What is UICollectionViewCompositionalLayout? When would you use it?

Short answer: Compositional layout is the declarative layout system for `UICollectionView`: you compose **items** into **groups** and groups into **sections**, each sized with fractional, absolute or estimated dimensions. A section provider closure can return a different layout per section (a carousel, then a grid, then a list), and it receives an `NSCollectionLayoutEnvironment` describing the container's current size and traits.

Key points:

- Building blocks: `NSCollectionLayoutSize` -> `NSCollectionLayoutItem` -> `NSCollectionLayoutGroup` (`.horizontal`, `.vertical`, `.custom`) -> `NSCollectionLayoutSection`.
- Dimensions: `.fractionalWidth/Height` (relative to the containing group), `.absolute`, `.estimated` (self-sizing).
- `UICollectionViewCompositionalLayout { sectionIndex, environment in ... }` builds sections on demand; use `environment.container.effectiveContentSize.width` (content size minus insets) to choose column counts.
- Per-section features: `orthogonalScrollingBehavior` (carousels, paging), boundary supplementary items (headers/footers, `pinToVisibleBounds`), decoration items (section backgrounds), `interGroupSpacing`, `visibleItemsInvalidationHandler`.
- Lists are compositional too: `NSCollectionLayoutSection.list(using:layoutEnvironment:)` or `UICollectionViewCompositionalLayout.list(using:)`.

Related to check: UICollectionView list configuration, Diffable data source, Custom UICollectionViewLayout, Size classes, Resizable iPhone apps / iPhone Duo, Cell reuse

---

Draw Cycle

Q: What's the difference between layout and draw? When does each happen?

Short answer: Layout decides *where* views are (frames), and display decides *what* their pixels look like. UIKit doesn't do either immediately: you invalidate with `setNeedsLayout()` or `setNeedsDisplay()`, and once per run-loop turn UIKit runs the update passes top-down, in order: constraints (`updateConstraints`), layout (`layoutSubviews`), then display (`draw(_:)` or layer contents). iOS 26 adds a properties pass, `updateProperties()`, that is coalesced with the next layout pass and runs before `layoutSubviews()`; it's where you push model state into views.

Key points:

- Invalidate: `setNeedsUpdateConstraints()`, `setNeedsUpdateProperties()` (iOS 26+), `setNeedsLayout()`, `setNeedsDisplay()`
- Force now: `updateConstraintsIfNeeded()`, `updatePropertiesIfNeeded()` (iOS 26+), `layoutIfNeeded()`; there is no "display if needed" you should call
- Order per frame: constraints and properties (iOS 26+), then layout, then display; the render server composites afterwards
- Layout runs when bounds, constraints, subviews or traits change; `draw(_:)` runs only when content is invalidated (or `contentMode = .redraw` and the bounds change)
- `draw(_:)` allocates a backing store the size of the view x scale, so it costs memory

Related to check: Layout cycle, View lifecycle (`viewIsAppearing`, `updateProperties()`), CALayer vs UIView, Auto Layout, UIKit observation tracking

---

View Hierarchy

Q: How does the view hierarchy work? What's the relationship between superview, subviews, and the window?

Short answer: Views form a tree: each view has at most one `superview` and an ordered `subviews` array, and the root of an on-screen tree is a `UIWindow`, which belongs to a `UIWindowScene`. The window's `rootViewController` installs its view as the window's content, and everything else hangs below it.

Key points:

- Scene -> `UIWindow` -> root view controller's `view` -> subviews. Apps built with the iOS 27 SDK must use the UIScene life cycle, so always reach the window through `view.window` / `view.window?.windowScene`, never `UIApplication.shared.windows`.
- `addSubview` appends on top; `insertSubview(_:at:)`, `insertSubview(_:aboveSubview:)` / `belowSubview:`, `bringSubviewToFront`, `sendSubviewToBack` control order; `removeFromSuperview()` detaches.
- Adding a view that already has a superview **moves** it (a view can't be in two places).
- Coordinates are relative to the superview (`frame`) or to the view itself (`bounds`); convert with `convert(_:to:)` / `convert(_:from:)`.
- Hooks: `willMove(toSuperview:)`, `didMoveToSuperview()`, `willMove(toWindow:)`, `didMoveToWindow()`.
- The **view controller hierarchy** is separate: use containment (`addChild`, `didMove(toParent:)`) whenever you add another controller's view, so appearance and trait callbacks reach it.

Related to check: Auto Layout, View lifecycle, Layout cycle, Hit testing, Responder chain, View controller containment, UIScene life cycle, View debugging

---

Segues

Q: What are segues? How do they work with Storyboards?

Short answer: A segue is a storyboard-defined transition between two view controllers. A button, a cell or code (`performSegue(withIdentifier:sender:)`) triggers it, UIKit instantiates the destination from the storyboard, and then it performs a show, present or popover transition.

Key points:

- Kinds: **Show** (push if inside a navigation controller, otherwise present), **Show Detail** (split view detail column), **Present Modally**, **Present As Popover**, plus embed and relationship segues for containers
- `prepare(for:sender:)`: configure `segue.destination` before it appears
- `@IBSegueAction`: `(NSCoder, Any?) -> ViewController?` builds the destination with dependencies
- `shouldPerformSegue(withIdentifier:sender:)`: cancel a segue (e.g. validation failed)
- Unwind: an `@IBAction func unwind...(_ segue: UIStoryboardSegue)` on the *destination*, wired to the Exit icon

Related to check: Storyboards vs programmatic UI, Present vs push, IBOutlet weak vs strong, View controller lifecycle, `UIHostingController`

UICollectionView List Configuration

Q: How do you create a list with UICollectionView?

Short answer: You give a collection view a compositional layout built from `UICollectionLayoutListConfiguration`, register `UICollectionViewListCell`s, and feed it with a diffable data source. The

result looks and behaves like a table view (separators, swipe actions, accessories, headers, insetGrouped and sidebar styles), but it's still a collection view, so list sections can sit next to grids and carousels and adapt to the container's width.

Key points:

- `UICollectionLayoutListConfiguration(appearance:)`: `.plain`, `.grouped`, `.insetGrouped`, `.sidebar`, `.sidebarPlain`.
- Whole-layout: `UICollectionViewCompositionalLayout.list(using:)`; per section: `NSCollectionLayoutSection.list(using:layoutEnvironment:)`.
- Headers/footers: `headerMode = .supplementary` (supplementary views) or `.firstItemInSection` (expandable outline headers); `footerMode`.
- Swipe actions: `leadingSwipeActionsConfigurationProvider` / `trailingSwipeActionsConfigurationProvider`, returning `UISwipeActionsConfiguration`.
- Cells: `UICollectionViewListCell` + `defaultContentConfiguration()` (`UIListContentConfiguration`), `accessories` (`.disclosureIndicator()`, `.checkmark()`, `.reorder()`, `.outlineDisclosure()`, `.delete()`).
- Outlines: `NSDiffableDataSourceSectionSnapshot` with `append(_:to:)` and `expand(_:)` for hierarchical sidebars.

Related to check: UITableView vs UICollectionView, Compositional layout, Diffable data source, Cell reuse, UISplitViewController, UITab and sidebar

---

Memory Warnings

Q: How do you handle memory warnings in a view controller?

Short answer: When the system is running low on memory, UIKit calls `didReceiveMemoryWarning()` on view controllers, `applicationDidReceiveMemoryWarning(_:)` on the app delegate, and posts `UIApplication.didReceiveMemoryWarningNotification`. The right response is to drop anything you can recreate: image and data caches, decoded thumbnails, off-screen content, while keeping visible UI and unsaved user data.

Key points:

- `didReceiveMemoryWarning()`: call `super`, then release caches. Since iOS 6, UIKit no longer unloads view controllers' views for you (`viewDidUnload` is long gone)
- App-wide caches observe the notification or are owned by an object that gets the app delegate callback; iOS 26 adds typed main-actor messages (`addObserver(of: UIApplication.shared, for: .didReceiveMemoryWarning)`)
- `NSCache` evicts under memory pressure on its own; set `countLimit` / `totalCostLimit`. A plain `Dictionary` cache never shrinks
- `os_proc_available_memory()` tells you how much headroom the process has before its limit
- iOS 27 adds `systemPrefersReducedResourceUsage` (iOS 27+) on `UIApplication` and as a trait: a broader signal to choose lighter work (lower-resolution assets, less prefetching), not a substitute for handling warnings
- Backgrounding: in `sceneDidEnterBackground`, persist state and release large resources (decoded images, media buffers); apps with smaller suspended footprints are less likely to be terminated

Related to check: ARC, NSCache, Image downsampling, UIApplication lifecycle, Out-of-memory terminations (jetsam), MetricKit

---

UIRefreshControl

Q: How do you implement pull-to-refresh in UITableView?

Short answer: Assign a `UIRefreshControl` to the scroll view's `refreshControl` property (any `UIScrollView`, so table views and collection views alike), react to its `.valueChanged` event, and call `endRefreshing()` when the reload finishes, whether it succeeded or failed. In modern code the handler is a `UIAction` that starts a `Task`; the task inherits the view controller's main-actor isolation, so it can `await` the network call and then update the UI and end refreshing directly, with `defer` guaranteeing the spinner stops.

Key points:

- `tableView.refreshControl = UIRefreshControl()`; no need to add it as a subview.
- `addAction(UIAction { ... }, for: .valueChanged)` (or the older `addTarget(_:action:for:)` + `@objc`).
- Always call `endRefreshing()`; put it in `defer` inside the task.
- `beginRefreshing()` only shows the spinner; it doesn't send `.valueChanged` and doesn't scroll to reveal it.
- Keep a reference to the refresh `Task` so it can be cancelled when the screen goes away or a second refresh starts.

Related to check: ScrollView delegate, ScrollView content insets, Diffable data source, UITableView prefetching, SwiftUI `.refreshable`

UIContextMenuInteraction

Q: How do you add context menus to views?

Short answer: For an arbitrary view, I add a `UIContextMenuInteraction` and return a `UIContextMenuConfiguration` from its delegate: an optional preview view controller and a `UIMenu` of `UIActions`. UIKit then handles the long press (or secondary click on iPad and Mac), the lift animation, the blur and dismissal.

Key points:

- `UIContextMenuInteraction(delegate:)` + `view.addInteraction(_:)`
- `contextMenuInteraction(_:configurationForMenuAtLocation:)` returns `UIContextMenuConfiguration(identifier:previewProvider:actionProvider:)`, or `nil` for no menu
- `UIMenu` children: `UIAction` (`attributes: .destructive / .disabled`, `state: .on`), nested `UIMenu` (`options: .displayInline`, `.singleSelection`), `UIDeferredMenuElement` for async items
- Collection views: `collectionView(_:contextMenuConfigurationForItemsAt:point:)` (supports multi-selection)
- Buttons and bar items: `menu` + `showsMenuAsPrimaryAction`, no interaction needed
- SwiftUI equivalent: `.contextMenu { } preview: { }`

Related to check: UIControl target-action (`UIAction`, `UIButton.menu`), UIGestureRecognizer delegate, UICollectionView delegate, Accessibility (custom actions), SwiftUI `.contextMenu`

UIPageViewController

Q: When would you use UIPageViewController?

Short answer: `UIPageViewController` is a container that pages between **full view controllers**, with a swipe gesture and either a scroll or page-curl transition. It fits when each page is a substantial screen with

its own lifecycle (onboarding steps, a document reader, a photo viewer where each page is a zoomable view controller).

Key points:

- `UIPageViewController(transitionStyle: .scroll | .pageCurl, navigationOrientation: .horizontal | .vertical)`.
- `dataSource` returns the view controller *before/after* a given one (lazy, linked-list style); `setViewControllers(_:direction:animated:)` jumps directly.
- `delegate didFinishAnimating(_:previousViewControllers:transitionCompleted:)` tells you which page is now showing; check `completed`, because a swipe can be cancelled.
- Built-in dots appear only with `.scroll` when you implement `presentationCount(for:)` / `presentationIndex(for:)`; many apps use their own `UIPageControl` for styling.
- It's a proper container: child view controllers get `viewWillAppear`/`viewIsAppearing` etc. as pages change.

Related to check: View controller containment, Compositional layout (orthogonal paging), ScrollView delegate, UIViewController transitions, SwiftUI TabView

---

Dynamic Type

Q: How do you support Dynamic Type in UIKit?

Short answer: Use text styles (`UIFont.preferredFont(forTextStyle:)`) and set `adjustsFontForContentSizeCategory = true`, so labels, text fields and text views rescale automatically when the user changes their text size. Custom fonts go through `UIFontMetrics` so they scale the same way.

Key points:

- System text styles: `.largeTitle`, `.title1`…`.body`, `.caption2`; `UIFont.preferredFont(forTextStyle:compat` to resolve for specific traits
- `adjustsFontForContentSizeCategory` on `UILabel`, `UITextField`, `UITextView`, and `UIButton.Configuration` text
- Custom fonts: `UIFontMetrics(forTextStyle: .body).scaledFont(for:)`, and `scaledValue(for:)` for spacing and icon sizes
- Symbols: `UIImage.SymbolConfiguration(textStyle: .body)` scales SF Symbols with text
- Clamp extremes only when necessary: `minimumContentSizeCategory` / `maximumContentSizeCategory` on any `UIView`
- React to changes for layout (not fonts) with `registerForTraitChanges([UITraitPreferredContentSizeCatego` or read the trait in `viewWillLayoutSubviews` and let automatic trait tracking re-run it

Related to check: Accessibility, UITraitCollection, Stack views, Intrinsic content size, UILabel vs UITextView, Self-sizing cells

---

UISplitViewController

Q: How does UISplitViewController work on iPad?

Short answer: `UISplitViewController` is the system container for list - detail interfaces. With the column-based API (`.doubleColumn` or `.tripleColumn` style) you assign view controllers to the `.primary`, `.supplementary` and `.secondary` columns, and iPadOS 26 adds an `.inspector` column on the trailing side.

Key points:

- `setViewController(_:for:)` per column; an optional `.compact` column supplies a dedicated view controller for the collapsed state instead of stacking the others
- Display: `preferredDisplayMode` (`.oneBesideSecondary`, `.oneOverSecondary`, `.secondaryOnly`, ...) and `preferredSplitBehavior` (`.tile`, `.overlay`, `.displace`); `show(_:)` / `hide(_:)` columns
- `displayModeButtonVisibility` controls the system sidebar toggle (column-style split views add it for you)
- Collapse and expand are driven by size class; the delegate's `splitViewController(_:topColumnForCollapsingT` picks what the user sees when collapsing, `splitViewController(_:displayModeForExpandingToProposedDisp` when expanding
- iOS 26+: `.inspector` column with `preferredInspectorColumnWidth`, `minimumInspectorColumnWidth` and `maximumInspectorColumnWidth`; `isShowing(_:)` to query a column; `didShowColumn` / `didHideColumn` delegate callbacks; width properties for the secondary column
- Liquid Glass (iOS 26): the sidebar floats over content; use `UIBackgroundExtensionView` (iOS 26+) to let hero content extend beneath it

Related to check: Size classes, UITraitCollection, View controller containment, UIScene multi-window, Tab bar sidebar (`UITab`), SwiftUI `NavigationSplitView` and `ArrangementView`

UITab and UITabGroup

Q: How do `UITab` and `UITabGroup` change the way you build a `UITabBarController`?

Short answer: Since iOS 18, you describe a tab bar controller's content as `UITab` objects instead of handing it an array of view controllers with `tabBarItems`. A `UITab` carries its title, image, badge and identifier and creates its view controller lazily through a provider closure; a `UITabGroup` nests tabs, which the controller shows as a section in a **sidebar** on iPad and in wide windows, while compact widths still get a tab bar.

Key points:

- **`UITab(title:image:identifier:viewControllerProvider:)`**: the provider runs lazily when the tab is first shown; `viewController` gives you the created controller later, and `UIViewController.tab` goes the other way
- **`UITabGroup(title:image:identifier:children:viewControllerProvider:)`**: a sidebar section; `allowsReordering`, `displayOrder`, `managingNavigationController` (children push into one navigation stack), `sidebarActions`
- **`UISearchTab`**: a system-styled search tab; `automaticallyActivatesSearch` (iOS 26+) focuses the field when selected and returns to the previous tab on cancel
- **`mode`**: `.automatic` (sidebar if any tab is a group and the platform supports it), `.tabBar`, `.tabSidebar`; `sidebar` for the sidebar object itself
- **Customization:** `UITab.Placement` (`.default`, `.optional`, `.movable`, `.pinned`, `.fixed`, `.sidebarOnly`), `allowsHiding`, `isHiddenByDefault`, `customizationIdentifier` for persistence, `compactTabIdentifiers` to filter what compact widths show
- **Selection:** `selectedTab`, `tab(forIdentifier:)`, delegate `tabBarController(_:shouldSelectTab:)` / `didSelectTab:previousTab:`
- **iOS 26+:** `tabBarMinimizeBehavior` (`.onScrollDown`, `.onScrollUp`, `.never`; minimizing is iPhone only), `bottomAccessory = UITabAccessory(contentView:)`, the `tabAccessoryEnvironment` trait (`.regular` above the bar, `.inline` beside the minimized bar)
- **iOS 27+:** `prominentTabIdentifier` gives one tab prominent treatment (when it's nil, a search tab with `automaticallyActivatesSearch` gets it); `performBatchUpdates(_:)` coalesces several tab changes into one animated update

Related to check: UISplitViewController, UINavigationBar customization, Liquid Glass (`UIGlassEffect`), SwiftUI TabView and `Tab`, Size classes, UITraitCollection, Safe area, iPhone Duo poses and vertical bars

---

UICollectionViewDiffableDataSource

Q: How does DiffableDataSource differ from traditional data source?

Short answer: Beyond "snapshots instead of index paths", the interesting part is **identity**: diffable compares item identifiers by `Hashable`, so the identifier should be a stable ID (`Contact.ID`), not the whole mutable model. Identity tells the diff *which* items were inserted, deleted or moved; content changes to an existing item are a separate signal you send with `reconfigureItems`, which re-runs the cell configuration on the existing cell.

Key points:

- **Identifiers, not models:** `UICollectionViewDiffableDataSource<Section, Item.ID>`; look up the current model by ID in the cell provider. Identifiers must be `Hashable & Sendable` and unique in a snapshot.
- **`reconfigureItems`** (preferred): keeps the existing cell, re-runs its configuration, preserves selection and in-flight state, and also updates prefetched cells. **`reloadItems`**: discards the cell and dequeues a new one (`prepareForReuse`, a cross-fade), only when you really need a fresh cell, such as a different cell type.
- **Main actor:** the diffable data source types are `@MainActor`, so calling `apply` from a background queue is a compile error in Swift 6 mode. (Older advice was "apply from one queue consistently, main or background"; Swift 6 settles it.)
- **`await dataSource.apply(snapshot)`** suspends until the update completes; `applySnapshotUsingReloadData` resets without diffing.
- Build heavy snapshot data off the main actor if needed (snapshots of `Sendable` IDs are cheap to pass), then apply on the main actor.

Related to check: Diffable data source (snapshot basics), Cell reuse, UICollectionView prefetching, Custom collection view layout, Observation in UIKit (`updateProperties`), MainActor

---

UIScrollView Content Insets

Q: How do content insets affect UIScrollView?

Short answer: `contentInset` adds scrollable padding around the content without changing the content size, so content can rest below a bar or above the keyboard but still scroll under them. On top of your own `contentInset`, UIKit adds system insets for the safe area and bars, controlled by `contentInsetAdjustmentBehavior`, and the total is `adjustedContentInset`, which is what actually determines the scroll limits.

Key points:

- `contentInset`: your padding. `adjustedContentInset` (read-only) = `contentInset` + system adjustments; override `adjustedContentInsetDidChange()` to react
- `contentInsetAdjustmentBehavior`: `.automatic` (default), `.scrollableAxes`, `.never`, `.always`
- Scroll indicators: `verticalScrollIndicatorInsets` / `horizontalScrollIndicatorInsets` (the `scrollIndicatorInsets` getter is deprecated; its setter still sets both); `automaticallyAdjustsScrollIndicatorInsets` keeps them in sync with the safe area
- The initial "scrolled to top" offset is `-adjustedContentInset.top`, not `0`
- `contentLayoutGuide` (content size) and `frameLayoutGuide` (visible frame) for Auto Layout inside scroll views; that's separate from insets
- iOS 26+: `topEdgeEffect` / `bottomEdgeEffect` (`UIScrollEdgeEffect`) style the blur/fade where content meets a bar (`.automatic`, `.soft`, `.hard`)

Related to check: Safe area, Keyboard handling, UIRefreshControl, Scroll view delegate, Liquid Glass bars (UINavigationBar customization)

View Debugging

Q: How do you debug view layout issues in Xcode?

Short answer: Start with Xcode's **view debugger** (Debug View Hierarchy in the debug bar): it pauses the app and captures the window as an exploded 3D hierarchy, so you can select any view, see its frame, constraints and properties in the inspectors, and spot views that are hidden, zero-sized, clipped or covered. Then confirm the cause in code: Auto Layout warnings in the console, ambiguity checks, and LLDB to print the hierarchy.

Key points:

- **Debug View Hierarchy:** 3D exploded view, clipped-content and constraint toggles, Object/Size inspectors with frames and constraints, "Show Constraints" for a selected view; purple runtime issues for ambiguous layouts.
- **Console:** "Unable to simultaneously satisfy constraints" logs; add a symbolic breakpoint on `UIViewAlertForUnsatisfiableConstraints` to stop where it happens. Give constraints `identifiers` so the log is readable.
- **Ambiguity:** `view.hasAmbiguousLayout`, `exerciseAmbiguityInLayout()`, and the private-but-useful LLDB call `_autolayoutTrace` (debug only).
- **Environment Overrides** (debug bar): change Dynamic Type size, light/dark appearance and accessibility settings live, without restarting.
- **SwiftUI:** the view debugger shows SwiftUI content inside its hosting view (names are less descriptive than UIKit classes); `let _ = Self._printChanges()` in `body` logs what triggered an update; `.border(.red)` or `.background(.red.opacity(0.2))` reveals a view's actual frame; the SwiftUI instrument in Instruments shows update counts and costs.
- **Device state:** Xcode 27's Device Hub hosts the resizable simulator; combine with the iPhone Duo simulator (Xcode 27.1) and iPhone Mirroring to reproduce size-dependent bugs.

Related to check: Auto Layout, Constraint priority, Hit testing, Safe area, Layout cycle, SwiftUI view identity, Resizable iPhone apps / iPhone Duo

---

UILabel vs UITextView

Q: When would you use UITextView instead of UILabel?

Short answer: `UILabel` is a lightweight, non-interactive view for displaying text: titles, captions, cell content, even multi-line attributed text. Reach for `UITextView` when the user must **edit** text, **select and copy** it, tap **links or data detectors**, or read long content that **scrolls**.

Key points:

- `UILabel`: display only, `numberOfLines`, `lineBreakMode`, `attributedText`, `adjustsFontForContentSizeCategory`; cheap and ideal in cells.
- `UITextView`: editing (`isEditable`), selection (`isSelectable`), `dataDetectorTypes` (only when not editable), link handling, scrolling, rich text editing, Writing Tools (`writingToolsBehavior`).
- Read-only text with tappable links: `isEditable = false`, `isScrollEnabled = false` (so it self-sizes via intrinsic content size), `textContainerInset = .zero`, `textContainer.lineFragmentPadding = 0` to align with labels.
- `UITextField` is the single-line *input* control; don't confuse it with either.

- TextKit 2 is the default for `UITextView`; touching `layoutManager` forces a fallback to TextKit 1, so use `textLayoutManager`.

Related to check: Dynamic Type, Intrinsic content size, ScrollView content insets, Accessibility, SwiftUI Text and TextEditor, Attributed strings

---

Custom UICollectionViewLayout

Q: When would you create a custom UICollectionViewLayout?

Short answer: Only when compositional layout can't express the geometry: a true masonry/waterfall where each item goes into the shortest column, a circular or radial arrangement, a timeline with items positioned by date, or layouts that need custom attributes and interactive physics. Compositional layout covers grids, lists, carousels, pinned headers and per-section variety, and custom group providers (`NSCollectionLayoutGroup.custom`) cover many "almost a grid" cases, so a subclass is the last resort.

Key points:

- Override `prepare()`, `collectionViewContentSize`, `layoutAttributesForElements(in:)`, `layoutAttributesForItem(at:)`.
- Cache attributes in `prepare()`; the query methods are called constantly while scrolling and must be cheap.
- Invalidate with `invalidateLayout()` when data changes; override `shouldInvalidateLayout(forBoundsChange:)` to return `true` only when the **width** changes (not on every scroll offset).
- Derive columns from the collection view's bounds width, which changes on rotation, iPad multitasking, iPhone Mirroring and the folded/unfolded iPhone Duo.
- Custom `UICollectionViewLayoutAttributes` subclasses carry extra data (must implement `copy(with:)` and `isEqual`).

Related to check: Compositional layout, UICollectionView prefetching, Diffable data source, SwiftUI custom Layout, View debugging

---

UIApplication Lifecycle

Q: What are the main UIApplication lifecycle events?

Short answer: Since iOS 13 the lifecycle is split in two. The **app delegate** handles process-level events: launch (`willFinishLaunching` / `didFinishLaunching`), scene configuration and discarding, memory warnings and termination.

Key points:

- States: not running -> inactive -> active inactive -> background -> suspended (and possibly terminated by the system without notice)
- App delegate: `application(_:didFinishLaunchingWithOptions:)` for process setup (logging, DI container, `BGTaskScheduler` registration); `application(_:configurationForConnecting:options:)`; `application(_:didDiscardSceneSessions:)`; `applicationDidReceiveMemoryWarning`; `applicationWillTerminate` (not guaranteed to run)
- Scene delegate: `scene(_:willConnectTo:options:)` builds the window and handles launch URLs or activities; `sceneWillEnterForeground`, `sceneDidBecomeActive`, `sceneWillResignActive`, `sceneDidEnterBackground`
- Once scenes are adopted, `applicationDidBecomeActive` and friends are **not called**. For app-wide observers, use `UIApplication.didBecomeActiveNotification` and the other app-level notifications

- URL handling moves too: `application(_:open:options:)` is deprecated (iOS 26) in favour of `scene(_:openURLContexts:)` plus `connectionOptions.urlContexts` on cold launch
- SwiftUI App life-cycle apps already use scenes and meet the requirement

Related to check: UIScene multi-window, State restoration (`stateRestorationActivity(for:)`), Memory warnings, Background tasks, App launch time

---

UITraitCollection

Q: What is UITraitCollection? How do you use it?

Short answer: A trait collection describes the environment a view is in: size classes, display scale, light or dark appearance, Dynamic Type size, layout direction, contrast and more. Traits flow down the hierarchy from the window scene through view controllers to views, and you can override them for a subtree with `traitOverrides`.

Key points:

- Built-in traits: `horizontalSizeClass`, `verticalSizeClass`, `userInterfaceStyle`, `displayScale`, `preferredContentSizeCategory`, `layoutDirection`, `accessibilityContrast`, `legibilityWeight`, `userInterfaceIdiom`
- `registerForTraitChanges([UITraitUserInterfaceStyle.self]) { (self: Self, previous) in ... }` replaces `traitCollectionDidChange`; registration is tied to the object's lifetime
- `traitOverrides` on a view, view controller, window or window scene changes traits for everything below it (replaces `setOverrideTraitCollection(_:forChild:)`)
- Custom traits: a `UITraitDefinition`, plus accessors on `UITraitCollection` and `UIMutableTraits`. They can bridge to SwiftUI's environment with `UITraitBridgedEnvironmentKey`
- iOS 27 adds `systemPrefersReducedResourceUsage` (iOS 27+), a trait that tells you the system would like lighter work
- `traitCollection.displayScale` replaces `UIScreen.main.scale`

Related to check: Size classes, Dynamic Type, UISplitViewController, View lifecycle (`viewIsAppearing`), Rotation and orientation, SwiftUI environment

---

UINavigationBar Customization

Q: How do you customize UINavigationBar appearance?

Short answer: Since iOS 26, the first answer is "customize as little as possible": apps rebuilt with the iOS 26+ SDK get **Liquid Glass** navigation bars automatically. The bar background disappears, content scrolls underneath with a scroll edge effect, and bar button items float in shared glass capsules.

Key points:

- **Appearance objects:** `UINavigationBarAppearance` for the bar, set on `navigationBar` (all screens in that controller) or on `navigationItem` (one screen, overrides the bar). `UIBarButtonItemAppearance` for item text styling.
- **Four slots:** `standardAppearance` (scrolled), `scrollEdgeAppearance` (content at the top edge), `compactAppearance` / `compactScrollEdgeAppearance` (compact-height bars).
- **Liquid Glass (iOS 26+):** items are grouped automatically into shared glass backgrounds. `UIBarButtonItem.fixedSpace()` splits a group, `hidesSharedBackground = true` removes the capsule behind one item (e.g. an avatar), and the `.prominent` style makes a tinted, stand-alone primary action. `navigationItem.subtitle` and `largeSubtitle` add a second title line, and `badge` puts a count on an item.

- **iOS 27+:** `visibilityPriority` keeps important items visible when space is tight (others move to overflow); `navigationItem.navigationBarMinimization` controls how the bar minimizes on scroll; `UIBarAppearance.overrideUserInterfaceStyle` forces light or dark bar content.
- **iPhone Duo (iOS 27.1 SDK):** apps run edge to edge and toolbars and tab bars lay out in a **vertical bar** below the status bar. `UIBarButtonItem.axisBehavior` (`.verticalPreferred`, `.horizontalOnly`), `navigationItem.verticalBarCompressionBehavior`, and overriding `preferredVerticalBarBehavior` to return `.disabled` (only for screens like a full-screen player or calculator) are the controls (all iOS 27.1+).

Related to check: Liquid Glass (UIGlassEffect), UIToolbar and SwiftUI toolbar, ScrollView delegate (scroll edge effect), Safe area, Resizable iPhone apps / iPhone Duo, Dynamic Type

---

Liquid Glass (UIGlassEffect)

Q: How do you adopt Liquid Glass in a UIKit app?

Short answer: Start by *removing* things: an app rebuilt with the iOS 26+ SDK already gets Liquid Glass in navigation bars, toolbars, tab bars, sheets, menus and standard controls, and the most common migration bug is old customization (opaque bar appearances, background images, custom tab bar backgrounds) hiding it. For custom floating controls, glass is a `UIVisualEffect`: put a `UIGlassEffect` (iOS 26+) in a `UIVisualEffectView`, add content to its `contentView`, and group neighbouring glass views inside a `UIVisualEffectView` whose effect is `UIGlassContainerEffect` so they sample together and blend.

Key points:

- **`UIGlassEffect(style:)`** with `.regular` (default, keeps content legible) or `.clear` (for controls over rich media; add dimming beneath); `isInteractive` for touch response; `tintColor` for emphasis
- **`UIGlassContainerEffect`**: `spacing` sets how close glass views must be before their shapes blend; glass views go in the container's `contentView`
- **Shape:** `cornerConfiguration` on `UIView` (iOS 26+): `.capsule()`, `.corners(radius: .fixed(16))`, or `.containerConcentric(minimum:)` so a nested view's corners stay concentric with its container
- **Buttons:** `UIButton.Configuration.glass()`, `.prominentGlass()`, `.clearGlass()`, `.prominentClearGlass()` (iOS 26+); bar button items get glass automatically, with `.prominent` style for the primary action
- **Scroll edges:** `scrollView.topEdgeEffect` / `bottomEdgeEffect` (`.style = .soft` / `.hard`, `isHidden`); `UIScrollEdgeElementContainerInteraction` makes a custom bar over a scroll view shape the edge effect like a system bar
- **`UIBackgroundExtensionView`**: its `contentView` (a hero image) stays in the safe area while mirrored, blurred copies fill the space under sidebars and insets

Related to check: UINavigationBar customization, Liquid Glass (SwiftUI `glassEffect`), UITab and tab groups (`tabBarMinimizeBehavior`), Safe area, Scroll view content insets, Design systems, Accessibility in UIKit

---

UIControl and Target-Action

Q: How does the target-action pattern work with UIControl?

Short answer: A `UIControl` (button, switch, slider, segmented control, text field) keeps a table of target, action and control-event entries. When an event such as `.touchUpInside` or `.valueChanged` happens, it calls `sendAction(_:to:for:)`, which routes through `UIApplication` to the target, or up the responder chain if the target is `nil`.

Key points:

- Classic: `addTarget(_:action:for:)` with an `@objc` method taking `()`, `(_ sender:)` or `(_ sender:, forEvent:)`
- Modern: `UIAction { [weak self] action in ... }`, `addAction(_:for:)`, `UIButton(configuration:primaryAction:)`, `UISwitch(frame:primaryAction:)`
- `.primaryActionTriggered` means "the control's main action" (tap for a button, Return for a text field)
- Targets are **not retained** by the control; `UIAction` closures **are**
- `nil` target = responder chain (first object that implements the selector)
- Remove: `removeTarget(_:action:for:)` or `removeAction(identifiedBy:for:)`

Related to check: Responder chain (nil-targeted actions), UIGestureRecognizer delegate, UIContextMenuInteraction / `UIMenu`, Retain cycles and capture lists, Liquid Glass buttons

---

UIGestureRecognizerDelegate

Q: When do you need UIGestureRecognizerDelegate?

Short answer: You need the delegate when the default rules for gestures aren't enough. By default only one recognizer wins a touch sequence, every recognizer sees every touch in its view, and there's no conditional "should this start now?".

Key points:

- `gestureRecognizer(_:shouldRecognizeSimultaneouslyWith:)`: let two recognize together (either side returning `true` is enough)
- `gestureRecognizerShouldBegin(_:)`: veto starting, e.g. pan only when horizontal
- `gestureRecognizer(_:shouldRequireFailureOf:)` / `shouldBeRequiredToFailBy:`: dynamic failure dependencies, including across view hierarchies
- `gestureRecognizer(_:shouldReceive:)` for a `UITouch`, and `shouldReceive` for a `UIEvent` (filter before the recognizer ever sees it)
- The protocol is `@MainActor`; the `delegate` property is **weak**

Related to check: Gesture recognizers, Hit testing, UIScrollView (`panGestureRecognizer`), UIControl target-action, UIContextMenuInteraction

---

UIView Animations

Q: How do you animate UIView properties?

Short answer: You set the final values of animatable properties inside a `UIView.animate` block, and UIKit records those changes and hands them to Core Animation to interpolate. For new code I use `UIView.animate(springDuration:bounce:)`, which describes a spring by how long it takes and how much it overshoots.

Key points:

- Animatable: `frame`, `bounds`, `center`, `transform`, `alpha`, `backgroundColor`, plus constraint changes via `layoutIfNeeded()`
- `animate(springDuration:bounce:initialSpringVelocity:delay:options:)`: `bounce` 0 = critically damped, 0.15 - 0.3 = gentle overshoot
- Legacy spring: `usingSpringWithDamping:initialSpringVelocity:` (harder to tune, still in older code)

- Options: `.curveEaseInOut`, `.allowUserInteraction`, `.beginFromCurrentState`, `.repeat` + `.autoreverse`, `.transitionCrossDissolve`, `.flushUpdates` (iOS 26+)
- `animateKeyframes` + `addKeyframe(withRelativeStartTime:relativeDuration:)` for choreographed steps
- `UIView.performWithoutAnimation { }` to opt a change out of an enclosing animation
- SwiftUI curves in UIKit: `UIView.animate(.smooth) { }` (requires `import SwiftUI`)

Related to check: Animations (overview), UIViewPropertyAnimator, Core Animation / CATransaction, Draw cycle / `updateProperties()`, UIUpdateLink, UIViewController transitions

---

UICollectionView Prefetching

Q: How does prefetching work in UICollectionView?

Short answer: A collection view's `prefetchDataSource` (`UICollectionViewDataSourcePrefetching`) is told which index paths are *about* to appear, based on scroll direction and speed, so you can start expensive work early: image downloads and decoding, database reads, network pages. When the user changes direction, `cancelPrefetchingForItemsAt` tells you those items are no longer likely, so you cancel.

Key points:

- `collectionView.prefetchDataSource = self`; `isPrefetchingEnabled` is on by default.
- `prefetchItemsAt` is a hint, not a guarantee: it isn't called for the first visible screen or for every item during a fast fling. The cell configuration must still be able to load on its own.
- Key work by **item identifier**, not index path: with diffable data source use `dataSource.itemIdentifier(for:)`, because index paths shift when the snapshot changes.
- Cancellation is cooperative: `task.cancel()` propagates into `URLSession.data(from:)`, `Task.sleep` and `Task.checkCancellation()`.
- Keep the cache/loader off the main actor (an `actor`, or decode with `byPreparingForDisplay()`), and keep the task bookkeeping on the main actor with the view controller.

Related to check: UITableView prefetching, Cell reuse, Diffable data source, Image downsampling, Lazy loading strategy, Task cancellation

---

UIViewController Transitions

Q: How do you create custom view controller transitions?

Short answer: First I check whether a built-in transition does the job. Since iOS 18, `preferredTransition = .zoom { ... }` gives a fluid, interactive zoom from a source view for both push and present, including interactive dismissal by swiping, with no animator code.

Key points:

- Built-in: `preferredTransition = .zoom(sourceViewProvider:)` (iOS 18), `.zoom(sourceBarButtonItemProvi` (iOS 26+), `.crossDissolve`, `.coverVertical`, `.flipHorizontal`, `.partialCurl`
- `UIViewControllerTransitioningDelegate`: `animationController(forPresented:presenting:source:)`, `animationController(forDismissed:)`, `presentationController(forPresented:presenting:source:)`
- `UIViewControllerAnimatedTransitioning`: `transitionDuration(using:)`, `animateTransition(using:)`, optional `interruptibleAnimator(using:)`
- Context: `containerView`, `view(forKey: .from/.to)`, `finalFrame(for:)`, `completeTransition(!transitionWasCancelled)`
- `UIPresentationController` subclass for chrome and sizing (dimming view, custom frame)

Related to check: Present vs push, Animations / `UIViewPropertyAnimator`, UIPresentationController and sheets, UINavigationController (Liquid Glass bars), SwiftUI zoom navigation transition

---

UIScene and Multi-Window

Q: How does UIScene support multiple windows on iPad?

Short answer: Each window is a `UIWindowScene` with its own `UISceneSession`, its own scene delegate, its own window hierarchy and its own foreground/background state. The app process is shared, so models and services live at app level while navigation and UI state live per scene.

Key points:

- `UISceneSession`: the persistent record of a window (`persistentIdentifier`, `role`, `configuration`, `stateRestorationActivity`, `userInfo`). It survives the scene being disconnected to save memory
- `UIWindowScene`: the live scene, which owns `windows`, `keyWindow`, `screen`, `traitCollection` and `effectiveGeometry`
- Scene configurations in the Info.plist (or returned from `application(_:configurationForConnecting:options:)`) choose a delegate class per role or activity type
- Open a window: `activateSceneSession(for: UISceneSessionActivationRequest(userActivity:))`, which replaces `requestSceneSessionActivation(_:userActivity:options:errorHandler:)`. Close one: `requestSceneSessionDestruction(_:options:errorHandler:)`
- Route new windows by `NSUserActivity` type: the activity arrives in `connectionOptions.userActivities`
- iPadOS 26 windowing: every app window can be freely resized and arranged, so a scene can be any size at any time
- iOS 26: `windowScene(_:didUpdateEffectiveGeometry:)` replaces the deprecated `didUpdateCoordinateSpace` callback for size and orientation changes

Related to check: UIApplication lifecycle (TN3187 scene requirement), UISplitViewController, State restoration, Size classes, Rotation and orientation (`effectiveGeometry`)

Migrating to the Scene Life Cycle

Q: Your legacy app uses only an AppDelegate. What breaks when you rebuild with Xcode 27, and how do you migrate?

Short answer: The short answer is that it stops launching. Per TN3187, iOS 18.4 started logging a warning for apps without the UIScene life cycle, iOS 26 strengthened it, and **an app built with the iOS 27 SDK that hasn't adopted scenes won't launch on iOS 27**.

Key points:

- **Step 1, manifest:** `UIApplicationSceneManifest` with `UIApplicationSupportsMultipleScenes = NO` and one `UIWindowSceneSessionRoleApplication` configuration naming `$(PRODUCT_MODULE_NAME).SceneDelegate` (and `UISceneStoryboardFile` if you use a main storyboard, in which case UIKit creates the window)
- **Step 2, window:** `UIWindow(windowScene:)` in `scene(_:willConnectTo:options:)`; `UIWindow(frame:)` is deprecated in iOS 26, and `AppDelegate.window` goes away
- **Step 3, inputs:** launch options become `UIScene.ConnectionOptions` (`urlContexts`, `userActivities`, `shortcutItem`) on cold launch; warm launches arrive in `scene(_:openURLContexts:)`, `scene(_:continue:)` and `windowScene(_:performActionFor:completionHandler:)`
- **Step 4, life-cycle work:** `applicationDidBecomeActive` / `WillResignActive` / `DidEnterBackground` / `WillEnterForeground` (deprecated in iOS 26) become `sceneDidBecomeActive` and

friends. After adoption the app-delegate versions **aren't called at all**. App-wide observers can use `UIApplication.didEnterBackgroundNotification` and the other app-level notifications
- **Step 5, restoration:** `stateRestorationActivity(for:)` on the scene delegate, read back from `session.stateRestorationActivity`
- **SwiftUI apps** using the `App` life cycle already use scenes; a `UIApplicationDelegateAdaptor` app delegate is fine

Related to check: UIApplication lifecycle (the states and callbacks), UIScene multi-window, App state restoration, Deep linking, Resizable iPhone apps, Discrete resizing and `UIRequiresFullScreen`, Backward compatibility (SDK-linked behavior)

Accessibility in UIKit

Q: How do you make UIKit views accessible?

Short answer: Every `UIView` takes part in the accessibility API: standard controls are accessible out of the box, and for custom views I set `isAccessibilityElement`, a short `accessibilityLabel`, an `accessibilityValue` for state, `accessibilityTraits`, and a hint only when the result of the action isn't obvious. Beyond VoiceOver, I support Dynamic Type with text styles and `adjustsFontForContentSizeCategory`, and I respect Reduce Motion and Reduce Transparency.

Key points:

- `accessibilityLabel` (what it is), `accessibilityValue` (current state, e.g. "50%"), `accessibilityHint` (what happens), `accessibilityTraits` (`.button`, `.header`, `.selected`, `.adjustable`)
- `isAccessibilityElement = true` on custom leaf views; containers group children with `accessibilityElements` or `shouldGroupAccessibilityChildren`
- `accessibilityCustomActions` instead of hidden swipe-only gestures; `accessibilityIncrement()`/`Decrement()` for `.adjustable`
- Dynamic Type: `preferredFont(forTextStyle:)`, `UIFontMetrics` for custom fonts, `numberOfLines = 0`, switch layouts for accessibility sizes
- `accessibilityIdentifier` is for UI tests and is never spoken; the label is what users hear
- Reduce Motion (`UIAccessibility.isReduceMotionEnabled`), Reduce Transparency (`isReduceTransparencyEna`
  Increase Contrast (`traitCollection.accessibilityContrast`)

Related to check: Dynamic Type, UITraitCollection (`registerForTraitChanges`), Liquid Glass (`UIGlassEffect`), UI testing with `XCUIApplication`, Animations (Reduce Motion)

SwiftUI

Observable Macro

Q: What is the @Observable macro? How does it replace ObservableObject?

Short answer: `@Observable` is a macro from the Observation framework that makes a class's stored properties trackable, with no `@Published` and no Combine. SwiftUI records which properties `body` actually reads and re-renders a view only when one of *those* properties changes, whereas `ObservableObject` invalidates every observing view on any `@Published` change.

Key points:

- **Per-property tracking:** a view depends only on the properties it read in `body`, so fewer re-renders than `ObservableObject`
- **Ownership:** `@State private var model = Model()`; passed in: plain `let`/`var`; need `$` bindings: `@Bindable`

- **Environment:** `.environment(model)` + `@Environment(Model.self) private var model`
- **`@ObservationIgnored`** excludes a property (caches, IDs, injected services)
- Classes only (not structs or actors); write UI models as `@MainActor @Observable final class`
- **Outside views:** `Observations { model.count }` (iOS 26+) is an `AsyncSequence` of changes; UIKit (iOS 26+) tracks `@Observable` reads in `layoutSubviews`, `updateProperties()` and similar methods automatically

Related to check: ObservableObject and @Published (the legacy model and migration), @StateObject vs @ObservedObject, @State / @Binding / @Bindable, sharing models via the environment (@EnvironmentObject), Combine integration, @MainActor, SwiftData and SwiftUI (`@Model` types are observable)

@State, @Binding, @Bindable, and (legacy) @ObservedObject

Q: Explain @State, @Binding, @Bindable, and @ObservedObject. When would you use each?

Short answer: `@State` is storage the view owns, for a simple value or for an `@Observable` model the view creates. `@Binding` is a two-way reference to a value someone else owns, which is how a child edits a parent's `@State`.

Key points:

- `@State`: view-owned source of truth; `private`; survives re-renders for the view's identity
- `@Binding`: no storage of its own; reads and writes the owner's value (`$count` creates one)
- `@Bindable`: wraps an `@Observable` reference to produce bindings; works on a property or as a local `@Bindable var model = model` inside `body`
- Read-only access to a passed-in `@Observable` model needs no wrapper: `let model: Model`
- `@ObservedObject` (legacy): passed-in `ObservableObject`; pairs with `@StateObject` for ownership

Related to check: Observable macro (the full model), @StateObject vs @ObservedObject, ObservableObject and @Published, sharing models via the environment (@EnvironmentObject), optional binding, forms and controls

---

@StateObject vs @ObservedObject

Q: What's the difference between @StateObject and @ObservedObject?

Short answer: Both observe an `ObservableObject`; the difference is ownership. `@StateObject` means the view owns the object: SwiftUI creates it once per view identity and keeps it alive across re-renders.

Key points:

- `@StateObject`: view owns it; its initial value is an `@autoclosure`, evaluated once per view identity
- `@ObservedObject`: view doesn't own it; the parent keeps it alive and passes it in
- Creating an object with `@ObservedObject` is the classic recreation bug: state resets whenever the parent's `body` runs
- Modern equivalents: `@State` (owns), `let model: Model` (passed in, read-only), `@Bindable var model: Model` (passed in, needs `$` bindings)

Related to check: Observable macro, ObservableObject and @Published (legacy model), @State / @Binding / @Bindable, view identity, view lifecycle

---

View Lifecycle

Q: What's the view lifecycle in SwiftUI? How does it differ from UIKit?

Short answer: A SwiftUI `View` is a cheap value, not an object with a lifecycle: its `init` and `body` run whenever SwiftUI needs a fresh description, which can be many times. The real lifecycle belongs to the view's *identity* in the render tree: it appears (`onAppear`, `.task` starts), updates when data it read changes (`body` re-runs, `onChange` fires), and disappears (`onDisappear`, `.task` cancelled).

Key points:

- `init`: runs on every parent update; keep it trivial, no side effects
- `body`: runs when observed state changes; must be pure and fast
- `.task { }`: the place for async loading; starts before the view appears, is cancelled when it disappears; `.task(id:)` restarts when a value changes
- `.onAppear` / `.onDisappear`: synchronous hooks; can fire more than once (tab switches, navigation pop, lazy rows scrolling)
- `.onChange(of:)` reacts to value changes; `@Environment(\.scenePhase)` for foreground/background
- Rough UIKit mapping: `viewDidLoad` ≈ first `.task`/`onAppear` (guard with a flag or load in the model), `viewWillAppear`/`viewIsAppearing` ≈ `onAppear`, `viewDidDisappear` ≈ `onDisappear`

Related to check: `.task` modifier, View identity, `body` as a computed property, `@State` / `@Observable`, UIKit view controller lifecycle (`viewIsAppearing`, `updateProperties()`), UIScene life cycle

---

body as Computed Property

Q: Why is body a computed property in SwiftUI? What are the implications?

Short answer: `body` is a computed property because a SwiftUI view is a *description* of UI for the current state, not the UI itself: SwiftUI calls `body` whenever a dependency changes, compares the new description with the old one, and updates only what differs in the render tree. A dependency is anything `body` reads: `@State`, `@Binding`, `@Environment` values, and with `@Observable` models exactly the properties it accessed.

Key points:

- Recomputed when something it *read* changes; the result is diffed, not redrawn wholesale
- With `@Observable`, tracking is per property: a view that reads `model.title` doesn't re-run when `model.count` changes (with `ObservableObject`, any `@Published` change invalidates every observer)
- `body` is `@MainActor` (as is all of `View`) and declared with `@ViewBuilder` implicitly
- No side effects: no network calls, no state mutation, no logging that assumes "once"
- Split large views into small subviews so each re-runs only for its own dependencies

Related to check: `@Observable` / `@State`, View identity, View lifecycle, `@ViewBuilder`, SwiftUI performance instrument, `Equatable` views

---

ObservableObject and @Published

Q: How do ObservableObject and @Published work together?

Short answer: `ObservableObject` plus `@Published` is the pre-iOS 17, Combine-based observation model. The protocol gives a class an `objectWillChange` publisher, and each `@Published` property calls `objectWillChange.send()` in its `willSet`; SwiftUI subscribes through `@StateObject`, `@ObservedObject` or `@EnvironmentObject` and invalidates the whole view on *any* published change.

Key points:

- Class-only protocol; `objectWillChange` is synthesized as an `ObservableObjectPublisher`
- `@Published` fires *before* the change (willSet); SwiftUI re-reads values on the next render pass
- Granularity is the whole object: any `@Published` change re-renders every observing view
- `$property` exposes a `Published.Publisher` for Combine (`debounce`, `removeDuplicates`, `sink`)
- Still appropriate: deployment target below iOS 17, or existing Combine pipelines built on `$property`

Related to check: Observable macro (the full modern model), @StateObject vs @ObservedObject, sharing models via the environment (@EnvironmentObject), Combine integration, @MainActor

---

Optional Binding

Q: How do you handle optional values and binding in SwiftUI?

Short answer: For display, unwrap with `if let` inside `body` and show a fallback otherwise. For *bindings*, SwiftUI gives you two tools: the failable initializer `Binding(_:)`, which turns a `Binding<T?>` into a `Binding<T>?` you can `if let`, and item-based presentation (`.sheet(item:)`, `navigationDestination(item:)`, and in Xcode 27 item-bound `alert` and `confirmationDialog`) where a non-nil value presents and setting it back to `nil` dismisses.

Key points:

- `if let value { ... } else { ... }` (shorthand `if let`) for optional content in `body`
- `if let text = Binding($note.subtitle) { TextField("Subtitle", text: text) }`
- `.sheet(item: $selection)` / `.fullScreenCover(item:)` / `navigationDestination(item:)` present while non-nil (sheet/cover need `Identifiable`)
- **Xcode 27 SDK (back-deploys):** `.alert(_:item:actions:)` and `.confirmationDialog(_:item:...)` bind the dialog to the item, replacing the `isPresented` + separate-state pattern
- Custom adapters: `Binding(get:set:)` to map `nil` to a default
- Optional `@Observable` model: unwrap, then `@Bindable var model = model` inside `body`

Related to check: @State and @Binding, @Bindable, Identifiable, sheets and full-screen covers, alerts and confirmation dialogs

---

NavigationStack and Navigation

Q: How does navigation work in SwiftUI? Explain NavigationStack and NavigationLink.

Short answer: `NavigationStack` is a push-style container whose state is data: a path of values. `NavigationLink(value:)` appends a value, and `.navigationDestination(for:)` maps each value type to a view, so the same push can come from a tap, a deep link or code.

Key points:

- `NavigationStack(path: $router.path)` + `NavigationLink(value:)` + `.navigationDestination(for: Route.self)`
- `NavigationPath` is type-erased (mixed value types, `Codable` via `path.codable` for state restoration); a typed `[Route]` array is simpler when one enum covers every screen
- Router: `@MainActor @Observable final class Router { var path = NavigationPath() }`, owned with `@State`, shared with `.environment(router)`
- **Zoom transition:** `.navigationTransition(.zoom(sourceID:in:))` on the destination + `.matchedTransitionSource(id:in:)` on the source

- **Tabs:** `TabView { Tab("Home", systemImage: "house") { ... } }`, `.tabViewStyle(.sidebarAdaptable)` (tab bar on iPhone, sidebar on iPad); each tab owns its own `NavigationStack`
- With the iOS 26+ SDK, navigation and tab bars adopt Liquid Glass automatically; `.tabBarMinimizeBehavior(.onScrollDown)` (iOS 26+) shrinks the tab bar while scrolling

Related to check: Toolbar, Sheets and fullScreenCover, matchedGeometryEffect / zoom transition, deep linking (`onOpenURL`), coordinator pattern, `NavigationSplitView`, `ArrangementView`, `@Observable`

TabView and Tab

Q: How do you build tab navigation in modern SwiftUI?

Short answer: Since iOS 18 a `TabView` is built from `Tab` values instead of views tagged with `.tabItem`: each `Tab` has a title, an image, an optional selection `value` and an optional `role`, and `TabSection` groups tabs. With `.tabViewStyle(.sidebarAdaptable)` the same declaration becomes a tab bar in compact width and a sidebar on iPad and in wide windows, and users can customize which tabs appear.

Key points:

- **Tab API** (iOS 18 baseline): `Tab("Home", systemImage: "house", value: .home) { ... }`; `TabView(selection: $selection)` binds to the values; `.badge(_:)` works on tabs
- **`Tab(role: .search)`** gets the system title, icon and placement (separated from the other tabs in the iOS 26 tab bar); `.tabViewSearchActivation(.searchTabSelection)` (iOS 26+) focuses the search field when the tab is selected
- **`TabSection`** groups tabs into sidebar sections; in a compact tab bar the section's tabs appear as ordinary tabs without the header, so keep the total small or hide some with `.defaultVisibility(.hidden, for: .tabBar)`
- **Customization:** `.customizationID(_:)` on tabs plus `.tabViewCustomization($customization)`; `TabViewCustomization` is `Codable`, so it persists in `@AppStorage`
- **iOS 26+:** `.tabBarMinimizeBehavior(.onScrollDown)` (or `.onScrollUp` for bottom-anchored content; iPhone only); `.tabViewBottomAccessory { }` with `@Environment(\.tabViewBottomAccessoryPlacem` (`.expanded` above the bar, `.inline` beside the minimized bar)
- **iOS 27+:** `TabRole.prominent` gives one tab prominent treatment (without one, a search tab may get it); `@Environment(\.isTabViewSidebarAvailable)` tells content whether a sidebar can appear

Related to check: NavigationStack, Liquid Glass, Toolbar, Searchable, UIKit `UITab` and `UITabGroup`, Deep linking / navigation architecture, App storage, Adaptive layout for iPhone Duo

---

Sheets and fullScreenCover

Q: How do you present sheets and full-screen modals in SwiftUI?

Short answer: Presentation is state-driven: `.sheet(isPresented:)` or `.sheet(item:)` shows a modal when a `Bool` becomes `true` or an optional `Identifiable` becomes non-`nil`, and SwiftUI resets that state when the user dismisses it. `.fullScreenCover` is the same API for immersive, non-swipe-dismissable flows.

Key points:

- `.sheet(item: $selected) { item in ... }` when the data drives presentation; one presentation per binding
- `@Environment(\.dismiss)` in the presented view; `.interactiveDismissDisabled()` for unsaved changes

- `presentationDetents([.medium, .large, .height(200), .fraction(0.3)])` + `presentationDragIndicator(.visible)`; bind `selection:` to track the detent
- `presentationBackgroundInteraction(.enabled(upThrough: .medium))` keeps the content behind interactive (Maps-style)
- `presentationSizing(.form / .page / .fitted)` controls size where sheets don't fill the window (iPad, Mac, large iPhone windows)
- **Liquid Glass (iOS 26+ SDK):** glass sheet backgrounds at partial detents; `presentationBackground(_:)` replaces the glass, so use it deliberately
- Zoom from the presenting control: `.navigationTransition(.zoom(sourceID:in:))` on the sheet content; toolbar items can be the source (iOS 26+)

Related to check: NavigationStack, Toolbar, alert / confirmationDialog (item-bound in iOS 27), zoom transition, Inspector, `@Environment(\.dismiss)`, UIKit `UISheetPresentationController`

@Environment and (legacy) @EnvironmentObject

Q: How do you share a model through the SwiftUI environment? How does @Environment(Model.self) compare with @EnvironmentObject?

Short answer: `@EnvironmentObject` is the `ObservableObject`-era way to share a model with a whole subtree without passing it through every initializer. Today you do the same with an `@Observable` model: inject it with `.environment(model)` and read it with `@Environment(Model.self) private var model`.

Key points:

- Modern: `.environment(model)` + `@Environment(Model.self)`; tracking is per property, like any `@Observable` read
- Lookup is by type; the nearest ancestor that injected that type wins
- Non-optional `@Environment(Model.self)` traps if nothing was injected; declare `Model?` to make it optional
- Legacy: `.environmentObject(model)` + `@EnvironmentObject var model: Model`, which also crashes if missing
- The owner (often the `App`) holds the model in `@State` and injects it

Related to check: Observable macro (the full model), environment values (custom `EnvironmentValues`), @State / @Binding / @Bindable, ObservableObject and @Published, dependency injection

---

GeometryReader

Q: What is GeometryReader? When would you use it?

Short answer: `GeometryReader` is a container that hands its content a `GeometryProxy` (size, safe-area insets, `frame(in:)`) and takes *all* the space it's offered, which is why it so often breaks layouts. In modern SwiftUI I reach for it last: to *read* a size or frame I use `onGeometryChange(for:of:action:)`, which doesn't affect layout; to size a child relative to its scroll view or container I use `containerRelativeFrame`; and for a genuinely custom arrangement I write a `Layout`.

Key points:

- `GeometryReader { proxy in }`: `proxy.size`, `proxy.safeAreaInsets`, `proxy.frame(in: .local / .global / .named("x"))`; it's greedy and places content at the top-leading corner
- **`onGeometryChange(for:of:action:)`**: observe a derived value (width, a `Bool` threshold, a frame) without wrapping the view; the action fires only when the transformed value changes
- **`containerRelativeFrame(_:count:span:spacing:)`**: size relative to the nearest container (scroll view, window), e.g. one card per page

- **`onScrollGeometryChange`** for content offset, instead of the old GeometryReader + `PreferenceKey` trick
- Make layout *decisions* from available width/height (or size classes), never from the device model, idiom or orientation

Related to check: `onGeometryChange`, `containerRelativeFrame`, `onScrollGeometryChange`, PreferenceKey, Layout protocol, `ViewThatFits` / `AnyLayout`, size classes, adaptive layout for iPhone Duo

---

Adaptive Layout for iPhone Duo and Resizable Windows

Q: How do you make a SwiftUI app work well on iPhone Duo and in resizable windows?

Short answer: I stop thinking in devices and design for the space the window gives me. In iOS 27 an iPhone app can be any size: it resizes in iPhone Mirroring and on iPad, and on iPhone Duo it goes from a phone-sized outer display to a 7.6" inner display when the user unfolds it.

Key points:

- **Structure:** `NavigationSplitView` shows columns in regular width and collapses to a stack in compact width; `preferredCompactColumn` chooses which column shows when collapsed. `TabView` with `.sidebarAdaptable` becomes a sidebar when there's room
- **Size classes for decisions, not identity:** `@Environment(\.horizontalSizeClass)` says how much room there is, not what the device is. A regular width can be an unfolded iPhone Duo, an iPad window or a stretched Mirroring window
- **Width for fine-tuning:** `onGeometryChange(for:of:action:)` transformed to a `Bool` or a column count, `containerRelativeFrame` to size relative to the container, `ViewThatFits` to pick the first arrangement that fits
- **Same identity, different layout:** `AnyLayout(HStackLayout())  AnyLayout(VStackLayout())` animates between arrangements and keeps `@State`; an `if/else` between two view trees resets it
- **Two-pane content:** `ArrangementView` (iOS 27.1+) arranges a primary and a secondary view (split or overlay) and adapts to size class and hardware features such as folding
- **SDK tiers on iPhone Duo:** built with the iOS 26 SDK the app runs centered with empty space around it; with the iOS 27 SDK it resizes to fill most of the inner display; with the iOS 27.1 SDK it runs edge to edge and toolbars and tab bars move into a vertical bar below the status bar

Related to check: GeometryReader (`onGeometryChange`, `containerRelativeFrame`), ArrangementView (iOS 27.1+), NavigationStack and Navigation, Toolbar (vertical bars on iPhone Duo), Size classes, Custom Layout, Testing resizable layouts, Designing an adaptive app for iPhone, iPhone Duo and iPad

---

ArrangementView

Q: What is ArrangementView and when would you use it?

Short answer: `ArrangementView` (iOS 27.1+) lays out two pieces of content, a primary and a secondary view, and lets the system choose the arrangement from the context: the available size, the size class and hardware features such as a foldable being folded or unfolded. You pick a style: `.split` puts the two views side by side along an axis it chooses, `.overlay` layers the primary over the secondary and can turn into a side-by-side layout when the context changes, and `.automatic` (the default) resolves to split.

Key points:

- `ArrangementView { primary } secondary: { secondary }` plus `.arrangementViewStyle(.split / .overlay / .automatic)`. Both closures are `ContentBuilders`

- **Split:** adapts its axis to available size and size class; `.split.axes(.vertical)` or `.axes(.horizontal)` restricts the axes it may use
- **Overlay:** primary over secondary in z-order, for full-screen experiences (media player, in-call UI). `.overlay.axes(_:)` sets which axes it may fall back to when it can no longer stay overlaid, for example when a foldable is folded
- **Tuning, applied to the arranged views:** `splitArrangementLayoutRatio(_:)` or the min/ideal/max horizontal and vertical ratio form, `splitArrangementLayoutSize(minWidth:idealWidth:maxWidt` `splitArrangementFixedLayoutSize(horizontal:vertical:)`; `layoutPriority` decides which view is sized first. `overlayArrangementEdge(.trailing / .bottom)` says where a view goes when the overlay becomes side by side
- **Reading the result:** `@Environment(\.splitArrangementAxis)` (`nil` outside a split arrangement) and `@Environment(\.overlayArrangementZIndex)` let content adapt to how it was placed
- **UIKit counterpart:** `UIArrangementViewController` with `.split` / `.overlay` arrangements and `setViewController(_:for: .primary / .secondary)` (iOS 27.1+)

Related to check: Adaptive layout for iPhone Duo, NavigationStack and Navigation (`NavigationSplitView`), GeometryReader and `ViewThatFits`, Custom Layout, UISplitViewController, iPhone Duo poses and vertical bars

---

Combine Integration

Q: How does SwiftUI integrate with Combine?

Short answer: SwiftUI's state no longer depends on Combine: `@Observable` models use the Observation framework, and only the legacy `ObservableObject` model is built on Combine (`objectWillChange`, `@Published`). Combine still matters for event streams such as timers and notifications, which you consume in a view with `.onReceive`, and for existing pipelines built on `$property` publishers.

Key points:

- `.onReceive(publisher)` runs a closure on the main thread for each value (timers, `NotificationCenter` publishers)
- `ObservableObject` uses `objectWillChange` (a Combine publisher); `@Published` exposes `$property`
- `@Observable` has no publishers; use `Observations { model.query }` (iOS 26+) for an `AsyncSequence` of changes
- Bridge a publisher into async code with `publisher.values`
- Debounce-and-search in views: `.task(id: query)` with `Task.sleep` replaces `$query.debounce(...).sink`

Related to check: Observable macro (Observations and the full model), ObservableObject and @Published, task modifier, @MainActor, async/await

---

Modifier Order

Q: Does modifier order matter in SwiftUI?

Short answer: Yes. Each modifier wraps the view it is applied to in a new view, so a chain is read inside out: every modifier acts on the result of everything above it.

Key points:

- Layout modifiers (`padding`, `frame`) change the size that later modifiers see
- Drawing modifiers (`background`, `overlay`, `border`) fill the size they receive at that point
- `clipShape` clips everything above it; `.cornerRadius(_:)` is deprecated: use `.clipShape(.rect(cornerRadius:))`

- `contentShape` and `allowsHitTesting` affect hit testing from that point
- Environment modifiers (`font`, `foregroundStyle`, `tint`) flow *down* to children, so their position relative to containers matters, not to sibling modifiers

Related to check: ViewModifier, View protocol, layout system (`frame`, `padding`), Liquid Glass (`glassEffect`), animations, hit testing

---

@MainActor

Q: What is @MainActor in SwiftUI? Why is it important?

Short answer: `@MainActor` is the global actor for the main thread; anything isolated to it can only run there, and the compiler enforces that in Swift 6 mode. SwiftUI's `View` protocol is `@MainActor`, so `body` and your view's methods are already main-actor isolated.

Key points:

- `View` is `@MainActor`: `body`, button actions and `.task` closures run on the main actor
- Mark UI models `@MainActor @Observable final class`; `@Observable` alone doesn't make a class `Sendable`, isolation does
- `await` inside a main-actor method suspends without blocking; the method resumes on the main actor
- Move heavy work off the main actor with `nonisolated` async functions or `@concurrent` (Swift 6.2)
- Swift 6.2 default MainActor isolation (new app targets) treats unannotated code as `@MainActor`

Related to check: Observable macro, task modifier, Combine integration, concurrency (actors, `Sendable`, async/await)

Identifiable

Q: What is Identifiable? When do you need it?

Short answer: `Identifiable` is a standard-library protocol with one requirement: an `id` whose type is `Hashable`. SwiftUI uses that id as the *identity* of each element in data-driven views, so it can tell which rows were inserted, removed, moved or simply updated, and animate and preserve state correctly.

Key points:

- `protocol Identifiable { associatedtype ID: Hashable; var id: ID { get } }`
- Classes that declare conformance get a default `id` (`ObjectIdentifier`); SwiftData `@Model` types are `Identifiable` (by `PersistentIdentifier`), and Xcode's Core Data codegen adds `Identifiable` to entity classes
- Without conformance: `ForEach(names, id: \.self)` for simple `Hashable` values
- Identity drives diffing, animations, `@State` preservation in rows, and scroll position
- **iOS 27+:** reorderable containers (`ForEach(...).reorderable()` + `.reorderContainer(for:move:)`) report moves by `Item.ID`, so stable ids matter even more

Related to check: View identity, ForEach and List, Hashable, `.sheet(item:)`, optional binding, reorderable containers (iOS 27)

PreferenceKey

Q: What is PreferenceKey? When would you use it?

Short answer: A `PreferenceKey` lets descendant views send values *up* the hierarchy, the opposite direction of the environment. Children attach a value with `.preference(key:value:)`, SwiftUI combines values from all children with the key's `reduce` function, and an ancestor reads the result with `.onPreferenceChange` or `overlayPreferenceValue` / `backgroundPreferenceValue`.

Key points:

- `protocol PreferenceKey { associatedtype Value; static var defaultValue: Value { get }; static func reduce(value: inout Value, nextValue: () -> Value) }`
- Implement `defaultValue` as `static let` (a `static var` stored property is an error in Swift 6)
- `reduce` merges sibling values: max, sum, append to an array, or merge a dictionary
- `Anchor<CGRect>` preferences (`anchorPreference`) pass geometry that the ancestor resolves with a `GeometryProxy`
- `onGeometryChange(for:of:action:)` reports a derived geometry value without a key or `GeometryReader`

Related to check: GeometryReader, onGeometryChange, Layout protocol, anchor preferences, environment (data flowing down)

@ViewBuilder

Q: What is @ViewBuilder? How does it work?

Short answer: `@ViewBuilder` is a result builder: the compiler rewrites a closure's list of statements into calls like `buildBlock`, `buildIf` and `buildEither`, which is how you can write several views in a row, plus `if`, `if let`, `switch` and `#available`, without `return` or wrapper types. The result is one concrete, statically typed view (`TupleView`, `_ConditionalContent`, `Optional<...>`), which lets SwiftUI know the structure and identity of every branch at compile time.

Key points:

- Result builder (`@resultBuilder`, SE-0289); `buildBlock` uses parameter packs, so the old 10-children limit is gone
- Supports `if`/`else`, `if let`, `switch`, `#available`, `ForEach`; not `for` loops or `guard`, and no arbitrary statements (a `let` is fine)
- Different branches produce `_ConditionalContent<A, B>`, so they're *different identities*: switching branches resets state
- Use it on custom container parameters: `init(@ViewBuilder content: () -> Content)` and store the built `Content`, not the closure
- **`ContentBuilder`** (`typealias ContentBuilder = ViewBuilder`): one builder for views, toolbar content, commands and more; `ViewBuilder` is not removed or renamed

Related to check: Result builders, `some View` / opaque types, View identity, custom containers (`ForEach(subviews:)`), Toolbar, `body` as a computed property

---

ScrollView vs List

Q: When would you use ScrollView vs List?

Short answer: `List` is the platform's row-based collection: lazy, backed by a collection view, with built-in selection, sections, swipe actions, edit mode, move/delete, pull-to-refresh and the system look. `ScrollView` is a plain scrolling container for any layout; it's eager unless you put a `LazyVStack`/`LazyVGrid` inside, and you build row behavior yourself. iOS 27 narrows the gap: reorderable containers (`.reorderable()` + `.reorderContainer`) work in stacks, grids and custom layouts, and `.swipeActionsContainer()` gives a custom `ScrollView` the same swipe coordination `List` has.

Key points:

- `List`: lazy, `selection:`, `Section`, `.swipeActions`, `.onMove`/`.onDelete` + `EditButton`, `.refreshable`, `.listStyle(.insetGrouped / .plain / .sidebar)`
- `ScrollView { LazyVStack { ForEach } }` for custom layouts; plain `VStack` builds every row up front
- **Reorder anywhere (iOS 27+):** `ForEach(items) { }.reorderable()` inside, `.reorderContainer(for: Item.self) { difference in }` on the container; the closure gets the moved ids (`sources`) and a `destination` (`.before(id)` or `.end`)
- **`.swipeActionsContainer()` (iOS 27+)** on a `ScrollView`: only one row's actions open at a time, scrolling or tapping elsewhere closes them (a no-op on `List`, which already does this)
- Both work with `ScrollPosition`, `scrollTargetLayout` and `onScrollGeometryChange`

Related to check: Lazy loading, Grid, ScrollViewReader / `ScrollPosition`, View identity, `ForEach`, `Identifiable`, swipe actions

---

Animations

Q: How do you animate in SwiftUI?

Short answer: SwiftUI animations are state-driven: you change state inside a transaction that carries an `Animation`, and SwiftUI interpolates every animatable property that changed. Explicit animation is `withAnimation(.snappy) { ... }`; implicit animation is `.animation(.smooth, value: x)`, scoped to one value; and the default animation is a spring, with presets `.smooth`, `.snappy` and `.bouncy`.

Key points:

- `withAnimation(_:_:completion:)` animates everything that changes in the closure; the completion closure runs when it finishes
- `.animation(_:value:)` animates only when `value` changes; `.animation(_:body:)` scopes the animation to the modifiers inside `body`
- Springs: `.spring(duration:bounce:)`, presets `.smooth` (no bounce), `.snappy`, `.bouncy`; springs keep velocity when interrupted
- Transitions: `.transition(.opacity)`, `.move(edge:)`, `.scale`, `.push(from:)`, combined with `.combined(with:)`; custom ones conform to the `Transition` protocol
- Custom animatable data: conform to `Animatable`, or let the `@Animatable` macro synthesize `animatableData` (exclude properties with `@AnimatableIgnored`)

Related to check: PhaseAnimator, ContentTransition, SymbolEffect, matchedGeometryEffect / zoom transition, Transactions, UIKit spring animations

---

UIViewRepresentable

Q: How do you use UIKit views in SwiftUI? What is UIViewRepresentable?

Short answer: `UIViewRepresentable` is a SwiftUI view that wraps a `UIView`. SwiftUI calls `makeUIView(context:)` once to create the view, `updateUIView(_:context:)` whenever the SwiftUI inputs change, and `dismantleUIView(_:coordinator:)` when it goes away.

Key points:

- `makeUIView` creates and configures once; `updateUIView` applies the current SwiftUI state (it runs often, so make it idempotent)
- `Coordinator`: delegates, target-action, and a reference back to the latest representable (`parent`) for bindings

- The protocol is `@MainActor`, so `makeCoordinator`, `make/update` and the `Coordinator` all run on the main actor; mark the coordinator `@MainActor` explicitly
- `context.environment` and `context.transaction` (animate UIKit changes when SwiftUI animates)
- `sizeThatFits(_:uiView:context:)` reports an ideal size for the proposal, instead of fighting intrinsic content size
- `UIGestureRecognizerRepresentable` for custom UIKit gestures on any SwiftUI view

Related to check: UIViewControllerRepresentable, UIHostingController, Coordinator pattern, MainActor and isolated conformances, adaptive layout (no `UIScreen.main`)

WebView

Q: How do you show web content in a SwiftUI app? When would you pick each option?

Short answer: It depends on whose content it is and what you need to control. For your own or tightly integrated content (help pages, terms, rich articles) on iOS 26+, SwiftUI's native `WebView` backed by an `@Observable` `WebPage` is the answer: no representable, and title, URL, progress and navigation are observable properties.

Key points:

- **`WebView(url:)`** (iOS 26+) for the simple case; **`WebView(page)`** with a `WebPage` (iOS 26+, `import WebKit`) when you need state or control
- **`WebPage`** is a `@MainActor` class that conforms to `Observable`: `title`, `url`, `isLoading`, `estimatedProgress`, `backForwardList`, `hasOnlySecureContent`; `load(_:)` accepts a `URL`, `URLRequest` or HTML string and returns an async sequence of navigation events (`.committed`, `.finished`, …); `reload()`, `stopLoading()`, and `callJavaScript(_:arguments:)` is async
- **Policies** with `WebPage.NavigationDeciding` (async `decidePolicy(for:preferences:)`), JavaScript dialogs with `DialogPresenting`, and `WebPage.Configuration` for the data store, user content controller and URL scheme handlers
- **View modifiers:** `.webViewBackForwardNavigationGestures(_:)`, `.webViewLinkPreviews(_:)`, `.webViewScrollPosition(_:)`, `.webViewContentBackground(_:)`
- **`SFSafariViewController`**: read-only browsing, private from your app, no JavaScript access; wrap it in `UIViewControllerRepresentable` and present it as a sheet
- **`ASWebAuthenticationSession`**: OAuth / OIDC in a system sheet that the app can't inspect; `.https(host:path:)` callbacks (iOS 17.4+) need associated domains

Related to check: UIViewRepresentable, OAuth 2.0 flow (PKCE), App Transport Security, Observation (`@Observable`), Sheets, Deep links / universal links

---

@Environment

Q: What is @Environment? How do you read and set environment values?

Short answer: The environment is a set of values SwiftUI passes implicitly down the view hierarchy. You read one with `@Environment(\.keyPath)` and override it for a subtree with `.environment(\.keyPath, value)`.

Key points:

- Read: `@Environment(\.dismiss) private var dismiss`
- Set for a subtree: `.environment(\.theme, .dark)`; the nearest ancestor wins
- Custom values: `extension EnvironmentValues { @Entry var theme: Theme = .standard }`

- `@Entry` also works on `FocusedValues`, `Transaction` and `ContainerValues`
- `@Observable` models use the type-based overload: `.environment(model)` + `@Environment(Model.self)`

Related to check: @Observable macro, @EnvironmentObject (legacy), PreferenceKey (data flowing up), size classes and adaptive layout, ContainerValues

---

.task Modifier

Q: What does the .task modifier do? How does it differ from .onAppear?

Short answer: `.task { }` starts an async task tied to the view's lifetime: it begins before the view appears and SwiftUI **cancels it automatically when the view disappears**. `.task(id:)` adds a restart rule: whenever the id changes, the running task is cancelled and a new one starts, which makes it the idiomatic tool for "load the detail for the current selection" or "debounce a search".

Key points:

- `.task { await model.load() }`: runs once per appearance of this identity, cancelled on disappear
- `.task(id: query) { }`: cancels and restarts when `query` changes (id must be `Equatable`)
- `.task(priority:)` sets the `TaskPriority` (default `.userInitiated`); in the Xcode 27 SDK, `.task(name:)` labels the task for the Swift Concurrency instrument and the debugger (by default it's named after the file and line)
- The body inherits main-actor isolation; `await` suspends without blocking the UI; CPU work goes in a `@concurrent` function
- `.onAppear` for cheap synchronous setup (analytics, focus), `.task` for anything async

Related to check: View lifecycle, View identity, async/await, task cancellation, `withTaskCancellationShield` (iOS 27), `@concurrent`, `@Observable`, `.refreshable`, `searchable`

Custom Layout (Layout Protocol)

Q: How do you create custom layouts in SwiftUI? What is the Layout protocol?

Short answer: The `Layout` protocol lets you write your own container, like a flow or radial layout, using the same propose-measure-place negotiation the built-in stacks use. You implement `sizeThatFits` (given the parent's proposal, how big am I?) and `placeSubviews` (where does each child go?), asking each subview for its size with `sizeThatFits` along the way.

Key points:

- `sizeThatFits(proposal:subviews:cache:)` and `placeSubviews(in:proposal:subviews:cache:)`; the proposal's width/height can be `nil` (ideal size), `0` (minimum) or `.infinity` (maximum)
- `subview.sizeThatFits(_:)` measures, `subview.place(at:anchor:proposal:)` positions; place relative to `bounds.minX/minY`, not zero
- **`AnyLayout(HStackLayout())` / `AnyLayout(VStackLayout())`** switches layout *without* changing child identity, so state is kept and the change animates
- **`ViewThatFits`** picks the first child that fits the available space
- **`ArrangementView { primary } secondary: { } (iOS 27.1+)`**: system-adaptive primary/secondary arrangement driven by size, size class and hardware; styles `.automatic` (resolves to split), `.split`, `.overlay`

Related to check: GeometryReader / `onGeometryChange`, Layout priorities, Grid, `ViewThatFits`, `AnyLayout`, `ArrangementView`, size classes, adaptive layout for iPhone Duo

---

Custom Containers

Q: How do you build your own container view in SwiftUI, one that takes arbitrary child views like `List` or `VStack` does?

Short answer: A custom container takes a `@ViewBuilder` content closure and then works with the *resolved subviews* rather than with data. `ForEach(subviews: content) { subview in ... }` iterates each child (static views and the expansions of any `ForEach` inside are flattened together), and `Group(subviews: content) { subviews in ... }` hands you the whole `SubviewsCollection` so you can count, slice or split it.

Key points:

- **`Subview`** is a `View` and `Identifiable` (identity comes from the child's own identity), so you can wrap and decorate it, and `ForEach` over a `SubviewsCollection` works directly
- **`ForEach(subviews:)`** for per-child decoration; **`Group(subviews:)`** when the layout depends on the collection (count, first item, "+N more", splitting into columns)
- **Container values** are scoped to the *direct* container, unlike environment values (flow down to everything) and preferences (bubble up to any ancestor)
- **Sections:** `ForEach(sections: content)` gives `SectionConfiguration` values with `header`, `content` and `footer` collections plus their own `containerValues`
- `Layout` subviews (`LayoutSubview`) can read `containerValues` too, so a custom `Layout` can use the same keys

Related to check: ViewBuilder / ContentBuilder, Custom Layout (`LayoutValueKey`), Environment values (`@Entry`), Preference keys, View identity, ForEach and `Identifiable`

Alert and confirmationDialog

Q: How do you show alerts and confirmation dialogs in SwiftUI?

Short answer: Both are view modifiers driven by state: `.alert` for an important message or a yes/no decision, `.confirmationDialog` for a set of choices about an action the user just started (the old action sheet). The actions are a `@ViewBuilder` of `Button`s with roles, and an optional `message:` closure.

Key points:

- `alert("Title", isPresented: $flag) { buttons } message: { Text(...) }`
- Item-bound (Xcode 27 SDK; back-deploys to earlier iOS): `alert("Delete?", item: $pendingDelete) { item in ... }`; T doesn't need to be `Identifiable`
- Before Xcode 27: `alert(_:isPresented:presenting:)` plus a separate Bool, or a hand-made `Binding<Bool>` derived from the optional
- Roles: `.destructive` (red), `.cancel` (the system places it; a confirmation dialog adds one if you omit it); `.confirm` and `.close` (iOS 26+)
- SwiftUI dismisses the dialog and resets the binding for you when any button is tapped

Related to check: @State, @Binding, sheets and fullScreenCover, Liquid Glass popovers, error handling in view models

---

AttributedString and Text

Q: How do you display formatted or attributed text in SwiftUI?

Short answer: For simple styling I use `Text` with inline Markdown or modifiers, and interpolate styled `Text` into other `Text` to mix styles. For runs with their own attributes (a link, a highlighted range, a color) I build a Swift `AttributedString`, a value type with type-safe attribute scopes, and pass it to `Text`.

Key points:

- `Text("This is **bold** and [a link](https://example.com)")`: string *literals* are `LocalizedStringKey`s and support Markdown
- Mix styles with interpolation: `Text("Total: \(Text(price).bold())")`; `Text + Text` is deprecated in iOS 26
- `AttributedString`: `var s = AttributedString("Hello")`, `s.font = .headline`, `s.foregroundColor = .red`, ranges via `s.range(of:)`, concatenation with `+`
- `try AttributedString(markdown:)` for Markdown from a runtime `String`
- Rich-text editing (iOS 26+): `TextEditor(text: $attributed, selection: $selection)`; change formatting with `transformAttributes(in: &selection)`

Related to check: Text, Markdown, Localization (String Catalogs), TextEditor, UITextView / UIViewRepresentable

---

Toolbar

Q: How do you add toolbar items in SwiftUI?

Short answer: Use `.toolbar { }` with `ToolbarItem` or `ToolbarItemGroup` and a *semantic* placement (`.primaryAction`, `.cancellationAction`, `.confirmationAction`) or a positional one (`.topBarLeading`, `.topBarTrailing`, `.bottomBar`, `.keyboard`); SwiftUI then puts items in the right bar for the platform and context, and the toolbar must sit inside a `NavigationStack` to show on iOS. With the iOS 26+ SDK, toolbars are Liquid Glass: adjacent items share one glass background, `ToolbarSpacer` (iOS 26+) splits them into separate groups, and bars should have no custom background. iOS 27 adds control over what overflows when space runs out: `.visibilityPriority(_:)` on items, `.toolbarOverflowMenu { }` and the `.topBarPinnedTrailing` placement.

Key points:

- `ToolbarItem(placement:)`, `ToolbarItemGroup`, `.principal` for a custom title view, `.keyboard` for an input accessory bar
- `.topBarLeading` / `.topBarTrailing` replace the deprecated `.navigationBarLeading` / `.navigationBarTrailing`
- **iOS 26+:** `ToolbarSpacer(.fixed)` / `ToolbarSpacer(.flexible)` separate glass groups; `.sharedBackgroundVisibility(.hidden)` takes an item out of the shared glass; `.buttonStyle(.glassProminent)` for the primary action; `.tabBarMinimizeBehavior(.onScrollDown)` on a `TabView` shrinks the tab bar while scrolling
- **iOS 27+:** `.visibilityPriority(.high)` keeps an item visible longer when space is tight (lower-priority items move to the overflow menu first); `.toolbarOverflowMenu { }` puts actions in the overflow menu permanently; `.topBarPinnedTrailing` pins an item to the trailing edge (it only overflows when search is active and space runs out)
- There is **no** `toolbarMinimizeBehavior`; the auto-minimize API is iOS 26's `tabBarMinimizeBehavior`

Related to check: NavigationStack, ControlGroup, Sheets, `TabView` / `tabBarMinimizeBehavior`, Liquid Glass (`glassEffect`, `.buttonStyle(.glass)`), UIKit `UINavigationBar` customization

---

Forms and Controls

Q: What is a Form in SwiftUI? How do you use it for settings-style UIs?

Short answer: `Form` is a container that styles its controls for data entry the platform way: on iOS it's a grouped, scrolling list of rows, like Settings. I put standard controls inside (`TextField`, `Toggle`, `Picker`,

Stepper, Slider, DatePicker, LabeledContent), group them with Section headers and footers, and bind them to an @Observable model through @Bindable.

Key points:

- Form { Section("Account") { ... } footer: { ... } }
- Bind controls to a @MainActor @Observable model with @Bindable (or @State for local values)
- Pickers default to a menu inside a Form; .pickerStyle(.navigationLink / .inline / .segmented) changes that
- LabeledContent("Version", value: "2.1") for read-only rows; .disabled(_:) for dependent settings
- Primary actions go in the toolbar (.confirmationAction / .cancellationAction placements) or a prominent button

Related to check: @FocusState, @Bindable, Picker, Toggle, toolbar placements, Liquid Glass (glassEffect, glass button styles), NavigationSplitView

---

View Identity

Q: What is view identity in SwiftUI? Why does it matter?

Short answer: Identity is how SwiftUI decides that a view in this update is "the same" view as in the last one. It ties together lifetime, state and animation: while identity stays the same, @State (and an @Observable model owned by @State) is kept, and changes animate; when identity changes, the old view is torn down (its .task cancelled, onDisappear called) and a new one starts with fresh state.

Key points:

- Structural: the two branches of an if/else are different views, so switching resets their state and transitions instead of animating
- Explicit: ForEach(items) uses Identifiable.id; .id(value) gives a new identity whenever value changes
- Prefer one view with changing *modifier values* (.opacity(isOn ? 1 : 0), AnyLayout) over if/else when you want state kept and a smooth animation
- AnyView erases the type, so SwiftUI can't use the type to compare structure; avoid it in hot paths
- Lifetime follows identity: @State initial values, .task, onAppear/onDisappear are all per identity, not per struct init

Related to check: @State / @Observable ownership, View lifecycle, .task modifier, body as a computed property, ForEach / Identifiable, AnyLayout, matchedGeometryEffect

---

Lazy Loading

Q: How do you implement lazy loading in SwiftUI? LazyVStack vs VStack?

Short answer: LazyVStack, LazyHStack, LazyVGrid and LazyHGrid create their child views only when they're about to scroll into view, while a VStack builds and measures every child immediately. List is lazy by default.

Key points:

- Lazy stacks/grids must be inside a ScrollView; VStack is right for small, fixed content
- Lazy containers create views on demand but don't recycle cells like UITableView; keep rows light and ids stable
- **Pagination:** trigger the next page from the last row's .task (or onScrollTargetVisibilityChange), and guard against duplicate loads in the @MainActor @Observable model

- **`AsyncImage`:** phases (`empty`, `success`, `failure`); **iOS 27+:** HTTP-compliant caching, `AsyncImage(request:)` for a cache policy, `.asyncImageURLSession(session)` to use your own session and `URLCache` for a view subtree
- `.task` on a row is cancelled when the row goes away, so in-flight work for scrolled-off rows stops

Related to check: ScrollView vs List, Grid, `.task` modifier, View identity, image caching, `URLCache`, pagination

---

Accessibility

Q: How do you make SwiftUI views accessible?

Short answer: Standard SwiftUI controls are accessible by default: a `Button("Save")` already has a label, a button trait and an action, so most of the work is refining custom views. I use `accessibilityLabel` / `Value` / `Hint` / `AddTraits`, group related views with `accessibilityElement(children:)`, and expose gestures as actions (`accessibilityAction`, `accessibilityAdjustableAction`).

Key points:

- Label: what it is ("Share"); Value: current state ("3 of 5 stars"); Hint: what happens when activated, only when it's not obvious
- Traits: `.isHeader` (rotor navigation), `.isSelected`, `.isButton` for tappable non-`Button` views
- `accessibilityElement(children: .combine)` merges a row into one element; `.ignore` + your own label replaces the children
- Custom gestures must have accessible equivalents: `accessibilityAction(named:)`, `accessibilityAdjustableAction` for swipe up/down
- Dynamic Type: use text styles (`.font(.body)`), check `dynamicTypeSize.isAccessibilitySize` and switch layout with `AnyLayout` or `ViewThatFits`
- Environment: `accessibilityReduceMotion`, `accessibilityReduceTransparency`, `colorSchemeContrast`, `accessibilityVoiceOverEnabled`

Related to check: ScaledMetric, Dynamic Type, accessibility identifiers (UI testing), Liquid Glass / `glassEffect`, Animations (Reduce Motion), UIKit accessibility (`UIAccessibility`)

#Preview and PreviewProvider

Q: How do you create SwiftUI previews with #Preview? How does it relate to PreviewProvider?

Short answer: Previews are declared with the `#Preview` macro: `#Preview("Name") { MyView() }` renders the view in Xcode's canvas, with an optional name and *traits* such as orientation or size-that-fits layout. `@Previewable` lets you declare `@State` (and other dynamic properties) right inside the preview so bindings are live, and a `PreviewModifier` builds shared, cached setup like an in-memory SwiftData container once for many previews.

Key points:

- `#Preview { ... }`, `#Preview("Dark mode") { ... }`, multiple previews per file
- Traits: `#Preview(traits: .landscapeLeft)`, `.sizeThatFitsLayout`, `.fixedLayout(width:height:)`, `.modifier(MyPreviewModifier())`
- `@Previewable @State var isOn = true` for live, interactive bindings
- `PreviewModifier`: `makeSharedContext()` (async, cached) + `body(content:context:)` for sample data and environment setup
- Devices come from the canvas device picker, not code; `.previewDevice("...")` is legacy
- Inject dependencies exactly as the app does: `.environment(model)`, `.modelContainer(...)`

Related to check: @Observable and @Environment, SwiftData and SwiftUI, adaptive layout, Xcode canvas, Dynamic Type

---

@FocusState

Q: How do you manage keyboard focus in SwiftUI?

Short answer: `@FocusState` is a property wrapper that both reads and drives which view has focus. I bind fields to it with `.focused($focus, equals: .email)` (or `.focused($isFocused)` for a single field), move focus by assigning a new value, and dismiss the keyboard by setting it to `nil` or `false`.

Key points:

- `@FocusState private var focus: Field?` with a `Hashable` enum for several fields; `Bool` for one
- `.focused($focus, equals: .password)` links a view to a value
- Assign to move focus; `nil` dismisses the keyboard
- `.onSubmit { }` + `.submitLabel(.next / .done)` for Return-key flow
- `.defaultFocus($focus, .email)` sets the initial focus declaratively

Related to check: Forms and controls, TextField / SecureField, Keyboard avoidance, `keyboardLayoutGuide` (UIKit), Commands and `@FocusedValue`

@FetchRequest and Core Data

Q: How do you integrate Core Data with SwiftUI?

Short answer: Inject the persistent container's `viewContext` into the environment with `.environment(\.managedObjectContext, ...)`, then use `@FetchRequest` in views: it runs the fetch, returns `FetchedResults` and keeps them up to date as the context changes. Writes go through the same `@Environment(\.managedObjectContext)`, and heavy work goes on a background context.

Key points:

- `@Environment(\.managedObjectContext)` provides the context (main-queue `viewContext`)
- `@FetchRequest(sortDescriptors:predicate:animation:)`; prefer Swift `SortDescriptor(\Item.timestamp)` over `NSSortDescriptor`
- `FetchedResults` is a live `RandomAccessCollection`; Xcode's generated entity classes conform to `Identifiable` (add `extension Item: Identifiable {}` for hand-written subclasses), so `List`/`ForEach` work directly
- Change filters at runtime through `items.nsPredicate` / `items.sortDescriptors`
- `@SectionedFetchRequest` for grouped lists
- SwiftData equivalent: `@Query(filter: #Predicate<Item> { !$0.isArchived }, sort: \Item.timestamp, order: .reverse)`

Related to check: SwiftData and SwiftUI (`@Query`), @Observable macro, Swift Concurrency and Sendable, Table and column, Identifiable

---

matchedGeometryEffect

Q: What is matchedGeometryEffect? When do you use it?

Short answer: `matchedGeometryEffect(id:in:)` links two views with the same id in the same `@Namespace`, so when one is removed and the other inserted inside an animation, SwiftUI interpolates the frame

between them: a "hero" transition *within one view hierarchy*, like a thumbnail expanding into a card or a selection pill sliding between tabs. It doesn't work across navigation pushes or sheets.

Key points:

- `@Namespace private var ns`; same `id` + same namespace on the outgoing and incoming view; change state inside `withAnimation`
- Only one view per id should be the *source* (`isSource: true`, the default) at a time
- `properties: .frame / .size / .position` chooses what's matched; `anchor:` sets the reference point
- **Zoom transition:** `NavigationLink` / `.sheet` / `.fullScreenCover` destination gets `.navigationTransition(.zoom(sourceID:in:))`; the tapped view gets `.matchedTransitionSource(id:in:)`
- Toolbar items can be zoom sources too (iOS 26+), e.g. a "Compose" button that zooms into a sheet
- UIKit equivalent: `preferredTransition = .zoom { context in sourceView }`

Related to check: NavigationStack, Sheets and fullScreenCover, `PhaseAnimator`, `contentTransition`, UIKit `preferredTransition = .zoom`, view identity

---

Layout Priorities

Q: How do layout priorities work in SwiftUI?

Short answer: SwiftUI layout is a negotiation: the parent proposes a size, each child chooses its own size, and the parent places it. A stack first gives every child its minimum, then hands out the remaining space starting with the *least flexible* children.

Key points:

- `.layoutPriority(1)` on the view that must keep its full size; others compress first
- Within the same priority, space goes to the least flexible child first (fixed text before `Spacer`, which is fully flexible)
- `Spacer(minLength:)` defaults to the system spacing as its minimum; `Spacer(minLength: 0)` lets it collapse fully
- **Alternatives** that are often clearer: `.fixedSize()` (never compress), `.lineLimit` + `.truncationMode`, `.minimumScaleFactor`, or `ViewThatFits` to swap to a different layout instead of truncating
- Custom `Layouts` can read a child's priority through `subview.priority`

Related to check: Custom Layout (`subview.priority`), `ViewThatFits`, `AnyLayout`, `fixedSize`, Grid, Dynamic Type / `ScaledMetric`, GeometryReader

---

@AppStorage

Q: What is @AppStorage? How does it differ from @State?

Short answer: `@AppStorage` is a property wrapper that reads and writes a single `UserDefaults` key and invalidates the view whenever that key changes, so the value survives relaunches and stays in sync across every view using the same key. `@State` is in-memory, private to one view's identity, and gone when the view leaves the hierarchy or the app quits.

Key points:

- Backed by `UserDefaults.standard` by default; `@AppStorage("key", store: UserDefaults(suiteName:))` for an App Group shared with widgets or extensions; `.defaultAppStorage(_:)` sets the store for a whole subtree

- Supported types: `Bool`, `Int`, `Double`, `String`, `URL`, `Data`, `Date`, their optionals, and `RawRepresentable` enums with `String` or `Int` raw values
- `$value` gives a `Binding`, so it plugs straight into `Toggle`, `Picker`, `TextField`
- The default value is used only when the key is missing; it is not written to disk until you set it
- Sibling: `@SceneStorage` persists per-scene UI state (selected tab, draft text) for state restoration, not app-wide preferences

Related to check: @State, @SceneStorage, @Observable macro, SwiftData and SwiftUI, WidgetKit integration (App Groups), Keychain

---

ScrollViewReader

Q: How do you programmatically scroll in SwiftUI?

Short answer: The modern answer is `ScrollPosition`: keep a `@State var position = ScrollPosition(...)`, bind it with `.scrollPosition($position)`, and call `position.scrollTo(id:)`, `scrollTo(edge:)` or `scrollTo(y:)`. It's two-way: when the user scrolls, the same value tells you which view or edge is showing and whether the user positioned it (`isPositionedByUser`).

Key points:

- `ScrollPosition(edge: .bottom)`, `ScrollPosition(idType: Message.ID.self)`; `.scrollPosition($position)` on the `ScrollView`
- Mark the scrolling content with `.scrollTargetLayout()` so `ForEach` ids become scroll targets and `position.viewID` updates as the user scrolls
- Wrap `scrollTo` in `withAnimation` for an animated scroll
- **`onScrollGeometryChange(for:of:action:)`**: derive a value (offset past a threshold, distance to bottom) from `ScrollGeometry`
- `onScrollVisibilityChange` (a view became visible, e.g. autoplay video), `onScrollPhaseChange` (idle / tracking / decelerating)
- `defaultScrollAnchor(.bottom)` for chat-style lists that start at the end
- Legacy: `ScrollViewReader { proxy in }` + `.id(_:)` + `proxy.scrollTo(id, anchor: .center)`

Related to check: ScrollView vs List, Lazy loading, GeometryReader / `onGeometryChange`, `scrollTargetBehavior`, View identity, Focus state

---

@ScaledMetric

Q: What is @ScaledMetric? When do you use it?

Short answer: `@ScaledMetric` is a dynamic property that scales a numeric value with the user's Dynamic Type setting. Text styles like `.body` already scale on their own; `@ScaledMetric` is for the *non-text* numbers around them, such as padding, spacing, icon frames and avatar sizes, so the layout grows in proportion with the text.

Key points:

- `@ScaledMetric(relativeTo: .body) private var avatarSize = 40.0`
- Re-evaluates when `dynamicTypeSize` changes, like any `DynamicProperty`
- `relativeTo:` matters: `.largeTitle` scales less steeply than `.caption` at large sizes
- SF Symbols sized with `.font(...)` and `Label`s already scale; don't double-scale them

Related to check: Dynamic Type, Accessibility, `ViewThatFits` / `AnyLayout`, `UIFontMetrics`, Adaptive layout

---

Canvas and Drawing

Q: How do you do custom drawing in SwiftUI?

Short answer: For a single custom shape I conform to `Shape` and build a `Path`: it's resolution-independent, can be filled, stroked and clipped, and animates when its data is `Animatable`. For many elements or immediate-mode drawing (charts, particles, a drawing surface) I use `Canvas`, which hands me a `GraphicsContext` and draws everything as one view instead of hundreds of views.

Key points:

- `Shape` / `Path`: vector, layout-aware (`path(in:)`), animatable (`@Animatable` or `animatableData`)
- `Canvas { context, size in ... }`: immediate mode, fill/stroke/draw images, text and resolved symbols; one view in the hierarchy
- `TimelineView(.animation)` + `Canvas` for per-frame animation
- `MeshGradient`: a grid of points and colors for smooth, organic gradients; animate the points
- Shaders: write a `[[ stitchable ]]` Metal function, call it via `ShaderLibrary`, apply with `.colorEffect` (per-pixel color), `.distortionEffect` (move pixels) or `.layerEffect` (sample the layer)

Related to check: Animations (`@Animatable`), Metal / GPU, `drawingGroup`, Liquid Glass, Accessibility, Swift Charts

---

Inspector

Q: What is the Inspector modifier? When do you use it?

Short answer: `.inspector(isPresented:content:)` adds a trailing panel for supplementary detail about the current content: properties, formatting, metadata, like the inspector in Keynote or Xcode. It adapts to the available width: in a regular-width window it's a resizable trailing column beside the content, and in compact width it's presented as a sheet.

Key points:

- `.inspector(isPresented: $show) { InspectorContent() }`, toggled from a toolbar button
- `inspectorColumnWidth(min:ideal:max:)` (or a fixed width) sizes the column in regular width
- In compact width it's a sheet, so presentation modifiers like `presentationDetents` apply there
- Attach it to the content inside a `NavigationStack`, or to the detail column of a `NavigationSplitView`, so its toolbar items merge with that column's toolbar
- It's supplementary: the primary content must work with the inspector closed

Related to check: `NavigationSplitView`, Sheets and fullScreenCover, Toolbar, `ArrangementView` (iOS 27.1), size classes, DocumentGroup, adaptive layout for iPhone Duo

---

ControlGroup and Control

Q: What is ControlGroup in SwiftUI? Is there a `Control` type?

Short answer: `ControlGroup` visually and semantically groups related controls, typically buttons, so the system can render them as one unit: a segmented-looking cluster in a toolbar, a row in a menu, or a single submenu when space is tight. There is **no `Control` protocol or view in SwiftUI**; "controls" are just views like `Button`, `Toggle` and `Picker`, and you customize a group through `ControlGroupStyle` (built-ins: `.automatic`, `.navigation`, `.menu`, `.compactMenu`, `.palette`).

Key points:

- `ControlGroup { Button... }` or `ControlGroup { ... } label: { Label("Format", systemImage: "textformat") }`
- `.controlGroupStyle(.navigation)` (back/forward pair), `.menu` (collapses into a menu), `.compactMenu` / `.palette` (icon rows inside menus)
- Custom look: conform to `ControlGroupStyle` and implement `makeBody(configuration:)` using `configuration.content` and `configuration.label`
- In menus and context menus, a `ControlGroup` shows its items as a compact horizontal row
- Correction: older material mentions a `Control` protocol; it doesn't exist

Related to check: Toolbar (`ToolbarItemGroup`, `ToolbarSpacer`, overflow), Menu / context menu, Inspector, Layout priorities, Liquid Glass

---

ContentTransition

Q: How do you animate content changes in SwiftUI?

Short answer: A content transition animates a change *inside* a view that stays in the hierarchy, such as new text or a different SF Symbol, rather than inserting or removing a view. You apply `.contentTransition(.numericText(value:))` to a counter or `.contentTransition(.symbolEffect(.replace))` to an icon, then change the state inside an animation.

Key points:

- `.numericText(value:)` rolls digits and picks the direction from whether the value went up or down; `.numericText(countsDown:)` sets it explicitly
- `.symbolEffect(.replace)` swaps SF Symbols; `.replace.magic(fallback:)` morphs badges and slashes between related symbols
- `.interpolate` morphs text glyphs (weight, size, color); `.opacity` cross-fades; `.identity` disables it
- It's triggered by an animation: `withAnimation { score += 1 }` or `.animation(.snappy, value: score)`

Related to check: SymbolEffect, Animations and transitions, PhaseAnimator, KeyframeAnimator, matchedGeometryEffect

---

Table and Column

Q: How do you create tables in SwiftUI?

Short answer: `Table` shows rows of `Identifiable` data in columns declared with `TableColumn`, with built-in single or multiple selection, sortable column headers through a `sortOrder` binding of `KeyPathComparator`s, and context menus for the selection. It's a multi-column layout for iPad and Mac.

Key points:

- `Table(items, selection: $selection, sortOrder: $sortOrder) { TableColumn("Name", value: \.name) ... }`
- `TableColumn(_:value:)` makes a column sortable; add a content closure to format the cell
- Selection: `Item.ID?` for single, `Set<Item.ID>` for multiple
- Sorting is *your* job: observe `sortOrder` and sort the data (`items.sort(using: sortOrder)`)
- Compact width collapses to the first column; use the size class to make that column richer

Related to check: List, Grid, `NavigationSplitView`, size classes and adaptive layout, Fetch request / `@Query` sorting, Diffable data source (UIKit)

Grid

Q: What is the Grid layout? How does it differ from LazyVGrid?

Short answer: `Grid` is an eager, two-dimensional layout: it builds and measures every cell, then sizes each column to its widest cell and each row to its tallest, so things line up like a table. `LazyVGrid`/`LazyHGrid` are scrolling containers: you declare the columns up front with `GridItems` and cells are created only as they scroll into view.

Key points:

- `Grid` + `GridRow`; cells align across rows automatically; a view outside a `GridRow` (e.g. `Divider`) spans the whole grid
- `gridCellColumns(_:)` spans columns, `gridColumnAlignment(_:)` sets a column's alignment, `gridCellUnsizedAxes(_:)` stops a cell from expanding the grid
- `LazyVGrid(columns:)` with `GridItem(.fixed)`, `.flexible(minimum:maximum:)` or **`.adaptive(minimum:)`** (fits as many columns as the width allows)
- Lazy grids must sit in a `ScrollView`; column widths come from the `GridItems`, not from cell content

Related to check: LazyVStack / lazy loading, ScrollView vs List, Layout priorities, `containerRelativeFrame`, reorderable containers (iOS 27), Table, adaptive layout for iPhone Duo

---

PhaseAnimator

Q: What is PhaseAnimator? When do you use it?

Short answer: `PhaseAnimator` (and the `.phaseAnimator` modifier) animates a view through an ordered sequence of phases. For each phase you describe how the view looks, and an `animation:` closure picks the animation used to move *into* that phase.

Key points:

- `phases`: any `Equatable` sequence, usually a `CaseIterable` enum; the first phase is the resting state
- Content closure: `(phase) -> some View`, or `(content, phase)` for the modifier form
- `animation: { phase in Animation? }` chooses the animation per step (`nil` = jump)
- `trigger:` runs the cycle once per change; no trigger = continuous loop (loading indicators, attention pulses)

Related to check: Animations, KeyframeAnimator, ContentTransition, SymbolEffect, matchedGeometryEffect

---

SymbolEffect

Q: How do you add effects to SF Symbols?

Short answer: SF Symbols have built-in, layer-aware animations that you apply with `.symbolEffect(_:)`. There are three kinds: **discrete** effects that play once when a value changes (`.bounce`, `.wiggle`, `.rotate`), **indefinite** effects that run while active (`.pulse`, `.variableColor`, `.breathe`, `.rotate`), and **transitions/replacements** for appearing, disappearing and swapping symbols (`.appear`, `.disappear`, `.replace`, and the iOS 26 `.drawOn` / `.drawOff`).

Key points:

- Discrete: `.symbolEffect(.bounce, value: count)` plays on every change of `value`
- Indefinite: `.symbolEffect(.pulse, isActive: isSyncing)`, or always-on with no `isActive`
- `.wiggle`, `.breathe` and `.rotate` join bounce, pulse, scale and variable color

- Swapping symbols: `.contentTransition(.symbolEffect(.replace))`; insert/remove: `.transition(.symbolEffect(.appear))`
- Draw On / Draw Off (iOS 26+): the symbol is drawn stroke by stroke, like handwriting, for symbols with SF Symbols 7 draw annotations

Related to check: ContentTransition, Sensory feedback, PhaseAnimator, Animations, SF Symbols app (layer and draw annotations)

Liquid Glass

Q: What is Liquid Glass, and how do you adopt it in a SwiftUI app?

Short answer: Liquid Glass is the iOS 26 design material: a translucent, light-bending layer that sits *above* content for navigation and controls, while content stays underneath. Most adoption is automatic: rebuild with the iOS 26+ SDK and standard components (navigation bars, toolbars, tab bars, sheets, menus, buttons and controls) render as glass, so the first step is removing custom bar backgrounds and opaque appearances that fight the material.

Key points:

- **Automatic adoption** with the iOS 26+ SDK for system bars, sheets, menus and controls; scroll views get a *scroll edge effect* under bars instead of an opaque bar background
- **`.glassEffect(_:in:)`** (iOS 26+): defaults to `.regular` in a capsule; any `Shape` works (`.circle`, `.rect(cornerRadius:)`)
- **`Glass` variants:** `.regular` (default, adapts for legibility), `.clear` (more transparent, for controls over rich media), `.identity` (no glass, handy for toggling); modifiers `.tint(_:)` for emphasis and `.interactive()` for touch response (scale, shimmer)
- **`GlassEffectContainer(spacing:)`**: glass can't sample other glass, so nearby elements must share a container; elements closer than `spacing` blend together
- **Morphing:** `.glassEffectID(_:in:)` with a `@Namespace` animates glass shapes into each other as views appear and disappear; `.glassEffectUnion(id:namespace:)` merges separate views into one shape; `.glassEffectTransition(_:)` picks `.matchedGeometry` or `.materialize`
- **Buttons:** `.buttonStyle(.glass)` and `.glassProminent` (iOS 26+); `.glass(_:)` taking a `Glass` value needs iOS 26.1
- **Around content:** `.backgroundExtensionEffect()` mirrors and blurs edge content under sidebars and insets; `.scrollEdgeEffectStyle(.soft / .hard, for:)` tunes the edge effect

Related to check: Toolbar (`ToolbarSpacer`), TabView and Tab (`tabBarMinimizeBehavior`), Design systems, UIKit Liquid Glass (`UIGlassEffect`), UINavigationBar customization, Matched geometry effect, Accessibility

---

Sensory Feedback

Q: What is sensory feedback in SwiftUI?

Short answer: Sensory feedback is SwiftUI's declarative API for haptics (and, on some platforms, sounds): `.sensoryFeedback(.success, trigger: value)` plays the feedback whenever the `Equatable` trigger changes. The feedback types are semantic (`.success`, `.warning`, `.error`, `.selection`, `.increase`, `.decrease`, `.impact`), so the system picks the right pattern per device and respects the user's System Haptics setting.

Key points:

- `.sensoryFeedback(_:trigger:)` fires on every change of `trigger`

- `.sensoryFeedback(_:trigger:condition:)` fires only when `condition(old, new)` returns true
- `.sensoryFeedback(trigger:) { old, new in SensoryFeedback? }` picks the feedback (or none) from the change
- Types: `.success`/`.warning`/`.error`, `.selection`, `.increase`/`.decrease`, `.start`/`.stop`, `.alignment`, `.levelChange`, `.impact(weight:intensity:)`, `.impact(flexibility:intensity:)`

Related to check: SymbolEffect, ContentTransition, Observation (`@Observable`), UIFeedbackGenerator, Accessibility

---

DocumentGroup

Q: How do you build document-based apps in SwiftUI?

Short answer: A `DocumentGroup` scene gives you the whole document experience for free: the system document browser, open, save, autosave, rename, share and iCloud Drive. You supply a document type that describes how to read and write the file, plus an editor view.

Key points:

- `DocumentGroup(newDocument:editor:)` for editable docs, `DocumentGroup(viewing:viewer:)` for read-only
- `FileDocument` (struct, `Sendable`): `readableContentTypes`, `init(configuration:)`, `fileWrapper(configuration:)`; the editor gets a `Binding` to it
- `ReferenceFileDocument` (class, `ObservableObject`): snapshot-based saving, needed for undo with large documents
- **iOS 27+:** `ReadableDocument` / `WritableDocument` (combined as `Document`) on an `@Observable` class; `DocumentGroup(editor:makeDocument:)`; `DocumentCreationSource` + `NewDocumentButton(_:source:)` for "New from template"
- Content types are `UTType`s; custom types must be declared as exported types in Info.plist
- `DocumentGroupLaunchScene` customizes the launch/browser screen with your own actions

Related to check: @Observable macro, @Bindable, Scenes and multiple windows, UTType, FileWrapper, Swift Concurrency (`@concurrent`, `sending`)

WidgetKit Integration

Q: How do you add widgets to a SwiftUI app?

Short answer: Add a Widget Extension target. Each widget is a `Widget` type whose configuration pairs a *timeline provider* with a SwiftUI view: the system asks the provider for a timeline of dated entries ahead of time and renders them, so widget views are snapshots, not live apps.

Key points:

- `StaticConfiguration` + `TimelineProvider` (completion-handler based, no options) or `AppIntentConfiguration` + `AppIntentTimelineProvider` (`async`, configurable via a `WidgetConfigurationIntent`)
- `Timeline(entries:policy:)` with `.atEnd`, `.after(date)` or `.never`; the app forces refreshes with `WidgetCenter.shared.reloadTimelines(ofKind:)`
- Share data with the app through an App Group (SwiftData store, `UserDefaults(suiteName:)`, files)
- Interactive widgets: `Button(intent:)` / `Toggle(isOn:intent:)`; after `perform()` the timeline reloads
- Controls (iOS 18): `ControlWidget` with `ControlWidgetButton` / `ControlWidgetToggle` for Control Center, the Lock Screen and the Action button

- `.containerBackground(_:for: .widget)` is required; the system removes it in some contexts (StandBy, accented modes)
- Live Activities: `ActivityConfiguration` in the same extension, started from the app with ActivityKit

Related to check: App Intents, App Groups and SwiftData, @AppStorage (shared suite), Live Activities (ActivityKit), Liquid Glass

---

SwiftData and SwiftUI

Q: How do you use SwiftData with SwiftUI?

Short answer: Mark model classes with `@Model`, attach a container to the scene with `.modelContainer(for:)`, read data in views with `@Query`, and write through `@Environment(\.modelContext)`. `@Query` is live: when the context inserts, deletes or changes matching models, the view updates.

Key points:

- `@Model final class` makes a class persistent and `@Observable`; relationships via properties, `@Relationship(deleteRule:)`, `@Attribute(.unique)`
- `.modelContainer(for: [Item.self])` on the `WindowGroup`; `ModelContainer(for:configurations:)` for custom setup (in-memory, App Group, CloudKit)
- `@Query(filter: #Predicate<Item> { ... }, sort: \.timestamp, order: .reverse)`
- `@Environment(\.modelContext)` to `insert`, `delete`, `save` (autosave is on for the main context)
- **`@ModelActor`** for imports and heavy work off the main actor
- iOS 18: `#Unique` (compound uniqueness), `#Index` (query performance), history tracking (`fetchHistory(_:)`), custom data stores

Related to check: @FetchRequest and Core Data, @Observable macro, Swift Concurrency (actors, Sendable), #Preview with PreviewModifier, @AppStorage, WidgetKit (App Groups)

Architecture

MVC

Q: Explain MVC. What are its strengths and weaknesses on iOS?

Short answer: MVC splits an app into the Model (data and business rules), the View (what's on screen) and the Controller (which reacts to input, updates the model, and updates the view). Apple's Cocoa MVC makes `UIViewController` the controller, but it also owns its view hierarchy and its life cycle, so in practice networking, formatting, validation, delegate and data source methods and navigation all pile into it: the "Massive View Controller".

Key points:

- Model: no UIKit; View: reusable, knows nothing about the model type; Controller: the glue
- Communication: target-action, delegates and data sources upward; the controller pushes data down
- Massive View Controller = presentation logic, networking and navigation all in one class
- Testing a view controller needs its life cycle, which makes unit tests slow and brittle

Related to check: MVVM, VIPER, coordinator pattern, view model design, view life cycle, SOLID

MVVM

Q: Explain MVVM. How does it improve on MVC?

Short answer: MVVM puts a view model between the view and the model: it owns the screen's state, runs presentation logic and calls services, while the view only renders that state and forwards user actions. Compared with MVC, logic moves out of the massive view controller into a plain class you can unit test without UI.

Key points:

- View model: screen state (loading, data, error) and intents (`load()`, `retry()`)
- View: renders state, sends actions; no business logic
- Binding: Observation tracks the properties `body` reads, no `@Published` or Combine needed
- Dependencies come in through `init` as protocols, so tests can pass fakes
- `@MainActor` because the state drives UI

Related to check: MVC, view model design, state management, @Observable macro, view model testing, dependency injection, coordinator pattern

---

VIPER

Q: What is VIPER? When would you use it?

Short answer: VIPER splits each screen or module into five roles: the **View** displays what it's told and forwards user events; the **Interactor** holds the business logic for the use case and talks to data services; the **Presenter** turns interactor results into view-ready values and decides what to do on user events; **Entities** are plain models; and the **Router** (wireframe) builds the module and handles navigation. Every boundary is a protocol, which makes each piece unit-testable and lets large teams work on separate parts, at the cost of a lot of boilerplate per screen.

Key points:

- View Presenter Interactor; Presenter -> Router; Router assembles the module
- Presenter holds a `weak` reference to the view; the view holds the presenter strongly
- Interactor = use case; Entities = domain models
- Everything UI-facing is `@MainActor`; interactors call `Sendable` services with async/await

Related to check: MVVM, MVC, clean architecture, use case, coordinator pattern, navigation architecture

---

Coordinator Pattern

Q: What is the Coordinator pattern? Why use it?

Short answer: A coordinator is an object that owns navigation for a flow: it creates screens, injects their dependencies and decides what comes next, so view controllers never push or present each other. Screens report intent ("item selected", "checkout finished") through closures or a delegate, and the coordinator turns that into navigation.

Key points:

- Coordinator creates view controllers and wires their callbacks; screens don't know their destinations
- **Child coordinators** for sub-flows (onboarding, checkout), removed from the parent when finished
- Coordinators are UI objects: `@MainActor final class`
- Callbacks capture the coordinator weakly; the navigation controller owns the screens
- **SwiftUI:** `@MainActor @Observable final class Router` with `path`, injected with `.environment(router)`, destinations mapped in `.navigationDestination(for:)`

Related to check: Navigation architecture, NavigationStack, deep linking, MVVM, delegate pattern, dependency injection

---

Dependency Injection

Q: What is dependency injection? What are the different types?

Short answer: Dependency injection means an object receives its collaborators from outside instead of creating them or reaching for singletons, so dependencies are explicit, swappable and testable. The main forms are **initializer injection** (the default: required dependencies, immutable, impossible to forget), **property injection** (for objects the framework creates, like storyboard view controllers) and **method injection** (a dependency needed for one call).

Key points:

- **Init injection** by default: `init(api: any APIClient)`; store as `let`
- **Property injection** when you don't control `init` (storyboards, `NSObject` subclasses), ideally set once before use
- **Method injection** for per-call collaborators: `func export(using formatter: any Formatter)`
- **`any P` vs generics:** `any` is simpler and flexible at runtime; a generic `<API: APIClient>` gives static dispatch but spreads the type parameter to users
- **Composition root:** one place (the `App` or scene delegate) creates the real services and passes them down

Related to check: Dependency graph, service locator, protocol-oriented architecture, @Entry environment values, mocking and test doubles, SOLID

Repository Pattern

Q: What is the Repository pattern? When would you use it?

Short answer: A repository is the single place the rest of the app asks for a kind of domain data (`UserRepository`), hiding where it comes from: an API, SwiftData, Core Data, a cache, or a mix. Callers depend on a `Sendable` protocol written in domain terms and receive the implementation through `init`, so view models and use cases can be tested with an in-memory fake.

Key points:

- Protocol in domain language: `user(id:)`, `save(_:)`, not `GET /users`
- Returns domain models (`Sendable` structs), never DTOs, `@Model` objects or `NSManagedObjects`
- Implementation is an `actor` when it holds a cache or in-flight requests; a `struct` when stateless
- Stored as `any UserRepository` (or generic) and injected
- Streams of changes can be exposed as `AsyncStream` when callers need live updates

Related to check: Use case, hexagonal architecture, clean architecture, caching strategy, offline-first, request de-duplication, actors

---

Delegate Pattern

Q: Explain the delegate pattern. When would you use it?

Short answer: With the delegate pattern, an object hands some decisions or event notifications to another object through a protocol, so a reusable component (a table view, a picker, a download manager) can ask "how many rows?" or report "the user picked this" without knowing who is listening. It's one-to-one, the

contract is explicit, and the delegating object holds the delegate `weak`, because the delegate usually owns it and a strong reference would create a retain cycle.

Key points:

- `protocol PickerDelegate: AnyObject`, so it can be held `weak`
- `weak var delegate: (any PickerDelegate)?` in the delegating object
- Methods pass the sender (`picker(_:didSelect:)`) so one delegate can serve several instances
- **Data source vs delegate:** the data source supplies content, the delegate handles behavior and events (UIKit splits them)
- **Isolation:** UI delegates are `@MainActor`; a main-actor type conforming to a *nonisolated* protocol uses an isolated conformance, `extension VC: @MainActor SomeDelegate` (Swift 6.2)

Related to check: Closures and retain cycles, NotificationCenter, event-driven architecture, continuations, coordinator pattern

---

NotificationCenter

Q: When would you use NotificationCenter vs Delegate vs Closure?

Short answer: They differ in how many listeners there are and how tightly the two sides know each other. A **delegate** is a one-to-one, typed, weak contract with several related callbacks (`UITableViewDelegate`); a **closure** is a one-to-one callback for a single event, often a completion or an "on finish" output from a screen or coordinator; **NotificationCenter** is one-to-many broadcast where the sender doesn't know who listens, which suits app-wide events like logout, locale or system changes.

Key points:

- Delegate: one listener, multiple methods, `weak` to avoid cycles, `AnyObject` protocol
- Closure: one listener, one event; capture `[weak self]` when the closure is stored
- NotificationCenter: many listeners, loose coupling, harder to trace and test
- Typed messages (iOS 26+): `NotificationCenter.MainActorMessage` (observer runs on the main actor, posted from the main actor) or `AsyncMessage` (`Sendable`, delivered asynchronously)
- `NotificationCenter.default.notifications(named:)` is an `AsyncSequence` for existing string-named notifications

Related to check: Delegate pattern, event-driven architecture, async sequences, retain cycles, @Observable

---

Clean Architecture

Q: What is Clean Architecture? What are the layers?

Short answer: Clean Architecture (Robert C. Martin) arranges code in concentric layers with one rule: **source dependencies point inward**.

Key points:

- **Entities:** core business types and rules (`Order`, validation), `Sendable` value types
- **Use cases (interactors):** one application action each, such as `PlaceOrder`, orchestrating entities and repository protocols
- **Interface adapters:** view models / presenters, repository implementations, DTO entity mapping
- **Frameworks and drivers:** SwiftUI/UIKit, `URLSession`, SwiftData/Core Data, third-party SDKs
- **Dependency rule:** inner layers never import outer ones; dependency inversion via protocols

Related to check: SOLID (dependency inversion), repository pattern, use case, hexagonal architecture, MVVM, VIPER, dependency graph, error-handling architecture

---

SOLID Principles

Q: Explain the SOLID principles. How do they apply to iOS?

Short answer: SOLID is five design guidelines for code that's easy to change and test. **Single responsibility**: a type has one reason to change, so a view model doesn't also parse JSON and cache images.

Key points:

- SRP: split by reason to change (UI state vs data access vs formatting), not by line count
- OCP: protocols and generics for extension points; `switch` over a closed enum is fine when the set really is closed
- LSP: a conformer must not strengthen preconditions or throw unexpected errors; `UIKit` subclass overrides that skip `super` break it
- ISP: `UserFetching` and `UserSaving` instead of a twelve-method `UserServiceProtocol`
- DIP: the core defines `protocol OrderStore: Sendable`; the network module implements it

Related to check: Protocol-oriented architecture, dependency injection, hexagonal architecture, clean architecture, use case, MVC

Protocol-Oriented Architecture

Q: How do you design with protocols in iOS architecture?

Short answer: I use protocols as the contracts at the seams of the app: between a view model and the services it calls, between the domain and persistence or networking, and between feature modules. Types depend on a small protocol and receive the concrete implementation through `init`, which makes them testable with fakes and lets you swap implementations.

Key points:

- Small, role-based protocols (`UserFetching`) over one big `UserServiceProtocol`
- Inject through `init`; store as `any UserFetching` or a generic `<S: UserFetching>`
- `Sendable` on protocols whose values cross actors or tasks
- Protocol extensions give defaults; composition (`A & B`) builds requirements without inheritance
- Associated types and primary associated types (`any Repository<User>`) for generic contracts

Related to check: Protocols and POP, opaque types, dependency injection, SOLID, repository pattern, Sendable

---

Use Case / Interactor

Q: What is a Use Case? When would you create one?

Short answer: A use case (called an interactor in Clean Architecture and VIPER) encapsulates one application action, such as "place order" or "load profile", by orchestrating repositories and services and applying the business rules for that action. It sits between the view model and the data layer, so the view model handles presentation state and the use case owns the "what happens" logic, which can then be shared between screens, App Intents, widgets and background tasks.

Key points:

- One action per use case, named as a verb phrase (`PlaceOrder`, `FetchUserProfile`)
- Depends on repository protocols, returns domain models; no UIKit/SwiftUI, no formatting
- `Sendable` struct (stateless) so it can run from any actor; state lives in repositories (actors)
- Run independent fetches concurrently with `async let`
- Testable with in-memory fakes and Swift Testing

Related to check: Repository pattern, clean architecture, hexagonal architecture, VIPER interactor, MVVM, typed throws

---

Modular Architecture

Q: What is modular architecture? How do you structure an app with modules?

Short answer: A modular app is split into Swift packages with explicit dependencies, usually in layers: **Core** modules (models, networking, persistence, design system), **Feature** modules that depend only on Core and on each other's *interfaces*, and a thin **App** target that composes everything and owns navigation. I use local SPM packages in the workspace, which gives each module its own build settings: its own Swift language mode (so modules can move to Swift 6 independently), the `package` access level for APIs shared inside a package but hidden from the app, and package traits for optional features.

Key points:

- Layers: App -> Features -> Core; dependencies point downward only, no cycles
- `public` for the module's API, `package` for helpers shared across targets in the same package, `internal` for everything else
- Feature-to-feature: depend on a small `FeatureXInterface` module (protocols + route types), not the implementation
- Each package sets its own Swift language mode and, for UI packages, can opt into default `@MainActor` isolation (Swift 6.2 tools)
- Package traits (Swift 6.1+) enable optional code, e.g. an analytics backend, as compilation conditions

Related to check: Access control incl. `package`, migration strategy, dependency graph, code organization, modular app architecture, large codebase structure

---

Red-Green-Refactor

Q: What is the Red-Green-Refactor cycle in TDD?

Short answer: Red-Green-Refactor is the inner loop of TDD: write one small failing test (**red**), make it pass with the simplest code possible (**green**), then improve the design with the tests as a safety net (**refactor**), and repeat. This answer focuses on what the cycle does to *architecture*: because each test has to construct the unit under test cheaply, the code that comes out of the loop takes its dependencies through `init` behind small protocols, keeps logic in plain types away from UIKit/SwiftUI, and has seams where you need them, not everywhere.

Key points:

- Red proves the test can fail; in Swift, "does not compile yet" counts as red
- Green means minimal, even hard-coded, until the next test forces generality (triangulation)
- Refactor both production and test code; all tests stay green throughout
- Short cycles (minutes), run with U or a focused test; Swift Testing runs tests in parallel by default

Related to check: TDD workflow, unit testing, refactoring for testability, parameterized tests, SOLID, use case

---

Feature Flags

Q: How would you implement feature flags? What are the considerations?

Short answer: A feature flag lets you change app behavior at runtime without shipping a new binary, which on iOS matters because App Review and update adoption make rollbacks slow. I'd define flags as a typed enum with safe defaults compiled in, resolve them from layers (local developer override, then cached remote values, then the default), and fetch remote values at launch without blocking the UI.

Key points:

- Typed keys (`enum Flag: String`) with compiled-in defaults, so a missing or failed fetch is safe
- Resolution order: debug override -> remote (cached on disk) -> default
- Fetch at launch and on foreground, apply on the next natural boundary (screen load), not mid-flow
- Evaluate targeting (percentage rollout, segment) on the server, or hash a stable user ID so a user doesn't flip between variants
- Inject the service; never read a global singleton from deep inside views

Related to check: Feature toggles, remote configuration, feature flags design, A/B testing architecture, migration strategy, dependency injection

---

Error Handling Architecture

Q: How do you structure error handling across layers?

Short answer: Each layer speaks its own error vocabulary and translates at the boundary. The data layer catches infrastructure errors (`URLError`, `DecodingError`, SwiftData errors) and maps them to a small domain error enum the rest of the app understands, such as `.offline`, `.notFound` or `.unauthorized`.

Key points:

- **Domain errors** are enums owned by the domain, `Sendable` and `Equatable` for testing
- **Map at boundaries:** repository -> domain error; view model -> user-facing state
- **Typed throws** for closed sets (`throws(ValidationError)`, a repository's own errors); untyped `throws` for open-ended or pass-through code
- **Cancellation is not an error to show:** catch `CancellationError` (and `URLError.cancelled`) and ignore it
- **Log** the underlying error with privacy annotations; show the user something actionable

Related to check: Error handling and typed throws (Swift fundamentals), Result type, circuit breaker, crash reporting, logging architecture, clean architecture

---

Offline-First

Q: How do you design an offline-first app?

Short answer: In an offline-first app the local database is the source of truth: the UI only ever reads and writes local data, and a separate sync engine reconciles it with the server whenever it can. I'd use SwiftData with `@Query` driving the views, so they render instantly from disk and update when sync writes new data.

Key points:

- UI local store only; network sync engine only
- Outbox: a `needsUpload` flag or a pending-operations table, retried with backoff
- Pull with a server cursor or change token (`updatedSince`), not "download everything"
- Conflict policy chosen per entity: last-writer-wins by timestamp, field-level merge, or ask the user
- Show sync state honestly: "Saved on this device", "Syncing…", "Last updated 5 min ago"

Related to check: Offline mode design, data synchronization, repository pattern, event sourcing, network reachability, caching strategy, idempotency

---

State Management

Q: How do you manage app state? What approaches are there?

Short answer: State is the data that drives the UI, and the core rule is a single source of truth with clear ownership. I pick the tool by scope: `@State` for values local to one view, `@Binding` to let a child edit a parent's value, an `@Observable` model for screen or feature state, and `.environment(model)` with `@Environment(Model.self)` for state shared across a subtree or the whole app.

Key points:

- **Local:** `@State private var isExpanded = false`, owned by the view
- **Child edits parent:** `@Binding`, or `@Bindable` to get bindings into an `@Observable` model
- **Screen/feature:** an `@Observable` model owned with `@State`, passed down as a plain reference
- **Shared:** `.environment(session)` high up, `@Environment(Session.self)` where needed
- **Persisted:** SwiftData `@Query` keeps the view in sync with the store
- Data flows down, events flow up; every value has exactly one owner

Related to check: MVVM, view model design, @Observable macro, @State and @Binding, SwiftData and SwiftUI, Redux/TCA

Navigation Architecture

Q: How do you structure navigation in a large app?

Short answer: I take navigation decisions out of individual screens and give each flow an owner. In SwiftUI that is a `@MainActor @Observable` router holding a `NavigationPath` (or a typed `[Route]` array) plus any sheet state; screens call intents like `router.showDetail(id)`, and `navigationDestination(for:)` maps a `Hashable` route value to a view in one place.

Key points:

- Routes are `Hashable` (and `Codable` for restoration) values, not views
- One router per navigation stack or tab; child flows get child routers/coordinators
- Screens emit intents; they never construct their destination screens
- Deep link: parse URL -> routes -> `router.path = ...`, from `onOpenURL` or `scene(_:openURLContexts:)`
- The router is `@MainActor` because it drives UI

Related to check: Coordinator pattern, deep linking, NavigationStack, app state restoration, modular architecture, VIPER router

---

Caching Strategy

Q: How do you design a caching layer?

Short answer: I design caching as layers checked from fastest to slowest: an in-memory cache, a disk cache, then the network, each with an explicit freshness policy. The usual policy is cache-aside with a TTL, often stale-while-revalidate (show cached data instantly, refresh in the background), and the cache is an `actor` so concurrent callers are safe under Swift 6 and the same key isn't fetched twice at once.

Key points:

- **Layers:** memory (`NSCache` or a dictionary in an actor) -> disk (`URLCache`, files in the Caches directory, SwiftData) -> origin
- **Policies:** cache-aside, write-through, TTL, stale-while-revalidate; HTTP validation with `ETag` / `If-None-Match` (304)
- **Keys:** include everything that changes the result (URL, query, user, locale, image size)
- **De-duplication:** keep in-flight `Tasks` per key so 10 cells asking for the same image cause one request
- **Bounds:** size limits and eviction (`NSCache` evicts under memory pressure; the system can purge `Caches/`)

Related to check: NSCache, URLCache, offline-first, image downsampling, request deduplication, actors

---

ViewModel Design

Q: How do you design a ViewModel? What should it contain?

Short answer: A view model turns domain data into view state and turns user intents into calls on services. It contains three things: the state the view renders, methods for each user action, and injected dependencies.

Key points:

- **State:** an enum like `idle / loading / loaded / failed`, or a few properties for simpler screens; `private(set)` so only the view model mutates it
- **Actions:** methods named after intents (`load()`, `refresh()`, `delete(_:)`), `async` where they do work
- **Dependencies:** repositories or use cases passed into `init` as protocols
- **Isolation:** `@MainActor` because state drives UI; `@Observable` alone doesn't make the class `Sendable`, the actor isolation does
- **Testability:** no UI types, no singletons, so a test can build it with fakes

Related to check: MVVM, state management, @Observable macro, repository pattern, use case, dependency injection, view model testing

---

Deep Linking

Q: How do you implement deep linking?

Short answer: Deep linking means any entry point (a universal link, a custom URL scheme, a notification, Spotlight, Siri or a widget) can open a specific screen. Architecturally, I funnel every entry point into one place: parse the input into a typed `Route` enum, then hand it to the router or coordinator that owns navigation, which rebuilds the stack.

Key points:

- **Receive:** SwiftUI `.onOpenURL { }` (both universal links and schemes); UIKit `scene(_:continue:)` for universal links (`NSUserActivity.webpageURL`), `scene(_:openURLContexts:)` for custom schemes
- **Cold start:** in UIKit the URL arrives in `connectionOptions.userActivities` / `urlContexts` in `scene(_:willConnectTo:options:)`, not in the callbacks above
- **Parse -> Route -> navigate:** a pure, testable parser; the router sets the navigation path
- **Universal link setup:** Associated Domains entitlement (`applinks:example.com`) plus the AASA file served over HTTPS
- **App Intents** expose the same routes to Siri, Shortcuts, Spotlight and widgets

Related to check: Coordinator pattern, navigation architecture, app lifecycle (scene callbacks), App Intents, universal links

---

App Lifecycle

Q: How does the app lifecycle affect architecture?

Short answer: The lifecycle forces an app to treat every screen as something that can be backgrounded, suspended and killed without warning, so state that matters must be saved as you go and restorable at launch. Today the unit of lifecycle is the **scene**, not the app: SwiftUI exposes it as `scenePhase` (`.active`, `.inactive`, `.background`), UIKit as `UISceneDelegate` callbacks.

Key points:

- Process states: not running -> (inactive) -> active inactive -> background -> suspended -> terminated (by the system, with no callback)
- **Scene life cycle:** `sceneWillEnterForeground`, `sceneDidBecomeActive`, `sceneWillResignActive`, `sceneDidEnterBackground`; SwiftUI `@Environment(\.scenePhase)`
- **App delegate still exists** for process-level work: launch setup, push token registration, background URL session events, scene configuration
- **State restoration:** `@SceneStorage` in SwiftUI, `NSUserActivity` / `stateRestorationActivity(for:)` in UIKit
- **Background time is short:** save on `.background`; use `BGTaskScheduler` or `BGContinuedProcessingTask` (iOS 26+) for longer work

Related to check: UIApplication lifecycle, UIScene and multiwindow, state restoration, background tasks, deep linking (scene URL callbacks)

Analytics Architecture

Q: How do you integrate analytics without coupling?

Short answer: Features depend on a tiny `Sendable` protocol, `AnalyticsTracking`, with one synchronous `track(_:)` method, and never import a vendor SDK. Events are a typed `Sendable` enum, so the compiler checks names and payloads.

Key points:

- `protocol AnalyticsTracking: Sendable { func track(_ event: AnalyticsEvent) }`, injected through `init`
- Typed events (`enum AnalyticsEvent: Sendable`), mapped to vendor names and properties in one place
- The facade is non-blocking: `track` yields into an `AsyncStream`, and an actor does the work off the main actor in order
- Multiple sinks behind one interface; swapping a vendor touches one file
- Consent gate before anything leaves the device

Related to check: Analytics events (schema, batching), A/B testing architecture, dependency injection, privacy manifests, crash reporting

---

Logging Architecture

Q: How do you structure logging in an app?

Short answer: I build logging on Apple's unified logging system through `os.Logger`: one subsystem (the bundle ID) and a category per feature or layer (`network`, `sync`, `auth`), declared once as static constants so every module logs consistently. `Logger` is `Sendable`, cheap when a level is disabled, and formats lazily, and its string interpolation takes **privacy** annotations: dynamic values are redacted in logs by default, so you mark only what is safe as `.public` and hash identifiers with `.private(mask: .hash)`.

Key points:

- `Logger(subsystem:category:)` per category; filter in Console.app or `log stream --predicate 'subsystem == "..."'`
- Privacy: interpolated strings are `.private` by default in release logs; never log tokens, passwords or PII
- `OSLogStore(scope: .currentProcessIdentifier)` to export recent app logs on user request
- Remote log shipping, if you need it, goes through a separate `Sendable` sink with sampling and scrubbing
- Don't wrap `os.Logger` in your own `protocol Logger`: it shadows the system type and loses lazy formatting, privacy and source location

Related to check: Logging levels, crash reporting, request/response logging, analytics architecture, secrets management

---

Config Management

Q: How do you manage configuration (API URLs, keys) across environments?

Short answer: I keep per-environment values (Debug, Staging, Release) in `.xcconfig` files attached to build configurations, expose them through Info.plist keys, and read them once at launch into a typed `AppConfig` struct that is injected like any other dependency. Code never branches on `#if DEBUG` for URLs scattered across files, and nothing secret is compiled into the binary: anything shipped in an IPA can be extracted.

Key points:

- **Build configurations + schemes:** Debug / Staging / Release, each with its own `.xcconfig`
- **xcconfig -> Info.plist:** `API_BASE_URL = ...` in the xcconfig, `$(API_BASE_URL)` in Info.plist, read via `Bundle.main.object(forInfoDictionaryKey:)`
- **Typed config:** a `Sendable` struct validated at startup (fail fast in debug if a key is missing)
- **Secrets:** CI secrets for build-time tokens (signing, symbol upload); runtime secrets come from your backend after the user authenticates, stored in the Keychain
- **Remote config** for runtime values, cached, with safe defaults

Related to check: Secrets management, feature flags, remote config, dependency injection, CI/CD build configurations

---

Design Systems

Q: How do you implement a design system in an iOS app?

Short answer: A design system is a shared vocabulary of **tokens** (semantic colors, typography, spacing, radii, motion) and **components** built from them, shipped as its own Swift package that every feature imports. Tokens are semantic (`Color.ds.textPrimary`, not `Color.gray`), backed by asset-catalog colors with light, dark and high-contrast variants, and typography is built on Dynamic Type text styles so it scales.

Key points:

- **Tokens:** semantic color, type scale mapped to `Font.TextStyle`, spacing scale, corner radii; generated from the design tool where possible
- **Dynamic Type:** `.font(.body)` or `Font.custom(_:size:relativeTo:)`, `@ScaledMetric` for sizes that should scale, layouts that reflow at accessibility sizes
- **Components:** styles and modifiers over wrapper views, so they compose with standard controls
- **Liquid Glass (iOS 26+):** prefer system components; `.buttonStyle(.glass)` / `.glassProminent`, `GlassEffectContainer` for grouped floating controls; remove custom bar backgrounds that fight the material
- **Distribution:** an SPM package with resources (colors, fonts), previews and snapshot tests

Related to check: Liquid Glass, Dynamic Type, accessibility, theming via environment, code organization, snapshot testing

---

Backward Compatibility

Q: How do you maintain backward compatibility when changing APIs?

Short answer: There are three directions: supporting older OS versions with newer SDK APIs, evolving your own module APIs without breaking callers, and evolving data formats and server contracts that old app versions still use. For the OS, I build with the latest SDK, keep a deployment target (iOS 18 in this book), and gate newer APIs with `if #available` and `@available`.

Key points:

- **Version numbering jumped:** after iOS 18 came **iOS 26** (Apple aligned all OS versions with the year in 2025), so a check for the newest features is `#available(iOS 26, *)`, and iOS 19 - 25 never existed
- `if #available(iOS 26, *) { ... } else { ... }` at runtime; `@available(iOS 26, *)` on declarations; `if #unavailable(iOS 26)` for the inverse
- Swift 6.4 also accepts `@available(anyAppleOS 26, *)` for code that shares the unified version number across Apple platforms
- Deprecation attributes with `renamed:` / `message:` give callers a migration path and Fix-its
- Test on the minimum supported OS, not just the latest simulator

Related to check: API versioning, availability checking, migration strategy, app lifecycle (scene requirement), adaptive layout and iPhone Duo

---

Code Organization

Q: How do you organize code in a large iOS project?

Short answer: I organize by **feature first**, then by layer inside each feature, so everything about checkout (views, view models, models, tests) lives together and has a clear owner. As the project grows, folders become **local Swift packages**: each feature is a package target that depends on shared `Core` targets (networking, design system, domain), and the app target is a thin shell that wires them together.

Key points:

- **Feature-based** (`Features/Checkout`, `Features/Profile`) beats layer-based (`Views/`, `ViewModels/`) at scale: changes stay local, ownership is obvious
- **Shared code** in focused modules (`Networking`, `DesignSystem`, `Domain`), never a grab-bag `Utils` or `Extensions` target
- **Access control:** `internal` by default, `public` for the module API, **`package`** for things shared between targets of one package but hidden from the app
- **Per-module settings:** Swift 6 language mode, strict concurrency, upcoming features (`swiftSettings`)

- **Tests** live next to the module they test (a test target per package target)

Related to check: Modular architecture, dependency graph, clean architecture, access control (`package`), design systems

---

Service Locator vs DI

Q: What's the difference between Service Locator and Dependency Injection?

Short answer: Both decouple a type from the concrete implementations it uses; the difference is who does the looking up. With **dependency injection** a type declares what it needs in its `init` and something outside (a composition root, a coordinator, a parent view) passes it in, so dependencies are explicit, checked at compile time, and trivially replaced in tests.

Key points:

- DI: `init(api: any APIClient)`; dependencies visible in the signature
- Locator: `resolve(APIClient.self)`; dependencies hidden in the implementation
- Locator failures are runtime (missing registration), DI failures are compile time
- In Swift 6 a global registry is shared mutable state: it must be `Sendable`, so a `Mutex` or actor around the storage, and registered services must be `Sendable`
- Swift Testing runs tests in parallel, so a global locator swapped per test causes flaky tests

Related to check: Dependency injection, dependency graph, testing singletons, thread-safe singleton, protocol-oriented architecture, environment values

---

Event-Driven Architecture

Q: What is event-driven architecture? When would you use it?

Short answer: In an event-driven design, components publish facts ("user logged out", "cart changed") without knowing who reacts, and subscribers react independently. That decouples modules: analytics, caches and UI can all respond to `UserDidLogOut` without the auth module importing any of them.

Key points:

- **Publish/subscribe:** publishers don't know subscribers; many-to-many
- **Typed events:** `Sendable` structs or enums, not string names plus `userInfo: [AnyHashable: Any]`
- **iOS 26+ typed notifications:** conform a struct to `NotificationCenter.MainActorMessage` (delivered on the main actor) or `AsyncMessage` (delivered asynchronously, must be `Sendable`); observe with `addObserver(of:for:)`; `AsyncMessages` can also be iterated with `messages(of:for:)`
- **Own bus:** an `actor` holding `AsyncStream` continuations per subscriber; `onTermination` removes them
- **Legacy:** selector/block `NotificationCenter` observers and Combine `PassthroughSubject` are still common in existing code

Related to check: NotificationCenter, delegate pattern, AsyncStream, actors, event sourcing, CQRS, Combine (legacy)

CQRS

Q: What is CQRS? How does it apply to mobile?

Short answer: Command Query Responsibility Segregation separates the code that **changes** state (commands) from the code that **reads** it (queries), and lets each side use its own model. On the server that can mean separate databases; on iOS it's usually lighter: writes go through a command handler (validation, persistence, sync outbox) that runs off the main actor, and screens read through observed queries such as SwiftData `@Query` or a read-optimized projection.

Key points:

- **Command:** intent to change (`PlaceOrder`), returns nothing or an ID; validates, persists, enqueues for sync
- **Query:** returns data shaped for a screen, never mutates
- **Separate models:** a write model with invariants; read models that are denormalized for display (an `OrderSummary` row with the total precomputed)
- **Mobile fit:** `@ModelActor` for writes, `@Query` / `FetchDescriptor` for reads; offline outbox of pending commands
- **Eventual consistency** when read models are rebuilt asynchronously

Related to check: Event sourcing, saga pattern, offline-first, repository pattern, SwiftData `@ModelActor`, BFF

Hexagonal Architecture

Q: What is hexagonal (ports and adapters) architecture?

Short answer: Hexagonal architecture puts the application core (domain models and use cases) in the middle with no framework imports, and lets it talk to the outside world only through **ports**: protocols the core defines in its own terms. **Adapters** implement those ports for a specific technology: URLSession, SwiftData, the Keychain, or a fake in tests.

Key points:

- Port = protocol owned by the core (`OrderStore`), not by the infrastructure (`URLSessionClient`)
- Adapter = concrete implementation in an outer module
- Driving (primary) vs driven (secondary) sides
- In Swift 6, ports that cross isolation are `Sendable`; adapters with mutable state are actors
- SPM makes the rule enforceable: the `Core` package has no dependency on `Infrastructure`

Related to check: Clean architecture, use case, repository pattern, SOLID / dependency inversion, modular architecture, dependency injection

---

BFF (Backend for Frontend)

Q: What is a BFF? When would an iOS app use one?

Short answer: A Backend for Frontend is a thin server layer owned by (or built with) the client team that exposes an API shaped for one client type, such as the iOS app, instead of making the app call several general-purpose microservices. It aggregates, trims and reshapes data, so a home screen is one request returning exactly what the screen renders rather than three or four round trips over a mobile network.

Key points:

- **Aggregation:** one call per screen; fewer round trips matters on high-latency cellular
- **Shaping:** only the fields the app needs, pre-formatted where it helps (localized strings, image URLs sized for the device)
- **Decoupling:** backend services can change while the BFF keeps the mobile contract stable

- **Server-driven behavior:** the BFF can pick experiment variants, order sections, or ship lightweight server-driven UI
- **Cost:** another service to build, deploy, monitor and version

Related to check: API versioning, GraphQL client, circuit breaker, request deduplication, A/B testing (server-side variants)

Event Sourcing

Q: What is event sourcing? When would you consider it?

Short answer: Event sourcing stores the sequence of things that happened (`ItemAdded`, `OrderPaid`) as an append-only log, and derives current state by replaying those events through a pure `apply` function. You get a full audit trail, "time travel" (state at any point), undo, and an easy sync format, at the cost of more moving parts: versioned event schemas, snapshots, and projections for queries.

Key points:

- Events are immutable facts in the past tense; the log is append-only
- State = `events.reduce(initial, apply)`; `apply` is pure and deterministic
- Snapshots cap replay cost; projections build read models (often paired with CQRS)
- The store that guards the log is shared mutable state, so make it an `actor`; events are `Sendable` value types

Related to check: CQRS, saga pattern, event-driven architecture, offline-first, state management, actors

---

Saga Pattern

Q: What is the Saga pattern? When is it used?

Short answer: A saga is a way to run a multi-step business transaction across systems that can't share one database transaction: each step commits on its own, and if a later step fails, the saga runs **compensating actions** for the steps that already succeeded, in reverse order (refund the payment, release the reserved seat). In **orchestration**, one coordinator runs the steps and compensations; in **choreography**, each service reacts to the previous one's events with no central owner.

Key points:

- Local commits plus compensations; no distributed locks or two-phase commit
- Compensations run in reverse order and must be idempotent (they may be retried)
- Orchestration is easier to reason about and test; choreography decouples services but scatters the flow
- The result is eventual consistency: there are moments when step 1 is done and step 2 isn't

Related to check: Event sourcing, CQRS, event-driven architecture, idempotency, cancellation, circuit breaker

---

API Versioning Strategy

Q: How do you version your API for mobile clients?

Short answer: Mobile is the hard case for API versioning because you can't force everyone to update: old app versions keep calling your API for months or years. So I version explicitly, either in the path

(`/v2/orders`) or a header (`API-Version: 2026-10-01`), keep old versions running until usage drops, and make the client tolerant of additive changes so most server changes don't need a new version at all.

Key points:

- **Path versioning:** simple, cache- and log-friendly, coarse. **Header or date-based versioning** (Stripe-style): finer-grained, one URL space
- **Additive changes don't need a version:** new fields, new endpoints, new enum cases (if clients decode tolerantly)
- **Breaking changes do:** removed or renamed fields, changed types or semantics
- **Client identification:** `User-Agent` / `X-App-Version` header with the app version and build
- **Deprecation:** `Deprecation` and `Sunset` response headers (RFC 9745, RFC 8594), plus a minimum supported version

Related to check: Backward compatibility, BFF, error-handling architecture, config management, networking layer design

---

Circuit Breaker

Q: What is the circuit breaker pattern? When do you use it?

Short answer: A circuit breaker wraps calls to an unreliable dependency and stops calling it once it's clearly failing, so the app fails fast instead of piling up timeouts, draining battery and hammering a struggling server. It has three states: **closed** (calls go through and failures are counted), **open** (calls fail immediately for a cool-down period), and **half-open** (after the cool-down, one trial call decides whether to close again or reopen).

Key points:

- Closed -> open after N consecutive failures (or a failure rate over a window)
- Open -> half-open after a cool-down (`Duration`, measured with `ContinuousClock`)
- Half-open: allow one probe; success closes, failure reopens
- Only count failures that mean "the service is unhealthy" (timeouts, 5xx), not 4xx or cancellation
- One breaker per dependency or host, not one global breaker

Related to check: Bulkhead pattern, retry strategies, error-handling architecture, offline-first, actors

---

Bulkhead Pattern

Q: What is the bulkhead pattern?

Short answer: A bulkhead isolates resources into separate pools so one overloaded or failing area can't sink the rest, like watertight compartments in a ship. In an iOS app, that means a burst of image downloads can't use up every connection and leave the checkout request waiting, and a slow sync job can't starve user-facing work.

Key points:

- **Separate pools:** API vs media vs background sync each get their own session, limits and timeouts
- **Concurrency caps:** at most N tasks in flight per pool
- **Load shedding:** when a pool's queue is full, fail fast instead of queuing forever
- **Priority:** user-initiated work gets its own pool and higher `TaskPriority`
- Pairs with the circuit breaker: bulkheads limit the blast radius, breakers stop calling what's already failing

Related to check: Circuit breaker, task groups, API rate limiting, task priority, connection pooling

---

Health Checks

Q: How do you implement health checks in an iOS app?

Short answer: Health checks are a server-side idea (liveness: "is the process up?", readiness: "can it serve traffic?"), and on iOS they translate into two things. First, a launch-time self-check of the app's own critical dependencies: the database opened and migrated, the Keychain is readable, required config is present.

Key points:

- Each check is a small `Sendable` type returning `.healthy`, `.degraded(reason)` or `.failed(reason)`
- Run them in parallel with `withTaskGroup`; a slow check must not delay the others
- Critical failures (store won't open) -> recovery screen; non-critical (status endpoint slow) -> banner
- Don't block first frame on network checks; show cached content and update

Related to check: Circuit breaker, feature toggles, crash reporting, network reachability, logging architecture

---

Feature Toggles

Q: How do feature flags differ from feature toggles?

Short answer: They are the same thing: "feature toggle" is Martin Fowler's term and "feature flag" is the more common one, and nobody reliably uses one for build time and the other for runtime. The useful distinction is between *kinds* of toggles, which differ in how long they live and who flips them: release toggles hide unfinished work, experiment toggles drive A/B tests, ops toggles (kill switches) turn off a misbehaving feature, and permission toggles gate features per user or plan.

Key points:

- Release toggle: short-lived (days to weeks), enables trunk-based development; delete after launch
- Experiment toggle: lives for the length of the experiment; assignment must be sticky per user
- Ops toggle / kill switch: may be long-lived; must default to the safe state and work from cache
- Permission toggle: long-lived, really product configuration; usually enforced server-side too
- Build-time (`#if`) vs runtime (remote): only runtime toggles can be flipped after App Store release

Related to check: Feature flags implementation, A/B testing architecture, migration strategy, config management, package traits

---

A/B Testing Architecture

Q: How do you architect A/B testing in an app?

Short answer: I put experiments behind a small `ExperimentService` that the rest of the app depends on through a protocol, so features ask "which variant of `checkout_flow` am I in?" and never talk to Firebase, Optimizely or Statsig directly. Assignments are fetched in the background and applied at the next session (or at a defined point), so launch is never blocked and a user never sees a variant flip mid-session.

Key points:

- **Assignment:** deterministic bucketing (hash of user ID + experiment key) server-side or in the SDK, so the same user always gets the same variant across devices and reinstalls
- **Sticky per session:** activate fetched values at launch, not while the screen is visible

- **Fallback:** typed variants with a `control` default, so a failed fetch is a safe outcome
- **Exposure vs assignment:** log exposure when the variant is rendered, not when the config arrives
- **Isolation:** the service is an `actor` (or `Sendable` struct over immutable data) injected through `init`

Related to check: Feature flags and toggles, analytics architecture, analytics events, config management, BFF, dependency injection

---

Secrets Management

Q: How do you manage secrets in an iOS app?

Short answer: The first rule is that an app binary can't keep a secret: anything compiled in, whether a string literal, an xcconfig value copied into Info.plist, an "obfuscated" byte array, or a bundled file, can be pulled out of the IPA with `strings` or a debugger. So real secrets (a payment provider's secret key, a third-party API's private key, an OAuth client secret) stay on the server, and the app calls your backend, which adds them.

Key points:

- No secrets in source, the binary, or the repo; scan the repo (pre-commit or CI) for leaks
- Third-party keys that must ship (maps, analytics) are *public identifiers*: restrict them server-side by bundle ID, quota and scope
- Backend proxy (or backend-for-frontend) for calls that need a real secret
- Keychain with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` (or `WhenUnlockedThisDeviceOnly`) for tokens; `ThisDeviceOnly` keeps them out of backups and off other devices
- Rotate: short-lived access tokens, refresh tokens the server can revoke

Related to check: Keychain, securing sensitive data, OAuth 2.0 flow, authentication, request signing, config management, BFF

Localization Architecture

Q: How do you structure an app for localization?

Short answer: I treat every user-facing string as a localizable resource from day one, stored in String Catalogs (`.xcstrings`), which Xcode fills by extracting keys from SwiftUI `Text`, `String(localized:)` and `LocalizedStringResource` at build time and which track translation state per language. Plurals and device variants live in the catalog: you write one interpolated key like `"\(count) new messages"` and add *one* / *few* / *many* / *other* variants per language, instead of branching in code.

Key points:

- One String Catalog per table, per module; Swift packages pass `bundle: .module`
- Keys are the English source text (SwiftUI default) or stable semantic keys with `defaultValue:` and a `comment:` for translators
- Plurals: one key with an integer interpolation, varied by plural in the catalog; never `count == 1 ? ... : ...`
- `LocalizedStringResource` for deferred localization; `String(localized:)` when you need a `String` now
- `FormatStyle` (`.currency(code:)`, `.dateTime`, `.measurement`) and `Text(_:format:)` for values

Related to check: String Catalogs, `LocalizedStringResource`, `FormatStyle`, Dynamic Type, accessibility, modular architecture (bundle per package), design systems

---

Crash Reporting Integration

Q: How do you integrate crash reporting?

Short answer: I combine Apple's own data with a third-party reporter. Apple gives you Xcode Organizer crash reports (from users who share analytics) and **MetricKit**, whose `MXDiagnosticPayload` delivers crash, hang, CPU-exception and disk-write diagnostics to the app itself.

Key points:

- **Symbolication:** each build's dSYM (matched by UUID) turns addresses into function names; upload them from CI for every archive, including App Store builds
- **MetricKit:** subscribe with `MXMetricManager.shared.add(_:)` and receive `didReceive(_: [MXDiagnosticPayload])` (delivered at the next launch rather than once a day since iOS 15). **iOS 27+** adds a Swift-native `MetricManager` whose `diagnosticReports` is an `AsyncSequence` of `Sendable`, `Codable` `DiagnosticReport` values, and the iOS 27 SDK marks the `MX...` subscriber API as to be deprecated
- **Context:** breadcrumbs, the current screen, app state, feature-flag and experiment variants as custom keys
- **Non-fatals:** record caught errors that matter (failed payments, decode failures) separately from crashes
- **Integration:** SPM package, initialized early at launch, behind a `CrashReporting` protocol

Related to check: Symbolication, MetricKit, logging architecture, analytics events, privacy manifests, error-handling architecture

---

Analytics Events

Q: How do you structure analytics events?

Short answer: An event is a small, versioned, typed record: a name from an agreed taxonomy (`object_action` in snake_case, like `checkout_completed`), a timestamp, and a few typed properties. I model properties as a `Sendable` value enum rather than `[String: Any]`, so events are `Codable`, can be persisted, and can cross into a background actor under Swift 6.

Key points:

- **Taxonomy:** a shared tracking plan (name, properties, owner, version) agreed with product and data; past-tense `object_action` names
- **Typed schema:** `AnalyticsValue` enum (`string`, `int`, `double`, `bool`) instead of `Any`
- **Context added centrally:** app version, OS, session ID, experiment variants, attached once by the pipeline, not by every caller
- **Batching:** buffer in an actor; flush at N events, every N seconds, and on `scenePhase == .background`
- **Privacy:** no PII, IDs hashed or pseudonymous, consent respected; declared in the privacy manifest

Related to check: Analytics architecture, A/B testing architecture (exposure events), crash reporting, logging architecture, actors and reentrancy

---

Logging Levels

Q: What logging levels do you use? When?

Short answer: I use the five levels of `os.Logger`, and I pick them by who needs the message and whether it should survive on the device: **debug** for developer detail while working on a feature, **info** for helpful context that's only kept around a problem, **notice** (the default) for the important milestones of a flow, **error**

for failures the app handled, and **fault** for bugs: states that should be impossible. The unified logging system decides persistence by level, so I don't need `#if DEBUG` around log calls: debug messages cost almost nothing when nobody is streaming them, and info messages stay in memory unless something goes wrong.

Key points:

- `debug` (alias `trace`): not persisted by default; visible when streaming in Xcode or Console
- `info`: held in memory; written out only when an error or fault happens nearby, or with a logging profile
- `notice` (default `log(_:)`): persisted to disk up to the storage limit; use sparingly
- `error` (alias `warning`): persisted; a recoverable failure (request failed, decode failed)
- `fault` (alias `critical`): persisted with extra process context; a programming error or corrupted state

Related to check: Logging architecture, crash reporting, error handling architecture, request/response logging, analytics events

---

Dependency Graph

Q: How do you visualize and manage dependencies?

Short answer: I think about two graphs. The **module graph** is which targets and packages import which, and it must be acyclic and layered: features depend on domain and core modules, never on each other, and the domain depends on nothing.

Key points:

- **Acyclic and layered:** App -> Features -> Domain/Core; arrows point toward stable, abstract modules
- **Dependency inversion breaks cycles:** the lower module defines a protocol, the higher module implements it
- **SPM enforces it:** cycles are a resolution error; undeclared imports can be flagged (`--explicit-target-dependency-import-check error`)
- **Third-party dependencies:** few, pinned by version (`Package.resolved` committed), wrapped behind your own protocols, and listed with their privacy manifests
- **Composition root:** the app target creates services and injects them; feature modules never reach for globals

Related to check: Dependency injection, modular architecture, code organization, clean architecture, hexagonal architecture, SOLID (dependency inversion)

Migration Strategy

Q: How do you migrate to a new architecture?

Short answer: Incrementally, never as a big-bang rewrite: I use the strangler fig pattern, putting a seam (a protocol or a router entry point) in front of the old code, building the new implementation behind it, and moving traffic over screen by screen behind a feature flag, with the old path kept as a rollback until metrics say the new one is healthy. For risky logic I run both paths in shadow mode and compare results before switching.

Key points:

- Seams first: protocols at module boundaries let old and new implementations coexist
- Feature flags select the path and give an instant rollback without a release
- Migrate leaf modules first (models, utilities), then features, then the app target
- Measure: crash-free rate, performance, and business metrics per path before deleting the old one

Related to check: Feature toggles, modular architecture, app lifecycle, backward compatibility, preconcurrency, dependency graph, health checks

---

App Intents Architecture

Q: How do you architect App Intents so the same actions work in Siri, Spotlight, Shortcuts and widgets?

Short answer: I treat App Intents as the app's **action layer**: a typed, system-visible API over the same use cases the UI calls. Each user-meaningful action is an `AppIntent` with `@Parameters` and an `async perform()`.

Key points:

- **Intents:** verbs. `static let title`, `@Parameters`, `perform() async throws -> some IntentResult` (add `& ProvidesDialog`, `& ReturnsValue<T>` or `& OpensIntent` as needed)
- **Entities:** nouns. `id`, `displayRepresentation`, `typeDisplayRepresentation`, `defaultQuery`. Expose only what the system needs; this is a projection of the domain model, not the model itself
- **Queries:** `EntityQuery` (`entities(for:)`, `suggestedEntities()`), `EntityStringQuery` for free-text lookup, `EnumerableEntityQuery` for small fixed sets
- **Shortcuts:** `AppShortcutsProvider` with phrases that include `\(.applicationName)`; they appear right after install
- **Dependencies:** register services once at launch with `AppDependencyManager.shared.add(dependency:)` and read them with `@Dependency` in intents and queries

Related to check: Deep linking, use case / interactor, dependency injection, WidgetKit integration, Siri and app schemas, SiriKit to App Intents migration, testing App Intents, modular architecture

---

Siri, App Schemas and View Annotations

Q: In iOS 27, how does Siri understand your app's content and act on it? What do you have to build?

Short answer: Siri with Apple Intelligence doesn't guess what your app contains from its UI. It works from three things the app declares through App Intents.

Key points:

- **Schemas:** `@AppEntity(schema: .notes.note)`, `@AppIntent(schema: .notes.createNote)`, `@AppEnum(schema:)`. These macros exist from iOS 18 and replace the deprecated `@AssistantEntity` / `@AssistantIntent` spellings; each domain is tagged with the OS that introduced it. The schema defines the shape (properties, parameters, return value) that Apple's models were trained on, so Siri can use your action without phrases you wrote
- **Domains:** iOS 18 introduced books, browser, camera, files, journal, mail, photos, presentation, reader, spreadsheet, system, whiteboard and word processor; iOS 27 adds messages, notes, reminders, calendar, clock, maps, phone and audio
- **Semantic index:** conform entities to `IndexedEntity` and call `CSSearchableIndex.default().indexAppEntitie` delete with `deleteAppEntities(identifiedBy:ofType:)` when content goes away
- **View annotations** is Apple's feature name, not a type or modifier. The APIs are SwiftUI `.appEntityIdentifier(EntityIdentifier(for: entity))` and `.appEntityUIElements { context in ... }` for visible and selected elements (iOS 18.4+); UIKit `UIView.appEntityIdentifier`, `appEntityUIElementProvider` and `UITableView.appIntentsDataSource` (iOS 18.4+); and `NSUserActivity.appEntityIdentifier` (iOS 18.2+)

Related to check: App Intents architecture, Siri AI integration (system design), SiriKit to App Intents migration, testing App Intents, Foundation Models integration, deep linking, Spotlight / Core Spotlight

---

Migrating from SiriKit to App Intents

Q: We still have SiriKit intents and an Intents extension. Should we keep them? How would you migrate to App Intents?

Short answer: SiriKit (the Intents and IntentsUI frameworks, `.intentdefinition` files and Intents extensions) is **legacy, not deprecated**. Apple's SiriKit page says the frameworks "continue to provide legacy support" and points new work to App Intents, which is what Apple Intelligence, Siri schemas, Spotlight, interactive widgets and controls build on.

Key points:

- **Status:** SiriKit still runs and still ships in the iOS 27 SDK; App Intents is the strategic API
- **Custom intents:** Xcode can convert a custom intent from an `.intentdefinition` file to Swift App Intent code; the generated type adopts `CustomIntentMigratedAppIntent` with `static let intentClassName`, which ties it to the old intent so existing shortcuts and donations keep resolving
- **What App Intents adds:** Swift-only definitions (no definition file, no code generation), App Shortcuts with no user setup, entities and queries, Spotlight indexing, schemas for Siri with Apple Intelligence, view annotations, and the same intents in widgets, controls and the Action button
- **System domains:** for messaging, calling, payments, workouts and similar domains that used SiriKit's built-in intents, check whether an App Intents schema domain exists (iOS 27 adds messages, phone, notes, reminders, calendar and others) before rewriting

Related to check: App Intents architecture, Siri and app schemas, testing App Intents, use case / interactor, migration strategy, WidgetKit integration

---

Integrating Foundation Models

Q: How would you integrate Apple's Foundation Models framework into an app?

Short answer: Foundation Models (iOS 26+) gives the app direct access to the on-device language model behind Apple Intelligence: no API key, no per-request cost, and the user's data never leaves the device. I wrap it in a feature service behind my own protocol.

Key points:

- **Session:** `LanguageModelSession(tools:instructions:)`. Instructions come from the developer and take priority over prompts; prompts can contain user input. The session keeps a `transcript`, so it is multi-turn
- **Guided generation:** `@Generable` types with `@Guide(description:_:)` constraints (`.range`, `.count`, `.anyOf`, regex patterns). `respond(to:generating:)` returns a typed `Response<T>`, and the schema is enforced while the model decodes
- **Streaming:** `streamResponse(to:generating:)` yields snapshots of `T.PartiallyGenerated`, where every property is optional until it arrives
- **Tools:** a `Tool` has a `@Generable Arguments` type and an `async call(arguments:)`; the model decides when to call it, so it can read app data (a pantry, a calendar) without the app stuffing everything into the prompt
- **Availability:** `.available`, or `.unavailable(.deviceNotEligible / .appleIntelligenceNotEnabled / .modelNotReady)`. The feature degrades instead of failing

Related to check: Testing language-model features (evaluations), on-device AI feature (system design), App Intents architecture, Siri and app schemas, Observation, Swift concurrency, privacy manifests, dependency injection

---

Networking

URLSession

Q: How do you use URLSession for networking? What are the key APIs?

Short answer: `URLSession` is Apple's HTTP stack, and in modern Swift you use its async methods: `data(for:)`, `upload(for:from:)`, `download(for:)` and `bytes(for:)`. A session is built from a `URLSessionConfiguration` (timeouts, caching, cookies, connectivity behavior, headers), and you usually create one long-lived session per configuration and inject it, rather than using `URLSession.shared` everywhere.

Key points:

- Async API: `let (data, response) = try await session.data(for: request)`; cancelling the Swift `Task` cancels the request
- URLSession does **not** throw for 404 or 500: check `HTTPURLResponse.statusCode` yourself
- Configurations: `.default` (disk cache, cookies, credentials), `.ephemeral` (in-memory only, nothing persisted), `.background(withIdentifier:)` (transfers continue while the app is suspended; delegate-only, no async methods)
- Useful settings: `timeoutIntervalForRequest` / `ForResource`, `waitsForConnectivity`, `requestCachePolicy` / `urlCache`, `httpAdditionalHeaders`, `allowsExpensiveNetworkAccess` / `allowsConstrainedNetworkAccess`
- Delegates: session delegate (`URLSessionDelegate`, `URLSessionTaskDelegate`, `URLSessionDataDelegate`, `URLSessionDownloadDelegate`, `URLSessionWebSocketDelegate`) or a per-task `delegate:` argument
- `URLSession` is `Sendable`, so a client struct that holds one can be `Sendable` too

Related to check: Timeout configuration, response validation, response caching, upload progress, request cancellation, background downloads, SSL pinning, REST API client design

---

A Networking Layer for Swift 6 Strict Concurrency

Q: Design a networking layer that is safe under Swift 6 strict concurrency.

Short answer: I start from an isolation map: every type in the layer is either **immutable and `Sendable`**, **an actor**, or **`@MainActor`**, and nothing else crosses a boundary. The client is a `struct` holding only `Sendable` configuration (a `URLSession` behind a `Sendable` transport protocol, a base URL), so it can be called from anywhere.

Key points:

- Client: `struct APIClient: Sendable` with `let` properties only; inject `any HTTPTransport` (a `Sendable` protocol) for tests
- DTOs: `struct ... : Decodable, Sendable`; generic methods constrain `T: Decodable & Sendable` so results can cross back to the caller's actor
- Actors for state: `TokenProvider` holding `Task<Token, any Error>?`, a loader holding `[Key: Task<Value, any Error>]` to share in-flight requests
- `Mutex` (iOS 18, `Synchronization`) for state touched from **synchronous** callbacks, like `URLSessionDelegate` methods, where you can't `await` an actor: a delegate can be a `final class ...: NSObject, URLSessionTaskDelegate, Sendable` whose only stored property is a `let Mutex`
- `@concurrent` on CPU-heavy entry points (decoding large payloads) so a `@MainActor` caller doesn't decode on main
- Cancellation is free: SwiftUI `.task` cancels the `Task`, URLSession cancels the request, and `CancellationError` / `URLError(.cancelled)` should not show an error UI

Related to check: REST API client design, request interceptors, authentication (single-flight refresh), request deduplication, mock networking, actors and reentrancy, `@concurrent` / approachable concurrency, `Mutex`

Codable and JSON

Q: How do you decode JSON with Codable? What about custom key mapping?

Short answer: I declare a model that conforms to `Decodable` (or `Codable` if I also send it), and the compiler synthesizes `init(from:)` and `CodingKeys` from the stored properties; then `JSONDecoder().decode(User.self, from: data)` does the rest. For key mapping I have two tools: `keyDecodingStrategy = .convertFromSnakeCase` when the whole API is snake_case, or a custom `CodingKeys` enum when names differ arbitrarily.

Key points:

- `Codable = Encodable & Decodable`; synthesis needs every stored property to be `Codable`
- `CodingKeys: String, CodingKey` renames keys; leaving a property out of `CodingKeys` excludes it (it then needs a default value)
- `.convertFromSnakeCase` turns `avatar_url` into `avatarUrl`, **not** `avatarURL`; with acronyms, use explicit `CodingKeys`
- Optional properties decode missing keys and `null` as `nil`; a default value on a non-optional property still requires the key
- `decodeIfPresent(_:forKey:) ?? default` in a custom `init(from:)` for "missing means default"

Related to check: Codable dates, Codable enums, nested Codable, decoding error handling, Sendable, JSON parsing performance

---

REST API Design

Q: How do you structure a REST API client in Swift?

Short answer: I separate **what** to call from **how** to call it. Each endpoint is a small value, `Endpoint<Response>`, that knows its method, path, query and body and carries its response type, so call sites are type-safe: `try await client.send(.user(id: 42))` returns a `User`.

Key points:

- Endpoints as values (or static factories), generic over the `Decodable` response
- One `send` path: build request -> middleware -> transport -> validate -> decode
- Inject the transport behind a `Sendable` protocol for tests
- REST semantics: resources as nouns, HTTP verbs (GET/POST/PUT/PATCH/DELETE), status codes, idempotency
- Shared `JSONDecoder` settings: date strategy, `keyDecodingStrategy` (or explicit `CodingKeys`)
- Versioning in the path or a header; base URL per environment

Related to check: URLSession, URLComponents, response validation, request interceptors, REST vs GraphQL, mock networking, Codable

---

Error Handling

Q: How do you handle networking errors?

Short answer: There are three failure layers and I handle each one deliberately. **Transport** failures arrive as a thrown `URLError` (offline, timeout, DNS, TLS).

Key points:

- Common `URLError.Codes`: `.notConnectedToInternet`, `.networkConnectionLost`, `.timedOut`, `.cannotFindHost`, `.secureConnectionFailed`, `.cancelled`, `.dataNotAllowed`
- `urlError.networkUnavailableReason` tells you if the failure was due to Low Data Mode (`.constrained`), cellular being disallowed, or an expensive network
- Status codes: 401 -> refresh/sign out, 403 -> no permission, 404 -> not found, 409/422 -> validation (show server message), 429/503 -> back off, 5xx -> retry with backoff
- Retry only transient errors (timeouts, connection lost, 5xx, 429) and only idempotent requests
- User-facing messages come from the domain error, never from raw `localizedDescription`

Related to check: Retry logic, decoding error handling, typed throws, request cancellation, network reachability, API rate limiting, authentication (401 handling)

Authentication

Q: How do you handle authentication in API requests?

Short answer: Signing in should use **passkeys** or **Sign in with Apple** where possible (phishing-resistant, no passwords to leak), or OAuth authorization code + PKCE through `ASWebAuthenticationSession` for third-party identity providers. The result is a short-lived access token and a refresh token, stored in the **Keychain**.

Key points:

- Passkeys: `ASAuthorizationPlatformPublicKeyCredentialProvider(relyingPartyIdentifier:)`, needs the `webcredentials:` associated domain; the server verifies the WebAuthn assertion
- Sign in with Apple: `ASAuthorizationAppleIDProvider`; send the identity token to your server; check credential state at launch
- Keychain with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` (background refresh works, not restored to another device). Never `UserDefaults`
- Token provider is an `actor` holding `Task<Token, any Error>?`; refresh proactively near expiry and reactively on 401
- Retry a 401 **once**; if refresh fails, sign out and clear the Keychain

Related to check: OAuth 2.0 flow (PKCE), Keychain, headers, request interceptors, actors, App Attest, certificate pinning

---

Passkeys

Q: How do passkeys work and how do you implement them on iOS?

Short answer: A passkey is a **WebAuthn / FIDO2 credential**: a public - private key pair created per account and per site. The private key stays in the user's password manager (iCloud Keychain or a third-party provider) and never leaves it unencrypted; your server stores only the public key.

Key points:

- **Registration:** server returns a challenge and a user ID -> `createCredentialRegistrationRequest(challenge:name:userID:)` -> send `credentialID`, `rawClientDataJSON` and `rawAttestationObject` to the server, which stores the public key

- **Sign-in:** server returns a challenge -> `createCredentialAssertionRequest(challenge:)` -> send `credentialID`, `rawClientDataJSON`, `rawAuthenticatorData`, `signature` and `userID`; the server verifies the signature, origin, RP ID and challenge
- **Relying party ID** = your domain (`example.com`). The app needs the Associated Domains entitlement `webcredentials:example.com`, and the site serves `apple-app-site-association` listing your app ID
- **UI modes:** a modal sheet (`performRequests()`), AutoFill-assisted (`performAutoFillAssistedRequests()` with a `.username` text field), and `.preferImmediatelyAvailableCredentials` to fall back silently when the device has no passkey
- **Upgrades:** `requestStyle: .conditional` (iOS 18) creates a passkey automatically right after a password sign-in, without a sheet
- **iOS 26+:** `ASAuthorizationAccountCreationProvider` for one-sheet account creation with a passkey; credential-manager signals (`reportUnknownPublicKeyCredential`, `reportAllAcceptedPublicKeyCredentials`, on `ASCredentialDataManager` from iOS 26.2) keep the user's password manager in sync with your server

Related to check: Authentication (tokens, Keychain, single-flight refresh), OAuth 2.0 flow (PKCE), Sign in with Apple, App Attest, universal links / associated domains, secrets management

---

App Attest and DeviceCheck

Q: What are App Attest and DeviceCheck, and how do you use them?

Short answer: Both live in the DeviceCheck framework and help your **server** decide whether to trust a request, because the server can't trust the client by default. **App Attest** (`DCAppAttestService`) proves that a request comes from a genuine, unmodified copy of your app on a real Apple device: the app generates a key pair in the Secure Enclave, Apple attests that key once, and from then on the app signs sensitive requests with it.

Key points:

- **App Attest flow:** `generateKey()` -> key ID (store it) -> server issues a one-time challenge -> `attestKey(_:clientDataHash:)` with the challenge hash -> server validates the attestation (Apple certificate chain, your App ID, the hash) and stores the public key -> per request `generateAssertion(_:clientDataHash:)` over a hash of the request data plus a fresh challenge -> server verifies the signature and an increasing counter
- Entitlement `com.apple.developer.devicecheck.appattest-environment: development` or `production` (attestations from one don't validate against the other)
- `isSupported` is false in the Simulator and in some extensions; design a fallback policy (allow with lower limits, step-up checks)
- Keys are per install: reinstalling or restoring to a new device means a new key; `DCError.invalidKey` means "generate and attest again"
- **DeviceCheck:** `DCDevice.current.generateToken()` -> your server calls Apple's DeviceCheck API (signed with your DeviceCheck key) to query or update the two bits and their last-update date

Related to check: Request signing, authentication, passkeys, certificate pinning, CryptoKit and the Secure Enclave, secrets management, API rate limiting

---

Caching Policy

Q: How does HTTP caching work with URLSession?

Short answer: `URLSession` stores eligible responses in the session's `URLCache` (memory plus disk) and, with the default `.useProtocolCachePolicy`, follows the server's HTTP caching headers. A fresh response

(`Cache-Control: max-age`) is served from the cache without touching the network; a stale one is revalidated with `If-None-Match` / `If-Modified-Since`, and a **304 Not Modified** is turned back into the cached 200 transparently.

Key points:

- Server controls freshness: `max-age`, `no-cache` (store, but revalidate every time), `no-store` (never store), `private`, `ETag`, `Last-Modified`, `Vary`
- Per-request `cachePolicy`: `.useProtocolCachePolicy` (default), `.reloadIgnoringLocalCacheData`, `.returnCacheDataElseLoad`, `.returnCacheDataDontLoad` (offline only)
- Session-wide: `URLSessionConfiguration.urlCache` and `.requestCachePolicy`; `.ephemeral` sessions cache in memory only
- Not everything is cached: only successful responses whose headers and policies allow it, and only if they're small relative to the cache (no more than about 5% of the disk capacity)
- Invalidate after mutations with `removeCachedResponse(for:)` or `removeAllCachedResponses()`

Related to check: Response caching (app-level), URLCache, offline-first, AsyncImage, headers, content negotiation (`Vary`)

Background Downloads

Q: How do you implement background downloads?

Short answer: I create a `URLSession` with `URLSessionConfiguration.background(withIdentifier:)` and a delegate, and start `downloadTask`s on it. The transfer is handed to a system daemon, so it continues while the app is suspended or even terminated by the system.

Key points:

- Background sessions are **delegate-only**: the async `download(for:)` / `data(for:)` conveniences and completion-handler tasks aren't supported
- Move the file in `didFinishDownloadingTo` **before returning**; the temporary file is deleted afterwards
- One session per identifier, created once (app launch or relaunch); `sessionSendsLaunchEvents = true`
- `isDiscretionary = true` lets the system wait for Wi-Fi and power; downloads started while the app is in the background are discretionary anyway
- Background Assets: `BAURLDownload` scheduled from a downloader extension (iOS 16.4+ API), or **managed asset packs** hosted by Apple via `AssetPackManager` (iOS 26+)

Related to check: Download resume, URLSession, background tasks (`BGTaskScheduler`, `BGContinuedProcessingTask`), app lifecycle (UIScene), file storage

---

Alamofire vs URLSession

Q: When would you use Alamofire over URLSession?

Short answer: For a new app I default to `URLSession` with async/await: it's built in, supports HTTP/2 and HTTP/3, background transfers and per-session configuration, and a small `Sendable` client covers request building, validation and decoding in about a page of code. Alamofire is still common and still a good choice when a codebase already uses it or needs its batteries: request interceptors (adapt and retry), a ready-made `RetryPolicy`, chained validation, upload/download progress, `ServerTrustManager` for pinning, and `EventMonitor` logging.

Key points:

- Both are async/await-first today: `AF.request(...).serializingDecodable(T.self).value` vs `session.data(for:)` + `JSONDecoder`
- Alamofire is a layer *on top of* `URLSession`; it doesn't add transport features, only ergonomics
- Alamofire's `RequestInterceptor` = `adapt` (add auth headers) + `retry` (refresh token, back off); in plain Swift that's an actor holding the token plus a retry loop
- `RetryPolicy` retries only idempotent methods by default, with exponential backoff
- Dependency cost: binary size, upgrade cadence, Swift 6 readiness of the version you pin; prefer SPM (CocoaPods trunk is going read-only)

Related to check: URLSession, request interceptors, retry logic, certificate pinning, API mocking strategy, error handling

---

SSL Pinning

Q: What is SSL pinning? How do you implement it?

Short answer: Pinning means the app trusts only specific keys for its own servers, on top of normal certificate validation, so a mis-issued certificate or a user-installed root CA can't be used to intercept its traffic. The simplest option is declarative: `NSPinnedDomains` in Info.plist (covered in the certificate pinning question).

Key points:

- Handle `NSURLAuthenticationMethodServerTrust` only; `.performDefaultHandling` for everything else
- Default validation first, then the pin check; never skip `SecTrustEvaluateWithError`
- `SecTrustCopyCertificateChain` -> `SecCertificateCopyKey` -> `SecKeyCopyExternalRepresentation` gives the raw key, **not** the SPKI; prepend the algorithm's ASN.1 header before hashing so the hash matches what `openssl` and other tools produce
- Pin the leaf key, an intermediate's key (more resilient), or both; include at least one backup key
- The delegate must be the session's delegate (or a per-task delegate), so `URLSession.shared` can't pin
- Network framework: `sec_protocol_options_set_verify_block` on the TLS options

Related to check: Certificate pinning (`NSPinnedDomains`), app transport security, URLSession delegates, proxy configuration, CryptoKit, OAuth 2.0

Retry Logic

Q: How do you implement retry logic for failed requests?

Short answer: Retry only failures that are **transient** (timeouts, dropped connections, 502/503/504, 429) and only requests that are **safe to repeat**: idempotent methods, or writes that carry an `Idempotency-Key` the server deduplicates. Between attempts, wait with **exponential backoff plus jitter** using `try await Task.sleep(for:)`, cap the delay and the number of attempts, and honor `Retry-After` when the server sends it.

Key points:

- Retry: `URLError.timedOut`, `.networkConnectionLost`, `.notConnectedToInternet`, `.cannotConnectToHost`; HTTP 429, 502, 503, 504
- Don't retry: 400, 401 (refresh the token instead), 403, 404, 422, decoding errors, `.cancelled`
- Backoff: `base x 2^attempt`, capped (e.g. 30 s), then **jitter** (randomize) so many clients don't retry in lockstep
- `Retry-After` overrides your computed delay

- Idempotency: GET, HEAD, PUT, DELETE, OPTIONS are idempotent; POST/PATCH only with an idempotency key
- Cancellation: `Task.sleep(for:)` throws `CancellationError`, which ends the loop

Related to check: Idempotency, timeout configuration, API rate limiting, request interceptors, error handling, request cancellation

---

Request Cancellation

Q: How do you cancel in-flight network requests?

Short answer: With async/await, you cancel the Swift `Task` that is awaiting the request: URLSession's async methods observe task cancellation, cancel the underlying `URLSessionTask`, and throw. In SwiftUI the easiest way is `.task` / `.task(id:)`, which cancels automatically when the view disappears or the id changes.

Key points:

- Cancel the `Task`; the awaiting `data(for:)` cancels its `URLSessionTask`
- The thrown error can be `CancellationError` or `URLError(.cancelled)`, depending on where cancellation lands: handle both
- `.task(id:)` restarts and cancels work when the id changes; `.task` cancels on disappear
- Structured children (`async let`, task groups) are cancelled with their parent; unstructured `Task { }` is not
- Wrapping callback APIs: `withTaskCancellationHandler(operation:onCancel:)` to forward cancellation
- Downloads: `cancel(byProducingResumeData:)` lets you resume later

Related to check: Task cancellation, `.task(id:)`, request deduplication, retry logic, download resume, idempotency

---

Upload and Download Progress

Q: How do you track upload/download progress?

Short answer: For uploads, implement `urlSession(_:task:didSendBodyData:totalBytesSent:totalBytesExpect` from `URLSessionTaskDelegate`. In modern code I pass that delegate **per task** to the async call, `session.upload(for:fromFile:delegate:)`, and have it yield fractions into an `AsyncStream`.

Key points:

- Upload: `didSendBodyData`, fraction = `totalBytesSent / totalBytesExpectedToSend`
- Download: `URLSessionDownloadDelegate.didWriteData`, or `bytes(for:)` + `response.expectedContentLen`
- Expected length can be unknown (–1 / `NSURLSessionTransferSizeUnknown`): show an indeterminate indicator
- Per-task delegates work with the async methods; the delegate must be `Sendable` in Swift 6
- Background sessions are delegate-only: use a session delegate, not the async methods
- `AsyncStream` with `.bufferingNewest(1)` drops stale values so the UI isn't flooded

Related to check: URLSession delegates, multipart upload, background downloads, download resume, `AsyncStream`, Observation, request cancellation

---

Combine and Networking

Q: How do you use Combine with networking?

Short answer: In Combine, `URLSession.shared.dataTaskPublisher(for:)` emits one `(data, response)` value or a `URLError`; I validate the status in `tryMap`, `decode` the body, `receive(on:)` the main queue, and `sink` into the model, keeping the `AnyCancellable`. That's still worth knowing because many codebases built in the iOS 13 - 16 era use it, but for new code I write the same thing with async/await: it's shorter, errors are typed by `throws`, cancellation comes from the task (or SwiftUI's `.task`), and Swift 6 checks isolation at compile time.

Key points:

- Pipeline: `dataTaskPublisher` -> `tryMap` (status check) -> `decode(type:decoder:)` -> `receive(on:)` -> `sink` / `assign(to:)`
- Cancellation: dropping or cancelling the `AnyCancellable` cancels the request
- `.retry(n)` resubscribes, which **re-sends** the request; only safe for idempotent calls
- Bridge: `for try await value in publisher.values` consumes a publisher in async code
- Async/await equivalent: `try await session.data(for:)`, check status, decode, assign on the `@MainActor` model

Related to check: Combine publishers, async/await, Observation (`@Observable`), `ObservableObject` (legacy), request cancellation, retry logic

---

Mocking for Tests

Q: How do you mock networking for unit tests?

Short answer: For unit tests I mock at a **protocol boundary I own**, not at the network. Feature code depends on a `Sendable` service protocol (`UserService`), which the production type implements with the API client and tests replace with a test double injected through `init`.

Key points:

- Two seams: the service protocol (for view models) and the transport protocol (for the API client)
- Stub (returns canned data), spy (records calls), fake (working in-memory implementation)
- Actor-based doubles are race-free under parallel tests; avoid `@unchecked Sendable` mocks
- Fixtures: JSON files in the test bundle, or inline literals for small cases
- Test failure paths too: `URLError(.notConnectedToInternet)`, 401, malformed JSON
- Don't mock types you don't own (`URLSession` subclasses); wrap them behind your own protocol

Related to check: API mocking strategy (`URLProtocol`, mock servers), dependency injection, Swift Testing, REST API client design (`HTTPTransport`), test doubles

---

URLComponents and URL Building

Q: How do you safely build URLs with query parameters?

Short answer: Never build URLs by string concatenation: user input with spaces, `&`, `#` or non-ASCII characters breaks the URL or, worse, injects extra parameters. Use `URLComponents` with `queryItems`, which percent-encodes each name and value, or the newer `URL` APIs, `appending(path:)` and `appending(queryItems:)`, which do the same with less code.

Key points:

- `URLComponents`: `scheme`, `host`, `port`, `path`, `queryItems`, then `.url`
- `URL.appending(path:)` (percent-encodes segments; `/` still separates) and `appending(queryItems:)`

- Encoded vs decoded accessors: `path` vs `percentEncodedPath`, `queryItems` vs `percentEncodedQueryItems`
- `+` in values: encode as `%2B` manually
- Reading: `URLComponents(url:resolvingAgainstBaseURL:)?.queryItems`
- Since iOS 17, `URL(string:)` percent-encodes invalid characters (such as spaces) instead of returning `nil`; use `URL(string:encodingInvalidCharacters: false)` for the strict behavior

Related to check: REST API client design, deep linking, OAuth 2.0 (authorize URL), request signing (canonical query), percent-encoding

---

HTTP Headers

Q: What HTTP headers should you set for API requests?

Short answer: I set headers in two places. Values that are constant for the app's lifetime (`Accept`, a descriptive `User-Agent` or app-version header, an API version) go on the session through `URLSessionConfiguration.httpAdditionalHeaders`.

Key points:

- Body description: `Content-Type: application/json` on requests that have a body
- What you accept: `Accept: application/json`; `Accept-Language` if the app's language differs from the device's
- Auth: `Authorization: Bearer <token>` per request, from the token provider (not session-wide)
- Tracing and safety: `X-Request-ID` (correlate client and server logs), `Idempotency-Key` for retried POSTs
- Caching: the server sends `ETag` / `Cache-Control`; `URLSession` adds `If-None-Match` itself with the default cache policy

Related to check: Authentication, content negotiation, caching policy, idempotency, request interceptors, request-response logging

---

Timeout Configuration

Q: How do you configure request timeouts?

Short answer: `URLSessionConfiguration` has two timeouts with different meanings. `timeoutIntervalForRequest` is an **idle timeout**: how long a task may wait for the *next* piece of data.

Key points:

- `timeoutIntervalForRequest` (idle, default 60 s): per session, or per request with `URLRequest.timeoutInterval`
- `timeoutIntervalForResource` (total, default 7 days): per session only
- A timeout surfaces as `URLError(.timedOut)` (code -1001)
- With `waitsForConnectivity = true`, the task waits for a network instead of failing; the resource timeout bounds that wait
- Swift Concurrency deadline: run the request and a `Task.sleep(for:)` in a task group, and cancel the loser

Related to check: URLSession, retry logic, network reachability (`waitsForConnectivity`), task groups and cancellation, `URLError`

App Transport Security

Q: What is App Transport Security (ATS)? When would you disable it?

Short answer: ATS is the system policy that makes the URL Loading System (`URLSession`, `WKWebView`, AVFoundation media) refuse insecure connections: it requires HTTPS with TLS 1.2 or later, forward-secret cipher suites, and certificates signed with SHA-256 and strong keys. It's on by default and configured in the `NSAppTransportSecurity` dictionary in Info.plist.

Key points:

- `NSAllowsArbitraryLoads` turns ATS off everywhere; App Review asks for justification. Avoid it
- Targeted keys: `NSExceptionDomains` (per domain: `NSExceptionAllowsInsecureHTTPLoads`, `NSExceptionMinimumTLSVersion`, `NSIncludesSubdomains`), `NSAllowsArbitraryLoadsInWebContent`, `NSAllowsArbitraryLoadsForMedia`
- `NSAllowsLocalNetworking` permits loads to local resources (unqualified host names, `.local` domains) without weakening anything else; ideal for a dev server or a LAN device
- `NSPinnedDomains` (same dictionary) adds declarative certificate pinning
- ATS doesn't apply to lower-level APIs such as the Network framework, which is why those need their own TLS configuration

Related to check: Certificate pinning, SSL pinning, privacy manifests, Network framework (`NetworkConnection` with TLS), proxy configuration, Info.plist

---

Decoding Error Handling

Q: How do you handle JSON decoding errors?

Short answer: `JSONDecoder` throws a `DecodingError` with one of four cases (`keyNotFound`, `typeMismatch`, `valueNotFound`, `dataCorrupted`), each carrying a `Context` whose `codingPath` points at the exact field. I catch it at the networking boundary, log the case and path (not the payload, which may contain personal data) with `os.Logger`, and convert it to a domain error such as `APIError.decoding`, so the UI shows a generic message while I can debug from logs.

Key points:

- `keyNotFound` (missing key), `typeMismatch` (`"42"` vs 42), `valueNotFound` (`null` for non-optional), `dataCorrupted` (invalid JSON, unknown enum value, bad date)
- `context.codingPath` + `debugDescription` + `underlyingError` explain where and why
- Check the HTTP status **before** decoding the success type; decode an error body for 4xx/5xx
- Lossy decoding: decode elements one by one and skip failures, but log them
- Typed throws (`throws(APIError)`) makes the boundary explicit for callers

Related to check: Codable and JSON, Codable enums, Codable dates, error handling in networking, typed throws, `os.Logger`

Pagination

Q: How do you implement pagination for API responses?

Short answer: Load the first page, then load the next one when the user scrolls near the end, and append it. I prefer **cursor-based** pagination (the server returns a `nextCursor`, and `nil` means the last page) because it stays correct when items are inserted or deleted, while **offset-based** (`?page=3&limit=20`) can skip or duplicate rows on a changing feed.

Key points:

- Offset/limit vs cursor (keyset) vs `Link` header (`rel="next"`, GitHub-style)
- Trigger: the last row's `.task` / `.onAppear` in SwiftUI; `UICollectionViewDataSourcePrefetching` or `willDisplay` in UIKit
- Guard against concurrent loads (`isLoading`) and stop at the end (`hasMore`)
- Deduplicate by `id` when appending (feeds can shift between pages)
- Pull-to-refresh resets the cursor and replaces the items
- Keep `isLoading` correct on error and on cancellation

Related to check: Infinite scroll, `UICollectionViewDataSourcePrefetching`, request cancellation, request deduplication, Observation, SwiftData caching

---

REST vs GraphQL

Q: What's the difference between REST and GraphQL?

Short answer: REST exposes **many resource endpoints** (`GET /users/42`, `GET /users/42/posts`) whose response shapes the server decides, and it leans on HTTP itself: verbs, status codes and HTTP caching. GraphQL exposes **one endpoint** with a typed schema, and the client sends a query describing exactly the fields it needs, often across several related objects in one round trip, which avoids over-fetching and under-fetching.

Key points:

- REST: multiple endpoints, server-defined shapes, HTTP caching and status codes, easy to debug with any tool
- GraphQL: single endpoint, client-defined shapes, strongly typed schema, introspection, subscriptions for real time
- Over-fetching (too many fields) and under-fetching (several round trips) are the REST pain points GraphQL targets
- GraphQL errors: partial data plus `errors`, usually with HTTP 200
- Caching: REST uses `URLCache` and `ETags`; GraphQL clients (e.g. Apollo iOS) keep a normalized cache keyed by object ID
- Code generation: Apollo iOS generates Swift types from `.graphql` files; for REST, swift-openapi-generator does the same from OpenAPI

Related to check: REST API client design, GraphQL client, gRPC on iOS, response caching, pagination, Codable

---

Nested Codable Structures

Q: How do you decode nested JSON structures?

Short answer: There are two approaches, and the choice is about what shape the app wants. If the JSON's nesting is meaningful, I mirror it: nested `Codable` structs (and arrays of them) decode automatically, with optionals for nullable objects.

Key points:

- Nested types decode recursively; each needs `Decodable` conformance
- Generic envelope: `struct APIResponse<T: Decodable>: Decodable { let data: T }` handles every endpoint's wrapper once
- Flattening: `container.nestedContainer(keyedBy: ProfileKeys.self, forKey: .profile)`; `nestedUnkeyedContainer` for nested arrays
- Optional nested objects: `decodeIfPresent` / optional property; a nested `null` is `nil`, but a missing required key inside it still throws

- Encoding mirrors it: `encoder.container(...).nestedContainer(...)` to write the same shape back

Related to check: Codable and JSON, Codable enums (discriminated unions), decoding error handling, Codable dates, GraphQL responses

---

Network Reachability

Q: How do you check network reachability?

Short answer: I use the Network framework's `NWPathMonitor`, which is an `AsyncSequence` of `NWPath` values, so a `@MainActor` model can simply `for await path in NWPathMonitor()` inside a SwiftUI `.task` and publish `isOnline`, `isExpensive` and `isConstrained`. But I don't use reachability as a gate before requests: a "satisfied" path doesn't mean your server is reachable, and an unsatisfied one can change a moment later.

Key points:

- `for await path in NWPathMonitor()`: the loop ends when the task is cancelled
- `path.status`: `.satisfied`, `.unsatisfied`, `.requiresConnection`
- `path.isExpensive` (cellular or hotspot), `path.isConstrained` (Low Data Mode), `path.usesInterfaceType(.wif`
- `NWPathMonitor(requiredInterfaceType: .wifi)` monitors one interface type
- URLSession: `waitsForConnectivity`, `allowsExpensiveNetworkAccess = false`, `allowsConstrainedNetworkAccess = false`; blocked requests fail with `URLError.networkUnavailableReason`
- `SCNetworkReachability` is deprecated (iOS 17.4); the old Apple "Reachability" sample and its ports are legacy

Related to check: Network framework, timeout configuration, retry logic, WebSocket reconnection, background downloads, Low Data Mode

---

Codable Enums

Q: How do you decode enums with Codable?

Short answer: An enum with a `String` or `Int` raw value gets `Codable` synthesized, and the JSON value must match a raw value exactly. That's fine for closed sets, but server enums grow, and a single unknown value throws `dataCorrupted` and fails the **whole** response.

Key points:

- `enum Status: String, Codable` maps cases to strings; customize with explicit raw values (`case inProgress = "in_progress"`)
- Unknown value -> `DecodingError.dataCorrupted` for the entire payload
- Forward-compatible: `case unknown(String)` (or a `default` case) via `singleValueContainer()`
- Discriminated unions: read `type` from a keyed container, then `try Payload(from: decoder)` on the same decoder
- Synthesized associated-value coding exists, but its shape is Swift's, not your API's

Related to check: Codable and JSON, decoding error handling, nested Codable, enums with associated values, API versioning

---

Request/Response Interceptors

Q: How do you add logging or modify all requests (e.g., add auth)?

Short answer: I route every request through one client that runs a **middleware chain**: each middleware receives the request and a `next` function, can modify the request (add the bearer token, a request ID, a user agent), call `next`, and then inspect or react to the response (log it, refresh the token on a 401 and retry once). Middlewares are small `Sendable` values composed in order, so auth, logging and retry are separate, testable pieces instead of code copied into every call.

Key points:

- Shape: `intercept(request, next:) async throws -> (Data, HTTPURLResponse)`
- Order matters: e.g. logging (outermost) -> retry -> auth -> transport
- Auth middleware: add the header, and on 401 refresh once (single-flight in an actor) and retry
- Static headers can go in `URLSessionConfiguration.httpAdditionalHeaders` instead
- `URLProtocol` subclasses registered in `protocolClasses` see every request on that session

Related to check: OAuth 2.0 token refresh, request/response logging, retry logic, request signing, API mocking strategy (`URLProtocol`), Alamofire

---

WebSockets

Q: How do you implement WebSocket connections?

Short answer: On iOS the built-in client is `URLSessionWebSocketTask`: create it with `session.webSocketTask(with:)`, call `resume()`, then use the async `send(_:)` and `receive()` methods. `receive()` returns **one** message, so you run a receive loop in a task and turn it into an `AsyncThrowingStream` that the UI layer can `for try await` over.

Key points:

- `URLSessionWebSocketTask.Message` is `.string(String)` or `.data(Data)`
- `receive()` must be called again for every message; stop the loop on error
- `sendPing(pongReceiveHandler:)` detects dead connections that TCP hasn't noticed
- `cancel(with: .goingAway, reason:)` closes cleanly; `closeCode` / `closeReason` tell you why the server closed
- `maximumMessageSize` (default 1 MB) makes larger messages fail the receive
- `URLSessionWebSocketDelegate` reports `didOpenWithProtocol` / `didCloseWith`
- Subprotocols: `webSocketTask(with: url, protocols: ["graphql-transport-ws"])`

Related to check: WebSocket reconnection, `AsyncStream`, actors, URLSession, chunked transfer / server-sent events, Network framework (`NetworkConnection`, iOS 26+)

Server-Sent Events

Q: How would you consume Server-Sent Events on iOS?

Short answer: Server-Sent Events (SSE) is a one-way stream from server to client over a normal HTTP response with `Content-Type: text/event-stream`. The body is lines of `field: value` (`data`, `event`, `id`, `retry`), and a **blank line** ends each event. iOS has no built-in `EventSource`, so I open the request with `URLSession.bytes(for:)`, split the bytes into lines, feed them to a small parser, and expose events as an async loop the UI consumes from `.task`.

Key points:

- Request: `Accept: text/event-stream`; response 200 with that content type; 204 means "stop, don't reconnect"
- Parsing: `data:` lines accumulate (joined with `\n`), `event:` sets the type (default `message`), `id:` sets the last event ID, `retry:` sets the reconnect delay in ms, lines starting with `:` are comments (often heartbeats)
- **`bytes.lines` drops empty lines**, so it loses event boundaries; split the bytes yourself (or use a library)
- Reconnect loop: send `Last-Event-ID`, honor `retry`, add exponential backoff with jitter, stop on task cancellation
- `timeoutIntervalForRequest` is an idle timeout between bytes, so it must be longer than the server's heartbeat interval

Related to check: Chunked transfer, URLSession (`bytes(for:)`), WebSockets, WebSocket reconnection, retry logic, AsyncSequence / AsyncStream, timeout configuration

---

Codable and Dates

Q: How do you handle date encoding/decoding?

Short answer: `JSONDecoder` and `JSONEncoder` have a `dateDecodingStrategy` / `dateEncodingStrategy` that applies to every `Date` property. The default, `.deferredToDate`, encodes a `Double` of seconds since **2001-01-01**, which almost no API uses, so I always set a strategy that matches the server: `.iso8601` for ISO 8601 strings, `.secondsSince1970` / `.millisecondsSince1970` for Unix timestamps, or `.custom` when the API mixes formats.

Key points:

- Strategies: `.deferredToDate` (default), `.iso8601`, `.secondsSince1970`, `.millisecondsSince1970`, `.formatted(DateFormatter)`, `.custom { decoder in ... }`
- `.iso8601` handles `Z` and offsets; don't rely on it for fractional seconds across the OS versions you support
- `Date.ISO8601FormatStyle` (a `Sendable` value) is the modern parser; `DateFormatter` needs `en_US_POSIX` locale and an explicit time zone
- The `.custom` closure is `@Sendable` in Swift 6, so capture only `Sendable` values
- Date-only fields (birthdays, "2026-10-17") aren't instants; consider decoding them as `String` or `DateComponents` rather than `Date`

Related to check: Codable and JSON, decoding error handling, nested Codable, formatting and localization, `FormatStyle`

Request Prioritization

Q: How do you prioritize network requests?

Short answer: There are two levels. At the transport level, `URLSessionTask.priority` (0.0 - 1.0, with `lowPriority`, `defaultPriority` and `highPriority` constants) is a **hint** URLSession uses when scheduling requests (not every protocol honors it, and changes after a task starts may be ignored).

Key points:

- `URLSessionTask.priority`: a hint, not a guarantee
- Async API: `session.data(for: request, delegate: PriorityDelegate(URLSessionTask.highPriority))`, setting it in `didCreateTask` (called synchronously before the task starts)
- `URLRequest.networkServiceType` (`.responsiveData`, `.background`, `.avStreaming`…) tells the system the traffic class

- Background or discretionary work: background sessions with `isDiscretionary = true`
- `httpMaximumConnectionsPerHost` limits parallel connections (HTTP/1.1 mostly)
- App-level scheduling: an actor that limits concurrency and orders waiting work by priority

Related to check: Request cancellation, HTTP/2 and HTTP/3, connection pooling, background downloads, image loading and prefetching, actors

---

Response Validation

Q: How do you validate HTTP responses?

Short answer: URLSession only throws for transport failures; a 404 or 500 comes back as a normal response, so the client must validate it before decoding. I cast to `HTTPURLResponse`, map the status code to a domain error (401 -> re-authenticate, 404 -> not found, 429 -> rate limited with `Retry-After`, 5xx -> server error carrying the decoded error body), check the content type, and only then decode, wrapping `DecodingError` so it's distinguishable from HTTP failures.

Key points:

- `guard let http = response as? HTTPURLResponse`
- 2xx success (watch for 204 No Content with an empty body); 3xx are followed automatically; 304 only appears with your own conditional requests
- Map 4xx/5xx to typed errors; decode the server's error payload when it has one
- `http.mimeType == "application/json"` (it strips parameters like `charset`)
- Decode after validating; wrap `DecodingError` separately
- Optional: verify `expectedContentLength`, required headers, or a response signature

Related to check: Error handling in networking, Codable error handling, retry logic, API rate limiting, REST API client design, typed throws

---

Multipart Form Upload

Q: How do you upload multipart form data (e.g., image + fields)?

Short answer: A `multipart/form-data` body is a list of parts separated by a **boundary** string. The request's `Content-Type` is `multipart/form-data; boundary=XYZ`; each part starts with `--XYZ` and CRLF, has its own headers (`Content-Disposition: form-data; name="..."`, plus `filename` and `Content-Type` for files), a blank line, then the bytes; the body ends with `--XYZ--`.

Key points:

- Boundary: random (`UUID`) so it can't appear in the content
- Delimiter is `--` + boundary; the final one is `--` + boundary + `--`
- Line endings must be `\r\n`, with a blank line between part headers and content
- File parts: `filename="photo.jpg"` and `Content-Type: image/jpeg` (get MIME types from `UTType(filenameExtension:)?.preferredMIMEType`)
- Small: `upload(for:from: Data)`; large or background: `upload(for:fromFile:)`
- Progress comes from the task delegate's `didSendBodyData`

Related to check: Upload progress, background downloads and uploads, URLSession, content negotiation, Alamofire, image downsampling before upload

---

HTTP/2 and HTTP/3

Q: What are HTTP/2 and HTTP/3? How do they affect iOS?

Short answer: HTTP/2 keeps HTTP's semantics but changes the wire format: binary frames, **multiplexing** many requests as streams over one TCP connection, and HPACK header compression. That removes HTTP/1.1's application-level head-of-line blocking, but a single lost TCP packet still stalls every stream.

Key points:

- HTTP/2: multiplexing, header compression, one connection per origin; negotiated with TLS ALPN
- HTTP/3: QUIC (UDP), per-stream loss recovery, faster handshakes, connection migration; discovered via `Alt-Svc` headers or HTTPS DNS records
- `URLSession` uses the best protocol the server offers with no code change; `URLRequest.assumesHTTP3Capable` lets a request try HTTP/3 on the first connection
- `URLSessionTaskTransactionMetrics.networkProtocolName` reports `"http/1.1"`, `"h2"` or `"h3"`
- Network framework supports QUIC directly for custom protocols (`NWProtocolQUIC`, and the iOS 26 `NetworkConnection` API)

Related to check: Connection pooling, gRPC on iOS, chunked transfer, Network framework (`NetworkConnection`, QUIC), timeout

---

Certificate Pinning Implementation

Q: How do you implement certificate pinning?

Short answer: Pinning means the app accepts a server only if its certificate chain contains a key I expect, on top of normal trust evaluation, so a mis-issued certificate or a user-installed proxy root can't intercept traffic. The first choice on iOS is **declarative pinning** with `NSPinnedDomains` in Info.plist: list SHA-256 hashes of the SubjectPublicKeyInfo (SPKI) for leaf or CA identities, and ATS enforces them with no code.

Key points:

- Pin the **public key (SPKI)**, not the whole certificate, so renewals that keep the key don't break the app
- `NSPinnedDomains` -> domain -> `NSPinnedLeafIdentities` or `NSPinnedCAIdentities` -> `SPKI-SHA256-BASE64`; `NSIncludesSubdomains` optional
- Programmatic: `SecTrustEvaluateWithError` **first**, then compare pins; return `.useCredential` with `URLCredential(trust:)` or `.cancelAuthenticationChallenge`
- `SecKeyCopyExternalRepresentation` returns the raw key, not the SPKI; prepend the algorithm's ASN.1 header before hashing so pins match `openssl` output
- Always a backup pin (next key, or an intermediate CA key) to survive rotation

Related to check: SSL pinning (concepts and threat model), App Transport Security, Network framework TLS, request signing, secrets management

---

Request Deduplication

Q: How do you deduplicate in-flight requests?

Short answer: Deduplication (single-flight) means that if three screens ask for `/me` at the same moment, only one HTTP request goes out and all three callers get its result. I implement it as an `actor` holding a dictionary of in-flight work, `[RequestKey: Task<Data, Error>]`.

Key points:

- Shared mutable state (the in-flight map) lives in an actor
- Key = method + URL (plus body hash or auth identity if they change the response)
- Only deduplicate safe, idempotent reads (GET/HEAD); never merge two POSTs
- Remove the entry when the task completes, success or failure
- Deduplication is not caching: it only shares work that is *currently* running
- The same pattern gives single-flight token refresh (`Task<Token, Error>?` in an auth actor)

Related to check: Actors and reentrancy, idempotency, response caching, request cancellation, OAuth 2.0 token refresh, connection pooling

---

Response Caching

Q: How does URLCache work? When do you use it?

Short answer: `URLCache` is URLSession's built-in HTTP cache, with a memory tier and a disk tier. With the default `.useProtocolCachePolicy`, it follows the server's HTTP caching headers: a fresh response (`Cache-Control: max-age`) is served from the cache without a network round trip, and a stale one with an `ETag` or `Last-Modified` is revalidated with a conditional request, where a `304 Not Modified` is turned back into the cached `200` for you.

Key points:

- `URLCache(memoryCapacity:diskCapacity:directory:)`, set on `URLSessionConfiguration.urlCache` (or `URLCache.shared`)
- Policies: `.useProtocolCachePolicy` (default), `.reloadIgnoringLocalCacheData`, `.returnCacheDataElseLoad`, `.returnCacheDataDontLoad`
- Only successful (2xx) responses that the headers and policies allow are stored, and only if small enough (Apple's guideline: about 5% of the disk cache per response); in practice it's for GETs
- Headers that matter: `Cache-Control` (`max-age`, `no-store`, `no-cache`, `private`), `ETag`, `Last-Modified`, `Vary`
- `.ephemeral` sessions cache in memory only
- Delegate hook: `urlSession(_:dataTask:willCacheResponse:)` to change the storage policy or refuse caching (delegate-based data tasks)

Related to check: Caching policy, request deduplication, content negotiation (`Vary`), URLSession configuration, image caching, offline-first with SwiftData

---

OAuth 2.0 Flow

Q: How do you implement OAuth 2.0 in an iOS app?

Short answer: A native app is a *public client*, so it uses the **authorization code flow with PKCE**: generate a random `code_verifier`, send its SHA-256 `code_challenge` (S256) and a random `state` to the authorize endpoint, and show the login page with `ASWebAuthenticationSession` (in SwiftUI, `@Environment(\.webAuthenticationSession)`, which is async). When the provider redirects back, check that `state` matches, then exchange the `code` plus the verifier for tokens at the token endpoint.

Key points:

- PKCE: 32 random bytes -> base64url verifier; `challenge = base64url(SHA256(verifier))`; `code_challenge_method=S256`
- `state` protects against CSRF / injected callbacks; reject the callback if it doesn't match
- `ASWebAuthenticationSession.Callback.https(host:path:)` uses a universal-link-style callback on a domain you own (needs the associated web-credentials domain); `.customScheme("myapp")` is the older scheme-based option

- `prefersEphemeralWebBrowserSession` / `.ephemeral` skips shared Safari cookies (and the consent prompt)
- Token exchange is a form-encoded POST: `grant_type=authorization_code`, `code`, `code_verifier`, `redirect_uri`, `client_id`
- Keychain item with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` (or stricter); not `UserDefaults`

Related to check: Authentication (passkeys, Sign in with Apple), Keychain, request interceptors (adding the bearer token), request deduplication, SSL pinning, request signing

---

GraphQL Client

Q: How would you implement a GraphQL client?

Short answer: A GraphQL client is a thin layer over HTTP: every operation is a POST to one endpoint with a JSON body of `query`, `variables` and `operationName`, and every response is a JSON object with `data` and/or `errors`. The important twist is that a **200 can still contain errors**, and partial data is legal, so the client must check `errors` rather than relying on the status code.

Key points:

- Request body: `{"query": "...", "operationName": "GetUser", "variables": {"id": "123"}}`
- Response: `{"data": {...}, "errors": [{"message": "...", "path": [...]}]}`; check `errors` even on 200
- Variables are typed: use a generic `Encodable` value, not `[String: String]`
- Caching: POSTs bypass `URLCache`; use a normalized client cache, or **persisted queries** (send a hash; GET-able and CDN-cacheable)
- Subscriptions run over WebSocket (`graphql-ws` protocol) or server-sent events

Related to check: REST vs GraphQL, nested Codable, response caching, WebSockets, pagination, API documentation

---

gRPC on iOS

Q: What is gRPC? When would you use it on iOS?

Short answer: gRPC is a schema-first RPC framework: you define services and messages in a `.proto` file, generate client code, and calls travel as compact Protocol Buffers over HTTP/2. Its strengths are a strict, versionable contract shared across many languages, small binary payloads, and first-class streaming (server, client and bidirectional).

Key points:

- Protobuf: binary, schema-evolved by field numbers (add fields, never reuse numbers)
- Four call types: unary, server streaming, client streaming, bidirectional streaming
- grpc-swift 2: `GRPCCore` + a transport package (`grpc-swift-nio-transport`) + `grpc-swift-protobuf` for codegen; requires an iOS 18 deployment target and Swift 6
- gRPC relies on HTTP/2 trailers that `URLSession` doesn't expose, so the client uses its own (SwiftNIO-based) transport, including a Network.framework-backed variant on Apple platforms
- Alternatives that work over plain HTTPS/`URLSession`: gRPC-Web or the Connect protocol (Connect-Swift)

Related to check: HTTP/2 and HTTP/3, API documentation (OpenAPI codegen), GraphQL client, WebSockets, AsyncSequence, connection pooling

---

WebSocket Reconnection

Q: How do you handle WebSocket reconnection?

Short answer: I put the connection in an `actor` that runs a **reconnect loop**: connect, receive until the socket fails, then wait with exponential backoff plus jitter (`Task.sleep(for:)`) and connect again, resetting the backoff once a connection is healthy. The loop runs inside a task owned by the screen or session (for example SwiftUI `.task`), so cancelling that task stops reconnecting and closes the socket.

Key points:

- Detect disconnects: `receive()` throws, the server closes (`closeCode`), or a ping gets no pong
- Backoff: 1 s, 2 s, 4 s… capped (e.g. 60 s) with jitter; reset after a successful connect
- Cancellation: `withTaskCancellationHandler` closes the socket, because `receive()` doesn't stop on task cancellation by itself
- Reconnect immediately when the network comes back (`NWPathMonitor`) or the app returns to the foreground
- Resync: resubscribe and fetch missed events by last seen ID; deduplicate by message ID
- Don't reconnect on deliberate closes (logout, `.normalClosure`, auth rejected)

Related to check: WebSockets, retry logic, network reachability (`NWPathMonitor`), actors and cancellation, API rate limiting, Network framework

---

Network Framework and NetworkConnection

Q: What is NetworkConnection in iOS 26, and when would you use the Network framework instead of URLSession?

Short answer: `URLSession` is the right tool for anything HTTP-shaped: REST, downloads, uploads, HTTP streaming, and WebSockets to a web server. The Network framework sits one level lower.

Key points:

- Stack builder: the outermost protocol is your application protocol, wrapped around what's below it: `Coder(Msg.self, using: .json) { TLS() }`; `TLS()` defaults to TCP below
- Stream protocols (`TLS`, `TCP`) give you bytes: `receive(atLeast:atMost:)` / `receive(exactly:)`. Message protocols (`WebSocket`, `TLV`, `Coder`, `UDP`) give you whole messages, and `messages` is an `AsyncThrowingStream`
- A received message is a `(content, metadata)` tuple, e.g. `message.content`
- It starts lazily on the first send or receive. State comes from `onStateUpdate { connection, state in }`: `.preparing`, `.ready`, `.waiting(error)` (no usable path yet), `.failed`, `.cancelled`
- `withNetworkConnection(to:using:_:)` scopes the connection to a closure, so it is cancelled when the closure returns
- Siblings: `NetworkListener` (accepts connections, `run { connection in ... }`), `NetworkBrowser` with the `Bonjour` provider (discovery), `QUIC` streams via `openStream()`

Related to check: URLSession, WebSockets, WebSocket reconnection, network reachability (`NWPathMonitor`), HTTP/2 and HTTP/3 (QUIC), certificate pinning, Sendable networking layer

---

Request Signing

Q: How do you sign API requests?

Short answer: Request signing proves a request wasn't altered and came from a holder of a key. The client builds a **canonical string** from the method, path, query, a timestamp, a nonce and a SHA-256 hash of the body, computes an HMAC-SHA256 over it with CryptoKit, and sends the signature plus the timestamp, nonce and key ID in headers.

Key points:

- Sign: method, path, canonical query, timestamp, nonce, body hash (and selected headers)
- `HMAC<SHA256>.authenticationCode(for:using:)` with a `SymmetricKey`
- Replay protection: timestamp window (e.g. ±5 minutes) + nonce cache on the server
- Asymmetric option: `SecureEnclave.P256.Signing.PrivateKey` signs, server verifies with the registered public key; the private key never leaves the device
- Standards: AWS Signature V4, HTTP Message Signatures (RFC 9421)
- TLS already gives integrity in transit; signing adds proof of the *client's key* and protects against replay through proxies and logs

Related to check: CryptoKit, Secure Enclave, App Attest, OAuth 2.0 flow, request interceptors, SSL pinning, secrets management

---

API Rate Limiting

Q: How do you handle API rate limiting client-side?

Short answer: Two halves: don't trigger limits, and react well when you do. To avoid them I throttle on the client (a token bucket in an actor, debounced search, deduplicated in-flight requests, batching).

Key points:

- 429 = you're over the limit; 503 + `Retry-After` = server overloaded. Some APIs also send `RateLimit-Remaining` / `X-RateLimit-Reset` style headers you can use to slow down early
- `Retry-After: 120` or `Retry-After: Wed, 21 Oct 2026 07:28:00 GMT`
- Backoff: `base * 2^attempt`, capped, plus random jitter; give up after a few attempts
- Client-side token bucket in an `actor`: callers `await` a permit before sending
- Retry only idempotent requests (GET/PUT/DELETE) or ones carrying an `Idempotency-Key`

Related to check: Retry logic, idempotency, request deduplication, error handling, circuit breaker, actors

---

Chunked Transfer

Q: What is chunked transfer encoding? When do you see it?

Short answer: `Transfer-Encoding: chunked` is the HTTP/1.1 way to send a body whose length isn't known up front: instead of `Content-Length`, the server writes a series of size-prefixed chunks and ends with a zero-length chunk. You see it with dynamically generated responses and streams: server-sent events, newline-delimited JSON, LLM token streams, long-running exports.

Key points:

- No `Content-Length` -> `response.expectedContentLength == -1`; progress can't be a percentage
- `URLSession` de-chunks transparently; you never parse chunk sizes yourself
- `session.bytes(for:)` returns `URLSession.AsyncBytes`; `.lines` yields complete lines as they arrive
- Chunk boundaries are **not** message boundaries: a JSON object can span chunks, or several can share one; frame by lines or by a protocol (SSE `data:` lines)

- Uploads: a streamed body (`httpBodyStream`, `uploadTask(withStreamedRequest:)`) is sent chunked over HTTP/1.1

Related to check: HTTP/2 and HTTP/3, AsyncSequence / AsyncStream, WebSockets, server-sent events, download resume, timeout

---

Connection Pooling

Q: Does URLSession pool connections?

Short answer: Yes. Each `URLSession` keeps its own pool of connections per host and reuses them across tasks, so after the first request you skip DNS, TCP and TLS setup.

Key points:

- Pooling is automatic and **per session**; two sessions never share connections
- `URLSession.shared` or one app-wide session (per distinct configuration) maximizes reuse
- `httpMaximumConnectionsPerHost` mostly affects HTTP/1.1 servers
- Idle connections are closed after a while; keep-alive is handled by the system
- `URLSessionTaskTransactionMetrics.isReusedConnection` shows whether a request reused a connection

Related to check: HTTP/2 and HTTP/3, timeout, chunked transfer, request deduplication, URLSession configuration

---

Proxy Configuration

Q: How do you configure a proxy for URLSession?

Short answer: By default URLSession uses the **system proxy settings** (Wi-Fi proxy, PAC files, MDM-configured proxies), and most apps should leave it that way. When an app must route its own traffic through a specific proxy, the modern API is `URLSessionConfiguration.proxyConfigurations`, an array of Network framework `ProxyConfiguration` values: HTTP CONNECT proxies (optionally over TLS), SOCKSv5, and privacy relays (HTTP/2 or HTTP/3 relay hops, Oblivious HTTP).

Key points:

- Default: system settings; no code needed
- `config.proxyConfigurations = [ProxyConfiguration(httpCONNECTProxy: endpoint, tlsOptions: ...)]`
- `ProxyConfiguration(socksv5Proxy:)`, `ProxyConfiguration(relayHops:)`, `ProxyConfiguration(obliviousHTTPRelay:...)`
- Scope with `matchDomains` / `excludedDomains`; `allowFailover` controls falling back to a direct connection
- Legacy: `connectionProxyDictionary` (`kCFNetworkProxiesHTTPEnable` / `HTTPProxy` / `HTTPPort`); the HTTPS-proxy keys are macOS-only
- Proxy authentication can also arrive as a challenge whose `protectionSpace.isProxy()` is true
- Network framework connections use `NWParameters.PrivacyContext.proxyConfigurations`

Related to check: URLSession configuration, SSL pinning, app transport security, request/response logging, Network framework, API mocking strategy

---

Download Resume

Q: How do you resume an interrupted download?

Short answer: `URLSessionDownloadTask` can produce **resume data**: an opaque blob that records the URL, the bytes already on disk and the validator (`ETag` or `Last-Modified`). I get it in two ways: by pausing with `cancel(byProducingResumeData:)` (or its async form `cancelByProducingResumeData()`), or from a failure, where `URLError.downloadTaskResumeData` carries it.

Key points:

- Resumable only for GET, when the server sends `Accept-Ranges: bytes` and an `ETag` or `Last-Modified`, and the partial file still exists
- Pause: `cancelByProducingResumeData()`; failure: `(error as? URLError)?.downloadTaskResumeData`
- Resume: `session.downloadTask(withResumeData:)` or `try await session.download(resumeFrom:)`
- Resume data is opaque; store it, don't parse it
- Background sessions retry transient failures themselves and report resume data in `didCompleteWithError`

Related to check: Background downloads, chunked transfer, connection pooling, caching policy (`ETag`), actors, error handling

---

Request/Response Logging

Q: How do you log requests and responses for debugging?

Short answer: I log at one choke point, the API client or a logging middleware, using `os.Logger` with a subsystem and a `network` category. Unified logging is cheap enough to leave in release builds, and its **privacy annotations** decide what's visible: interpolated strings are `private` (redacted) by default outside a debugger, so I mark only safe values like the method, path and status as `.public`, and I never log tokens, passwords or full bodies at all.

Key points:

- `Logger(subsystem:category:)`; levels `debug` (not persisted), `info`, `notice` (default), `error`, `fault`
- Privacy: `\(value, privacy: .public)`, `.private`, `.private(mask: .hash)` to correlate without revealing
- Never log `Authorization`, cookies, tokens, or PII; redact by allow-list, not deny-list
- Metrics: `urlSession(_:task:didFinishCollecting:)` gives DNS/TLS/request timings and the protocol (`h2`, `h3`)
- Reading logs: Xcode console filters, Console.app, `log stream --predicate 'subsystem == "..."'`, and `OSLogStore` to export the app's own logs for a bug report
- `print` is unstructured, has no levels or privacy, and ships everything

Related to check: Logging architecture and levels, request interceptors, secrets management, privacy manifests, Instruments (network and signposts), proxy configuration

---

API Mocking Strategy

Q: How do you mock APIs for development?

Short answer: I mock at the lowest layer that still gives fast, deterministic feedback, and I pick the layer per purpose. For view models and previews, a `Sendable` `HTTPClient` (or generated `APIProtocol`) protocol with a fixture-backed fake is simplest.

Key points:

- Layer 1: protocol fake injected through `init` (unit tests, SwiftUI previews)
- Layer 2: `URLProtocol` on an ephemeral session's `protocolClasses` (tests the real client; no global `registerClass`)
- Layer 3: launch argument / environment variable that selects fixtures (UI tests, demos, offline development)
- Layer 4: mock server from the OpenAPI spec (WireMock, Prism, etc.) for integration and manual QA
- Fixtures are real JSON captured from staging and checked into the test bundle

Related to check: URLProtocol (testing), mock networking, API documentation (OpenAPI fixtures), dependency injection, test isolation, SwiftUI previews

---

Content Negotiation

Q: What is content negotiation? How does Accept header work?

Short answer: Content negotiation is how one URL can serve different representations: the client states what it can handle in `Accept*` request headers, and the server picks one and labels it with the matching response header. `Accept: application/json` asks for JSON; a list with **quality values** (`application/json, application/xml;q=0.5`) states preferences, where q ranges from 0 to 1 and defaults to 1.

Key points:

- Request: `Accept` (format), `Accept-Language` (locale), `Accept-Encoding` (compression); `Content-Type` describes **the body you send**
- Response: `Content-Type`, `Content-Language`, `Content-Encoding`, `Vary`
- `URLSession` adds `Accept-Encoding` (gzip, deflate, br) and decompresses transparently; it also sends an `Accept-Language` based on the user's preferred languages
- Status codes: **406** (can't produce what you accept), **415 Unsupported Media Type** (server can't read your `Content-Type`)
- Media-type versioning: `Accept: application/vnd.example.v2+json` as an alternative to `/v2/` in the URL

Related to check: HTTP headers, caching policy (`Vary`), API versioning, error handling, localization

---

Idempotency

Q: What is idempotency? Why does it matter for APIs?

Short answer: An operation is idempotent if doing it twice has the same effect on the server as doing it once. It matters on mobile because networks fail **ambiguously**: a timeout doesn't tell you whether the server processed the request, so the only safe retry is one that can't double-charge or double-post.

Key points:

- Safe (no side effects: GET, HEAD) idempotent (same effect when repeated: plus PUT, DELETE)
- The key identifies the **operation**, not the attempt: generate it once, reuse it on every retry and after relaunch
- The server stores key -> response for a window (often 24 hours) and rejects the same key with a different payload
- Retry logic: automatically retry idempotent methods, and POST/PATCH only when they carry a key
- Offline queues persist the key with the payload so replayed operations stay deduplicated

Related to check: Retry logic, API rate limiting, request deduplication, offline persistence, error handling, HTTP/2 and HTTP/3

API Documentation

Q: How do you keep API docs in sync with the client?

Short answer: I make the API description, usually an OpenAPI document, the single source of truth and generate the client from it instead of hand-writing request code. On iOS that's Apple's **swift-openapi-generator**: a SwiftPM build plugin that turns `openapi.yaml` into a typed `Client`, request/response types and an `APIProtocol`, so a breaking server change becomes a compile error rather than a runtime decoding failure.

Key points:

- Spec first: `openapi.yaml` lives in a shared repo (or is fetched in CI) and both server and client are checked against it
- swift-openapi-generator generates at **build time** (no checked-in generated code); runtime is `OpenAPIRuntime` + a transport such as `OpenAPIURLSession`
- Generated responses are enums per status code (`.ok`, `.notFound`, `.undocumented`), which forces you to handle each documented case
- The generated `APIProtocol` is easy to fake in tests and previews, no URLProtocol needed
- Contract tests (or consumer-driven contracts such as Pact) catch drift the compiler can't: the server returning something the spec doesn't describe

Related to check: API mocking strategy, contract testing, API versioning, Codable, error handling, gRPC on iOS (protobuf as another schema-first approach)

Concurrency

async/await

Q: Explain async/await in Swift. How does it work?

Short answer: `async` marks a function that can suspend, and `await` marks each point where it might. While a function is suspended it gives up its thread, so other work runs, and it resumes later, possibly on a different thread, but always back on its actor if it's isolated to one.

Key points:

- Call `async` functions only from an async context: another async function, a `Task`, or SwiftUI's `.task`
- Suspension doesn't block the thread; the function is split into continuations the runtime schedules
- Isolation decides *where* code runs: a `@MainActor` async method runs on main between suspensions
- `Task { }` inherits the current actor, so inside `@MainActor` code no `MainActor.run` is needed
- Swift 6.2: CPU-heavy work goes in a `@concurrent` function; plain `nonisolated async` functions run on the caller's actor with Approachable Concurrency
- Structured concurrency: `async let` and task groups for parallel work, with automatic cancellation propagation

Related to check: Task, MainActor, actors, structured concurrency, `@concurrent`, continuations

---

MainActor

Q: What is MainActor? When do you need it?

Short answer: `@MainActor` is the global actor that represents the main thread. Code isolated to it can only run there, and in Swift 6 language mode the compiler enforces that at compile time: calling main-actor code from elsewhere requires `await`, and touching main-actor state from the wrong context is an error.

Key points:

- Apply `@MainActor` to a type (all members isolated), a single method or property, or a closure (`{ @MainActor in ... }`)
- `View`, `UIView`, `UIViewController` are `@MainActor`; so are `body`, button actions and `.task` closures
- `Task { }` **inherits** the current isolation: started from a `@MainActor` method, its body runs on the main actor, so `await MainActor.run` there is redundant
- `await` inside a main-actor method suspends without blocking; the method resumes on the main actor
- Isolation makes a class `Sendable`; `@Observable` alone does not
- Swift 6.2: default MainActor isolation per module (new app targets yes, packages no); opt out with `nonisolated` or `@concurrent`
- Isolated conformances (Swift 6.2): `extension Model: @MainActor SomeProtocol` lets a main-actor type conform to a nonisolated protocol

Related to check: Actors, `nonisolated` and `@concurrent`, `Sendable`, Task inheritance, Observable macro, `@preconcurrency`

Approachable Concurrency (Swift 6.2)

Q: What is 'approachable concurrency' in Swift 6.2? What actually changes when you turn it on?

Short answer: Approachable concurrency is Swift 6.2's set of defaults that make data-race safety fit how app code is really written: mostly single-threaded on the main actor, with a few deliberate trips to the background. It's three changes.

Key points:

- Xcode's **Approachable Concurrency** setting (`SWIFT_APPROACHABLE_CONCURRENCY`) enables the upcoming features `NonisolatedNonsendingByDefault` and `InferIsolatedConformances`; in Swift 5 mode it also enables three that Swift 6 mode already includes (`DisableOutwardActorInference`, `GlobalActorIsolatedTypesUsability`, `InferSendableFromCaptures`)
- **Default Actor Isolation** (`SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor`, compiler flag `-default-isolation MainActor`) is a *separate* setting
- Xcode 27's project templates: Approachable Concurrency = Yes for every target, Default Actor Isolation = MainActor for **app** targets only, and most templates still set the Swift 5 language mode (`SWIFT_VERSION = 5.0`)
- SwiftPM packages get none of this by default: add `.enableUpcomingFeature(...)` and, for UI modules only, `.defaultIsolation(MainActor.self)`
- In a default-MainActor module, opt out per declaration with `nonisolated` (data types, DTOs, pure helpers) or `@concurrent` (CPU-heavy async work)
- Under nonsending, passing main-actor state to a `nonisolated async` helper is no longer a "sending … risks causing data races" error, because nothing leaves the actor

Related to check: `@concurrent`, `nonisolated`, MainActor, isolated conformances, Swift 6 migration, `sending` and region-based isolation, `@preconcurrency`

Isolated Conformances

Q: What are isolated conformances in Swift 6.2? What problem do they solve, and what are their limits?

Short answer: An isolated conformance (SE-0470, Swift 6.2) lets a type that's isolated to a global actor, usually `@MainActor`, conform to a protocol whose requirements are nonisolated, by declaring that the *conformance itself* is main-actor isolated: `extension SignUpViewController: @MainActor FormValidating`. Before 6.2 you had two bad options: mark every requirement `nonisolated` (and lose access to the type's state) or use `@preconcurrency` (and get a runtime trap if it's called off the main actor).

Key points:

- Syntax: put the global actor on the conformance, `extension VC: @MainActor P { ... }`; the witnesses stay main-actor isolated and can read `self`'s state
- With `InferIsolatedConformances`, `@MainActor final class Settings: Equatable` gets an isolated `Equatable` conformance without the annotation; without the feature, that line is the error "conformance … crosses into main actor-isolated code"
- To get an ordinary conformance instead, write `nonisolated` on the conformance (`extension Token: nonisolated Hashable`) and make the witnesses `nonisolated`
- The conformance can be used anywhere the code is already on the main actor: calling a generic `some P` function, storing as `any P`, ==, `Set`/`Dictionary` operations
- **Limits:** it can't be used from a nonisolated or `@concurrent` context, and it can't satisfy a generic requirement that also needs `Sendable` or `SendableMetatype` (`T: P & Sendable`), because that type could be used on another thread
- Protocols that are already `@MainActor` (most UIKit delegates and data sources) don't need this; it's for plain protocols: your own, `Equatable`, `Hashable`, `Codable`, `Identifiable`

Related to check: MainActor, approachable concurrency, `@preconcurrency`, `nonisolated`, `@concurrent`, `Sendable`, protocols and generics, delegate pattern

---

Task

Q: What is Task? How do you use it?

Short answer: A `Task` is a unit of asynchronous work, and `Task { }` is how you start async work from synchronous code such as a button action or a delegate callback. It's an *unstructured* task: it **inherits the current actor isolation, priority and task-local values**, but it isn't a child of anything, so nobody cancels it for you.

Key points:

- `Task { }` inherits actor isolation: started from a `@MainActor` method, its body runs on the main actor, so no `await MainActor.run` is needed
- It also inherits priority and task-locals; `Task.detached` inherits none of them
- The handle is `Task<Success, Failure>`: `await task.value` (or `try await` for throwing tasks), `task.cancel()`
- Dropping the handle doesn't cancel the task; it keeps running until it finishes
- `Task(name: "...") { }` gives the task a name that shows up in debugging and Instruments
- `Task.immediate { }` (iOS 26+) starts running synchronously on the current actor until its first suspension, instead of being enqueued
- In SwiftUI, prefer the `.task` modifier, which is cancelled automatically when the view disappears

Related to check: Unstructured vs structured concurrency, task groups, `async let`, cancellation, task-local values, `.task` modifier

---

Grand Central Dispatch

Q: What is Grand Central Dispatch? How do you use it?

Short answer: GCD (libdispatch) is Apple's C-level concurrency library: you submit closures to dispatch queues, and the system runs them on a thread pool it manages. Queues are serial or concurrent; `DispatchQueue.main` is serial and runs on the main thread, and the global queues are concurrent, one per QoS class.

Key points:

- `DispatchQueue.main` (serial, main thread), `DispatchQueue.global(qos:)` (concurrent), custom queues via `DispatchQueue(label:attributes:)`
- `async` returns immediately; `sync` blocks the caller until the work finishes (never `sync` onto the current queue)
- Tools: `DispatchGroup`, barriers, `DispatchWorkItem`, `DispatchSemaphore`, `DispatchSource`, `asyncAfter`
- **Swift 6:** closures passed to `async` are `@Sendable`, so captured values must be `Sendable`; hop to main for UI work
- **Modern Swift:** `DispatchQueue.main.async` -> `@MainActor`; `DispatchQueue.global().async` -> `@concurrent func`; group -> `async let` / task group; `asyncAfter` -> `Task.sleep(for:)`

Related to check: Serial vs concurrent queues, QoS, DispatchGroup, dispatch barriers, DispatchWorkItem, OperationQueue, thread vs queue, @MainActor, @concurrent, deadlock

---

OperationQueue

Q: What is OperationQueue? How does it differ from GCD?

Short answer: `OperationQueue` is Foundation's object-oriented layer over GCD: you add `Operation` objects (or `BlockOperation`), and the queue runs them respecting dependencies, priorities and a concurrency limit. Compared with raw GCD it adds dependencies between operations, `maxConcurrentOperationCount`, cancellation state, and KVO-observable progress.

Key points:

- `op2.addDependency(op1)`: `op2` starts only after `op1` finishes (even across queues)
- `maxConcurrentOperationCount`: bound parallelism (1 makes it serial)
- `cancel()` / `cancelAllOperations()`: sets `isCancelled`; operations must check it
- `qualityOfService` and `queuePriority`; `OperationQueue.main` for main-thread work
- **Modern Swift:** dependencies -> sequential `await`s; concurrency limit -> task group that adds a child as one finishes; cancellation -> `Task.cancel()`

Related to check: GCD, task groups, cancellation, DispatchWorkItem, QoS, structured concurrency

---

Actors

Q: What are actors in Swift? When would you use them?

Short answer: An actor is a reference type that protects its mutable state by serializing access: only one task runs on a given actor at a time, and code outside the actor must `await` to call it. That makes data races on its state impossible, and in Swift 6 language mode the compiler enforces it.

Key points:

- `actor Store { var state }`: inside, access is synchronous; outside, every call and property read is `await` (and you can't set its properties from outside)

- Actors are `Sendable` by construction; what you pass in and out must be `Sendable` or `sending`
- Actors are **reentrant**: while a method is suspended at an `await`, other calls can run on the actor, so state can change across an `await` (see `reentrancy.md`). This also means two actors awaiting each other don't deadlock
- Each `await` into an actor is a potential **hop** between executors; group related work in one actor method rather than many small calls
- `nonisolated` members and immutable `let`s skip isolation; an `isolated` parameter runs a whole function on an actor (see `actor-isolation-rules.md`)
- Custom executors: an actor can run on a specific `SerialExecutor` (e.g. a `DispatchSerialQueue`) by overriding `unownedExecutor`, mainly for interop with code that already owns a queue
- `isolated deinit` (Swift 6.2) lets a deinit call isolated methods; it needs **iOS 18.4+** on custom actors (it back-deploys for `@MainActor` classes)

Related to check: Actor isolation rules, actor reentrancy, MainActor, global actors, `Sendable`, `Mutex`, `nonisolated`

Sendable

Q: What is Sendable? Why does it matter for concurrency?

Short answer: `Sendable` is a marker protocol for types whose values can safely cross isolation boundaries (between actors, tasks and the main actor) without creating data races. In Swift 6 language mode the compiler checks this at compile time and reports violations as errors, not warnings.

Key points:

- No requirements; it's a compile-time promise checked by the compiler
- Structs and enums whose stored members are all Sendable are inferred `Sendable` when non-public; public types must declare it
- A `final class` can conform if all stored properties are `let` and Sendable; mutable state needs a `Mutex` (`import Synchronization`)
- Actors and `@MainActor` classes are Sendable because isolation serializes access; `@Observable` alone doesn't make a class Sendable
- `@unchecked Sendable` is a promise the compiler can't verify, for legacy lock-based classes
- Region-based isolation (SE-0414) lets a freshly created non-Sendable value be passed to another isolation domain; `sending` (SE-0430) marks parameters and results that transfer ownership

Related to check: `@Sendable` closures, actors, MainActor, `Mutex`, region-based isolation, `@preconcurrency`

Region-Based Isolation and `sending`

Q: What is region-based isolation in Swift 6, and what does the `sending` keyword do?

Short answer: Region-based isolation (SE-0414) is how the Swift 6 compiler lets a non-`Sendable` value cross an isolation boundary safely: it tracks which values can reach which others (a *region*), and allows the transfer if, at that point, nothing the sender keeps can still reach the value. A freshly created object is in its own *disconnected* region, so you can hand it to an actor; using it again afterwards is the error "sending 'x' risks causing data races".

Key points:

- Every non-`Sendable` value lives in a region that is either **disconnected**, **actor-isolated** (reachable from an actor's or `@MainActor` state), or **task-isolated** (a parameter of a nonisolated function)

- Regions **merge** when values become linked: appending a to `list` puts both in one region, so sending `list` sends a too
- Sending a value transfers its whole region; any later use in the sender is an error. Actor-isolated or task-isolated values can't be sent at all, only disconnected ones
- `Sendable` = safe to share at any time; `sending` = safe to transfer *once*; `@Sendable` = a closure that captures only `Sendable` state
- Standard library uses: `Task { }` and `Task.detached`'s `operation` closures are `sending`, `Checked–Continuation.resume(returning:)` takes a `sending` value, task groups' `addTask` closures are `sending`
- Passing a disconnected value to a `nonisolated` or `@concurrent` callee as an ordinary (non-`sending`) parameter only *lends* it: you can keep using it after the `await`

Related to check: `Sendable`, actor isolation rules, `@Sendable` closures, actors, `@concurrent` and `non–isolated(nonsending)`, continuations, Swift 6 migration

---

Thread Safety

Q: How do you achieve thread safety in Swift?

Short answer: In Swift 6 language mode, data-race safety is checked at compile time: sharing unprotected mutable state across isolation domains is an error, not a runtime surprise. So thread safety starts with choosing where each piece of state lives.

Key points:

- **Swift 6 mode:** races are compile errors; Swift 5 mode with complete checking reports warnings. Migration is per module
- **Immutable `Sendable` values:** structs and enums of `Sendable` members, `final class` with only `let` `Sendable` properties
- **`@MainActor`:** UI types and view models; the main actor serializes access on the main thread
- **Actors:** shared mutable state with async access; callers `await`
- **`Mutex` / `Atomic`** (Synchronization, iOS 18+): synchronous access from any context; a class holding one can be `Sendable`
- **Legacy:** serial `DispatchQueue`, `NSLock`, `OSAllocatedUnfairLock`; the owning class becomes `@unchecked Sendable`

Related to check: Sendable, actors, @MainActor, NSLock and Mutex, race conditions vs data races, deadlock, Thread Sanitizer, serial vs concurrent queues

---

Retain Cycles in Async Code

Q: How can you avoid retain cycles in async code?

Short answer: A `Task { }` closure captures `self` **strongly** (you don't even have to write `self.`), and keeps it alive until the task completes. For short work that's fine: the reference is released when the task finishes, so it's a delayed release, not a leak.

Key points:

- `Task {}` captures `self` strongly until completion; so does `Task.detached`
- Short one-shot tasks: strong capture is usually fine (and often wanted, e.g. a save that must finish)
- Long-running or infinite tasks: hold the handle and cancel it; cancellation ends `for await` loops and `Task.sleep`
- `[weak self]` followed by `guard let self` **at the top** re-strongs `self` for the whole task, so nothing changes; unwrap inside each iteration or after each `await` instead

- `deinit { task?.cancel() }` only runs if the task doesn't hold `self` strongly
- SwiftUI's `.task` modifier is cancelled when the view disappears, which avoids the problem for views

Related to check: ARC, closures and capture lists, `Task`, cancellation, `.task` modifier, unstructured concurrency

---

TaskGroup

Q: What is TaskGroup? When would you use it?

Short answer: A task group runs a **dynamic number of child tasks** concurrently and lets you collect their results as they finish. You create one with `withTaskGroup` (or `withThrowingTaskGroup` when children can throw), add children with `group.addTask`, and iterate the group with `for await`.

Key points:

- `withTaskGroup { group in ... }`: since Swift 6.1 the child result type is inferred, so `of: User.self` is optional
- `group.addTask { }` starts a child; `for await result in group` yields results **in completion order**, not submission order
- All children are awaited before the closure returns, even ones you never iterated
- Cancellation of the parent propagates to every child; `group.cancelAll()` cancels them explicitly
- `withDiscardingTaskGroup` for fire-and-forget children whose results you don't need (no result buffering, so it suits long-running loops like servers or listeners)
- Limit concurrency with a **sliding window**: start *n* children, then add one each time one finishes

Related to check: Throwing task groups, `async let`, structured concurrency, cancellation, `Task`

---

async let

Q: What is async let? How does it differ from TaskGroup?

Short answer: `async let` starts a child task immediately and lets you `await` its result later, so a few independent calls run in parallel with almost no ceremony. It's structured concurrency: the child is tied to the enclosing scope, inherits priority, task-locals and cancellation, and can't outlive that scope.

Key points:

- `async let x = work()` starts `work()` as a child task right away; `try await x` reads the result (once)
- Several `async let`s run concurrently; awaiting them one after another still finishes in roughly the time of the slowest
- Each binding can have its own type; a task group needs one child result type
- Leaving the scope without awaiting is allowed: the child is **implicitly cancelled and then awaited**, so `async let` is never fire-and-forget
- Cancelling the parent cancels every `async let` child
- Only in async contexts; the binding can't escape (no storing it, no capturing it in an escaping closure)

Related to check: Structured concurrency, task groups, `Task`, cancellation, async throws, task-local values

---

Continuations

Q: What are continuations? How do you bridge callback-based APIs to async/await?

Short answer: A continuation is a handle to a suspended task. `withCheckedContinuation` (or `withCheckedThrowingContinuation`) suspends the current task, hands you the continuation, and the task resumes when the callback calls `resume(returning:)`, `resume(throwing:)` or `resume(with: result)`.

Key points:

- Checked continuations trap on a second resume and log a warning if one is destroyed without being resumed; the cost is tiny, so use them by default
- `withUnsafeContinuation` skips those checks; use it only for measured hot paths once the code is proven
- `resume(with: Result)` maps a `Result`-based callback in one line
- The continuation is `Sendable`, so it can be resumed from any thread or queue
- Results are `sending`: `resume(returning:)` accepts a non-Sendable value only if nothing else still refers to it (a value created fresh in the callback is fine)
- The body closure runs synchronously on the **caller's** actor (Swift 6.0 added an `isolation: #isolation` parameter; current SDKs declare the functions `nonisolated(nonsending)`), so non-Sendable state isn't forced across a boundary
- Many SDK APIs already have async versions (`URLSession.data(for:)`, `UNUserNotificationCenter`, `CLLocationUpdate.liveUpdates()`); check before wrapping

Related to check: Sync to async bridge, cancellation, AsyncStream, `Sendable` and `sending`, async throws

---

Race Conditions and Data Races

Q: What's the difference between a race condition and a data race?

Short answer: A data race is two threads accessing the same memory at the same time, at least one of them writing, with no synchronization; it's undefined behavior. A race condition is a logic bug where the result depends on timing or ordering, such as check-then-act, and it can happen even when every access is properly synchronized.

Key points:

- **Data race:** memory-level, unsynchronized concurrent access with a write; undefined behavior (crashes, torn values)
- **Race condition:** logic-level, outcome depends on interleaving; possible with actors and locks
- **Swift 6 mode:** unprotected shared mutable state (like a global `var` mutated from GCD closures) is a compile **error**
- **Fixes for data races:** actor, `Mutex`, `Atomic`, or not sharing (immutable `Sendable` values)
- **Runtime data races are still possible** through `@unchecked Sendable`, `nonisolated(unsafe)`, unsafe pointers, and C/ObjC code; Thread Sanitizer finds them

Related to check: Thread safety, actors and reentrancy, Sendable, NSLock and Mutex, Thread Sanitizer, deadlock

---

Structured Concurrency

Q: What is structured concurrency?

Short answer: Structured concurrency means every concurrent task is a **child of a scope**, like a variable's lifetime is tied to a block: the scope can't finish until its children have finished. In Swift that's `async let` (a fixed number of children) and task groups (a dynamic number).

Key points:

- `async let` starts a child immediately; you `await` it later. Leaving the scope without awaiting cancels it and then waits for it
- `withTaskGroup` / `withThrowingTaskGroup` for N children; `withDiscardingTaskGroup` when you don't need results
- Cancelling the parent cancels every child (cooperatively); a thrown error cancels siblings
- Children inherit priority and task-local values from the parent
- SwiftUI's `.task` modifier gives you the same lifetime guarantees at the view level
- `Task {}` and `Task.detached {}` are unstructured: no parent, no automatic cancellation

Related to check: `async let`, task groups, unstructured concurrency, cancellation, task-local values, `.task` modifier

---

Task Cancellation

Q: How does task cancellation work? How do you handle it?

Short answer: Call task.cancel() to request cancellation. Cancellation is cooperative - it only sets a flag; your code must check Task.isCancelled or call try Task.checkCancellation() and stop.

Key points:

- task.cancel() requests cancellation
- Check Task.isCancelled in loops
- CancellationError thrown only by code that checks (Task.checkCancellation, Task.sleep, URLSession)
- Child tasks are cancelled when parent is
- Async cleanup: `defer { await connection.close() }` is allowed since Swift 6.4 (SE-0493)
- Cleanup that must finish even though the task is cancelled: wrap it in `withTaskCancellation–Shield { }` (iOS 27+, SE-0504); inside it, `Task.isCancelled` reads false and cancellation-checking calls don't throw

Related to check: Task, TaskGroup, structured concurrency

Serial vs Concurrent Queue

Q: What's the difference between serial and concurrent queues in GCD?

Short answer: A serial queue runs one block at a time in FIFO order; a concurrent queue starts blocks in FIFO order but can run many at once, so they can finish in any order. The main queue is serial; the global queues are concurrent.

Key points:

- **Serial:** one at a time, FIFO; `DispatchQueue(label:)` is serial by default; `DispatchQueue.main` is serial
- **Concurrent:** `attributes: .concurrent` or `DispatchQueue.global()`; start order is FIFO, completion order isn't
- A serial queue isn't a thread: consecutive blocks may run on different threads (except main)
- Serial queue as a lock: `sync` for reads, `async` for writes; concurrent queue plus barriers for reader - writer

- **Modern Swift:** serial queue protecting state -> `actor` or `Mutex`; concurrent queue for parallel work -> task group or `@concurrent` functions

Related to check: GCD, dispatch barriers, actors, NSLock and Mutex, thread vs queue, thread safety, task groups

---

Dispatch Barrier

Q: What is a dispatch barrier? When would you use it?

Short answer: A barrier block submitted to a custom concurrent queue waits for every block submitted before it to finish, runs alone, and only then lets later blocks start. That gives you the reader - writer pattern: reads run in parallel with `sync`, writes go through `async(flags: .barrier)`.

Key points:

- `queue.async(flags: .barrier) { }` (or `sync(flags: .barrier)`); C API `dispatch_barrier_async`
- Only works on a **custom concurrent** queue; on a global queue it's an ordinary block, on a serial queue it adds nothing
- Barrier waits for earlier blocks and holds back later ones
- Reader - writer: concurrent `sync` reads, exclusive barrier writes
- **Modern Swift:** actor (async callers) or `Mutex` (sync callers) instead

Related to check: Serial vs concurrent queues, GCD, NSLock and Mutex, actors, thread safety

---

DispatchSemaphore

Q: What is DispatchSemaphore? When would you use it?

Short answer: `DispatchSemaphore` is a GCD counting semaphore: `wait()` decrements the count and **blocks the calling thread** while it's zero, and `signal()` increments it and wakes a waiter. It was the classic way to cap concurrent work (e.g. at most 3 downloads) or to make an async callback "synchronous".

Key points:

- `DispatchSemaphore(value: n)` (0 to signal between threads, `n` to cap concurrency); `wait()` blocks if the count is 0, `signal()` increments; always pair them (`defer { signal() }`)
- The cooperative pool has roughly one thread per CPU core and assumes tasks never block; `wait()` breaks that promise
- Swift 6: `wait()` is unavailable from async contexts (a compile error), and the compiler suggests awaiting a task instead
- There's no `AsyncSemaphore` in the standard library or Foundation; third-party packages provide one
- Modern alternatives: sliding-window `withTaskGroup` (see TaskGroup), an actor that hands out slots, or `Mutex` for short synchronous critical sections

Related to check: TaskGroup (sliding window), actors, `Mutex`, `DispatchGroup`, deadlock, cooperative thread pool

---

Combine Publishers and async/await

Q: How do you use Combine publishers with async/await?

Short answer: Every Combine publisher has a `.values` property that turns it into an `AsyncSequence`: `AsyncPublisher` when `Failure == Never` (iterate with `for await`) and `AsyncThrowingPublisher`

otherwise (`for try await`). Cancelling the task that runs the loop cancels the underlying subscription, so you don't keep an `AnyCancellable`.

Key points:

- `publisher.values`: `for await v in model.$query.values { }`; breaking out of the loop or cancelling the task cancels the subscription
- First value only: `for try await v in publisher.values { return v }`
- `.values` requests **one value at a time**; a `PassthroughSubject` or `@Published` that emits while the loop body is busy has its values **dropped**. Add `.buffer(size:prefetch:whenFull:)` before `.values` when every value matters
- Combine operators (`debounce`, `removeDuplicates`, `combineLatest`) can be applied before `.values`; the pure-async equivalents live in the Swift Async Algorithms package
- There's no built-in AsyncSequence -> Publisher; feed a subject from a `Task` if legacy code needs a publisher
- Combine is legacy for observation: `ObservableObject` + `@Published` is "older code / iOS 16 and earlier"; `@Observable` doesn't produce publishers
- `Observations { model.query }` (Swift 6.2, iOS 26+) is an `AsyncSequence` of transactional changes to `@Observable` properties

Related to check: AsyncSequence, AsyncStream, Observation (`@Observable`, `Observations`), Combine, cancellation, `.task` modifier

---

AsyncSequence

Q: What is AsyncSequence? How do you use it?

Short answer: `AsyncSequence` is the asynchronous counterpart of `Sequence`: each element may take time to arrive, so you iterate with `for await` (or `for try await` when it can throw), and the loop suspends between elements without blocking a thread. Foundation ships many of them (`url.lines`, `URLSession.bytes(for:)`, `NotificationCenter.notifications(named:)`), you get the familiar operators (`map`, `filter`, `prefix`, `first(where:)`), and you build your own with `AsyncStream` or by conforming to the protocol.

Key points:

- `for await x in seq { }`; `for try await` when the sequence can throw
- Since Swift 6.0, `AsyncSequence` has a `Failure` associated type, so you can write `some AsyncSequence<Int, Never>`
- Built-in operators are lazy: `seq.filter { ... }.map { ... }.prefix(10)` does nothing until iterated
- Create one with `AsyncStream.makeStream(of:)` (push) or `AsyncStream(unfolding:)` (pull), or conform to `AsyncSequence` + `AsyncIteratorProtocol`
- Cancellation: the consuming task's cancellation ends iteration of well-behaved sequences; your own iterators should check it
- `Observations { model.value }` (iOS 26+) turns `@Observable` state into an async sequence

Related to check: AsyncStream, AsyncIteratorProtocol, `Observations`, Combine publishers, cancellation, `.task` modifier

---

Global Actors

Q: What are global actors? How do you create a custom one?

Short answer: A global actor is a single, app-wide actor whose isolation you can attach to declarations *anywhere*: types, methods, properties, global variables and closures. An ordinary actor protects the state inside one instance; a global actor protects state spread across many types and files, and everything marked with it shares one serial domain, so those pieces can call each other synchronously.

Key points:

- Declaration: `@globalActor actor AudioActor { static let shared = AudioActor() }` (the `GlobalActor` protocol requires `shared`)
- Usage: `@AudioActor` on a type, member, global/static variable or closure; how that applies is covered in `global-actor-isolation.md`
- Everything isolated to the same global actor is one domain: synchronous calls between them, `await` from anywhere else
- Classes isolated to a global actor are `Sendable`
- `MainActor` is itself declared as a `@globalActor final actor`, with an executor bound to the main thread
- Instance actor vs global actor: two `Cache()` instances can run in parallel; everything on `@AudioActor` is serialized together
- Because actors are reentrant, global actors don't deadlock when they await each other, but they also don't keep state consistent across an `await`

Related to check: Global actor isolation, MainActor, actors, actor isolation rules, custom executors, default actor isolation

---

nonisolated

Q: What does nonisolated mean? When would you use it?

Short answer: `nonisolated` opts a declaration out of its enclosing actor's isolation, so callers can use it synchronously without `await`, but it can no longer touch that actor's mutable state. I use it for members that don't need the actor (pure helpers, immutable `let`s, synchronous protocol requirements like `description` or `hash(into:)`), and in a Swift 6.2 default-MainActor module it's how I mark code that shouldn't be on the main actor.

Key points:

- On an actor or `@MainActor` member: callable synchronously from anywhere; accessing isolated state is a compile error
- Immutable `let` properties of a `Sendable` type can be `nonisolated`
- Swift 6.1: `nonisolated` on a whole type or extension (`nonisolated struct Point`, `nonisolated extension Store`) applies to all members
- Swift 6.2 (`NonisolatedNonsendingByDefault`): a plain `nonisolated async func` runs on the **caller's** actor; spell it `nonisolated(nonsending)` explicitly if you like
- `@concurrent` marks an async function that always runs on the concurrent pool, off any actor (the modern answer for CPU work)
- `nonisolated(unsafe)` turns off checking for a stored variable: a last resort when you guarantee synchronization yourself

Related to check: Actors, MainActor, `@concurrent`, `Sendable`, default actor isolation, isolated conformances

---

@concurrent

Q: What does `@concurrent` do? In Swift 6.2, how do you run work off the main actor?

Short answer: `@concurrent` (Swift 6.2, SE-0461) marks an `async` function or closure that always runs on the global concurrent executor, off whatever actor called it. It exists because Swift 6.2 flipped the default: with *Approachable Concurrency*, a plain `nonisolated async` function runs on the **caller's** actor, so "async" no longer implies "background".

Key points:

- Only valid on `async` functions and closures ("cannot use @concurrent on non-async global function")
- It *is* an isolation: it can't be combined with `@MainActor` or another global actor, and on a member of an actor or `@MainActor` type it makes that member nonisolated, so it can't touch the type's state
- Arguments and results cross an isolation boundary, so they must be `Sendable` or in a disconnected region; passing main-actor-owned non-`Sendable` state is an error ("sending 'self.x' risks causing data races")
- It doesn't create a task; it's a function hop inside the current task, so priority, task-locals and cancellation carry over
- Closures: `Task { @concurrent in ... }` and `group.addTask { @concurrent in ... }` are valid when you need an unstructured or child task that starts off the actor
- Under the old (pre-6.2 or feature-off) rules, `nonisolated async` already ran on the pool; `@concurrent` states that intent explicitly and keeps it when nonsending is enabled

Related to check: Approachable concurrency, `nonisolated` and `nonisolated(nonsending)`, MainActor, `Task.detached` and unstructured tasks, task groups, `Sendable` and `sending`, cancellation

---

Deadlock

Q: What causes deadlock? How do you avoid it?

Short answer: A deadlock is two or more pieces of work each waiting for something the other holds, so none can proceed. In GCD the classic cases are `sync` onto the serial queue you're already on (including `DispatchQueue.main.sync` from the main thread) and two locks acquired in opposite orders.

Key points:

- **Same-queue `sync`:** the outer block waits for the inner, which waits for the outer; recent libdispatch often detects this and traps instead of hanging
- **Lock ordering:** thread 1 holds A and wants B, thread 2 holds B and wants A; fix with one global acquisition order
- **Non-recursive locks:** re-locking a `Mutex` or `NSLock` on the same thread deadlocks
- **Actors:** `await` releases the actor while suspended, so actors calling each other don't deadlock (they can have reentrancy bugs instead)
- **Blocking async code:** `DispatchSemaphore.wait()`, `DispatchGroup.wait()`, `Thread.sleep` in async code can starve the cooperative pool (one thread per core) or block the main actor

Related to check: GCD, serial vs concurrent queues, NSLock and Mutex, semaphores, actors and reentrancy, MainActor.assumeIsolated, sync/async bridging

---

Quality of Service

Q: What is QoS in GCD? What are the priority levels?

Short answer: Quality of Service tells the system how important work is to the user, so it can schedule CPU time, I/O and energy accordingly. The classes, highest first, are `userInteractive`, `userInitiated`, `default`, `utility` and `background`.

Key points:

- **userInteractive:** main-thread work, animations, event handling; must finish almost instantly
- **userInitiated:** the user tapped and is waiting (opening a document, loading a screen)
- **default:** fallback when nothing is specified
- **utility:** long-running work with visible progress (downloads, imports)
- **background:** invisible maintenance (prefetching, backups, indexing); may be heavily throttled, especially in Low Power Mode
- **Modern Swift:** `Task(priority: .utility) { }`, `group.addTask(priority:)`, `Task.currentPriority`; `Task { }` inherits the caller's priority

Related to check: Task priority, GCD, Task vs Task.detached, @concurrent, deadlock and priority inversion, energy efficiency

---

Actor Reentrancy

Q: What is actor reentrancy? Why does it matter?

Short answer: When an actor method hits an `await`, it suspends and gives the actor back, so other calls can run on the same actor before it resumes. That's reentrancy: the actor still never runs two pieces of code *at the same time*, but state you read before the `await` may have changed by the time you continue.

Key points:

- Every `await` inside an actor method is an interleaving point; code between two `await`s runs atomically
- Data races are still impossible; what you lose is *invariants across a suspension* (a logic race, not a memory race)
- Re-validate after `await`: re-read the cache, re-check flags, don't trust local copies of actor state
- Deduplicate expensive work by storing a `Task` in a dictionary before awaiting it; later callers `await` the same task
- Reentrancy applies to `@MainActor` too (see `reentrancy-actors.md` for the design side and other patterns)

Related to check: Reentrancy and actors (design patterns), actors, `Task`, cancellation, suspension points, image caching

---

RunLoop and Main Thread

Q: What is the RunLoop? How does it relate to the main thread?

Short answer: A run loop is an event-processing loop attached to a thread: it waits for input sources (touches, ports, `perform` requests) and timers, dispatches them, and sleeps when there's nothing to do. Every thread can have one, created lazily, but only the main thread's run loop runs automatically; it drives UI events, `Timer`s, `CADisplayLink`, and drains the main dispatch queue, which is also the main actor's executor.

Key points:

- `RunLoop.main` processes UI events, timers, display links and main-queue work (so `@MainActor` code runs there too)
- Secondary threads have a run loop only if you access it, and it does nothing unless you run it
- **Modes:** `.default`; `.tracking` while scrolling; `.common` covers both. A timer added in `.default` pauses during scrolling
- `Timer.scheduledTimer` adds to the current run loop in `.default` mode
- **Modern Swift:** UI work belongs on `@MainActor`, not `DispatchQueue.main.async`; periodic async work can be a `Task` loop with `Task.sleep(for:)`

Related to check: Main thread and @MainActor, GCD, Timer, CADisplayLink, thread vs queue, UI responsiveness

---

Thread vs Queue

Q: What's the difference between a thread and a queue in GCD?

Short answer: A thread is an OS execution context with its own stack; a dispatch queue is a scheduling abstraction, an ordered list of work that GCD runs on threads from a pool it manages. You submit closures to queues and GCD decides which thread runs them, so a queue is not a thread: a serial queue runs one block at a time, but not necessarily always on the same thread.

Key points:

- **Thread:** OS-scheduled, has a stack and optionally a run loop; `Thread` / pthreads, you manage its lifetime
- **Queue:** FIFO list of work; serial or concurrent; GCD maps it onto pool threads
- Only the main queue is tied to one thread (the main thread)
- **Swift Concurrency:** tasks and actors are not threads; isolation is the guarantee, not thread identity (only `@MainActor` maps to a specific thread)
- In Swift 6, `Thread.current` is unavailable in async contexts, because the answer can change at the next `await`

Related to check: GCD, serial vs concurrent queues, RunLoop, @MainActor, actors, task-local values, @concurrent

---

Delaying Work: Task.sleep vs asyncAfter

Q: How do you delay execution in Swift? asyncAfter vs Task.sleep?

Short answer: In modern Swift you delay work by suspending the current task: `try await Task.sleep(for: .seconds(1))`. It doesn't block a thread, it respects actor isolation (after the sleep you're back on the same actor), and it's **cancellable**: cancelling the task makes the sleep throw `CancellationError` right away, so the delayed work never runs.

Key points:

- `try await Task.sleep(for: .seconds(1))` / `.milliseconds(300)`: a `Duration`, measured on `ContinuousClock`
- `Task.sleep(until: .now + .seconds(2), clock: .continuous)` for a deadline; `tolerance:` lets the system coalesce timers
- `ContinuousClock` keeps counting while the device sleeps; `SuspendingClock` doesn't
- Cancellation: store the `Task` handle and call `cancel()`; the sleep throws and the rest of the task is skipped
- Legacy: `DispatchQueue.main.asyncAfter(deadline: .now() + 1) { ... }`; cancel via `DispatchWorkItem.cancel()`
- In SwiftUI, `.task(id:)` restarts and cancels delayed work automatically (debounced search)

Related to check: Task, cancellation, `Clock` / `Duration`, `DispatchWorkItem`, `Timer`, `.task(id:)` modifier

---

Thread-Safe Singleton

Q: How do you implement a thread-safe singleton?

Short answer: In Swift, `static let shared = Manager()` is lazily initialized exactly once, and the runtime makes that initialization thread-safe (it uses `swift_once`, the equivalent of `dispatch_once`). In Swift 6 language mode that's only half the answer: the compiler also checks that the *instance* is safe to share, so `static let shared` must hold a `Sendable` type or be isolated to a global actor, and `static var shared` is an error.

Key points:

- `static let` initialization is lazy, one-time and thread-safe; no double-checked locking needed
- Swift 6: `static let shared` of a non-Sendable class is an error ("non-'Sendable' type … may have shared mutable state")
- `static var shared` is an error ("nonisolated global shared mutable state") unless it's `@MainActor`-isolated or `nonisolated(unsafe)`
- Fixes: make the class `@MainActor`, make it an `actor`, or make it `Sendable` with immutable state or `Mutex`-protected state
- `nonisolated(unsafe)` exists but is a last resort; you're promising your own synchronization
- In a Swift 6.2 default-MainActor app target, an unannotated singleton is `@MainActor` automatically

Related to check: Actors, MainActor, `Sendable`, `Mutex`, dependency injection, lazy initialization

---

async throws

Q: How do async and throws work together?

Short answer: `async` and `throws` are independent effects that compose: an `async throws` function can suspend *and* fail, and the caller acknowledges both with `try await`. Errors propagate across suspension points exactly as they do in synchronous code, up through callers, child tasks and task groups, until a `do`/`catch` handles them.

Key points:

- Declaration order is `async throws`; call-site order is `try await` (`await try` only gets a warning and a fix-it)
- Errors cross suspension points and actor hops unchanged; a `catch` after an `await` sees the original error
- In structured concurrency, child errors propagate to the parent: `async let` at the `await`, task groups through `next()` / iteration
- Unstructured tasks capture the error in their type: `Task<Feed, any Error>`, surfaced by `try await task.value`
- Cancellation usually arrives as a thrown `CancellationError` (from `Task.sleep`, `URLSession`, `Task.checkCancellation()`); treat it as "stop", not as a failure to show
- Typed throws: `async throws(FeedError)` gives callers an exhaustive error type, but you must map every underlying error (network, decoding, cancellation) into it

Related to check: Error handling and typed throws, async/await, `async let`, task groups, cancellation, continuations

---

Task Local Values

Q: What are task-local values? When do you use them?

Short answer: A task-local value is a piece of context bound to the current task for the duration of a scope, readable by any code that runs in that task without passing it as a parameter. You declare it with `@TaskLocal static var`, bind it with `$value.withValue(x) { ... }`, and read it like a normal static property; outside any binding you get the default.

Key points:

- Declare: `@TaskLocal static var requestID: String?` (static or global; needs a default, here `nil`)
- Bind: `try await Context.$requestID.withValue("req-42") { ... }`; the binding ends when the closure returns, and nested `withValue` calls shadow outer ones
- Read anywhere, including synchronous code called from the task: `Context.requestID`
- Inherited by child tasks (`async let`, `addTask`) and copied by `Task { }`; not by `Task.detached`
- Values are immutable within a binding; to "change" one, bind a new value for a narrower scope
- The value should be `Sendable`, since child tasks read it concurrently

Related to check: Task, structured concurrency, `async let`, task groups, task priority, logging, Swift Testing parallelism

---

Task.yield

Q: What does Task.yield do? When would you use it?

Short answer: `await Task.yield()` voluntarily suspends the current task and puts it back in the queue, so other tasks waiting for the same executor get a chance to run before it continues. That matters because Swift Concurrency is *cooperative*: the global pool has roughly one thread per CPU core, and a task only gives up its thread at a suspension point.

Key points:

- Suspends and reschedules the task; if nothing else is waiting, it may resume almost immediately
- Useful in long CPU-bound loops with no other `await` (parsing, indexing, image processing)
- Doesn't throw and doesn't check cancellation; pair it with `try Task.checkCancellation()`
- On an actor (including `@MainActor`), yielding lets other queued calls run on that actor, which means **reentrancy**: state can change across the yield
- Pointless in loops that already suspend on real async work (network, file I/O)
- Not a replacement for moving heavy work off the main actor: put it in a `@concurrent` function first, then yield inside it if needed

Related to check: Cooperative thread pool, `@concurrent`, cancellation, actor reentrancy, task groups, async to sync bridge

---

Unstructured Concurrency

Q: What is unstructured concurrency? When do you use it?

Short answer: Unstructured concurrency means creating a task with `Task { }` or `Task.detached { }`: the new task has no parent, so it isn't cancelled when the code that created it is cancelled, and nothing waits for it to finish. It's the right tool at a **sync->async boundary** (a button action, a delegate callback, `viewDidLoad`) where there's no async context to attach a child to.

Key points:

- `Task { }` inherits actor isolation, priority and task-locals but not cancellation; `Task.detached` inherits nothing

- No automatic cancellation and no automatic awaiting: the creator must hold the `Task` handle to cancel it or read `value`
- Legitimate uses: UI event handlers, bridging from delegates/callbacks, work that must outlive the current scope (e.g. a save that should finish after the screen closes)
- In SwiftUI use `.task { }` / `.task(id:)`: it's still started for you, but cancelled when the view disappears or the id changes
- The operation closure is `sending @isolated(any)`: the compiler checks captured values can be handed over, and the closure carries its isolation so the task can be enqueued directly on the right actor

Related to check: Structured concurrency, `Task`, task groups, `async let`, cancellation, retain cycles in async code

---

AsyncStream

Q: How do you use AsyncStream? When is it useful?

Short answer: `AsyncStream` is the easiest way to turn push-based code (delegates, callbacks, notifications, sensor updates) into an `AsyncSequence` you consume with `for await`. The modern way to create one is `let (stream, continuation) = AsyncStream.makeStream(of: T.self)`: the producer calls `continuation.yield(_:)` and `finish()`, and `continuation.onTermination` tells you when the consumer stopped so you can tear down the underlying source.

Key points:

- `AsyncStream.makeStream(of:bufferingPolicy:)` returns the stream and its continuation as a pair, so the producer can live outside the builder closure
- The continuation is `Sendable`; it's safe to call `yield` from any thread or callback
- `onTermination` fires on `finish()`, on consumer cancellation, or when the stream is deallocated; unregister observers there
- Buffering: `.unbounded` (default), `.bufferingNewest(n)` (keep the latest, e.g. location), `.bufferingOldest(n)`; `yield` returns `.enqueued`, `.dropped` or `.terminated`
- Designed for **one consumer**; for several, create a stream per subscriber
- `AsyncStream(unfolding:)` for pull-based producers (each `next()` calls your closure)

Related to check: AsyncSequence, AsyncThrowingStream, continuations, `Observations`, Combine publishers, cancellation

---

MainActor.assumeIsolated

Q: What is MainActor.assumeIsolated? When is it safe?

Short answer: `MainActor.assumeIsolated { }` synchronously asserts that the current code is already running on the main actor and, inside the closure, lets you touch `@MainActor` state without `await`. It's for nonisolated *synchronous* callbacks that are documented to arrive on the main thread (delegates, C callbacks, framework notifications) where the compiler can't see that guarantee.

Key points:

- Synchronous, no suspension: the closure runs right now, on the current thread, and can return a value
- Runtime check: a crash if called off the main actor, which is better than a silent data race
- Safe only when the caller *guarantees* main-thread delivery (documented API, delegate queue set to `.main`)

- `MainActor.preconditionIsolated()` / `assertIsolated()` check without granting access; every actor has `assumeIsolated` / `preconditionIsolated` too (useful with custom executors)
- Alternatives: `extension VC: @MainActor Delegate` (isolated conformance, Swift 6.2) for Swift callers, or `@preconcurrency` conformance (Swift 6.0), where the compiler inserts the same runtime check for you
- If the callback can arrive on any thread, don't assume: hop with `Task { @MainActor in ... }`

Related to check: MainActor, `nonisolated`, isolated conformances, `@preconcurrency`, actor isolation rules, custom executors

---

@Sendable Closures

Q: When must closures be Sendable?

Short answer: A closure must be `@Sendable` when it can be called concurrently or from a different isolation domain than the one that created it, for example a completion handler that a framework calls on an arbitrary queue. A `@Sendable` closure can only capture Sendable values, and it can't capture mutable local variables.

Key points:

- `@Sendable` function types: captures must be Sendable; captured `var`s are an error; calls may happen concurrently
- Required for escaping callbacks that cross isolation (many framework completion handlers are `@Sendable`)
- `Task { }` / `addTask`: `sending @escaping @isolated(any)` closures; captured values are *transferred*, checked by region-based isolation
- `Task { }` inherits the current actor, so inside `@MainActor` code it can capture `self` and main-actor state
- Two task-group children can't both capture the same non-Sendable value: it would be shared between concurrent tasks
- `withCheckedContinuation`'s body is a plain non-escaping closure that runs immediately; the `CheckedContinuation` is Sendable, so you can capture it in a `@Sendable` callback
- `@isolated(any)` closures expose their isolation via `.isolation`

Related to check: Sendable, `sending`, region-based isolation, Task, TaskGroup, continuations, `@preconcurrency`

Cooperative Cancellation

Q: How do you implement cooperative cancellation?

Short answer: Cancellation in Swift is cooperative: `task.cancel()` only sets a flag, and your code decides where it's safe to stop. You add checkpoints with `try Task.checkCancellation()` (throws `CancellationError`) or `Task.isCancelled` (return early or return a partial result), clean up on the way out, and let the error propagate so callers and child tasks stop too.

Key points:

- Checkpoints: `try Task.checkCancellation()` to bail out with `CancellationError`, `Task.isCancelled` to stop gracefully
- Check at natural boundaries (each item, each page, every N iterations of a hot loop), not inside every line
- Cleanup goes in `defer`; since Swift 6.4 it can `await` (SE-0493)
- Cleanup that must not be cancelled itself: `withTaskCancellationShield { }` (iOS 27+, SE-0504)

- `withTaskCancellationHandler` reacts immediately, e.g. to cancel a wrapped callback-based request
- Structured children are cancelled with their parent; unstructured tasks need their owner to call `cancel()`

Related to check: Task.cancel, CancellationError, TaskGroup

Actor Isolation Rules

Q: What are the actor isolation rules?

Short answer: Every piece of code and mutable state in Swift 6 belongs to an *isolation domain*: a specific actor instance, a global actor such as `@MainActor`, or nonisolated. Code may touch an actor's mutable state synchronously only while it is running on that actor; from anywhere else it has to `await`, which hops onto the actor, and anything it passes across must be `Sendable` or provably transferred.

Key points:

- Three kinds of domain: an actor instance (`self` inside an `actor`), a global actor (`@MainActor`, custom `@globalActor`), or nonisolated
- Inside the domain, access is synchronous; from outside, every call or mutable-property read is an `await` (a possible suspension point)
- You can never *set* an actor's property from outside; call a method on the actor instead
- Immutable `let` properties of a `Sendable` type can be read without `await` within the same module; `nonisolated` members are callable synchronously but can't touch isolated state
- An `isolated` parameter (`func f(_ a: isolated Ledger)`) makes the whole function run on that actor; `isolated (any Actor)? = #isolation` makes it run on the **caller's** actor
- Values crossing domains must be `Sendable`, or non-`Sendable` values in a disconnected region (SE-0414) that are `sending` (SE-0430) and never used again by the sender
- Swift 6.2: `nonisolated async` functions run on the caller's actor (`nonisolated(nonsending)`), and `@concurrent` runs them off any actor

Related to check: Actors, MainActor, `nonisolated` and `@concurrent`, `Sendable` and `sending`, global actor isolation, actor reentrancy, `MainActor.assumeIsolated`

Global Actor Isolation

Q: How does global-actor isolation apply to types, members, globals and closures? What gets inferred?

Short answer: You declare an actor with the `@globalActor` attribute and a `static let shared` instance, and the actor's name becomes an attribute you can put on declarations: `@DatabaseActor`. The interesting part is how that isolation *applies*: on a type it covers every member, on a method or property just that one, on a global variable it protects that variable, and on a closure or function type it says where that closure runs.

Key points:

- Creation: `@globalActor actor DatabaseActor { static let shared = DatabaseActor(); private init() {} }`
- On a **type**: all members, including stored properties and initializers, are isolated; opt individual members out with `nonisolated`
- On a **member** of an otherwise unisolated type: only that method or property
- On a **global or static variable**: the usual Swift 6 fix for mutable global state, alongside `@MainActor`, `Mutex` or `let`

- On a **closure** or function type: `Task { @DatabaseActor in ... }`, `let job: @DatabaseActor @Sendable () -> Void`
- **Inference:** subclasses inherit the class's global actor; extensions of an isolated type are isolated; conforming to a `@DatabaseActor` protocol *in the type's own declaration* makes the whole type `@DatabaseActor` (conforming in an extension doesn't); property wrappers no longer infer isolation in Swift 6 (SE-0401)
- **Isolated conformances** (Swift 6.2): `extension RecordStore: @DatabaseActor Persisting` lets an isolated type satisfy a nonisolated protocol
- Classes isolated to a global actor are `Sendable`

Related to check: Global actors, MainActor, actor isolation rules, `nonisolated`, isolated conformances, default actor isolation

---

AsyncIteratorProtocol

Q: What is AsyncIteratorProtocol? How does it relate to AsyncSequence?

Short answer: `AsyncSequence` is the thing you loop over; `AsyncIteratorProtocol` is the cursor that actually produces the values. A sequence's only job is `makeAsyncIterator()`, and the iterator's only job is `mutating func next() async -> Element?`, returning `nil` when it's finished.

Key points:

- `AsyncSequence` -> `makeAsyncIterator()`; `AsyncIteratorProtocol` -> `mutating func next() async throws -> Element?`
- `for await` / `for try await` desugars to `var it = seq.makeAsyncIterator(); while let x = await it.next() { ... }`
- `associatedtype Failure` (Swift 6.0, iOS 18): inferred from your `next()`: non-throwing means `Never`, `throws(ParseError)` means `ParseError`. That's what makes `some AsyncSequence<Int, Never>` and typed `catch` work
- Swift 6.0 added `next(isolation:)` with a default implementation; `for await` calls it so iteration stays on the caller's actor and non-Sendable iterators don't have to cross isolation. When you wrap another iterator, forward it: `await base.next(isolation: #isolation)` (plain `base.next()` is a "sending 'self.base' " error under Swift 6.2's `NonisolatedNonsendingByDefault`)
- Iterators are usually single-pass and owned by one task; the iterator itself is typically not `Sendable`
- For push-based sources (delegates, callbacks), don't hand-write an iterator: use `AsyncStream.makeStream(of:)`

Related to check: AsyncSequence, AsyncStream, typed throws, cancellation, actor isolation rules (`#isolation`), Combine publishers

---

TaskGroup and ThrowingTaskGroup

Q: When do you use ThrowingTaskGroup?

Short answer: Use `withThrowingTaskGroup` when child tasks can throw. When you iterate with `for try await` (or call `try await group.next()`), the first child error is rethrown in the parent; if that error leaves the group's closure, the group **cancels all remaining children** and waits for them before the error reaches the caller.

Key points:

- `try await withThrowingTaskGroup { group in ... }`; child type is inferred since Swift 6.1 (`of:` optional), but spell it out when a child closure has several statements

- An error is rethrown only when you *consume* that child's result (`for try await`, `next()`, `waitForAll()`)
- Throwing out of the closure cancels the remaining children and awaits them; cancellation is cooperative, so they stop at their next check
- `group.cancelAll()` for "first success wins" (e.g. race mirrors, take the first result)
- `withThrowingDiscardingTaskGroup`: the first child error cancels the group automatically and is thrown
- Same sliding-window technique as `withTaskGroup` to limit concurrency

Related to check: TaskGroup, `async let`, structured concurrency, cancellation, error handling, typed throws

---

@preconcurrency

Q: What is @preconcurrency? When do you need it?

Short answer: `@preconcurrency` is a migration tool: it tells the compiler that some code predates Swift Concurrency, so concurrency diagnostics involving it are relaxed instead of blocking your Swift 6 build. On an import it downgrades `Sendable` errors for types from a module that hasn't been audited yet; on a protocol conformance it lets a `@MainActor` type adopt a nonisolated protocol with a runtime isolation check; and library authors put it on declarations so new `Sendable` or `@MainActor` annotations don't break older clients.

Key points:

- `@preconcurrency import Module`: suppresses or downgrades `Sendable` diagnostics for that module's unaudited types; the compiler warns if it has no effect, so stale ones are easy to find
- `@preconcurrency` conformance (`extension VC: @preconcurrency LegacyDelegate`): compiles, and inserts a runtime check that traps if the requirement is called off the main actor
- Swift 6.2 *isolated conformances* (`extension VC: @MainActor LegacyDelegate`) are the checked alternative when the protocol is only used on the main actor
- `@preconcurrency` on a declaration: a library adds concurrency annotations while Swift 5 clients still see the old signature
- Migration is per module: Swift 5 mode + *Strict Concurrency Checking* = *Complete* (warnings), fix them, then switch that target to Swift 6 language mode (errors)
- Upcoming-feature flags (`-enable-upcoming-feature X`, SwiftPM `.enableUpcomingFeature("X")`) adopt future behaviors one at a time
- Xcode 26+ *Approachable Concurrency* turns on Swift 6.2 features, notably `NonisolatedNonsendingByDefault` and `InferIsolatedConformances`; *Default Actor Isolation* = *MainActor* is a separate setting

Related to check: Sendable, MainActor, isolated conformances, Swift 6 language mode, `nonisolated(nonsending)`, `@concurrent`

Migrating to Swift 6 Language Mode

Q: How would you migrate a large, existing iOS app to the Swift 6 language mode?

Short answer: I treat it as an incremental, module-by-module project rather than a flag flip. First I turn on *complete* concurrency checking while staying in Swift 5 mode, so every data-race issue shows up as a warning without breaking the build; then I fix one module at a time, starting with leaf packages (models, networking, persistence) and ending with the app target, and switch each one to Swift 6 mode once it's clean, so regressions become errors.

Key points:

- Language mode is per module: an Xcode target (`SWIFT_VERSION`) or a SwiftPM target (`.swiftLanguageMode(.v6)`); Swift 5 and Swift 6 modules link together fine
- Phase 1: `SWIFT_STRICT_CONCURRENCY = complete` in Swift 5 mode (warnings). Phase 2: `SWIFT_VERSION = 6` for that module (errors)
- Order: leaf modules first, so their `Sendable` and isolation annotations are in place before the modules that import them
- Approachable Concurrency (Xcode 26+): `NonisolatedNonsendingByDefault` and `InferIsolatedConformances`; set default MainActor isolation for UI modules. Use each feature's *Migrate* mode (or `swift package migrate --to-feature ...`) to get fix-its that preserve behaviour
- The common error families: global/static mutable state; main-actor state touched from nonisolated code; non-`Sendable` values crossing isolation ("sending … risks causing data races"); non-`Sendable` captures in `@Sendable`/`sending` closures; protocol conformances that cross isolation
- Fixes in order of preference: isolate it (`@MainActor`, an actor), make it a value type or immutable, protect it (`Mutex`), transfer it (`sending`, fresh values), and only then escape hatches (`@preconcurrency import`, `nonisolated(unsafe)`, `@unchecked Sendable`) with a comment

Related to check: Swift 6 migration plan (system design), approachable concurrency, `@preconcurrency`, `Sendable`, region-based isolation and `sending`, MainActor, `@concurrent`, isolated conformances, `Mutex`

Task Priority

Q: How does task priority work? Can you change it?

Short answer: Every task has a `TaskPriority` (`.high`, `.medium`, `.low`, `.background`, with `.userInitiated` and `.utility` as aliases for `.high` and `.low`) that maps onto the system's QoS classes and tells the scheduler what matters to the user. Priority is inherited: `Task { }`, `async let` and task-group children start at their parent's priority unless you pass one explicitly.

Key points:

- Set at creation: `Task(priority: .background) { }`, `group.addTask(priority: .utility) { }`
- Inherited by `Task { }` and structured children; `Task.detached` without a priority gets the default
- `Task.currentPriority` reflects escalation; `Task.basePriority` is what it was created with
- **Escalation:** awaiting a task's `.value`, or enqueuing higher-priority work on an actor that's busy, raises the waited-on work's priority (it doesn't come back down until that work finishes)
- Escalation is how a `.background` prefetch that the user is now waiting for gets finished quickly
- iOS 26+: `task.escalatePriority(to:)` escalates manually, and `withTaskPriorityEscalationHandler` lets you forward escalation to work Swift can't see (a legacy job queue, a C library)
- Mapping to QoS: `.high` ≈ `userInitiated`, `.medium` ≈ `default`, `.low` ≈ `utility`, `.background` ≈ `background` (see `quality-of-service.md`)

Related to check: Quality of Service, Task, task groups, actors, task-local values, `withTaskCancellationHandler`

Reentrancy and Actors

Q: Why are Swift actors reentrant, and how do you design around it?

Short answer: Swift actors are deliberately *reentrant*: when an actor-isolated method suspends at an `await`, the actor is free to run other queued work until that method resumes. Swift chose this because a

non-reentrant actor would sit idle while waiting on the network and could deadlock when two actors await each other; reentrancy guarantees progress and avoids those deadlocks.

Key points:

- No deadlocks from actors awaiting each other (or themselves), because a suspended method doesn't hold the actor
- Atomic: code between two `await`s. Not atomic: an async method as a whole
- The `@MainActor` is reentrant too: a double tap can start two overlapping `async` submissions
- Calls aren't guaranteed to run in submission order (priority and scheduling decide)
- Patterns: keep mutations synchronous; check-and-set a flag with no `await` in between; store an in-flight `Task` (see `reentrancy.md`); chain tasks to serialize whole operations

Related to check: Actor reentrancy (caching and dedupe), actors, MainActor, `Task`, `Mutex`, task-yield, Swift Concurrency instrument

---

Sync to Async Bridge

Q: How do you call async code from sync context?

Short answer: You can't `await` in a synchronous function, so you start a `Task { }`, which runs the async work and lets the sync caller return immediately. The result comes back by updating state (a `@MainActor` model the UI observes) or through a completion handler, never by blocking the sync caller until the task finishes: Swift has no supported "run blocking", and a semaphore waiting on a `Task` can deadlock the cooperative pool or freeze the main thread.

Key points:

- `Task { }` inherits actor isolation, priority and task-locals; from a `@MainActor` method its body runs on main, so it can update UI state directly
- Keep the handle (`Task<Void, Never>?`) if the work can be superseded or cancelled; dropping it doesn't cancel anything
- SwiftUI: `.task { }` (and `.task(id:)`) runs async work tied to the view's lifetime; `.refreshable` and many other modifiers already take async closures
- `Task.immediate { }` (iOS 26+) runs synchronously until the first suspension, for state that must change before the sync caller returns
- For legacy callers, expose a completion-handler wrapper that starts a `Task`; `@objc` async methods are already exposed to Objective-C with a completion-handler signature
- No synchronous waiting: `DispatchSemaphore.wait()` or `DispatchGroup.wait()` on a `Task` blocks a thread the task may need
- The opposite directions: callback API -> async is `continuations.md`; sync code called from async is `async-sync-bridge.md`

Related to check: Task, continuations, async to sync bridge, cancellation, `.task` modifier, MainActor

---

Async to Sync Bridge

Q: How do you call sync code from async context?

Short answer: Usually you just call it: a synchronous function called from an `async` function runs right there, on the current isolation, with no `await`. The questions that matter are *where* it runs and *how long* it holds the thread.

Key points:

- Plain sync call from async code: no `await`, runs on the current executor (main actor, another actor, or the concurrent pool)
- Sync function isolated to a different actor: call it with `await` (it becomes an implicit hop); use `await MainActor.run { }` only to group several main-actor statements from **non-isolated** code
- Inside a `@MainActor async` function you're already on main, so `MainActor.run` is redundant
- CPU-heavy sync work -> wrap it in a `@concurrent` function (Swift 6.2); `async` alone doesn't move work off the actor
- Blocking waits (`DispatchSemaphore.wait()`, `Thread.sleep`, sync network or file APIs that wait) occupy one of a small, fixed number of cooperative threads; run them on a dedicated queue and resume a continuation
- Shared state reached from both sync and async code: a `Mutex` gives synchronous, non-suspending access from either side

Related to check: Sync to async bridge, continuations, MainActor, `@concurrent`, `Mutex`, Task.yield, cooperative thread pool

---

DispatchWorkItem

Q: What is DispatchWorkItem? When do you use it?

Short answer: `DispatchWorkItem` wraps a closure in an object you can submit to a queue, cancel, wait on, or attach a completion to with `notify`. Its classic use is cancellable delayed work, such as debouncing a search field: cancel the previous item and schedule a new one with `asyncAfter`.

Key points:

- Create with a closure (optionally `qos:` and `flags:`), submit with `queue.async(execute:)` or `asyncAfter(deadline:execute:)`
- `cancel()`: if not started, it won't run; if running, it only sets `isCancelled`
- `notify(queue:execute:)`: run something after it finishes; `wait()` blocks until it does
- `perform()` runs it synchronously on the current thread
- **Modern Swift:** `DispatchWorkItem.cancel()` -> `Task.cancel()`; `asyncAfter` -> `Task.sleep(for:)`

Related to check: GCD, asyncAfter, Task cancellation, cooperative cancellation, OperationQueue, debouncing

---

DispatchGroup

Q: How do you use DispatchGroup? When?

Short answer: A `DispatchGroup` tracks a set of work items so you can act when all of them finish. You call `enter()` before each async operation and `leave()` exactly once when it completes, then `notify(queue:)` runs a closure once the count reaches zero; `wait()` does the same but blocks the calling thread.

Key points:

- `enter()` / `leave()` must balance; `async(group:)` on a queue enters and leaves for you
- `notify(queue:)`: non-blocking completion callback
- `wait()` / `wait(timeout:)`: blocks the thread; never on the main thread, never in async code
- Results have to be collected by hand into thread-safe storage
- **Modern Swift:** `async let` for a few known calls, `withTaskGroup` / `withThrowingTaskGroup` for N calls

Related to check: async let, task groups, GCD, semaphores, deadlock, structured concurrency, cancellation

---

NSLock and Mutex

Q: When would you use a lock (Mutex, NSLock) instead of an actor?

Short answer: For new code the lock is `Mutex` from the Synchronization framework (iOS 18+), with `Atomic` for simple counters and flags. I reach for a lock instead of an actor when the access must be **synchronous**: a nonisolated protocol requirement, a delegate callback, a hot path where an `await` hop costs too much, or a tiny critical section like a cache lookup.

Key points:

- **`Mutex<Value>`** (`import Synchronization`): owns the state it protects; access only through `withLock { }`. It's `Sendable` and noncopyable, so store it as a `let` in a `final class: Sendable`
- **`Atomic<Int>`** / `Atomic<Bool>`: lock-free counters and flags (`add(_:ordering:)`, `load`, `store`, `compareExchange`)
- **Actor vs Mutex:** actor = async, serialized, callers `await`, can suspend inside; Mutex = sync, no `await` inside the closure, keep it short
- **Older options:** `OSAllocatedUnfairLock` (iOS 16+, `Sendable`, can own state), `NSLock` / `NSRecursiveLock` (the compiler can't verify them, so the owning class needs `@unchecked Sendable`), `os_unfair_lock` (unsafe to use directly from Swift)

Related to check: Actors, thread safety, Sendable, race conditions, deadlock, dispatch barriers (reader - writer), Thread Sanitizer

---

Memory & Performance

Retain Cycles

Q: What causes retain cycles? How do you find and fix them?

Short answer: ARC frees an object when its strong reference count reaches zero, so if two objects hold each other strongly, directly or through a chain, neither count ever reaches zero and both leak. In Swift the usual culprits are escaping closures stored on the object that capture `self`, delegates declared as strong properties, `Timer`s and block-based notification observers that retain their target, and long-running `Task`s stored on `self` that capture `self`.

Key points:

- Cycle = A -> B -> A of strong references; ARC has no cycle collector
- Closures: a stored escaping closure that uses `self` captures it strongly; `[weak self]` breaks it. Non-escaping closures (`map`, `forEach`, `withAnimation`) can't cause cycles
- Delegates: `weak var delegate: (any MyDelegate)?` with a class-constrained protocol (`AnyObject`)
- `Timer` retains its target or block: invalidate it, and capture `self` weakly in the block
- `Task { }` retains everything it captures **until it finishes**; a stored task running an infinite `for await` loop that captures `self` is a cycle until cancelled. Capture weakly inside the loop, cancel it, or in SwiftUI use `.task`, which cancels automatically
- Combine (legacy): `sink` closures stored in `self.cancellables` that capture `self` strongly

Related to check: ARC, Weak references, Memory Graph Debugger, Leaks instrument, Capture lists, Swift Concurrency tasks

---

ARC

Q: How does ARC work? What are strong, weak, and unowned references?

Short answer: ARC (Automatic Reference Counting) manages the lifetime of class instances: the compiler inserts retain and release calls, and when an object's strong reference count reaches zero its `deinit` runs and its memory is freed. A **strong** reference (the default; there is no `strong` keyword in Swift) keeps the object alive.

Key points:

- Strong: default; `var ref: SomeClass` increments the count
- `weak var` (or `weak let`, Swift 6.2): optional, zeroed on deallocation; use when the other object can die first (delegates, parent back-references)
- `unowned`: non-optional, no zeroing; use only when the referenced object always outlives this one (e.g. a card that can't exist without its customer)
- Applies to class instances (and closures, actors); structs and enums are values, though their class-typed fields are still reference counted
- Cycles: object object, or object -> closure -> `self`; break them with `weak`/`unowned` or capture lists `[weak self]`

Related to check: Retain cycles, weak references, capture lists, Debug Memory Graph, Leaks instrument, autoreleasepool, heap vs stack, copy-on-write

---

Instruments

Q: What Instruments would you use to debug memory and performance issues?

Short answer: I pick the instrument from the symptom, always on a real device with a Release build. For **slowness or hangs** I start with Time Profiler, which in current Xcode includes the Hangs track, and use Processor Trace when sampling isn't precise enough.

Key points:

- **Time Profiler:** sampled call stacks of where CPU time goes; the Hangs track flags main-thread unresponsiveness
- **Processor Trace:** records every branch on supported Apple silicon devices for exact, non-sampled execution history
- **Animation Hitches:** frame-by-frame commit and render hitches; **SwiftUI** instrument: view-body updates, their causes and cost
- **Allocations** (growth, generations) · **Leaks** (unreachable cycles) · **VM Tracker** (dirty / swapped memory)
- **Power Profiler:** system power and per-subsystem impact (CPU, GPU, display, networking); replaces the old Energy Log workflow
- **Swift Concurrency** (Xcode 27): task lifetimes and states, actor contention, main-actor hops; **run comparison** for regressions
- Others worth naming: **App Launch**, **Network**, **File Activity**, **Data Persistence**, **Metal System Trace**, **System Trace** (threads, syscalls, VM faults), Points of Interest via `OSSignposter`

Related to check: Time Profiler, Instruments Allocations, Allocations vs Leaks, Debug Memory Graph, energy log / Power Profiler, animation performance, app launch, MetricKit, Thread Sanitizer

---

Finding SwiftUI Performance Problems

Q: How do you find SwiftUI performance problems?

Short answer: SwiftUI problems come in two shapes: a view `body` that is **too slow** (expensive work inside it), or bodies that run **too often** (more views invalidated than needed). I profile a Release build on a device with the **SwiftUI** template in Instruments (the SwiftUI instrument is new in Xcode 26), which records every view-body update next to the Time Profiler and Hangs tracks.

Key points:

- **Template:** *SwiftUI* = the SwiftUI instrument plus Time Profiler, Hangs and Hitches tracks (`xcrun xctrace list templates` shows it). For devices on older OS versions, Instruments still has the *View Body (Legacy)* and *View Properties (Legacy)* instruments
- **Lanes:** update groups, long view body updates, long platform-view (representable) updates and other long updates, flagged by how likely they are to cause a hitch
- **Too slow:** select a long update, check the Time Profiler samples under it: formatters, sorting, filtering, image decoding or string building in `body`
- **Too often:** the update count per view type and the cause-and-effect graph; then fix dependencies
- **Quick check in code:** `let _ = Self._printChanges()` in `body` (debug only) prints what changed

Related to check: body as a computed property, `@Observable` vs `ObservableObject`, view identity, view hierarchy performance, scroll performance, animation performance, hangs and hitches, Instruments run comparison

---

Debugging Swift Concurrency Performance

Q: How do you debug Swift Concurrency performance?

Short answer: Slow async code is usually not slow *code*: it's tasks **waiting**, for an actor that's busy, for the main actor, or for too many tasks competing for the cooperative thread pool. The Time Profiler only shows CPU samples, so I record with the **Swift Concurrency** template, which adds the Swift Tasks and Swift Actors instruments: task lifetimes and states (running, suspended, waiting to run on an actor), how many tasks are alive, and per-actor queues that show contention.

Key points:

- **Template:** *Swift Concurrency* (check with `xcrun xctrace list templates`) = *Swift Tasks* (task states, lifetimes, the task forest) and *Swift Actors* (actor queue size, actor execution), plus Time Profiler, Hangs and Points of Interest
- **Actor contention:** many tasks enqueued on one actor while it runs long jobs. Fix: keep isolated sections short; do CPU work in a `@concurrent` function and only touch state on the actor
- **Main-actor pressure:** many small hops to `@MainActor` (or work that never left it) show up as main-thread time and hangs; batch UI updates and keep heavy work off main
- **Thread pool starvation:** blocking calls (`DispatchSemaphore.wait()`, synchronous I/O, locks held across slow work) inside tasks pin cooperative threads, so other tasks can't run
- **Task explosion:** thousands of short-lived tasks cost more in scheduling than they save; use task groups with a limit
- **Names help:** `Task(name: "...")` makes tasks identifiable in Instruments and LLDB; `OSSignposter` intervals mark feature boundaries

Related to check: Actors and reentrancy, `@concurrent` / approachable concurrency, structured concurrency, task priority, Time Profiler, main thread, hangs and hitches, Instruments run comparison, Thread Sanitizer

---

Image Loading and Caching

Q: How do you efficiently load and cache images?

Short answer: I treat image loading as a pipeline with a cache at each expensive step: check an in-memory cache of *decoded, display-sized* images first; then the disk/HTTP cache (`URLCache` honoring server cache headers, or a custom disk cache); and only then the network. After download, decode and downsample off the main actor to the size the view needs, store the result in an `NSCache` with a cost, and hand it to the UI on the main actor.

Key points:

- Memory cache: `NSCache` (thread-safe, evicts under memory pressure) with `totalCostLimit` and per-image cost (pixel bytes)
- Disk cache: `URLCache` with a sensible disk capacity, or a file cache in `Caches/` for full control
- Decode + downsample off main (`byPreparingThumbnail(ofSize:)` or ImageIO); cache the *result*
- Shared state (in-flight map) lives in an `actor`; the loader is `Sendable`
- SwiftUI `AsyncImage`: fine for simple cases; no in-memory cache or downsampling. iOS 27 adds `.asyncImageURLSession(_:)` so it loads through your own `URLSession` and cache configuration

Related to check: Image downsampling, async image decoding, lazy loading strategy, NSCache, URL-Cache / response caching, actors, memory warnings

---

Scroll Performance

Q: How do you optimize scrolling performance in UITableView/UICollectionView?

Short answer: Smooth scrolling means every frame is ready on time: about 8.3 ms per frame at 120 Hz on ProMotion devices, and late frames show up as **hitches**, measured as milliseconds of hitch per second of scrolling (Apple treats under 5 ms/s as good and over 10 ms/s as something users notice). Most of that budget goes on the main thread in the commit phase: dequeuing and configuring cells, Auto Layout, text measurement and image decoding.

Key points:

- Reuse: cell registrations + `dequeueConfiguredReusableCell`, diffable data sources; reset in `prepareForReuse` (and cancel in-flight loads)
- Prefetching: `UICollectionViewDataSourcePrefetching` / `UITableViewDataSourcePrefetching` start loads before cells appear, and `cancelPrefetching...` stops them
- Images: decode and downsample off main (`byPreparingThumbnail(ofSize:)`, ImageIO), cache decoded results in `NSCache`
- Layout: self-sizing cells are fine if constraints are simple; cache heights for complex content; avoid layout work in `scrollViewDidScroll`
- Rendering: `shadowPath`, no clipping of image content, opaque backgrounds where possible
- SwiftUI: `List` / `LazyVStack` create rows on demand; give rows stable `Identifiable` IDs, avoid `AnyView` and expensive computed properties in `body`, and load per-row data in `.task(id:)`

Related to check: Offscreen rendering, Image downsampling, Main thread, NSCache, View hierarchy performance, MetricKit

---

Lazy Loading

Q: What is lazy loading? When would you use it?

Short answer: Lazy loading means deferring the creation of something expensive until the first time it's actually needed, so you don't pay for it at launch or at all if it's never used. In Swift that shows up at several levels: `lazy var` properties, globals and `static let` (which are always lazy and initialized exactly once, thread-safely), lazy sequences (`.lazy`) that compute elements on demand, and lazy UI such as `LazyVStack`, `List` and cell reuse that only build what's on screen.

Key points:

- `lazy var` runs its initializer on first access and stores the result; it must be `var`, and the initializer can use `self`
- `lazy var` is **not** thread-safe: two threads touching it first can both run the initializer. In Swift 6 that's a compile-time concern: it's mutable state, so it needs isolation (`@MainActor`, an actor) or it can't live in a `Sendable` type
- Globals and `static let` are lazy **and** thread-safe (initialized once via `swift_once`): the idiomatic shared formatter or lookup table
- `.lazy` on collections chains `map`/`filter` without intermediate arrays and can stop early (`first(where:)`)
- UI: lazy containers (`LazyVStack`, `LazyVGrid`, `List`) create rows as they scroll in; `UIViewController.view` is itself loaded lazily
- App level: defer non-launch services, load large assets on demand (Background Assets), see *Lazy loading strategy* for images

Related to check: Lazy loading strategy (images), Startup optimization, Scroll performance, Observation (`@ObservationIgnored`), Swift 6 isolation

---

Autoreleasepool

Q: What is autoreleasepool? When would you use it?

Short answer: An autorelease pool holds objects that Objective-C code has "autoreleased" (released later rather than immediately) and releases them all when the pool drains. The main run loop drains its pool once per iteration and GCD drains one per work item, so normally you never think about it.

Key points:

- Autorelease is an Objective-C runtime mechanism; pure Swift objects are released by ARC immediately and never enter a pool
- Pools exist per thread: the main run loop and GCD drain them for you
- Use `autoreleasepool { }` inside long loops that create many ObjC-backed temporaries; it returns the closure's value
- The win is lower *peak* memory (and fewer jetsam terminations), not faster code

Related to check: ARC, Allocations instrument, image downsampling, OOM / jetsam, memory warnings, @concurrent

---

NSCache

Q: What is NSCache? How does it differ from Dictionary?

Short answer: `NSCache` is a Foundation key-value store designed for data you can recreate: it evicts entries automatically when the system is under memory pressure or when you exceed the count or cost limits you set, and it's safe to use from multiple threads. A `Dictionary` is a plain value type that holds every entry strongly until you remove it, never shrinks on its own, and isn't thread-safe when shared.

Key points:

- Automatic eviction under memory pressure, plus `countLimit` and `totalCostLimit` (with `setObject(_:forKey:cost:)`); the limits are guidelines, not strict, and eviction order isn't guaranteed
- Thread-safe: you can add, remove and query from different threads without your own locking
- **Does not copy keys** (an `NSDictionary` copies them); keys and values must be class types: `NSString`, `NSURL`, or your own wrapper class

- Not enumerable, no `count`, no way to list contents: it's a cache, not storage
- Works with `NSDiscardableContent` (e.g. `NSPurgeableData`) so the system can discard object contents; `evictsObjectsWithDiscardedContent` controls removal
- `NSCacheDelegate.cache(_:willEvictObject:)` for logging or cleanup when something is evicted

Related to check: Image downsampling, Memory warnings, OOM, URLCache, Lazy loading strategy, Sendable

---

Main Thread Best Practices

Q: Why should you avoid blocking the main thread? What are the consequences?

Short answer: The main thread runs the run loop that handles touches, layout, drawing and animation commits, so anything slow on it delays every frame. On a ProMotion display the budget is about **8.3 ms per frame at 120 Hz** (16.7 ms at 60 Hz); miss it and you get **hitches** (late frames, measured in ms of hitch per second of scrolling or animation), and block it for longer and you get a **hang**: the app stops responding to input.

Key points:

- Frame budget: 8.3 ms at 120 Hz, 16.7 ms at 60 Hz, and the app only gets part of it (the render server needs the rest)
- Hitch = a frame delivered late; hang = the main run loop not servicing events (>= 250 ms is a reportable hang)
- Common blockers: synchronous I/O and decoding, JSON parsing of big payloads, image decoding and resizing, Core Data fetches on the view context, locks and semaphores waiting on other threads
- **`async` does not mean "background"**: a `@MainActor async` method runs on the main thread between suspension points, and `Task { }` started on the main actor *inherits* main-actor isolation
- Off-main CPU work: a `@concurrent` function (Swift 6.2). `await` on a network call is fine on main: it suspends instead of blocking
- Watchdog: blocking launch, resume or scene transitions for too long terminates the app (`0x8badf00d` in crash reports)

Related to check: MainActor, `@concurrent` / approachable concurrency, Time Profiler, Scroll performance, Startup optimization, MetricKit

---

Hangs vs Hitches

Q: What's the difference between a hang and a hitch, and how do you measure each?

Short answer: Both come from the main thread missing a deadline, but at very different scales and with different symptoms. A **hitch** is a frame shown later than expected during scrolling or animation: a few milliseconds over the 8.3 ms budget at 120 Hz, which the user sees as stutter.

Key points:

- **Hitch:** frame late by n ms. Ratio: under 5 ms/s is good, 5 - 10 ms/s is noticeable, over 10 ms/s is bad. Two kinds: *commit* hitches (your main-thread work: layout, `body`, image decoding, text measurement) and *render* hitches (the render server: offscreen passes, huge layer trees, blur and mask effects)
- **Hang:** main run loop not handling events. Can be *busy* (CPU work on main) or *blocked* (waiting on a lock, semaphore, synchronous I/O or another thread); Time Profiler only shows the busy kind, so check thread states for the blocked kind
- **Watchdog:** a long hang during launch, resume or scene transitions gets the app terminated (`0x8badf00d` in crash reports)

- **Lab tools:** Hangs instrument (250 ms default threshold, configurable; the Animation Hitches template lowers it to 33 ms), Hitches instrument in the Animation Hitches template, SwiftUI instrument for long view updates, Thread Performance Checker in the scheme diagnostics, on-device hang detection in Settings › Developer
- **Field tools:** Organizer Hangs report; MetricKit `applicationResponsivenessMetrics` (hang-time histogram), `hangDiagnostics` (stack + duration), `animationMetrics.scrollHitchTimeRatio` and `hitchTimeRatio` (iOS 26+)
- **Tests:** `XCTOSSignpostMetric.scrollingAndDecelerationMetric`, and `XCTHitchMetric(application:)` (iOS 26+) in XCTest performance tests

Related to check: Main thread, scroll performance, animation performance, offscreen rendering, Time Profiler, SwiftUI performance instrument, MetricKit, performance testing, Instruments run comparison

---

Offscreen Rendering

Q: What causes offscreen rendering? How do you avoid it?

Short answer: Offscreen rendering happens when Core Animation can't draw a layer straight into the frame and has to render it into a separate buffer first, then composite that buffer back. It's triggered by effects that need the whole layer's rendered content before they can be applied: masks (`mask`, or `masksToBounds` with rounded corners over image or sublayer content), shadows without a `shadowPath` (the shape has to be computed from the rendered alpha), group opacity, and some blur and filter effects.

Key points:

- Triggers: `layer.mask`, `masksToBounds`/`clipsToBounds` + `cornerRadius` on content (images, sublayers), shadows without `shadowPath`, `allowsGroupOpacity` with alpha < 1 over sublayers, `shouldRasterize`, visual-effect blurs
- **`cornerRadius` alone is cheap** when it only rounds the layer's `backgroundColor` and border; the cost comes from clipping *content*
- `shadowPath` turns an expensive alpha-derived shadow into a cheap known shape; update it in `layoutSubviews`
- `shouldRasterize` caches the rendered layer as a bitmap: good for static, complex content; bad for content that changes (re-rasterizes every frame). Set `rasterizationScale` to `traitCollection.displayScale`
- Detect: Simulator › Debug › *Color Off-screen Rendered* (yellow), the Animation Hitches and Core Animation instruments
- SwiftUI: `.clipShape(.rect(cornerRadius:))` and `.shadow` have the same costs; `.drawingGroup()` deliberately renders a complex subtree offscreen into one Metal layer, which helps for many overlapping vector shapes but hurts simple views

Related to check: Scroll performance, View hierarchy performance, Core Animation, Metal and GPU, Liquid Glass

---

Debug Memory Graph

Q: How do you use the Debug Memory Graph to find retain cycles?

Short answer: The Debug Memory Graph pauses the running app and snapshots every live heap object and the references between them. My workflow: open and close the suspect screen a few times, click the memory graph button in the debug bar, and filter the navigator to my own types.

Key points:

- Start: debug bar memory-graph button (or Debug -> Debug Workflow -> View Memory Graph Hierarchy) while running
- Navigator filters: your workspace's types only, leaked blocks only, search by type name
- Instance counts per type: "3 x DetailViewModel after closing the screen three times" is the bug report
- Graph: bold = strong, light = weak/unowned; follow strong edges back to a root or a cycle
- Enable **Malloc Stack Logging** (Scheme -> Diagnostics, *Live Allocations Only*) to see the backtrace that allocated each object
- File -> Export Memory Graph saves a `.memgraph` you can inspect with the `leaks`, `heap` and `vmmap` command-line tools or attach to a bug

Related to check: Retain cycles, ARC, weak references, Leaks instrument, Allocations vs Leaks, memory graph debugger, Task cancellation

---

App Launch Optimization

Q: How do you optimize app launch time?

Short answer: I break launch into phases and measure each one. **Pre-main** is dyld loading and binding dynamic libraries plus static initializers (`+load`, global constructors); **app init** is `App.init`/`didFinishLaunching` and scene connection; **first frame** is building and rendering the first screen; **extended launch** is until the content the user came for is visible.

Key points:

- Launch types: **cold** (process not running, caches cold), **warm** (recently run, caches warm), **resume** (already in memory; not a launch)
- Pre-main: fewer dynamic frameworks, no `+load`/static-initializer work; **mergeable libraries** (Xcode 15+) keep dylibs in Debug but merge them into the app binary in Release
- App init: cheap `init`s, no synchronous network or disk-heavy work, lazy services, no main-thread blocking
- First frame: a simple first screen, cached data shown immediately, real content loaded in `.task`
- Measure: App Launch / Time Profiler instruments, `XCTApplicationLaunchMetric` performance tests, Organizer launch metrics, MetricKit `MXAppLaunchMetric` (`histogrammedTimeToFirstDraw`, `histogrammedOptimizedTimeToFirstDraw`, `histogrammedExtendedLaunch`)

Related to check: Startup optimization, MetricKit, Time Profiler, reduce binary size, lazy loading, Instruments, Hangs, SwiftData/Core Data migration

---

Battery Optimization

Q: How do you optimize for battery life?

Short answer: Energy is spent by the subsystems your app keeps awake: CPU, GPU, the display, the network radios, and location hardware. The biggest cost is often not the work itself but the *wake-ups*: a radio or CPU that's woken every few seconds never gets back to its low-power state.

Key points:

- **CPU:** no busy-waiting or polling loops; push instead of poll; cancel work for screens that are gone; coalesce timers (tolerance)
- **Network:** batch requests, cache responses, use background/discretionary `URLSession` for non-urgent transfers; each radio wake costs energy beyond the bytes sent
- **Location:** lowest acceptable accuracy, one-shot `requestLocation()` or significant-change/region monitoring instead of continuous GPS; stop updates promptly
- **GPU/display:** no needless 120 Hz animations or constant redraws; dark UI saves energy on OLED

- **Background:** BackgroundTasks framework, not keep-alive tricks; respect Low Power Mode and thermal state

Related to check: Energy log / Power Profiler, background fetch and BGTaskScheduler, CADisplayLink, Instruments, MetricKit, location services, URLSession background sessions

---

Copy-on-Write

Q: What is copy-on-write? How does it apply to Swift collections?

Short answer: Copy-on-write (COW) is how Swift gives large value types cheap copies. `Array`, `Dictionary`, `Set`, `String` and `Data` keep their elements in a reference-counted heap buffer; assigning or passing the value only shares that buffer, and the actual copy happens at the first *mutation* while the buffer is shared.

Key points:

- Copy is O(1) until a shared buffer is mutated; then it's O(n) once
- Standard-library collections, `String` and `Data` are COW; your own structs are not unless you implement it
- Custom COW: a struct wrapping a `final class` storage + `isKnownUniquelyReferenced(&storage)` before writes
- A struct holding a plain class reference *shares* that object: that's reference semantics, not COW
- Swift 6.2+ alternatives when you don't want COW: `InlineArray` (Swift 6.2, iOS 26+, fixed size, inline, eager copies), `~Copyable` types (no implicit copies at all), `Span` (read without copying, iOS 26+), `UniqueArray` (Swift 6.4, iOS 27+, a noncopyable growable array)

Related to check: Value vs reference types, heap vs stack, ARC, noncopyable types, InlineArray / Span, Sendable, string performance

---

Time Profiler

Q: How do you use the Time Profiler to find performance bottlenecks?

Short answer: The Time Profiler samples every thread's call stack at a high, fixed rate (about once per millisecond) while you reproduce the slow interaction, then aggregates the samples so you can see where CPU time went. I profile a Release build on a real device, select the time range of the slow moment in the timeline, and read the call tree: inverted with *Hide System Libraries* to find the heaviest leaf functions in my code, or top-down to see which feature they belong to.

Key points:

- Sampling: cheap and statistical; short functions called millions of times and blocked time (waiting on locks or I/O) look different. Blocked threads don't use CPU, so pair it with the **Hangs** instrument and *Thread States* or System Trace for waiting
- Call tree options: *Separate by Thread*, *Invert Call Tree*, *Hide System Libraries*, *Flatten Recursion*, and focusing or pruning a subtree (charge to caller)
- Weight = time on CPU; *self weight* = time in the function itself
- Main thread focus: select the main thread and the hitch or hang interval
- Mark your own regions with `OSSignposter` intervals; they appear as a track aligned with the samples
- Related templates: Swift Concurrency (tasks and actors; the Swift Concurrency instrument in Xcode 27), SwiftUI (body updates, Xcode 26), Animation Hitches, App Launch, CPU Profiler (cycle-accurate counters)

Related to check: Main thread, Scroll performance, Startup optimization, Symbolication, Instruments, Swift Concurrency instrument

---

Proving a Performance Fix Worked

Q: How do you prove a performance fix actually worked?

Short answer: One fast recording after the change proves nothing, because run-to-run noise on a phone (thermal state, background work, caches, network) is often as large as the improvement. I define the scenario and the metric up front (for example "search results visible, signposted interval, ms"), record **several runs before and after** on the same device and build configuration, and compare medians against the spread of the runs.

Key points:

- **Fix the scenario:** same device model and OS, Release build, same data, same steps (ideally automated with XCUITest), device cool and charged, nothing else running
- **Measure the thing you changed:** an `OSSignposter` interval around the operation beats eyeballing a whole trace
- **Several runs per side** (5 - 10): `xctrace record --append-run --run-name ...` keeps them in one trace; compare medians, not best or first runs
- **Noise band:** if the before/after difference is smaller than the spread between runs, you haven't shown anything yet; repeat or reduce noise
- **Guard against regressions:** XCTest `measure(metrics:options:)` with `XCTOSSignpostMetric`, `XCTClockMetric`, `XCTCPUMetric`, `XCTMemoryMetric`; baselines are stored per device, with an allowed deviation
- **Field confirmation:** MetricKit (`MXMetricPayload` on iOS 18 - 26, `MetricManager().metricReports` on iOS 27+) and Organizer per app version: launch time, hang rate, hitch ratio, memory

Related to check: Instruments, Time Profiler, performance testing (XCTest metrics), MetricKit, hangs and hitches, startup optimization, os_signpost, Swift Concurrency instrument, SwiftUI performance instrument

---

JSON Parsing Performance

Q: How do you optimize JSON parsing?

Short answer: I use `Codable` with `JSONDecoder` and make sure decoding never runs on the main actor: the network call and the decode live in a `@concurrent` function, and only the finished model is handed back to the UI. Then I reduce the work itself: model only the fields the app needs (unknown keys are skipped cheaply), use explicit `CodingKeys` instead of `.convertFromSnakeCase`, and keep payloads small with pagination or field selection on the server.

Key points:

- Decode off the main actor (`@concurrent` function); assign results on the main actor
- Model only needed fields; flatten with custom `init(from:)` only where it saves real work
- Explicit `CodingKeys` beat key-conversion strategies, which transform every key
- Large payloads: paginate or NDJSON; no streaming `JSONDecoder` exists
- Measure with Time Profiler on a Release build before tuning

Related to check: Codable, @concurrent and Swift Concurrency, URLSession async APIs, pagination, Sendable, Core Data batch inserts, Time Profiler

---

View Hierarchy Performance

Q: How does view hierarchy depth affect performance?

Short answer: In UIKit every `UIView` is backed by a `CALayer`, so each extra view adds work in four places: Auto Layout (more constraints to solve and more views to lay out), hit testing (more views to walk on every touch), rendering (more layers to commit and composite, and more blending if they're translucent), and memory (layer objects, plus a backing bitmap for views that draw their own content). Depth itself is rarely the problem; the **total number of views and constraints** and the effects on them are, especially in cells that are created and configured while scrolling.

Key points:

- Cost scales with views x constraints x how often layout runs; a deep but static hierarchy laid out once is fine
- `UIStackView` doesn't flatten anything: it's a non-rendering container that adds its own constraints; nested stacks in every cell add up
- Replace spacer and grouping views with `UILayoutGuide`; use `directionalLayoutMargins` instead of padding wrappers
- Opaque views with a background color composite faster than translucent ones (Simulator › *Color Blended Layers*)
- For very complex, static content, one view drawing with Core Graphics or a pre-rendered image can replace dozens of subviews
- SwiftUI: keep `body` cheap, split views so state changes invalidate small subtrees, prefer `@Observable` (per-property tracking) and stable identity; `Self._printChanges()` shows why a body re-ran

Related to check: Auto Layout performance, Scroll performance, Offscreen rendering, SwiftUI Observation, Constraint performance

---

Background Fetch and Processing

Q: How do you perform work in the background without draining battery?

Short answer: You don't run your own background loops; you ask the system and let it pick the moment. The BackgroundTasks framework has three task types: **`BGAppRefreshTask`** for short, frequent content refreshes (about 30 seconds), **`BGProcessingTask`** for longer deferrable maintenance that can require power and network, and **`BGContinuedProcessingTask`** (iOS 26+) for work the *user* started, like an export, that should keep going after they leave the app, with progress shown in system UI.

Key points:

- Register identifiers in Info.plist (`BGTaskSchedulerPermittedIdentifiers`) and enable the Background Modes capability (fetch / processing)
- `earliestBeginDate` is a lower bound, not a schedule; the system may run the task much later or not at all (e.g. after the user force-quits)
- SwiftUI: `.backgroundTask(.appRefresh("id")) { ... }` on the scene; the async closure is cancelled when time runs out
- UIKit: `BGTaskScheduler.shared.register(forTaskWithIdentifier:using:)` before launch finishes, set `expirationHandler`, always call `setTaskCompleted(success:)`
- `BGProcessingTaskRequest`: `requiresExternalPower`, `requiresNetworkConnectivity`; good for database cleanup, ML model updates
- `BGContinuedProcessingTask` (iOS 26+): submitted while in the foreground, must report `progress`, can be stopped by the user; wildcard identifiers (`com.example.app.export.*`)

Related to check: Battery optimization, Power Profiler / energy log, background URLSession downloads, Background Assets, silent push notifications, scene phase, cancellation

---

When to Use Weak References

Q: When should you use weak references? Give examples.

Short answer: A `weak` reference points to a class instance without keeping it alive, and ARC sets it to `nil` automatically when the instance is deallocated. I use it whenever the referencing object doesn't own the target and holding it strongly would create a cycle or extend its lifetime: delegates and data sources, back-references from child to parent, closures stored on the object they capture, observers, and caches or registries that shouldn't keep objects alive.

Key points:

- Delegates: `weak var delegate: (any Delegate)?`; the protocol must be `AnyObject`-constrained
- Parent/child: the parent owns children strongly; the child's `parent` is weak
- Closures: `[weak self]` for escaping closures stored by `self` or by something `self` owns, and for long-lived tasks
- Weak collections: wrap in a `struct WeakBox { weak var value: T? }`, or `NSHashTable.weakObjects()` / `NSMapTable` in Objective-C-style code
- **`weak let`** (Swift 6.2): the reference can't be reassigned, though it still becomes `nil`; a `final class` whose stored properties are `let` and Sendable (including `weak let`) can conform to `Sendable`
- `unowned` is the non-optional alternative for references that can't outlive the owner (it traps if they do); `weak` is the safe default

Related to check: ARC, Retain cycles, Memory Graph Debugger, Delegation, Sendable, Swift 6.2

Heap vs Stack

Q: What's the difference between heap and stack allocation?

Short answer: The stack is per-thread memory for function frames: allocating is just moving the stack pointer, it's freed automatically when the function returns, and it's small (on the order of 1 MB for the main thread and less for secondary threads). The heap is shared, dynamically sized memory: allocating means finding a free block (with synchronization), and objects need reference counting to know when to free them, so it's much more expensive.

Key points:

- Stack: LIFO, per thread, fast, fixed size, lifetime tied to scope
- Heap: shared, flexible lifetime, `malloc`/`free` + atomic retain/release (ARC)
- Classes, actors and closures' captured context: heap (the optimizer can promote non-escaping objects to the stack)
- Value types: inline in their container; collections (`Array`, `String`, `Dictionary`) are small structs pointing to a heap buffer
- Existentials (`any P`) store values up to 3 words inline and box larger ones on the heap; `some P`/generics avoid the box
- Swift 6.2: **`InlineArray`** (fixed size, no heap buffer) and **`Span`** (safe, non-escaping view) need an iOS 26+ runtime; **`~Copyable`** types avoid copies and give unique ownership

Related to check: Value vs reference types, ARC, copy-on-write, noncopyable types, existentials (`any` vs `some`), compiler optimizations, Allocations instrument, address sanitizer

---

Animation Performance

Q: How do you ensure smooth animations at 60 - 120 Hz on ProMotion displays?

Short answer: Smoothness is a frame-budget problem: at 120 Hz (ProMotion) each frame has **8.3 ms**, at 60 Hz 16.7 ms, and that budget is shared between your app's commit on the main thread and the render server. A frame that misses its deadline is a **hitch**, and Apple measures hitches as a *hitch time ratio* in ms of hitching per second.

Key points:

- Budget: 8.3 ms at 120 Hz, 16.7 ms at 60 Hz; hitches measured in ms/s (aim for under ~5 ms/s)
- Cheap: `transform`, `alpha`/`opacity`, position; they animate in the render server without per-frame app work
- Expensive: animating `frame`/`bounds`/constraints (layout each frame), shadows without `shadowPath`, masks, blurs, large blended layers (offscreen passes)
- Main thread: no decoding, parsing or synchronous I/O during animations; move CPU work to a `@concurrent` function
- SwiftUI: animate render-only modifiers (`scaleEffect`, `opacity`, `offset`) instead of changing layout sizes; keep `body` cheap with `@Observable` per-property tracking

Related to check: Offscreen rendering, CADisplayLink / UIUpdateLink, CATransaction, scroll performance, async image decoding, SwiftUI instrument, MetricKit, main thread

---

Auto Layout Performance

Q: How can Auto Layout affect performance? How do you optimize it?

Short answer: Auto Layout turns constraints into a system of linear equations that the layout engine solves incrementally. Since iOS 12 the engine scales roughly linearly for independent views, so the number of constraints is rarely the real problem.

Key points:

- Create constraints once (in `init`/`viewDidLoad`); mutate `constant`, `priority` (non-required) or `isActive` afterward
- Swap only the constraints that differ between layouts; never "remove all and rebuild"
- Batch changes and call `setNeedsLayout()`; call `layoutIfNeeded()` only when you need the frames now (e.g. inside an animation)
- `UIStackView` is a convenience, not an optimization: it generates constraints internally
- Text measurement (`UILabel`, `UITextView`) and custom `intrinsicContentSize` run often; keep them cheap and call `invalidateIntrinsicContentSize()` only when content really changes

Related to check: Auto Layout, UIStackView, self-sizing cells, scroll performance, animation performance, SwiftUI instrument, adaptive layout / iPhone Duo

---

Handling Memory Warnings

Q: What should you do when the app receives a memory warning?

Short answer: A memory warning means the device is under memory pressure and your app is a candidate for termination, so you shed anything you can recreate: in-memory caches, decoded images, prefetched data and off-screen content. You keep what the user would notice losing: the visible UI, unsaved input and navigation state.

Key points:

- **Release:** `NSCache` contents (it also evicts by itself), decoded image dictionaries, prefetched pages, precomputed layouts, hidden child view controllers' data, Metal/AV buffers you can rebuild
- **Keep:** what's on screen, user-entered data (save drafts if needed), and state needed to restore navigation
- The OS limit is on the app's **footprint** (dirty + compressed memory), not on virtual size; clean, file-backed memory (mapped files, code) doesn't count the same way
- Apps that have just shrunk their footprint are less likely to be chosen for termination; suspended apps with large footprints are reclaimed first
- **iOS 27+:** `UIApplication.systemPrefersReducedResourceUsage` (and the matching trait) asks apps to choose lighter work proactively: lower-resolution assets, less prefetching, fewer live effects. It isn't a memory warning, and you shouldn't start expensive work when it changes
- Headroom check: `os_proc_available_memory()` before a large allocation

Related to check: Memory Warning Handling (callbacks), UIKit memory warnings, OOM, NSCache, Image downsampling, VM Tracker

---

Reducing Object Allocations

Q: How do you reduce object allocations in performance-critical code?

Short answer: Every heap allocation costs a `malloc`, reference-counting traffic for its lifetime, and a `free`, and in a hot loop that adds up to measurable time and memory churn. I start by measuring with the Allocations instrument (transient allocation counts, not just live bytes), then remove allocations where they're avoidable: value types instead of classes, generics and `some` instead of `any` existentials, `reserveCapacity` instead of repeated growth, reusing expensive objects like formatters and cells, and no escaping closures in tight loops.

Key points:

- Classes, escaping closure contexts, `indirect` enums and growing arrays allocate on the heap; structs and enums of value types usually don't
- `any Protocol` values larger than three words are boxed on the heap; `some Protocol` and generics are specialized and avoid the box
- Collections: `reserveCapacity(n)` once, instead of reallocating as they grow; copy-on-write means copies are free until one is mutated
- Strings up to 15 UTF-8 bytes are stored inline (small-string optimization)
- Reuse: cells (`dequeueReusableCell`), `static let` formatters, preallocated buffers
- Swift 6.2 / iOS 26+: `InlineArray<N, T>` (fixed size, stored inline), `Span` (non-owning, no ARC); `~Copyable` types avoid copies and refcounting; `UniqueArray` (Swift 6.4, iOS 27+) is a noncopyable growable array with unique ownership
- `withUnsafeTemporaryAllocation` for scratch buffers that live only within a scope (often stack-allocated)

Related to check: Heap vs stack, Copy-on-write, String performance, Noncopyable types, Allocations instrument, Scroll performance

---

Core Data Performance

Q: How do you optimize Core Data performance?

Short answer: Most Core Data performance problems come from loading too much into memory or doing it on the wrong context. I fetch only what the screen needs (`fetchBatchSize`, `fetchLimit`, predicates on indexed attributes, `propertiesToFetch`), avoid fault storms by prefetching the relationships a list displays, and keep heavy imports and saves on a background context using `try await context.perform { }`.

Key points:

- `fetchBatchSize`: rows are loaded in pages as you access them (pairs well with lists and `NSFetchedResultsController`)
- Indexes (model editor *Indexes*, or fetch indexes) on attributes used in predicates and sort descriptors
- `relationshipKeyPathsForPrefetching` avoids one fault-fire round trip per row
- Background contexts for imports; `viewContext` (main queue) for UI; `automaticallyMergesChangesFromParent = true` on the view context
- Batch requests skip validation and change tracking: merge results with `mergeChanges(fromRemoteContextSave:i` or persistent history tracking
- `NSManagedObject` is not `Sendable`: pass `NSManagedObjectID` (or plain `Sendable` DTOs) across isolation

Related to check: Core Data, NSPersistentContainer, NSFetchedResultsController, SwiftData (`FetchDescriptor`, `@ModelActor`), persistent history tracking, Sendable, Allocations instrument

---

Compiler Optimizations

Q: What compiler optimizations affect iOS app performance?

Short answer: Debug builds use `-Onone` so code is debuggable, and Swift at `-Onone` can be 10x or more slower than optimized code, mostly because of unspecialized generics, ARC traffic and dynamic dispatch. Release builds use `-O` (optimize for speed) with **whole-module optimization**, which lets the compiler see every file in a module at once: it can devirtualize calls, inline, specialize generics for concrete types, remove redundant retain/release pairs, and strip dead code.

Key points:

- Levels: `-Onone` (Debug), `-O` (Release, speed), `-Osize` (smaller binary, slightly slower); avoid `-Ounchecked` (removes overflow and precondition checks)
- Whole-module optimization (Compilation Mode = Whole Module) is the Release default; incremental is for Debug build speed
- Optimizations: inlining, generic specialization, devirtualization, ARC optimization, dead-code elimination, stack promotion of objects that don't escape
- Help it: `final` classes, `private`/`fileprivate` members, structs and enums, `some` (static) over `any` (boxed, dynamic)
- Across modules: `@inlinable` / `@usableFromInline` publish bodies to clients; recent toolchains also apply a conservative cross-module optimization to small functions automatically

Related to check: Time Profiler, reduce binary size, generics and opaque types, static vs dynamic dispatch, heap vs stack, copy-on-write, modular architecture

---

MetricKit

Q: What is MetricKit? How do you use it?

Short answer: MetricKit is Apple's on-device telemetry framework: the system measures your app in the field and hands you the results, so you see real users' launch times, hangs, hitches, memory, battery and terminations instead of only what you can reproduce. It delivers two kinds of data: **metrics**, aggregated over roughly a day (`MXMetricPayload`), and **diagnostics** with call stacks for crashes, hangs, CPU and disk-write exceptions and slow launches (`MXDiagnosticPayload`), which arrive soon after the event, typically on the next launch.

Key points:

- iOS 18 baseline: `MXMetricManager.shared.add(subscriber)` with `didReceive(_: [MXMetricPayload])` and `didReceive(_: [MXDiagnosticPayload])`; keep a strong reference to the subscriber
- Metrics: `applicationLaunchMetrics` (time to first draw, resume time), `applicationResponsivenessMetrics` (hang time), `animationMetrics` (scroll hitch ratio; `hitchTimeRatio` iOS 26+), `memoryMetrics` (peak and suspended memory), `applicationExitMetrics` (memory-limit, watchdog and other terminations), CPU, GPU, disk, network, signposts
- Diagnostics: `crashDiagnostics`, `hangDiagnostics`, `cpuExceptionDiagnostics`, `diskWriteExceptionDiagnostics`, `appLaunchDiagnostics`, each with an **unsymbolicated** call-stack tree
- Custom intervals: `mxSignpost(.begin/.end, log: MXMetricManager.makeLogHandle(category:), name:)` (or `MetricManager.logHandle(category:)`, iOS 27+) so your own operations show up in metrics
- iOS 27+: `MetricManager().metricReports` / `.diagnosticReports`, results as `MetricResult` / `DiagnosticResult` enums (including `.memoryException`), `trackLaunchTask(id:)` for extended launch tasks
- Only from devices where users share analytics with developers; the data matches Xcode Organizer's, but you get it raw and per release

Related to check: Startup optimization, OOM, Symbolication, Scroll performance (hitch ratio), Main thread (hangs), Xcode Organizer

---

OOM (Out of Memory)

Q: What causes OOM crashes? How do you prevent them?

Short answer: An OOM termination happens when the app's memory **footprint** (dirty plus compressed memory) crosses the per-process limit and the kernel's memory manager, jetsam, kills it. The limit depends on the device's RAM and the app's state, so there's no single number to design against, and foreground apps get far more than suspended ones.

Key points:

- Jetsam kills on footprint, not virtual size; clean file-backed memory (code, mapped read-only files) can be dropped by the OS and is cheap
- No callback at the moment of death; memory warnings may come first, but a fast spike can skip them
- `os_proc_available_memory()` returns the bytes left before *this* process hits its limit: use it to scale work (decode size, batch size, cache limits)
- Entitlements: `com.apple.developer.kernel.increased-memory-limit` asks for a higher limit on supported devices (still check headroom at runtime); `extended-virtual-addressing` is for address space, not footprint
- Production signals: `MXAppExitMetric.foregroundExitData.cumulativeMemoryResourceLimitExitCount` (iOS 18 baseline), and on iOS 27+ `MetricResult.foregroundTermination` (`memoryLimitTerminationCount`) and `DiagnosticResult.memoryException` with a call stack
- Background: suspended apps with large footprints are evicted first (`cumulativeMemoryPressureExitCount` in background exit data)

Related to check: Memory warnings, Image downsampling, NSCache, VM Tracker, Leaks instrument, MetricKit

---

Async Image Decoding

Q: Why decode images off the main thread? How do you do it?

Short answer: A `UIImage` created from JPEG or PNG data is *not* decoded yet; the compressed bytes are expanded into a bitmap the first time it's drawn, which by default happens on the main thread during the Core Animation commit. For a large photo that can take tens of milliseconds, far more than the 8.3 ms frame budget at 120 Hz, so scrolling hitches.

Key points:

- Decoding is lazy and, by default, happens on main at first render
- Decoded size is `width x height x 4` bytes regardless of file size (a 12 MP photo ≈ 48 MB)
- `byPreparingForDisplay()` (async) / `preparingForDisplay()` (sync): decode at full size
- `byPreparingThumbnail(ofSize:)` / `preparingThumbnail(of:)`: decode *and* shrink in one step
- Assign on the main actor; cancel stale work when cells are reused

Related to check: Image downsampling, image loading and caching, lazy loading strategy, animation performance, scroll performance, NSCache, @concurrent

---

String and Collection Performance

Q: What are performance considerations for Strings and collections in Swift?

Short answer: Swift `String` is a Unicode-correct value type stored as UTF-8, with copy-on-write and an inline small-string representation for up to 15 bytes. Its main performance trait is that `Character` is a grapheme cluster, so `count` and moving an index are O(n), and strings are indexed with `String.Index`, not `Int`; when you need speed you work on the `utf8` view (or, on iOS 26+, a `UTF8Span`).

Key points:

- `str.count` walks the whole string; cache it or compare `isEmpty` instead of `count == 0`
- `Substring` shares the original's storage: great for parsing, but convert to `String` before storing so you don't keep a huge base string alive
- Appending with += to a uniquely owned `var` is amortized O(1); `s = s + x` and interpolation in a loop create new strings. `joined(separator:)` is clear and efficient
- Byte-level work (parsing, hashing, ASCII checks) on `s.utf8` avoids grapheme breaking
- Bridging: `NSString` APIs on a Swift `String` may convert between UTF-8 and UTF-16 storage
- Collections: `reserveCapacity`, `Set` for membership instead of `array.contains` in a loop, `removeFirst()` on `Array` is O(n) (use an index or a deque), `lazy` to avoid intermediate arrays
- Swift 6.2, iOS 26+: `String.utf8Span` / `UTF8Span` and `Span` give borrowed, bounds-checked access to contiguous UTF-8 without copying or retaining

Related to check: Copy-on-write, Object allocation, Heap vs stack, `InlineArray` / `Span`, Time Profiler

---

Memory Graph Debugger

Q: How do you use the Memory Graph Debugger?

Short answer: The Memory Graph Debugger pauses the running app and captures every live heap object plus the references between them. You open it from the *Debug Memory Graph* button in Xcode's debug bar; the navigator lists every type with its instance count, and selecting an instance shows the graph of who references it, back to a root.

Key points:

- Capture: debug bar button (three connected circles) while the app runs; the app is paused while you inspect
- Navigator filters: *Show only leaked blocks*, *Show only content from workspace*, search by type name
- The graph shows incoming references; bold lines are strong, light lines are weak/unowned

- Turn on **Malloc Stack Logging** (Scheme › Run › Diagnostics, “Live Allocations Only”) to see the allocation backtrace in the inspector
- `File › Export Memory Graph` saves a `.memgraph` for `leaks`, `heap`, `vmmap`, `footprint` and `malloc_history`
- Same leak-detection engine as the Leaks instrument, but interactive and point-in-time; Instruments is better for growth over time

Related to check: Debug Memory Graph for retain cycles, Leaks instrument, Retain cycles, VM Tracker, Allocations vs Leaks

---

Instruments Allocations

Q: How do you use the Allocations instrument?

Short answer: Allocations records every heap allocation and free, plus anonymous VM regions, with the call stack that made each one. The core technique is **generation analysis**: get the app to a steady state, press *Mark Generation*, perform an action that should leave memory unchanged (open and close a screen), mark again, and repeat several times.

Key points:

- Profile (I) on a device with a Release build -> *Allocations* (or *Leaks*, which includes it)
- Statistics: *All Heap & Anonymous VM*, *Persistent* vs *Transient* counts, *Persistent Bytes*
- Lifespan filter: *Created & Persistent* (still alive) vs *All Allocations*
- **Mark Generation** between repeated actions; steady per-iteration growth = problem; one-time growth = cache or warm-up
- Call Tree: *Invert Call Tree*, *Hide System Libraries* to land on your code
- Optional *Record reference counts* to see each retain/release on an object (for “who’s holding this?”)

Related to check: Allocations vs Leaks, Leaks instrument, Debug Memory Graph, VM Tracker, image downsampling, OSSignposter, MetricKit, OOM / jetsam

---

Leaks Instrument

Q: How does the Leaks instrument work?

Short answer: The Leaks instrument periodically takes a snapshot of the process’s heap and scans it, starting from the roots (globals, thread stacks, registers), for anything that still points at each allocation. A block that nothing reachable points to is reported as leaked, and a group of objects that only point at each other is reported as a cycle in the *Cycles & Roots* view, with the allocation backtrace if malloc stack logging is on.

Key points:

- Leak = unreachable from any root. Reachable-but-useless memory is not a “leak” to the tool
- The scan is *conservative*: any word that looks like a pointer keeps a block alive, so it can miss leaks (false negatives) but rarely reports false ones
- In Swift the usual result is a reference cycle: closure owner, delegate held strongly, `Timer` target, a stored `Task` capturing `self`
- The *Cycles & Roots* graph shows the cycle; the stack trace shows **where it was allocated**, not who retains it. Use the Memory Graph’s reference list for that
- Run the flow several times (open/close the screen) so leaks accumulate and stand out
- Xcode 27 adds **run comparison** in Instruments: compare a before/after trace to confirm a fix and catch regressions

Related to check: Memory Graph Debugger, Allocations vs Leaks, Retain cycles, Weak references, VM Tracker

---

VM Tracker

Q: What does the VM Tracker show?

Short answer: VM Tracker is the Instruments tool that shows the app's **virtual memory regions** by type, with how much of each is dirty, compressed and resident, captured as periodic snapshots. That matters because the Allocations instrument only sees `malloc` heap objects, while a lot of real memory lives elsewhere: decoded images and GPU textures in IOSurface and IOKit regions, Core Animation backing stores, memory-mapped files, and the app's own `__DATA` segments.

Key points:

- **Dirty**: pages the app has written (heap, decoded images, globals); can't be discarded, only compressed
- **Compressed (shown as Swapped)**: dirty pages the OS has compressed in RAM; iOS doesn't swap to disk, and compressed pages still count toward the footprint
- **Resident**: pages currently in physical memory, clean or dirty
- **Virtual**: reserved address space; large numbers here are normal and not a problem on their own
- Region types to know: `MALLOC_*` (heap), `IOSurface` / `IOKit` / `IOAccelerator` (images, video, GPU), `CoreAnimation`, `CG raster data`, `mapped file`, `__DATA` / `__TEXT`
- Snapshots are periodic (turn on automatic snapshotting); compare snapshots before and after a flow

Related to check: Allocations vs Leaks, OOM, Image downsampling, Memory Graph Debugger, Memory warnings

---

Allocations vs Leaks

Q: What's the difference between Allocations and Leaks?

Short answer: They answer different questions. **Leaks** periodically scans the heap and reports memory that nothing reachable points to any more: in Swift that is almost always a retain cycle, such as an object and a closure that own each other after the screen is gone.

Key points:

- **Leaked memory:** unreachable from any root. Leaks instrument, Memory Graph debugger (purple "!" badges)
- **Abandoned memory:** reachable but never used again. Only visible as growth: Allocations generation analysis
- **Cached memory:** reachable and reused on purpose; should be bounded (`NSCache`, size caps) and released under memory pressure
- Allocations tracks persistent vs transient counts, total bytes, and the allocating call stack
- Run both from the **Leaks** template (it includes Allocations), on a device, with a Release-like build

Related to check: Instruments Allocations (generation workflow), Leaks instrument, Debug Memory Graph, retain cycles, NSCache, memory warnings, OOM/jetsam, MetricKit

---

Symbolication

Q: How do you symbolicate a crash log?

Short answer: A crash report from a Release build contains only memory addresses: each frame is a binary image (identified by its build **UUID**) plus an offset. Symbolication maps those addresses back to function names, files and line numbers using the **dSYM** produced by that exact build, so the whole job comes down to keeping the right dSYM and matching UUIDs.

Key points:

- Build setting: *Debug Information Format* = **DWARF with dSYM File** for Release; archive every build you distribute
- UUID match is mandatory: `dwarfdump --uuid` on the dSYM must equal the UUID in the report's binary images section; a rebuild of "the same" code has a different UUID
- Organizer › Crashes: symbolicated automatically for App Store/TestFlight builds uploaded with symbols; you can open a crash in Xcode at the faulting line
- Device crash logs: `.ips` (JSON, iOS 15+); Xcode's *Devices and Simulators* › device logs symbolicates them when dSYMs are indexed by Spotlight
- Command line: `atos` for addresses, and Xcode's `CrashSymbolicator.py` for whole `.ips` files; `symbolicatecrash` is the legacy script for the old text format
- Bitcode was removed in Xcode 14, so the dSYM from your archive is final; there's no need to download recompiled dSYMs from App Store Connect anymore
- MetricKit `MXDiagnosticPayload` (and the iOS 27+ `DiagnosticReport`) call-stack trees contain binary UUIDs and offsets for server-side symbolication

Related to check: MetricKit, Reduce binary size (stripping), Crash reporting, OOM, Main thread (watchdog)

Address Sanitizer

Q: What does Address Sanitizer detect?

Short answer: Address Sanitizer (ASan) is a compiler-plus-runtime tool that catches memory-corruption bugs the moment they happen: use-after-free, heap/stack/global buffer overflows, double free, and use-after-scope. The compiler instruments every memory access and the runtime keeps "shadow memory" plus poisoned red zones around allocations, so a bad access stops the app with a report showing where the memory was allocated, freed and touched.

Key points:

- Enable: Scheme -> Run (or Test) -> Diagnostics -> Address Sanitizer; rebuilds with `-sanitize=address`
- Detects: heap-use-after-free, heap/stack/global buffer overflow, double free, use-after-scope (stack-use-after-return is optional)
- Cost: roughly 2 - 3x CPU and 2 - 3x memory; Debug and CI only
- Not a leak finder on iOS: use the Leaks instrument / Memory Graph for leaks, Thread Sanitizer for races
- **Strict memory safety** (Swift 6.2, opt-in via `-strict-memory-safety` / build setting): warns on every use of unsafe constructs until you acknowledge it with `unsafe`, so unsafe code is visible and auditable

Related to check: Thread Sanitizer, UBSan, Leaks instrument, Memory Graph debugger, Span / InlineArray (heap-vs-stack), unsafe pointers, test plans

---

Thread Sanitizer

Q: What does Thread Sanitizer detect?

Short answer: Thread Sanitizer (TSan) is a runtime tool that instruments memory accesses and reports data races: two threads touching the same memory without synchronization, at least one writing. In Swift

6 language mode the compiler already rejects most data races, so TSan's job shifts to the places the compiler can't see or was told to trust: `@unchecked Sendable`, `nonisolated(unsafe)`, unsafe pointers, and C/Objective-C code.

Key points:

- Detects data races (with both stack traces and the thread that created each), and some lock misuse
- **Enable:** Edit Scheme -> Run or Test -> Diagnostics -> Thread Sanitizer; or `xcodebuild test -enableThreadSanitizer YES`
- Runs on macOS and simulators, not physical iOS devices; can't be combined with Address Sanitizer
- Significant overhead (several times slower, much more memory), so use it in debug and CI, never release builds
- **Complements Swift 6 checking:** compile time covers checked Swift code; TSan covers the escape hatches and non-Swift code

Related to check: Race conditions vs data races, thread safety, Sendable, NSLock and Mutex, Address Sanitizer, Main Thread Checker

---

UBSan

Q: What does Undefined Behavior Sanitizer catch?

Short answer: The Undefined Behavior Sanitizer instruments **C, Objective-C and C++** code to catch operations the language standard leaves undefined, which can silently produce wrong results or behave differently under optimization: signed integer overflow, shifts by too many bits, misaligned pointer access, null pointer dereference, out-of-bounds indexing of fixed-size arrays, invalid `BOOL`/enum values, and division by zero. You turn it on in the scheme (Run or Test › Diagnostics › *Undefined Behavior Sanitizer*), and it reports the exact line at runtime as an Xcode runtime issue.

Key points:

- Checks are compiled into C-family code; there's no effect on Swift source
- Catches: signed overflow, invalid shifts, misaligned loads/stores, null dereference, array bounds (known size), invalid enum/bool loads, integer divide by zero, unreachable code reached, some invalid casts
- Low overhead compared with ASan (roughly 20% in Apple's guidance), so it's reasonable to leave on for Debug and test runs
- Combinable with **Address Sanitizer** or **Thread Sanitizer** (ASan and TSan can't run together)
- Swift's equivalents are built in: overflow traps (use `&+`, `&*` or `addingReportingOverflow` when wrapping is intended); unsafe pointer misuse is caught by ASan and, at compile time, by Swift 6.2's opt-in **strict memory safety** checking

Related to check: Address Sanitizer, Thread Sanitizer, Strict memory safety (Swift 6.2), Symbolication

---

Memory Warning Handling

Q: How do you handle didReceiveMemoryWarning?

Short answer: `didReceiveMemoryWarning()` is the `UIViewController` hook UIKit calls on the main thread when the system is short of memory; you call `super` and then drop whatever that controller can rebuild, such as decoded images, cached layouts or off-screen child content. Since iOS 6 UIKit no longer unloads views for you, so the override only frees what you free.

Key points:

- Three delivery points, all on the main thread: `UIViewController.didReceiveMemoryWarning()`, `UIApplicationDelegate.applicationDidReceiveMemoryWarning(_:)`, and the `didReceiveMemoryWarningNotification`
- There is **no** `sceneDidReceiveMemoryWarning`; memory warnings are app-wide, not per scene
- iOS 26+: `NotificationCenter.default.addObserver(of: UIApplication.shared, for: .didReceiveMemoryWarning) { ... }` delivers a typed `MainActorMessage`, so the closure is main-actor isolated in Swift 6
- Outside UIKit (extensions, frameworks): `DispatchSource.makeMemoryPressureSource(eventMask: [.warning, .critical], queue:)`
- `NSCache` already evicts on its own; you only need handlers for plain dictionaries, arrays of decoded data, and resources NSCache can't see
- Keep the handler cheap: release references, don't allocate or start new work

Related to check: Handling Memory Warnings (what to release), UIKit memory warnings in view controllers, NSCache, OOM, Image downsampling

---

Image Downsampling

Q: How do you downsample images to reduce memory?

Short answer: An image's memory cost is its *decoded* size, `width x height x 4` bytes, not its file size: a 4032 x 3024 photo is a ~1 MB JPEG but ~48 MB in memory once drawn, even inside a 100-point thumbnail. Downsampling means decoding straight to the pixel size you'll display, so the full-resolution bitmap never exists.

Key points:

- Memory = decoded pixels; target pixel size = points x `displayScale`
- ImageIO: `kCGImageSourceShouldCache: false` on the source, then a thumbnail with `...MaxPixelSize`, `...CreateThumbnailFromImageAlways`, `...WithTransform` (EXIF orientation), `...ShouldCacheImmediately` (decode now, off main)
- Works from a URL or `Data` (`CGImageSourceCreateWithData`) without loading a full bitmap
- UIKit alternative: `preparingThumbnail(of:)` / `byPreparingThumbnail(ofSize:)` on a `UIImage`
- Scale comes from the view's trait collection (`UIScreen.main` is deprecated and wrong for external displays and resizable windows)

Related to check: Async image decoding, image loading and caching, lazy loading strategy, NSCache, OOM / memory warnings, adaptive layout, VM Tracker

---

Lazy Loading Strategy

Q: What's a good lazy loading strategy for images?

Short answer: Load an image only when its cell is about to be visible, start slightly *before* that with prefetching, and cancel the moment it's no longer needed. In UIKit that's `UICollectionViewDataSourcePrefetching`: `prefetchItemsAt` warms the cache for upcoming rows and `cancelPrefetchingForItemsAt` cancels them when the user changes direction; the cell itself loads in `configure` and cancels in `prepareForReuse`.

Key points:

- On demand: nothing loads for rows that never come near the screen
- Prefetch a small window ahead; cancel prefetches that scroll out of range
- Cancel per cell/row: `prepareForReuse` (UIKit), `.task(id:)` (SwiftUI, automatic)
- Shared loader (an `actor`): memory cache (`NSCache`) + disk/HTTP cache + in-flight dedupe

- Downsample to the cell's pixel size (`displayScale`), decode off main

Related to check: Image loading and caching, image downsampling, async image decoding, lazy loading, scroll performance, NSCache, pagination, Task cancellation

---

CADisplayLink

Q: What is CADisplayLink? When do you use it?

Short answer: `CADisplayLink` is a timer synchronized to the display's refresh: it calls you once per frame, just before the frame is rendered, with `timestamp` (the current frame) and `targetTimestamp` (when the next frame will be shown). You use it for frame-by-frame work that the system animation APIs can't express: game loops, physics, custom drawing, or syncing to a scrubbing gesture.

Key points:

- Fires once per display frame on the run loop you add it to (`.main`, mode `.common` so it keeps firing during scrolling)
- Use `targetTimestamp - previous` for delta time; never assume a fixed frame rate
- `preferredFrameRateRange` (`CAFrameRateRange`) replaces the deprecated `preferredFramesPerSecond`
- Above 60 Hz on iPhone requires `CADisableMinimumFrameDurationOnPhone = YES` in Info.plist
- Your callback must finish within the frame budget: 8.3 ms at 120 Hz, 16.7 ms at 60 Hz
- **`UIUpdateLink`** (iOS 18): tied to a view or window scene, `requiresContinuousUpdates`, `UIUpdateInfo.modelTime` / `completionDeadlineTime`, low-latency options for drawing apps

Related to check: Animation performance, CATransaction, UIView animations, retain cycles, Metal / GPU, TimelineView, battery optimization

---

CATransaction

Q: How do you batch Core Animation changes?

Short answer: Every change to a layer tree happens inside a `CATransaction`. If you don't create one, Core Animation opens an *implicit* transaction for you and commits it at the end of the current run-loop pass, so all changes made in that pass reach the render server together in one commit.

Key points:

- Implicit transaction per run-loop pass; explicit `begin()`/`commit()` to scope settings
- `setDisableActions(true)`: apply property changes instantly (standalone layers otherwise animate implicitly for 0.25 s)
- `setAnimationDuration`, `setAnimationTimingFunction`: defaults for implicit animations in the group
- `setCompletionBlock`: called on the main thread when all animations *added after it* in the group finish
- Transactions nest; the outermost commit is what reaches the render server

Related to check: Animation performance, CADisplayLink, UIView animations, Core Animation layers, SwiftUI Transaction, offscreen rendering

---

Energy Log and Power Profiler

Q: What does the Energy Log show, and what do you use instead in Xcode 26 and later?

Short answer: The Energy Log was the older Instruments template (and on-device logging option) that showed an *energy impact* level over time plus the activity behind it: CPU, network, location, GPU, display, and foreground/background state. Its job was to tell you which subsystem was keeping the device awake.

Key points:

- Energy Log (legacy): energy impact rating over time, CPU activity, network, location, GPU, display brightness, sleep/wake and app state
- **Power Profiler** (Xcode 26): system power plus per-subsystem impact for your app; runs alongside Time Profiler and other instruments in one trace
- Quick check: Xcode's debug navigator **Energy Impact** gauge (overhead, CPU, network, location, GPU, background)
- Field data: Xcode Organizer energy reports and MetricKit metrics / CPU-exception diagnostics
- The output is "what woke up and when"; the fix is in your code (polling, timers, location, rendering)

Related to check: Battery optimization, Instruments, Time Profiler, background fetch, CADisplayLink, MetricKit, os_signpost, location services

---

Metal and GPU

Q: When would you use Metal for performance?

Short answer: Metal is Apple's low-level GPU API, and I reach for it when the work is massively parallel and the higher-level frameworks don't cover it: custom rendering and games, real-time video or camera filters, large image or signal processing, and custom compute kernels. Most apps never need raw Metal, because Core Image, Metal Performance Shaders, Core ML, RealityKit, SpriteKit and SwiftUI shader effects already run on the GPU for you.

Key points:

- Good fits: per-pixel or per-element work over large data, rendering at 60 - 120 fps, real-time video, ML-style math not covered by Core ML
- Try higher-level first: Core Image filters, MPS/MPSGraph, Accelerate (vDSP/BNNS) for CPU SIMD, SwiftUI `.colorEffect` / `.layerEffect` shaders
- Apple silicon has **unified memory**: `.storageModeShared` buffers avoid copies between CPU and GPU
- Create the device, queue and pipeline states **once**; per frame you only encode commands
- Never stall the CPU waiting on the GPU on the main thread (`waitUntilCompleted`); use completion handlers and multiple in-flight frames (triple buffering)
- Tools: Xcode GPU frame capture and the Metal debugger, the Metal performance HUD, and the *Metal System Trace* and *Game Performance* Instruments templates

Related to check: Image downsampling, Offscreen rendering, Scroll performance, VM Tracker (IOSurface memory), CADisplayLink

---

Reduce Binary Size

Q: How do you reduce app binary size?

Short answer: I start by measuring what users actually download, because the App Store thins every build per device: App Store Connect shows download and install sizes per device, and exporting with app thinning produces an *App Thinning Size Report* locally. Then I attack the biggest contributors, which are

usually assets and third-party SDKs rather than my own code: compress and de-duplicate images in asset catalogs, move large optional content out of the bundle (Background Assets, or on-demand resources in older apps), and drop dependencies I barely use.

Key points:

- Measure first: App Store Connect size per device, the App Thinning Size Report, and the linker map file (`LD_GENERATE_MAP_FILE`) to see which modules are big
- App thinning: the store delivers only the slices and asset variants (scale, GPU class, memory class) a device needs, so asset catalogs beat loose files
- Code settings (Release): Dead Code Stripping, Strip Linked Product + Strip Style *All Symbols*, Strip Swift Symbols, `SWIFT_OPTIMIZATION_LEVEL = -Osize` where speed allows
- Assets: asset catalog compression (`ASSETCATALOG_COMPILER_OPTIMIZATION = space`), HEIF/lossy where acceptable, vector PDFs/SF Symbols instead of many PNGs, remove unused assets and localizations you don't ship
- Large content: Background Assets (managed asset packs, iOS 26+) or a download after first launch, instead of bundling
- Dependencies: each SDK adds code and often resources; static linking lets the linker dead-strip unused code

Related to check: Startup optimization, Symbolication, Lazy loading strategy, Image downsampling, Swift Package Manager

---

Startup Optimization

Q: How do you measure and improve startup time?

Short answer: I measure first, in three places: in the field with Xcode Organizer's launch-time report and MetricKit's `MXMetricPayload.applicationLaunchMetrics` (time to first draw, resume time), locally with the Instruments *App Launch* template on a real device with a Release build, and in CI with an XCTest launch performance test so regressions fail a build. Then I work through the two phases.

Key points:

- Launch types: cold (after reboot or long idle), warm (binary cached), resume (from background, not a launch); measure cold and warm separately
- Pre-main: each dynamic framework costs load and bind time; prefer static linking or **mergeable libraries** (Xcode 15+); avoid global initializers with side effects
- Post-main: no synchronous I/O, network or heavy decoding before the first frame; lazily create services; start deferred work from `.task` on the root view, off the main actor where possible
- Watchdog: blocking launch for too long gets the app killed (`0x8badf00d`)
- Field data: MetricKit launch histograms and `appLaunchDiagnostics`; iOS 27+ `MetricManager.trackLaunchTask(id:)` measures extended launch tasks (work the user waits on after the first frame)
- Prewarming: the system may start your process early, so don't time launch from `main()` yourself

Related to check: MetricKit, Lazy loading, Reduce binary size, Main thread, Time Profiler, App launch

---

Data Persistence

UserDefaults

Q: What is UserDefaults? When would you use it?

Short answer: `UserDefaults` is a persistent key-value store for small preferences and app state: settings, flags, the last selected tab. It's backed by a property list in the app's `Library/Preferences`, cached in memory, and safe to call from any thread.

Key points:

- Property-list types only: `String`, numbers, `Bool`, `Date`, `Data`, `URL`, and arrays/dictionaries of them; encode `Codable` values to `Data` (or use `RawRepresentable` with `@AppStorage`)
- `register(defaults:)` sets fallback values in memory (not saved); `bool(forKey:)` and `integer(forKey:)` return `false`/`0` for missing keys
- Suites: `UserDefaults(suiteName: "group.com.example.app")` shares values with widgets and extensions through an App Group
- `@AppStorage("key")` (optionally `store:`) binds a view to a key and refreshes the view when the value changes
- `synchronize()` is unnecessary (Apple documents it as slated for deprecation); writes are persisted automatically
- Not for secrets (plain plist, included in backups) or large data (the whole domain is loaded into memory)

Related to check: When to Use UserDefaults, Keychain, `@AppStorage`, File Storage, Securing Sensitive Data, testing UserDefaults

---

When to Use UserDefaults

Q: When should you use UserDefaults versus Core Data or Keychain?

Short answer: Pick storage by what the data *is*. Small, non-sensitive preferences go in `UserDefaults`.

Key points:

- `UserDefaults`: settings, flags, small values; plain plist, loaded into memory, backed up, a required-reason API
- Keychain: tokens, passwords, keys; encrypted, per-item accessibility, can be shared with extensions
- SwiftData / Core Data: entities, relationships, predicates, migrations, iCloud sync; SwiftData is the default for new SwiftUI apps
- Files: images, downloads, documents, exports; Documents vs Application Support vs Caches, plus Data Protection
- Memory caches (`NSCache`) for re-creatable data that should disappear under memory pressure

Related to check: UserDefaults, Keychain, Core Data, File Storage, SQLite on iOS, SwiftData and SwiftUI

Data Modeling for Mobile

Q: How do you design a local data model for a mobile app that caches server data, works offline and syncs? What would you put on every entity?

Short answer: A mobile schema is not a copy of the server's tables: it's shaped by the screens that read it and by the fact that it syncs. I keep the **source-of-truth entities normalized** (one row per real thing, referenced by a stable ID) and add **denormalized fields or cache tables** only where a hot screen needs them, such as a count or a preview.

Key points:

- IDs: client-generated UUIDs (or ULIDs/UUIDv7 if you want time ordering) so objects can be created offline and referenced before the server has seen them; never use the database's internal ID (`PersistentIdentifier`, `NSManagedObjectID`, SQLite rowid) as an external identifier

- Timestamps: `createdAt`, `updatedAt` for change tracking; a server revision or ETag for optimistic concurrency; store dates in UTC and treat device clocks as untrustworthy for ordering across devices
- Soft deletes: `deletedAt` hides a row, the sync layer uploads the tombstone, and a retention job purges acknowledged tombstones later
- Uniqueness: `#Unique` on the UUID (iOS 18) makes server imports idempotent upserts instead of duplicates
- Denormalize deliberately: cached counts and previews must be updated in the same save as the change they summarize

Related to check: Offline Persistence, Data Synchronization, Data Deletion and Retention, Core Data Migration, SQLite on iOS, offline-first architecture, repository pattern

---

SwiftData vs Core Data

Q: We're starting a new feature. Should we use SwiftData or Core Data, and what would make you pick Core Data in 2026?

Short answer: For a new app or feature on an iOS 18+ target, SwiftData is the default: the schema lives in Swift, models are `Observable`, `@Query` drives SwiftUI, and `@ModelActor` plus Swift 6 checking keep concurrency honest. Core Data is still the right call when you need what SwiftData doesn't expose: store-level batch inserts and updates, custom merge policies, aggregate fetches, `NSFetchedResultsController` on older OS versions, or CloudKit **sharing** and **public** databases.

Related to check: Migrating from Core Data to SwiftData, SwiftData Basics, Core Data, SwiftData and CloudKit, SwiftData Custom Data Store, Core Data in Swift 6, SQLite on iOS, Realm

---

SwiftData Basics

Q: What is SwiftData, and how does it actually work under the hood?

Short answer: SwiftData is Apple's Swift-native persistence framework: you declare the schema in code with `@Model` classes, a `ModelContainer` owns the schema and the stores, and a `ModelContext` tracks inserts, edits and deletes until you save. The `@Model` macro rewrites each stored property into accessors backed by the model's persistent backing data and makes the class `Observable`, which is why SwiftUI and Observation see changes.

Key points:

- **`@Model`** generates `PersistentModel` + `Observable` conformance. Attributes can be any `Codable` value type: enums, and structs (stored as composite attributes). Properties that hold other models become relationships. `@Attribute(...)` adds options (`.unique`, `.externalStorage`, `.allowsCloudEncryption`, `.preserveValueOnDeletion`, `.spotlight`) and `originalName:` for renames; `@Transient` excludes a property
- **`ModelContainer`**: schema + one or more `ModelConfigurations` (stores) + an optional migration plan. It's `Sendable`; create one per store and share it
- **`ModelContext`**: the unit of work: `insert`, `delete`, `fetch`, `save`, `rollback`, `hasChanges`, an optional `undoManager`. `container.mainContext` is `@MainActor` and autosaves. Contexts are **not** `Sendable`
- **Identity:** `persistentModelID` (`PersistentIdentifier`) is `Sendable` and `Codable`. It's **temporary until the first save**, then replaced by a permanent one (`isTemporary` (iOS 27+) tells you which)
- **Reading:** `FetchDescriptor` + `#Predicate` anywhere; `@Query` in SwiftUI. iOS 27 adds `@Query(..., sectionBy:)` and `ResultsObserver`, an `Observable` live result set for UIKit and other non-SwiftUI code (iOS 27+)

Related to check: SwiftData and SwiftUI, ModelContainer Configuration, ModelActor and Concurrency, Predicate and FetchDescriptor, SwiftData Relationships, SwiftData Attributes, When to Save, SwiftData vs Core Data, Core Data

---

ModelContainer and ModelConfiguration

Q: How do you configure a SwiftData ModelContainer? Say you need a store shared with a widget, a local cache that never syncs, and an in-memory setup for tests.

Short answer: A `ModelContainer` is the schema plus one or more `ModelConfigurations`, and each configuration describes one store: its name, which models it holds, where it lives, whether it's read-only or in memory, and whether it syncs with CloudKit. You build the container once at launch, handle the error properly, and inject it with `.modelContainer(_:)`.

Key points:

- `ModelContainer(for: Item.self)` uses defaults: one store named `default.store` in Application Support, CloudKit `.automatic`, group container `.automatic`
- `ModelConfiguration(_ name:, schema:, isStoredInMemoryOnly:, allowsSave:, groupContainer:, cloudKitDatabase:)`, or the `url:` initializer to choose the file location yourself
- **App Group:** `groupContainer: .identifier("group.com.example.app")` puts the store where your widget and extensions can open it
- **CloudKit:** `cloudKitDatabase: .private("iCloud....")`, `.automatic` (uses the CloudKit container in your entitlements, if there is one) or `.none`. SwiftData offers only the private database
- **Multiple configurations:** give each one a `schema` listing its models; the container's schema is the union. Each configuration gets its own store identifier, and iOS 27 adds `storeIdentifier(forConfigurationNamed:)` and `configurationName(forStoreIdentifier:)` to map between them (iOS 27+)
- `isStoredInMemoryOnly: true` for previews and tests; `allowsSave: false` for a read-only seed store shipped in the bundle

Related to check: SwiftData Basics, SwiftData and CloudKit, SwiftData History Tracking (widget and extension changes), SwiftData Migration, App Groups and Shared Containers, Testing Persistence, Persistent Container (the Core Data equivalent), SwiftData and SwiftUI, WidgetKit (App Groups)

SwiftData Attributes

Q: What can you configure with `@Attribute` in SwiftData? And how do you store an enum, a Codable struct, or a property you don't want persisted at all?

Short answer: Every stored property of a `@Model` class becomes an attribute automatically; `@Attribute` is only for changing *how* it's stored. The options I reach for are `.unique` (an upsert key), `.externalStorage` (large `Data` kept outside the SQLite rows), `.transformable(by:)` (a `ValueTransformer` for a class type like `UIColor`), `.preserveValueOnDeletion` (keep the value in history after a delete), and `originalName:` (rename a property without losing data).

Key points:

- `@Attribute(.unique)`: inserting a model whose unique value already exists **updates** the existing row instead of adding a duplicate. For multi-property keys use `#Unique` (see SwiftData Index and Unique)
- `@Attribute(.externalStorage)`: a hint that the store may keep a large value as a separate file beside the database; use it for images and documents, not for small blobs

- `@Attribute(.transformable(by: MyTransformer.self))`: for non-Codable class types (`UIColor`, `NSAttributedString`); subclass `NSSecureUnarchiveFromDataTransformer` and register it before creating the container
- `@Attribute(.preserveValueOnDeletion)`: the value is kept in the history tombstone when the model is deleted, so a sync process can read, say, the `serverID` of what was deleted
- `@Attribute(originalName: "fullName")`: the old column name, so a rename is a lightweight migration instead of drop-and-add
- `@Attribute(.allowsCloudEncryption)`: encrypt the value in CloudKit. `@Attribute(.spotlight)`: index the value for Spotlight
- **Codable values**: an enum, a struct, or an array of them is stored without any attribute. A Codable struct is stored as a *composite* attribute (its fields become columns). `@Attribute(.codable)` **(iOS 27+)** stores it as a single encoded blob instead
- `@Transient`: excluded from the schema and never persisted; it needs a default value

Related to check: SwiftData Basics, SwiftData Index and Unique, SwiftData Migration, SwiftData History Tracking (`preserveValueOnDeletion`), SwiftData and CloudKit, Predicate and FetchDescriptor, File Storage

---

SwiftData Relationships

Q: How do relationships work in SwiftData? Explain delete rules and inverses, and how you'd keep the order of items in a playlist.

Short answer: A relationship is just a property whose type is another model (`var playlist: Playlist?`) or an array of them (`var entries: [PlaylistEntry]`). `@Relationship` configures the details: the delete rule, the inverse key path, count limits and `originalName` for renames.

Key points:

- Delete rules: `.nullify` (default: clear the other side's reference), `.cascade` (delete related objects too), `.deny`, `.noAction`
- Inverses: one-to-many pairs are inferred, but name them anyway. For many-to-many, put `@Relationship(inverse: \Tag.playlists)` on one side; without it you get two independent relationships and the other side stays empty
- Order: the array you get back after a reload is in arbitrary order. Store a `position` (or a timestamp) and sort, usually on a **join model** that can also carry data such as `addedAt`
- Make relationships optional (`Playlist?`, `[PlaylistEntry]? = []`) if you might sync with CloudKit, which requires it
- Predicates can traverse relationships (`$0.playlist?.name == name`, `tags.contains { ... }`), and `relationshipKeyPathsForPrefetching` loads them up front for lists

Related to check: SwiftData Basics, Predicate and FetchDescriptor, SwiftData Model Inheritance, SwiftData and CloudKit, SwiftData Migration (`originalName`), Data Modeling for Mobile, Core Data (delete rules and inverses come from the Core Data model)

#Predicate and FetchDescriptor

Q: How do you query SwiftData outside of @Query? Walk me through #Predicate and FetchDescriptor, and what goes wrong with them.

Short answer: `FetchDescriptor<T>` is a SwiftData query: a `#Predicate` filter, `SortDescriptors`, paging (`fetchLimit` / `fetchOffset`) and loading hints, run with `context.fetch`, `fetchCount`, `fetchIdentifiers` or `enumerate`. `#Predicate` is a macro that turns a Swift closure into an expression tree, which the store translates to SQL, so the filter runs in the database, not in memory.

Key points:

- Filters: comparisons, && / || / !, optional chaining (`$0.folder?.name == name`), `localizedStandardContains`, `contains`, `starts(with:)`, to-many `contains { }` and `.count` on relationships (test these; `.count` on an optional `[T]?` relationship threw at runtime)
- Captured values must be local constants: copy `self.searchText` or a model's ID into a `let` before the macro
- Compare relationships by ID: `$0.folder?.persistentModelID == folderID`. Comparing to a model instance doesn't compile
- Compose predicates with `other.evaluate($0)` inside a new `#Predicate`
- `fetchCount` for badges, `fetchIdentifiers` when you only need IDs (for example, to hand to another actor), `fetch(_:batchSize:)` for a lazily faulted collection, `enumerate(_:batchSize:)` to walk huge result sets in memory-bounded batches, `delete(model:where:)` for a store-level batch delete

Related to check: SwiftData Basics, SwiftData Index and Unique, Dynamic SwiftData Queries, SwiftData Batch Operations, ModelActor and Concurrency, SwiftData Model Inheritance (querying subclasses), Fetch Request (Core Data's `NSPredicate` equivalent), SwiftData and SwiftUI (dynamic `@Query`)

Dynamic SwiftData Queries

Q: Your list screen has a search field, a 'pinned only' toggle and a sort picker. How do you drive a SwiftData `@Query` from that state?

Short answer: `@Query`'s filter and sort are fixed when the property is created, so you can't just mutate them from `body`. The standard pattern is to keep the user's choices as `@State` in a parent view and pass them into a child view, whose `init` builds a new `Query(filter:sort:)` from them; when the inputs change, SwiftUI re-creates the child and the query re-runs, still live.

Key points:

- Parent owns `@State` (search text, toggles, sort); child's `init` assigns `_items = Query(filter:sort:animation:)`
- Build the `#Predicate` from **local copies** of the inputs (`let term = ...`); the macro captures values, not `self`
- Express optional criteria inside the predicate: `term.isEmpty || $0.title.localizedStandardContains(term)`
- Sort options map to `[SortDescriptor<Model>]`; `SortDescriptor(\.title, comparator: .localizedStandard)` for user-facing text
- `Query(_ descriptor:)` with a `FetchDescriptor` when you need `fetchLimit`, `fetchOffset` or `relationshipKeyPathsForPrefetching`
- **iOS 27+:** `@Query(sort:sectionBy:)` returns `SectionedResults<Model, String>`; `ResultsObserver` is an `@Observable` fetch for view models and UIKit

Related to check: Predicate and FetchDescriptor, SwiftData Basics, SwiftData Index and Unique, SwiftData and SwiftUI, Fetch Request (`@FetchRequest` with `nsPredicate` in Core Data), `.searchable`, `.task(id:)`

SwiftData Index and Unique

Q: How do #Unique and #Index work in SwiftData? What actually happens when you insert a model whose unique key already exists?

Short answer: `#Unique` declares one or more uniqueness constraints on a model (each can span several properties), and `#Index` declares database indexes for the properties you filter and sort on. Both are iOS 18 macros that become real SQLite `UNIQUE INDEX` / `INDEX` definitions in the default store.

Key points:

- `#Unique<Article>([\.remoteID], [\.feed, \.slug])`: two separate constraints, the second one compound. `@Attribute(.unique)` is the single-property shorthand
- `#Index<Article>([\.publishedAt], [\.feed, \.publishedAt])`: one single-column and one compound index; `#Index<Place>(.binary([\.name]), .rtree([\.lat, \.lon]))` picks the index type
- Upsert rules: the new values win for **all** attributes (defaults included); a collision on **any** constraint replaces the row; two inserts with the same key in one save leave the last one
- Model instances already in memory for that row show the new values after the save
- Unique constraints aren't supported with CloudKit sync

Related to check: SwiftData Attributes (`@Attribute(.unique)`), Predicate and FetchDescriptor, SwiftData Migration (deduplicating before adding a constraint), SwiftData Batch Operations, Data Modeling for Mobile, SwiftData and CloudKit, SQLite on iOS

ModelActor and SwiftData Concurrency

Q: How do you do background work with SwiftData in Swift 6? What can and can't cross an actor boundary?

Short answer: A `ModelContext` and every model it vends are confined to one isolation domain, so background work goes through a **`@ModelActor`**: an actor whose executor is backed by its own `ModelContext`. What crosses the boundary is `Sendable` data: the `ModelContainer`, `PersistentIdentifiers` (after a save), and plain value DTOs.

Key points:

- `@ModelActor actor Importer { ... }` generates `init(modelContainer:)`, `modelContainer` and `modelExecutor`; inside, use `modelContext` and the `self[id, as: Article.self]` subscript to load a model by ID
- Sendable: `ModelContainer`, `PersistentIdentifier`, `ModelConfiguration`, `FetchDescriptor`, your DTO structs. **Not** Sendable: `ModelContext`, `@Model` instances
- On the main actor, resolve IDs with `container.mainContext.model(for: id) as? Article` (or a fetch). Each context has its own instance of the row
- After the actor calls `save()`, the main context, `@Query` and other contexts on the same container see the changes; there's no manual merge step
- In a module with *default MainActor isolation* (new Xcode 26+ app targets), `@Model` classes still aren't inferred `@MainActor`, so they work inside a model actor

Related to check: SwiftData Basics, Predicate and FetchDescriptor, SwiftData History Tracking, SwiftData Batch Operations, Offline Persistence, Core Data (`perform` and `NSManagedObjectID`, the Core Data equivalent), SwiftData and SwiftUI, Swift Concurrency (actors, Sendable, `@concurrent`)

SwiftData Batch Operations

Q: You need to import 200,000 records into SwiftData, and later delete or update large parts of them. How do you do that without freezing the UI or running out of memory?

Short answer: Do the work in a `@ModelActor` so it runs off the main actor with its own context, and bound memory by working in chunks: insert a chunk, save it, and drop it before the next one. SwiftData gives you the pieces for this: `transaction { }` to make a chunk atomic and save it, `delete(model:where:)` to delete by predicate directly in the store, `enumerate(_:batchSize:)` to walk every matching row a batch at a time, and `fetchIdentifiers` / `fetchCount` when you only need IDs or numbers.

Key points:

- `@ModelActor actor Importer`: created with `Importer(modelContainer:)`; pass in `Sendable` DTOs and return IDs or counts, never models
- `try context.transaction { ... }`: runs the block and saves at the end (verified: another context sees the rows right after it returns, with no explicit `save()`)
- `try context.delete(model: Note.self, where: #Predicate { ... })`: a batch delete that goes straight to the store (verified: no `save()` needed); `includeSubclasses` defaults to `true`
- `try context.enumerate(descriptor, batchSize: 500) { model in ... }`: bounded-memory iteration; you still `save()` afterwards
- `fetch(_:batchSize:)` returns a lazily faulting `FetchResultsCollection`; the descriptor needs `includePendingChanges = false`, otherwise it throws `includePendingChangesWithBatchSize` (verified)
- `fetchIdentifiers(_:)` and `fetchCount(_:)` avoid materialising models entirely

Related to check: ModelActor and Concurrency, Predicate and FetchDescriptor, SwiftData Index and Unique, SwiftData Undo, Persistent Container (`NSBatchDeleteRequest`), Core Data Faulting, background tasks

---

SwiftData Schema Migration

Q: You shipped a SwiftData app, and the next release renames a property and makes email unique, but existing users have duplicate emails. How do you migrate their data safely?

Short answer: Every shipped schema becomes a frozen `VersionedSchema`, and a `SchemaMigrationPlan` lists those versions in order plus a stage for each step. A **lightweight** stage covers changes SwiftData can infer: new properties with defaults or optionals, removed properties, renames declared with `@Attribute(originalName:)`.

Key points:

- Nest each version's `@Model` classes inside its `VersionedSchema` enum (`SchemaV1.Contact`, `SchemaV2.Contact`) and point the app at the latest with `typealias Contact = SchemaV2.Contact`
- `MigrationStage.lightweight(fromVersion:toVersion:)` / `.custom(fromVersion:toVersion:willMigrat` both closures receive a `ModelContext` and must be `@Sendable`
- Renames: `@Attribute(originalName: "name") var fullName` (and `@Relationship(originalName:)`) keep the data; a plain rename looks like delete + add and drops the column
- Stages chain, so a user upgrading from V1 straight to V4 runs every step in order
- Pass the plan to `ModelContainer(for:migrationPlan:configurations:)` (and `DocumentGroup(..., migrationPlan:)` for document apps)

Related to check: Core Data Migration (staged migration, and the basic SwiftData lightweight example), SwiftData Index and Unique, Testing Persistence (migration tests), ModelContainer Configuration, SwiftData and CloudKit (additive-only schemas), SwiftData Basics

---

SwiftData History Tracking

Q: Your widget and App Intents write to the same SwiftData store as the app. How does the app find out exactly what changed while it wasn't looking?

Short answer: The default SwiftData store records every saved **transaction**: which models were inserted, updated or deleted, by which process and **author**. A reader keeps a **history token** for the last transaction it processed, then calls `fetchHistory` with a `HistoryDescriptor` whose predicate asks for transactions after that token.

Key points:

- Tag writers: `context.author = "widget"` (or `"intent"`, `"sync"`), so readers can skip their own transactions
- `DefaultHistoryTransaction` has `token`, `author`, `timestamp`, `bundleIdentifier`, `processIdentifier` and `changes`. Each change is `.insert`, `.update` (with `updatedAttributes`) or `.delete` (with a `tombstone`)
- Tombstones only keep attributes marked `@Attribute(.preserveValueOnDeletion)`, which is how you learn *what* was deleted (a server ID, a title)
- `DefaultHistoryToken` is `Codable` and `Comparable`: persist it (App Group `UserDefaults` or a file) and compare in the predicate
- `deleteHistory(_:)` purges; a reader whose token now points at purged history gets `SwiftDataError.historyTokenExpired` and must do a full reload

Related to check: Core Data Persistent History, App Groups and Shared Containers, SwiftData Attributes (`preserveValueOnDeletion`), ModelActor and Concurrency, Data Synchronization, SwiftData Custom Data Store (`HistoryProviding`), WidgetKit, App Intents

---

Undo and Redo with SwiftData

Q: How would you add undo and redo to a SwiftData app? How does it interact with autosave?

Short answer: `ModelContext` has an `undoManager` property: when one is attached, the context registers an undo action for every insert, change and delete, and `undo()` reverts them in the context. In SwiftUI the easy path is `.modelContainer(for:isUndoEnabled: true)`, which connects the main context to the window's `UndoManager`, so the system gestures (shake, three-finger swipe) and `@Environment(\.undoManager)` buttons just work.

Key points:

- SwiftUI: `.modelContainer(for: Note.self, isUndoEnabled: true)` (the default is `false`)
- Your own container: `container.mainContext.undoManager = UndoManager()` (or the window's manager)
- Read it in views with `@Environment(\.undoManager)`; call `undo()` / `redo()`; label actions with `setActionName(_:)`
- Changes made within one event are grouped automatically (`groupsByEvent`); use `beginUndoGrouping()` / `endUndoGrouping()` to make a multi-step operation a single undo
- Contexts you create yourself, including the one inside a `@ModelActor`, have **no undo manager** by default, and changes merged in from other contexts aren't undoable
- Autosave and undo are independent; `save()` does not clear the undo stack

Related to check: SwiftData Basics, Model Container Configuration, ModelActor and Concurrency, SwiftData Batch Operations, Core Data (`NSManagedObjectContext.undoManager`), `UndoManager`, document-based apps

---

SwiftData Model Inheritance

Q: SwiftData supports class inheritance since iOS 26. How do you model a hierarchy, how do queries behave across subclasses, and when would you avoid it?

Short answer: From iOS 26 a `@Model` class can subclass another `@Model` class (iOS 26+). The subclass inherits the parent's persisted properties and adds its own, and the store keeps the hierarchy as one entity tree.

Key points:

- The base is a non-`final` `@Model class`; subclasses are `@Model final class Book: LibraryItem` and call `super.init`
- Every `@Model` subclass needs an explicit `@available(iOS 26, *)` (and other platforms you build for), even when the deployment target is already iOS 26; without it the macro fails with "A PersistentModel Subclass is required to have platform availability specified"
- `FetchDescriptor<LibraryItem>` / `@Query var items: [LibraryItem]` return books and podcasts mixed; downcast in Swift with `switch item { case let book as Book: ... }`
- `FetchDescriptor<Book>` filters on both subclass and inherited properties
- `#Predicate<LibraryItem> { $0 is Podcast }` works; `delete(model:where:includeSubclasses:)` controls whether a base-type batch delete reaches subclasses

Related to check: SwiftData Basics, Predicate and FetchDescriptor, SwiftData Migration, SwiftData Relationships, Data Modeling for Mobile, SwiftData and SwiftUI

SwiftData Custom Data Store

Q: SwiftData lets you plug in your own store. How does a custom DataStore work, and when would you write one instead of using the default SQLite store?

Short answer: Since iOS 18, SwiftData separates the model layer (`@Model`, `ModelContext`, `@Query`) from storage through the **`DataStore`** protocol. A store is paired with a **`DataStoreConfiguration`**, and the container talks to it in **snapshots**: on `fetch` you return snapshots of the matching rows, and on `save` you receive inserted, updated and deleted snapshots, write them, and hand back permanent identifiers for the new rows.

Key points:

- `DataStoreConfiguration`: `Hashable`, with `name`, `schema` and an associated `Store` type; `validate()` has a default. Pass it to `ModelContainer(for:configurations:)`
- `DataStore` requirements: `identifier`, `schema`, `configuration`, `init(_:migrationPlan:)`, `fetch(_:)` and `save(_:)`. `fetchCount`, `fetchIdentifiers`, `erase`, `cachedSnapshots` and the editing-state hooks have default implementations you can override for speed
- Snapshots: `DefaultSnapshot` is a `Codable`, `Sendable` value holding a model's properties and its `persistentIdentifier`; define your own `DataStoreSnapshot` type for a different encoding
- Identity: inserted snapshots arrive with **temporary** IDs. Mint permanent ones with `PersistentIdentifier.identifier(for: storeIdentifier, entityName:, primaryKey:)` and return the mapping in `remappedIdentifiers`
- Throw `DataStoreError.preferInMemoryFilter` / `.preferInMemorySort` when you can't translate a predicate or sort; SwiftData then applies it to the snapshots you return
- Optional protocols: `DataStoreBatching` for `delete(model:where:)`, `HistoryProviding` for `fetchHistory` / `deleteHistory`

Related to check: SwiftData Basics, Predicate and FetchDescriptor, SwiftData History Tracking (`HistoryProviding`), ModelContainer Configuration, SQLite on iOS (GRDB), Codable File Persistence, SwiftData vs Core Data

---

SwiftData and CloudKit

Q: How do you sync a SwiftData app across a user's devices with iCloud? What constraints does that put on your schema, and what are the limits?

Short answer: SwiftData can mirror a store to the user's **private** CloudKit database with almost no code: add the iCloud capability with a CloudKit container plus the Remote notifications background mode, and give the `ModelConfiguration` a `cloudKitDatabase`. It uses Core Data's CloudKit mirroring, so the

rules are the same as `NSPersistentCloudKitContainer`: every attribute optional or defaulted, every relationship optional with an inverse, no unique constraints, no `.deny` delete rule.

Key points:

- `cloudKitDatabase: .private("iCloud.com.example.app")`, `.automatic` (the container in your entitlements) or `.none`. SwiftData has **no** public or shared database option
- Schema rules, enforced when the store loads (the error lists every offending property): attributes optional or with a default **on the declaration**; relationships optional **and** with inverses; no `#Unique` / `.unique`; no `.deny`
- `@Attribute(.allowsCloudEncryption)` encrypts a field in CloudKit; `.externalStorage` data still syncs
- Development vs production: record types are created in the development environment as data syncs (or all at once with `initializeCloudKitSchema()`), then you **deploy the schema to production** in the CloudKit Console before release
- Production schemas are additive: you can add record types and fields, not delete or change them, so migrations must be additive too

Related to check: Data Synchronization, CKSyncEngine, ModelContainer Configuration, SwiftData Relationships, SwiftData Index and Unique, SwiftData Migration, SwiftData History Tracking, Persistent Container (`NSPersistentCloudKitContainer`)

Core Data

Q: What is Core Data? How does it differ from a database?

Short answer: Core Data is Apple's object-graph and persistence framework: it manages the lifecycle, relationships, validation, undo and change tracking of model objects, and it can save them to a store (usually SQLite). It isn't a database.

Key points:

- Stack: `NSManagedObjectModel` (schema) -> `NSPersistentStoreCoordinator` (store access) -> `NSManagedObjectContext` (scratchpad of objects); `NSPersistentContainer` bundles all three
- Database vs Core Data: a database stores rows and answers queries; Core Data keeps an in-memory object graph with inverse relationships, delete rules, validation, undo, faulting and change notifications, and only persists on `save()`
- Contexts are confined to their own queue: `viewContext` on the main queue, background contexts on private queues, and all access goes through `perform`
- Swift 6: `NSManagedObjectContext` and `NSManagedObjectID` are `Sendable`; `NSManagedObject` is **not**. Pass object IDs across isolation and re-fetch
- Batch requests (`NSBatchInsertRequest`, `NSBatchDeleteRequest`, `NSBatchUpdateRequest`) work directly on the store, skipping the object graph
- `NSPersistentCloudKitContainer` adds iCloud sync; SwiftData and Core Data can share one store

Related to check: NSManagedObject, Persistent Container, Fetch Request, Faulting, Migration, SwiftData and SwiftUI, SQLite

---

NSManagedObject

Q: What is NSManagedObject? How does it relate to Core Data entities?

Short answer: `NSManagedObject` is the base class for Core Data model objects. An entity in the `.xcdatamodeld` describes the schema; each managed object is one instance of that entity, living in exactly one `NSManagedObjectContext`, which tracks its changes until you save.

Key points:

- Entity = schema (attributes, relationships, delete rules); `NSManagedObject` subclass = runtime instance
- Xcode generates subclasses (Codegen: *Class Definition* or *Category/Extension*), or you write them by hand with `@NSManaged` properties
- Each object belongs to one context and must only be touched on that context's queue
- `objectID` is the stable identity; it starts *temporary* and becomes permanent on save (or `obtainPermanentIDs(for:)`)
- Life-cycle hooks: `awakeFromInsert()` for defaults, `willSave()`, `prepareForDeletion()`
- Can be a fault (`isFault`) until a property is read

Related to check: Core Data, Persistent Container, Fetch Request, Faulting, SwiftData and SwiftUI (`@Model`, `PersistentIdentifier`)

Persistent Container

Q: What is NSPersistentContainer? How do you set it up?

Short answer: `NSPersistentContainer` wraps the whole Core Data stack: it loads the `.xcdatamodeld` by name, creates the persistent store coordinator, and gives you a main-queue `viewContext` plus factories for background contexts. You configure it through `persistentStoreDescriptions` before calling `loadPersistentStores`, which is where you choose the store URL (for example an App Group container), history tracking and migration options.

Key points:

- `NSPersistentContainer(name:)` finds the model in the bundle; `init(name:managedObjectModel:)` when the model lives in a package or framework
- `NSPersistentStoreDescription`: URL, type (SQLite by default, `NSInMemoryStoreType` for previews and tests), options
- **Lightweight migration is on by default**: the description's `shouldMigrateStoreAutomatically` and `shouldInferMappingModelAutomatically` are both `true`
- `viewContext`: main queue, for the UI. `newBackgroundContext()`: long-lived private-queue context. `performBackgroundTask`: one-off private context (closure or `async` form)
- The container, contexts and `NSManagedObjectID` are `Sendable`, so a `Sendable` persistence controller can hold the container as a `let`

Related to check: Core Data, NSManagedObject, Core Data Migration, Data Synchronization (`NSPersistentCloudKitContainer`), SwiftData and SwiftUI (`ModelContainer`)

Fetch Request

Q: What is NSFetchRequest? How do you use it to query Core Data?

Short answer: `NSFetchRequest` describes a Core Data query: which entity, an `NSPredicate` filter, sort descriptors, and tuning options such as limit, offset and batch size. You execute it on a context with `context.fetch(_:)` (or `count(for:)`), and Core Data translates it to SQL against the store, then merges the results with unsaved changes in that context.

Key points:

- Typed requests: `NSFetchRequest<User>` (generated `User.fetchRequest()`)
- `predicate`: `NSPredicate(format: "name CONTAINS[cd] %@", text)`; `[cd]` = case- and diacritic-insensitive; `%K` for key names

- `sortDescriptors`: `NSSortDescriptor(keyPath: \User.name, ascending: true)` (key-path form is type-checked)
- `fetchLimit` / `fetchOffset` for paging; `fetchBatchSize` loads rows in batches as you iterate
- `resultType`: managed objects (default), `.managedObjectIDResultType`, `.dictionaryResultType` (+ `propertiesToFetch`), or use `count(for:)` instead of fetching just to count
- `relationshipKeyPathsForPrefetching` avoids N+1 faulting; `includesPendingChanges`, `returnsObjectsAsFaults` for tuning

Related to check: Core Data, Faulting, NSManagedObject, @FetchRequest in SwiftUI, SwiftData and SwiftUI (`FetchDescriptor`, `#Predicate`)

Core Data Faulting

Q: What is Core Data faulting? Why does it matter?

Short answer: Faulting is Core Data's lazy loading. A fault is a placeholder managed object whose identity is known but whose property values haven't been loaded yet; the first time you read a property, the fault "fires" and Core Data fetches the row (from the row cache or the store).

Key points:

- Fetch results can come back as faults (`returnsObjectsAsFaults` is `true` by default), and to-many relationships are faults until accessed
- Reading any attribute fires the fault; `isFault` tells you the state without firing it
- `fetchBatchSize` returns a proxy array that loads rows batch by batch as you iterate
- `relationshipKeyPathsForPrefetching` loads related objects in the same round trip
- `context.refresh(object, mergeChanges: false)` turns an object back into a fault to free memory; `refreshAllObjects()` for the whole context
- SwiftData models fault the same way (it's built on the same store technology)

Related to check: Core Data, Fetch Request, NSManagedObject, Core Data performance, memory management

---

Core Data Migration

Q: What is Core Data migration? How do you handle schema changes?

Short answer: When the data model changes, the on-disk store has to be migrated to match it, or it won't open. You add a new model version in the `.xcdatamodeld`, make it current, and let Core Data migrate: **lightweight (inferred) migration** handles most changes and is on by default with `NSPersistentContainer`.

Key points:

- Versioned models: *Editor -> Add Model Version*, then set the current version; never edit a shipped version
- Lightweight handles: add/remove attributes and entities, optional non-optional with a default, renames via the *Renaming ID*, many relationship changes. `NSPersistentStoreDescription` has `shouldMigrateStoreAutomatically` and `shouldInferMappingModelAutomatically` set to `true` by default
- Staged migration: `NSLightweightMigrationStage` and `NSCustomMigrationStage` (with `willMigrateHandler` / `didMigrateHandler`) referencing models by `NSManagedObjectModelReference` + version checksum; pass the manager under `NSPersistentStoreStagedMigrationManagerOptionKey`

- Legacy heavyweight: `NSMappingModel` + `NSEntityMigrationPolicy`; slow and memory-hungry because both stores are loaded
- SwiftData: `VersionedSchema` per release, `SchemaMigrationPlan` with `.lightweight` and `.custom` stages

Related to check: Core Data, Persistent Container, NSManagedObject, SwiftData and SwiftUI, Data Synchronization (CloudKit schema rules)

Core Data in Swift 6

Q: We're moving a Core Data app to the Swift 6 language mode. What changes, and which Core Data types are actually `Sendable`?

Short answer: Core Data's threading rule hasn't changed: every managed object belongs to one context and is touched only on that context's queue. What Swift 6 adds is compile-time checking around it.

Key points:

- **Sendable** in the SDK: `NSPersistentContainer`, `NSPersistentCloudKitContainer`, `NSPersistentStoreCoordinator`, `NSManagedObjectContext`, `NSManagedObjectID`, `NSPersistentHistoryToken`. **Not Sendable:** `NSManagedObject` (marked non-Sendable), `NSPersistentStoreDescription`, `NSFetchRequest`
- `try await context.perform { ... }` returns the closure's value; `perform(schedule: .enqueued)` always queues the block, `.immediate` (default) runs it right away if you're already on the context's queue
- `viewContext` is a main-queue context: from `@MainActor` code you can use it directly; everywhere else use `perform`
- Merge policies: the old globals (`NSMergeByPropertyObjectTrumpMergePolicy`) are mutable global `var`s and are a **Swift 6 error**; use `NSMergePolicy.mergeByPropertyObjectTrump` (and `.mergeByPropertyStoreTrump`, `.overwrite`, `.rollback`, `.error`)
- A repository that only stores the container as a `let` can be `final class ...: Sendable` without `@unchecked`

Related to check: Core Data, NSManagedObject, Persistent Container, Core Data Persistent History, ModelActor and Concurrency, Sendable, region-based isolation, default actor isolation

---

Core Data Persistent History

Q: Your app and its widget both write to the same Core Data store. How does the app find out what the widget changed, and what is persistent history tracking?

Short answer: Persistent history tracking makes the store record every saved transaction: which objects were inserted, updated or deleted, by which context author and process. Each process remembers a **history token** for the last transaction it has processed; when the store changes (another process wrote, or a batch request ran), it fetches the transactions after that token, merges the affected object IDs into its `viewContext`, and stores the new token.

Key points:

- Enable it on the store description before loading: `NSPersistentHistoryTrackingKey`; add `NSPersistentStoreRemoteChangeNotificationPostOptionKey` to get a notification when another process (or a batch request) writes
- Tag writers: `context.transactionAuthor = "app"` / `"widget"` / `"importer"`, so each reader can skip its own transactions

- Fetch: `NSPersistentHistoryChangeRequest.fetchHistory(after: token)` -> `[NSPersistentHistoryTransaction]`, each with `changes` (`changeType`, `changedObjectID`), `author`, `token`
- Merge: `NSManagedObjectContext.mergeChanges(fromRemoteContextSave:into:)` with the inserted/updated/deleted ID sets
- Persist each process's last token (it's `NSSecureCoding` and `Sendable`) in the App Group container
- Purge: `NSPersistentHistoryChangeRequest.deleteHistory(before: token)` using the **oldest** token any reader still needs

Related to check: Persistent Container, App Groups and Shared Containers, Core Data in Swift 6, SwiftData History Tracking, Data Synchronization (`NSPersistentCloudKitContainer`), WidgetKit

---

Migrating from Core Data to SwiftData

Q: We have a large Core Data app. How would you move it to SwiftData without a big-bang rewrite or losing users' data?

Short answer: You don't need to convert the store: SwiftData can open the same SQLite file Core Data writes, so both stacks can run side by side and you migrate screen by screen. The requirements are that both describe the same schema (SwiftData class names match entity names, with the same attribute names, types and optionality), both point at the same store URL, and persistent history tracking is on in the Core Data stack, because SwiftData uses history.

Key points:

- Generate `@Model` classes from the `.xcdatamodeld` (Xcode: *Editor -> Create SwiftData Code…*), then review them
- **Entity name matching:** SwiftData's entity name is the class name; rename the Core Data *class* (e.g. `CDTrip`, entity still `Trip`) so the two Swift types don't collide
- Same store URL: `NSPersistentStoreDescription(url:)` and `ModelConfiguration(url:)`; for widgets, both in the App Group container
- Enable `NSPersistentHistoryTrackingKey` on the Core Data store description
- Keep one owner of schema changes at a time; while both stacks are live, change the schema in both together

Related to check: SwiftData vs Core Data, SwiftData Migration, Core Data Migration, Core Data Persistent History, Model Container Configuration, SwiftData Model Inheritance, Persistent Container

---

SQLite on iOS

Q: What is SQLite? When would you use it directly instead of Core Data?

Short answer: SQLite is an embedded, single-file, transactional SQL database that ships with iOS; Core Data and SwiftData both use it as their default store. Using it directly means you own the schema, the SQL and the migrations, and in return you get full control over queries, indexes and performance, plus a schema you can share with Android or a server.

Key points:

- Single file, no server, ACID transactions, WAL mode for concurrent readers with one writer
- C API (`import SQLite3`): `sqlite3_open_v2`, `prepare_v2`, `bind_*`, `step`, `finalize`; you check every result code
- A connection must not be used from several threads at once: wrap it in an actor (or use a library's serialized queue/pool)

- GRDB: `DatabaseQueue` / `DatabasePool` (WAL), `Codable` records (`FetchableRecord`, `PersistableRecord`), a query interface plus raw SQL, `DatabaseMigrator`, and `ValueObservation` for live queries; GRDB 7 targets Swift 6 concurrency
- Pick **SwiftData** for new SwiftUI apps with an object model, iCloud sync and little SQL; **Core Data** for existing stacks, batch operations and `NSPersistentCloudKitContainer`; **SQLite/GRDB** for SQL-heavy work (reporting, full-text search with FTS5, complex joins), a cross-platform schema, or predictable performance on large data

Related to check: Core Data, SwiftData and SwiftUI, Realm, File Storage, Offline Persistence

---

Realm

Q: What is Realm? How does it compare to Core Data?

Short answer: Realm is a cross-platform embedded object database: you declare model classes with `@Persisted` properties and Realm stores them in its own storage engine, not SQLite. Its selling points were a simple API, live auto-updating objects and results, and fast reads through memory mapping.

Key points:

- Object database: models subclass `Object` with `@Persisted` properties; no ORM over SQLite
- Writes happen in `realm.write { }` transactions; reads are lazy and zero-copy
- Live objects and `Results` update automatically; `observe` gives change sets for UI diffing
- Threading: live objects are confined to the thread (or actor) that opened the Realm; cross with the primary key, `ThreadSafeReference`, or frozen (immutable) objects; newer SDK versions support actor-isolated Realms
- Migrations: bump `schemaVersion` in `Realm.Configuration` and handle changes in a `migrationBlock`
- **Vs Core Data/SwiftData:** Realm is cross-platform (shared model with Android) and had a simpler threading story than legacy Core Data; Core Data and SwiftData are first-party, maintained with each OS release, integrate with SwiftUI (`@FetchRequest`, `@Query`), and sync through CloudKit. With Device Sync gone, apps that relied on it need a new sync layer (CloudKit via `CKSyncEngine`, their own backend, or a third-party service).

Related to check: Core Data, SQLite on iOS, SwiftData and SwiftUI, Data Synchronization, Offline Persistence

---

File Storage

Q: How do you store files on iOS? What are the main directories?

Short answer: Each app has a sandboxed container, and you read and write it with `FileManager` and `Data`/`URL` APIs. Where a file goes decides whether it's backed up, visible to the user, or purgeable: **Documents** for user-created content, **Library/Application Support** for app-managed data, **Library/Caches** for anything you can re-create, and **tmp** for short-lived scratch files.

Key points:

- Modern URLs: `URL.documentsDirectory`, `.applicationSupportDirectory`, `.cachesDirectory`, `.temporaryDirectory` (instead of `FileManager.urls(for:in:)[0]`); build paths with `appending(path:)`
- Documents: backed up, and shown in the Files app if you set `UIFileSharingEnabled` and `LSSupportsOpeningDocumentsInPlace`
- Application Support: backed up, hidden from users; create the directory yourself, it doesn't exist by default

- Caches: not backed up and can be purged when storage is low; tmp: purged even more aggressively
- `isExcludedFromBackup` for large, re-downloadable data outside Caches
- App Groups: `FileManager.default.containerURL(forSecurityApplicationGroupIdentifier:)` to share files with widgets and extensions

Related to check: Keychain, Securing Sensitive Data, UserDefaults, Offline Persistence, `NSCache`, document-based apps

---

Codable File Persistence

Q: You need to persist a small model to disk without a database. How would you design a Codable file store that's safe across crashes, concurrent access and app updates that change the format?

Short answer: For a few hundred kilobytes of state (settings, a draft, a cached response, a small list), I encode a `Codable` value to JSON and write it **atomically** to Application Support. I wrap the payload in an envelope with a `schemaVersion`, so a future release can read old files and migrate them in code.

Key points:

- `data.write(to:options: [.atomic, ...])` writes to a temporary file and renames it, so a crash leaves either the old file or the new one, never half of each
- A version envelope (`{"schemaVersion": 2, "payload": {...}}`) lets you decode a small header first and then pick the right decoder
- Additive changes don't need a version bump if new properties are optional or decoded with `decodeIfPresent` and a default; renames, type changes and restructuring do
- An `actor` store: one writer, no torn reads, an in-memory cache, and encoding happens off the main actor
- Pick the location by semantics: Application Support for app state, Caches for re-creatable data, and an explicit protection class

Related to check: File Storage, UserDefaults, SQLite on iOS, When to Save, Testing Persistence, `Codable`, actors, golden-file testing

---

Document-Based Persistence

Q: You're building a document-based app where each document has text plus large image attachments. How do you design the file format and the save path so opening and autosave stay fast, and how does the iOS 27 `Document` API change that compared with `FileDocument` or `UIDocument`?

Short answer: In a document app the user owns the file, so persistence means designing a **file format** and letting the system handle when to save: autosave, iCloud Drive, file coordination and versions. For anything with large or many parts I use a **package**, a directory that appears as one file, with a small versioned manifest plus one file per attachment, so a save rewrites only the manifest and new attachments instead of the whole document. iOS 27's `Document` protocols (`ReadableDocument` & `WritableDocument`) fit this well: reading and writing run off the main actor through a `DocumentReader` and `DocumentWriter`, and the writer receives the previous `FileWrapper`, so I can reuse unchanged children instead of re-serializing everything.

Key points:

- Flat file (one JSON, text or binary blob) for small documents; **package** (UTType conforming to `.package`, a directory `FileWrapper`) for attachments, media or anything over a few MB
- iOS 27+: `reader(configuration:)` produces a snapshot off the main actor, `apply(snapshot:previous:)` updates the model on the main actor; `snapshot(contentType:)` captures state on the main actor, `writer(configuration:)` writes it off the main actor

- `FileWrapperDocumentReader` / `FileWrapperDocumentWriter` cover file-wrapper formats; for formats you stream yourself (a database, a huge log), write your own `DocumentReader` / `DocumentWriter`, which receive the file URL and a `Subprogress` for progress reporting
- `FileDocument` (struct) can reuse `configuration.existingFile` too, but its `init(configuration:)` and `fileWrapper(configuration:)` are synchronous and the whole struct is copied on edit; `ReferenceFileDocument` and UIKit's `UIDocument` do snapshot-based saving with more manual work
- Version the manifest exactly like any other persisted format

Related to check: DocumentGroup (scene setup and `NewDocumentButton`), File Storage, Codable File Persistence, When to Save, `FileWrapper`, `UTType`, `NSFileCoordinator`, iCloud Drive

---

When to Save

Q: When should an app actually save its data? How do you avoid losing edits if the app is killed, without saving so often that it hurts performance?

Short answer: I save at **meaningful points** rather than on a timer or after every property change: when the user finishes an action (Done, send, leaving an editor), after a short pause in continuous editing, and always when the scene leaves the foreground, because iOS can terminate a suspended app without any further callback. SwiftData's main context autosaves, but I still save explicitly at those points so the timing is mine.

Key points:

- Lifecycle: save when `scenePhase` becomes `.inactive` or `.background` (SwiftUI) or in `sceneWillResignActive` / `sceneDidEnterBackground` (UIKit); `applicationWillTerminate` is not called for suspended apps, so it's not a save point
- SwiftData: the container's `mainContext` has `autosaveEnabled` on and saves periodically; contexts you create yourself and `@ModelActor` work should call `save()` explicitly; check `hasChanges` first
- Core Data: `viewContext.save()` after user actions; background imports in batches of a few hundred with `reset()` between them; `NSBatchInsertRequest` for very large imports
- Debounce continuous edits (text, sliders) with a cancellable `Task`; save immediately on explicit commits
- Atomic units: the change and anything that must accompany it (an outbox entry, a denormalized count) go in the **same** save

Related to check: Core Data, Persistent Container, Offline Persistence, Codable File Persistence, Document-Based Persistence, scene life cycle, background tasks

---

App Groups and Shared Containers

Q: Your widget, share extension and App Intents need the same data as the app. How do you share it, and what can go wrong?

Short answer: The app and each extension run as separate processes with separate sandboxes, so sharing goes through an **App Group**: an entitlement (`group.com.example.notes`) that every target lists, giving them a shared container directory and a shared `UserDefaults` suite. I put the SwiftData or Core Data store, shared files and small widget values there.

Key points:

- Add the *App Groups* capability with the same identifier to the app, widget, intents and other extension targets

- Files: `FileManager.default.containerURL(forSecurityApplicationGroupIdentifier:)` (returns `nil` if the entitlement is missing)
- Settings: `UserDefaults(suiteName: "group....")`; `@AppStorage("key", store: UserDefaults(suiteName: ...))` in SwiftUI
- SwiftData: `ModelConfiguration(groupContainer: .identifier("group...."))`. The default, `.automatic`, uses the app's App Group container when the entitlement is present, which surprises people who add a group later
- Core Data: set `NSPersistentStoreDescription.url` to a file inside the group container and enable persistent history
- Secrets go in a shared **Keychain access group** (a different entitlement), not in the group container

Related to check: UserDefaults, File Storage, Keychain, Keychain Sharing, Persistent Container, Core Data Persistent History, Model Container Configuration, SwiftData History Tracking, iCloud Documents (file coordination), WidgetKit Integration, Privacy Manifest

---

Data Synchronization

Q: How would you sync data across devices or with a backend?

Short answer: Make the local store the source of truth and treat sync as a background process that reconciles it with the server. The UI reads and writes locally, every change is recorded as pending, and a sync engine pushes pending changes and pulls remote changes using change tokens, resolving conflicts with a deliberate policy.

Key points:

- **SwiftData + CloudKit:** add the iCloud capability and `ModelConfiguration(cloudKitDatabase: .private("iCloud.com.example.app"))` (or `.automatic`); models must have defaults or optionals, no `#Unique`/`.unique`, and optional relationships
- **`NSPersistentCloudKitContainer`:** the Core Data equivalent; mirrors the store to a private (or shared/public) database with persistent history; same schema rules
- **`CKSyncEngine`:** you keep your own store (SwiftData, Core Data, GRDB, files); the engine schedules fetch and send, handles change tokens, push notifications, account changes and retries; you persist its `State.Serialization`
- Custom backend: delta sync (`updatedSince` cursor or server change token), idempotent mutations, tombstones for deletes
- Conflict policies: server wins, last-writer-wins by timestamp, field-level merge, or ask the user

Related to check: Offline Persistence, Core Data, Persistent Container, Core Data Migration, SwiftData and SwiftUI, background tasks, CloudKit

---

CKSyncEngine

Q: You're using `CKSyncEngine` to sync your own store with iCloud. What exactly do you have to persist, and how do you handle zones, account changes and conflicts?

Short answer: `CKSyncEngine` (iOS 17) owns the *mechanics* of CloudKit sync: scheduling, push subscriptions, change tokens, batching and retrying. My code owns the *data*: I tell the engine which record IDs have pending changes, build `CKRecord`s on demand in `nextRecordZoneChangeBatch`, and apply what it fetched in `handleEvent`.

Key points:

- Persist `stateSerialization` on every `.stateUpdate`; pass it back in `CKSyncEngine.Configuration` at launch. Losing it means a full re-fetch, not data loss

- Record changes: `state.add(pendingRecordZoneChanges: [.saveRecord(id)])`/`.deleteRecord(id)` after each local save; zones: `state.add(pendingDatabaseChanges: [.saveZone(zone)])`
- Use a custom zone in the private (or shared) database; one engine per database
- Store `encodeSystemFields(with:)` next to each local row and rebuild the record with `CKRecord(coder:)`, so saves carry the server's change tag
- Events to handle: `.stateUpdate`, `.accountChange`, `.fetchedRecordZoneChanges`, `.sentRecordZoneChanges`, plus `.fetchedDatabaseChanges` (zone deleted remotely)
- The delegate is `Sendable`: an actor, or a `final class` holding only `Sendable` state

Related to check: Data Synchronization (the overall sync design and a minimal engine), SwiftData and CloudKit, iCloud Key-Value Store, Offline Persistence, CloudKit sharing, `NSPersistentCloudKitContainer`

---

iCloud Key-Value Store

Q: How would you sync a few user preferences, like the last reading position or a font size, across a user's devices? What are the limits?

Short answer: For small values I'd use `NSUbiquitousKeyValueStore`, iCloud's key-value store: it has a `UserDefaults`-like API, needs only the iCloud *Key-value storage* capability, and the system syncs it to every device signed in to the same Apple Account. The limits are strict: at most **1024 keys** and **1 MB** of values in total (a single value can use the whole megabyte), and keys of up to 128 UTF-16 characters.

Key points:

- `NSUbiquitousKeyValueStore.default`: `set(_:forKey:)`, `string(forKey:)`, `double(forKey:)`, `dictionary(forKey:)`, `data(forKey:)` …; values must be property-list types
- Entitlement `com.apple.developer.ubiquity-kvstore-identifier`, added by the iCloud capability's *Key-value storage* option
- Observe `NSUbiquitousKeyValueStore.didChangeExternallyNotification`; `userInfo` has the change reason and the changed keys
- Reasons: server change, initial sync, **quota violation**, account change
- `synchronize()` at launch reconciles memory with the local copy; it doesn't force an upload, and the system decides when to sync
- Works without an iCloud account: values stay local and sync once the user signs in

Related to check: UserDefaults, When to Use UserDefaults, Keychain (iCloud Keychain for secrets), CKSyncEngine, iCloud Documents, Data Synchronization

---

iCloud Documents

Q: How does an app store documents in iCloud Drive? What are file coordination and `NSMetadataQuery` for, and how do you handle conflicts?

Short answer: With the iCloud Documents capability, the app gets a **ubiquity container**: a local directory that the system keeps in sync with iCloud, found with `FileManager.url(forUbiquityContainerIdentifier:)`. Files under its `Documents` folder can also appear in the Files app.

Key points:

- Capability: iCloud -> *iCloud Documents* with a container (`iCloud.com.example.notes`); to show it in iCloud Drive, declare it in `NSUbiquitousContainers` with `NSUbiquitousContainerIsDocumentScopePublic`

- `url(forUbiquityContainerIdentifier:)` returns `nil` when iCloud is unavailable and can block on first call, so call it off the main actor; `FileManager.default.ubiquityIdentityToken` tells you whether a user is signed in
- **Coordination:** `NSFileCoordinator().coordinate(readingItemAt:...)` / `coordinate(writingItemAt:options:...)`; use the URL passed to the block
- **Presenters:** `NSFilePresenter` (`presentedItemURL`, `presentedItemOperationQueue`, `presentedItemDidChange()`); `UIDocument` is one
- **Discovery:** `NSMetadataQuery` with `NSMetadataQueryUbiquitousDocumentsScope`; results include not-yet-downloaded items; call `startDownloadingUbiquitousItem(at:)` and check `ubiquitousItemDownloadingStatus`
- **Conflicts:** `NSFileVersion.unresolvedConflictVersionsOfItem(at:)`, mark versions `isResolved`, then `removeOtherVersionsOfItem(at:)`; `UIDocument.documentState` contains `.inConflict`

Related to check: Document-Based Persistence, File Storage, iCloud Key-Value Store, CKSyncEngine, App Groups and Shared Containers, Data Synchronization, `UIDocument`, `DocumentGroup`

Offline Persistence

Q: What is offline persistence? How do you design for it?

Short answer: Offline persistence means the app keeps working without a network: it shows the last known data instantly, accepts edits, and reconciles later. The design is local-first: a local database (SwiftData, Core Data or SQLite) is the single source of truth the UI observes, the network only ever updates that store, and user edits go into the store plus an **outbox** of pending changes that a sync process drains when connectivity returns.

Key points:

- UI reads only from the local store (`@Query`, `@FetchRequest`, an observed repository); network responses are written into the store, never straight into views
- Writes are optimistic: save locally, enqueue a pending change in the **same transaction**, update the UI immediately
- The outbox is drained in order, with idempotency keys so retries can't duplicate, and exponential backoff with jitter
- Don't gate requests on reachability; just try. Use `NWPathMonitor` (an `AsyncSequence`) to trigger a retry and to show an offline banner
- Show sync status per item (pending, failed) and the age of cached data

Related to check: Data Synchronization, Core Data, SwiftData and SwiftUI, File Storage, caching, network reachability, background tasks

---

Keychain

Q: What is the Keychain? When would you use it?

Short answer: The Keychain is the system's encrypted store for small secrets: passwords, OAuth tokens, API credentials, certificates and cryptographic keys. Items are encrypted with keys tied to the device and the passcode, and you choose *when* each item is readable through an accessibility class.

Key points:

- Item classes: `kSecClassGenericPassword` (tokens, most app secrets), `kSecClassInternetPassword`, `kSecClassKey`, `kSecClassCertificate`, `kSecClassIdentity`; items are identified by service + account

- Accessibility: `kSecAttrAccessibleWhenUnlocked` (default), `AfterFirstUnlock` (needed for background refresh), `WhenPasscodeSetThisDeviceOnly` (strictest); the **`ThisDeviceOnly`** variants are never migrated to another device through backups or iCloud Keychain, which is what you want for tokens and device-bound keys
- Sharing: a Keychain Sharing access group (`kSecAttrAccessGroup`) or the App Group shares items with your extensions and apps from the same team
- Access control: `SecAccessControlCreateWithFlags` with `.userPresence` / `.biometryCurrentSet` requires Face ID / Touch ID / passcode per use
- Keys: CryptoKit `SecureEnclave.P256` keys are generated inside the Secure Enclave and can't be exported

Related to check: Securing Sensitive Data, UserDefaults, File Storage (data protection), OAuth and PKCE, passkeys, Sign in with Apple

---

Keychain Sharing

Q: Your app, its widget and a second app from your company all need the same login token. How do you share Keychain items, and when should an item sync through iCloud Keychain?

Short answer: Keychain items live in **access groups**, and an app can read only the groups listed in its entitlements. To share, I add the Keychain Sharing capability to every target that needs the item (the app, its extensions, and any other app from the **same team**), which writes a `keychain-access-groups` entitlement like `$(AppIdentifierPrefix)com.example.shared`, and I pass that group as `kSecAttrAccessGroup` when adding and querying.

Key points:

- An app's groups are, in order: its `keychain-access-groups`, its application identifier (`TeamID.bundleID`), and its App Groups; **the first `keychain-access-groups` entry is the default** for new items when you don't pass `kSecAttrAccessGroup`
- An App Group (`group.com.example.shared`) also works as an access group, handy when targets already share an App Group container
- Only apps signed by the same team can share; there is no cross-team Keychain sharing (use passkeys, Sign in with Apple, or your server instead)
- `kSecAttrSynchronizable` is part of the item's identity: queries must pass `true` or `kSecAttrSynchronizableAny`, otherwise synced items are invisible
- Synced items can't use a `...ThisDeviceOnly` accessibility class, and updating or deleting a synced item affects every device

Related to check: Keychain, Securing Sensitive Data, Backups and Restore, Secure Enclave and CryptoKit, App Groups and Shared Containers, app extensions, passkeys, Sign in with Apple

---

Securing Sensitive Data

Q: How do you secure sensitive data in an iOS app?

Short answer: Think in layers: data at rest, data in transit, data leaving through side channels, and the integrity of the app itself. At rest, secrets go in the Keychain, files get a Data Protection class, and anything extra-sensitive is encrypted with CryptoKit using a key kept in the Keychain or Secure Enclave.

Key points:

- **At rest:** Keychain for tokens and credentials (`...ThisDeviceOnly` classes); `.completeFileProtection` for sensitive files; CryptoKit `AES.GCM` / `ChaChaPoly` for app-level encryption; Secure Enclave keys for signing

- **Secrets in the app:** anything in the binary, `Info.plist` or an `.xcconfig` can be extracted. Keep real secrets on your backend and hand the app short-lived tokens; xcconfig/CI variables only keep keys out of git
- **In transit:** ATS on by default; `NSPinnedDomains` for declarative pinning; OAuth with PKCE via `ASWebAuthenticationSession`; passkeys instead of passwords
- **App integrity:** App Attest (`DCAppAttestService`) to attest the app instance and sign requests; DeviceCheck for per-device bits
- **Side channels:** `os.Logger` with `privacy: .private` (never log tokens), hide sensitive screens in the app-switcher snapshot, mark secure text fields, avoid putting secrets on the pasteboard
- **Privacy manifest** (`PrivacyInfo.xcprivacy`): collected data types, tracking domains, and required-reason APIs (`UserDefaults`, file timestamps, disk space, boot time); third-party SDKs must ship their own

Related to check: Keychain, File Storage, UserDefaults (required-reason API), certificate pinning, App Transport Security, OAuth and PKCE, passkeys

---

File Protection Classes

Q: Explain the Data Protection classes on iOS. What happens to your files when the device is locked, and how does that affect background work?

Short answer: Every file on iOS is encrypted with a per-file key, and its **protection class** decides when the system can unwrap that key: `.complete` files are unreadable while the device is locked, `.completeUnlessOpen` files stay usable if they were already open, `.completeUntilFirstUserAuthentication` files (the default) are readable from the first unlock after boot onward, and `.none` files are always readable. You pick the class per file or per directory, and the trade-off is always the same: stricter protection versus code that has to run while the phone is in a pocket.

Key points:

- Set the class on write (`Data.WritingOptions.completeFileProtection` and friends), per item with `FileAttributeKey.protectionKey`, on a directory (new files inherit it), or app-wide with the Data Protection entitlement
- `.completeUntilFirstUserAuthentication` is the default for app files and for Core Data stores (`NSPersistentStoreFileProtectionKey`); it protects against a powered-off, stolen device, not against a locked one
- Shortly after lock the `.complete` class key is discarded; reads and writes then fail, and `UIApplication.ProtectedDataWillBecomeUnavailable` (typed `.protectedDataWillBecomeUnavailable` message, iOS 26+) gives you a moment to close files
- Check `UIApplication.shared.isProtectedDataAvailable` before background work touches protected data
- Keychain parallels: `WhenUnlocked` ≈ `.complete`, `AfterFirstUnlock` ≈ `.completeUntilFirstUserAuthentication`; `WhenPasscodeSetThisDeviceOnly` is stricter than any file class; there is no Keychain equivalent of `.completeUnlessOpen`

Related to check: File Storage, Keychain, Encryption at Rest, Securing Sensitive Data, Backups and Restore, background tasks, notification service extensions

---

Encryption at Rest

Q: iOS already encrypts the disk. When would you add your own encryption for data at rest, and how would you do it?

Short answer: Data Protection already encrypts every file with hardware-backed keys, and with `.complete` the data is unreadable while the device is locked, so for most apps the right answer is "pick the right protection class and keep secrets in the Keychain". I add **app-level encryption** only for a specific threat Data Protection doesn't cover: data that must stay encrypted while the device is *unlocked* (against other code paths, exports, backups or forensic copies), data that must be unreadable until the user re-authenticates in the app, or data I need to be able to destroy instantly by deleting a key.

Key points:

- Data Protection: free, transparent, per-file keys, tied to the passcode; protects a locked or powered-off device, not data inside a running, unlocked app
- App-level encryption: CryptoKit `ChaChaPoly.seal` / `AES.GCM.seal` give confidentiality **and** integrity; the sealed box's `combined` form carries nonce, ciphertext and tag
- Key storage: a `SymmetricKey(size: .bits256)` in the Keychain (`WhenUnlockedThisDeviceOnly`, or with `SecAccessControl` + biometrics to require Face ID), or wrapped by a Secure Enclave key
- Use authenticated data (file name, record ID, format version) so ciphertext can't be swapped between files undetected
- Layer them: app encryption **plus** a protection class, not instead of one

Related to check: File Protection Classes, Keychain, Securing Sensitive Data, Secure Enclave and CryptoKit, SQLite on iOS, Data Deletion and Retention

---

Secure Enclave and CryptoKit

Q: What is the Secure Enclave, and how would you use it from Swift? For example, to require Face ID before the app signs a sensitive request.

Short answer: The Secure Enclave is a separate processor with its own memory and keys. A private key generated there **never leaves it**: the app only gets an encrypted handle (`dataRepresentation`) that works on that one device, and every signature or key agreement runs inside the enclave.

Key points:

- Supported keys: P-256 signing and key agreement since iOS 13; **iOS 26+** adds post-quantum `SecureEnclave.MLKEM768` / `MLKEM1024` (key encapsulation) and `SecureEnclave.MLDSA65` / `MLDSA87` (signatures)
- Store the handle (`dataRepresentation`) in the Keychain and recreate the key with `init(dataRepresentation:aut` it's useless on any other device
- Access control: `SecAccessControlCreateWithFlags` with `.privateKeyUsage` plus `.biometryCurrentSet` (invalidated when the user enrolls a new face or finger), `.biometryAny`, `.userPresence` (biometrics or passcode) or `.devicePasscode`
- Pass an already-evaluated `LAContext` as `authenticationContext` to control the prompt text and avoid double prompts; `touchIDAuthenticationAllowableReuseDuration` allows a short reuse window
- Check `SecureEnclave.isAvailable` and have a fallback path

Related to check: Keychain, Encryption at Rest, Securing Sensitive Data, Backups and Restore, App Attest, passkeys, LocalAuthentication

---

Privacy Manifest

Q: What is a privacy manifest, and what goes in it?

Short answer: A privacy manifest is a property-list file, `PrivacyInfo.xcprivacy`, that an app or SDK ships in its bundle to declare what data it collects, whether it tracks, which domains it uses for tracking,

and why it calls **required-reason APIs**: APIs such as `UserDefaults`, file timestamps, system boot time, disk space and active keyboards that could be misused for fingerprinting. Since May 1, 2024, App Store Connect rejects uploads that use those APIs without an approved reason code, including uses inside third-party SDKs.

Key points:

- `NSPrivacyTracking` (Bool) and `NSPrivacyTrackingDomains` (array): domains listed there are blocked until the user grants App Tracking Transparency
- `NSPrivacyCollectedDataTypes`: one dictionary per data type with `NSPrivacyCollectedDataType`, `...Linked`, `...Tracking` and `...Purposes` (e.g. `NSPrivacyCollectedDataTypePurposeAppFunctionality`, `...Analytics`)
- `NSPrivacyAccessedAPITypes`: one dictionary per category with `NSPrivacyAccessedAPIType` and `NSPrivacyAccessedAPITypeReasons` (reason codes)
- **Third-party SDKs** include their own manifest; SDKs on Apple's list of commonly used SDKs must also be **signed** when you add them as binary dependencies, and Xcode shows and validates the signature
- **Privacy report:** Organizer -> right-click an archive -> *Generate Privacy Report* (a PDF aggregating all manifests)

Related to check: UserDefaults, File Storage (timestamps and disk space), App Groups and Shared Containers, Securing Sensitive Data, App Transport Security, App Tracking Transparency, App Store privacy labels

---

Data Deletion and Retention

Q: Your app lets users create an account. What does 'delete my account' have to do on iOS, both on the device and in iCloud, and how do you decide how long to keep data in general?

Short answer: Since mid-2022, App Review requires any app that supports account creation to let users **initiate account deletion inside the app** (guideline 5.1.1(v)), and deletion means deleting the account and its personal data, not just deactivating it. The flow is: confirm and re-authenticate, call the server to delete (and, for Sign in with Apple, have the server revoke the user's tokens with Apple's REST API), then wipe everything the app stored locally: the database, files, Keychain items, defaults, caches, cookies and web data.

Key points:

- In-app entry point that's easy to find (usually Settings -> Account); tell users what happens and how long it takes; data you must keep for legal reasons is the exception and should be disclosed
- Server first, local second: if the server call fails, the account still exists, so don't wipe and pretend it worked
- Local wipe: `ModelContainer.erase()` (or destroy the Core Data store), App Group container, Documents / Application Support / Caches / tmp, `UserDefaults` domains, `URLCache`, `HTTPCookieStorage`, `WKWebsiteDataStore`
- **Keychain items survive app deletion** on iOS, so delete them explicitly (and clear stale ones on the first launch after a reinstall)
- iCloud: delete the record zones your app owns in the private database (CloudKit), and the files in your iCloud Drive container

Related to check: Keychain, Keychain Sharing, Encryption at Rest, Backups and Restore, Data Modeling for Mobile, UserDefaults, App Groups and Shared Containers, Sign in with Apple, privacy manifests, CloudKit

---

Backups and Restore

Q: What parts of your app's data end up in an iCloud backup, and what can go wrong when a user restores that backup onto a new iPhone?

Short answer: iCloud and Finder backups include the app's container (Documents, Library and Application Support, Preferences) **except** Library/Caches, tmp and anything marked `isExcludedFromBackup`; the app binary itself is re-downloaded. Keychain items come along in encrypted and iCloud backups only if their accessibility class doesn't end in **`ThisDeviceOnly`**, and Secure Enclave keys never move.

Key points:

- Put re-creatable data in Caches (purgeable, not backed up); large re-downloadable data that must not be purged in Application Support with `isExcludedFromBackup`; user content in Documents or Application Support (backed up)
- App Review rejects apps that back up large amounts of re-downloadable data (the iOS Data Storage Guidelines)
- `ThisDeviceOnly` Keychain items restore only to the same device; non-`ThisDeviceOnly` items migrate in encrypted backups (and iCloud Keychain sync is a separate mechanism)
- Secure Enclave keys, App Attest keys and APNs device tokens are per device: re-create or re-register after a restore
- A restored container can come from an **older app version** while the installed app is the latest, so migrations must handle every old format

Related to check: File Storage, Keychain, Keychain Sharing, File Protection Classes, Secure Enclave and CryptoKit, Data Deletion and Retention, App Attest, push notifications

---

Testing Persistence

Q: How do you write fast, reliable tests for code that uses SwiftData or Core Data? How would you test a schema migration?

Short answer: I test persistence against the **real** framework with an **in-memory store**, not a mock of `ModelContext`: SwiftData's `ModelConfiguration(isStoredInMemoryOnly: true)` or a Core Data store at `/dev/null`. In Swift Testing each test gets a new suite instance, so I build the container in the suite's `init()` and every test starts from an empty, private store, which keeps tests independent even though they run in parallel.

Key points:

- One container per test (built in `init()`), stored on the suite so it stays alive for the whole test; never a shared static container
- Give each in-memory configuration a unique name (`UUID().uuidString`) so no two tests can ever point at the same store
- `@MainActor` suites when the code under test uses `mainContext`; for a `@ModelActor` repository, create it from the container and `await` its methods
- Assert through a fresh fetch, not through the objects you inserted, so you test what was saved
- Core Data: load the `NSManagedObjectModel` **once** per process (a `nonisolated(unsafe) static let`, since the type isn't `Sendable` but an in-use model is immutable) and pass it to `NSPersistentContainer(name:managedObjectModel:)`; set the store URL to `/dev/null` (SQLite semantics, no file) rather than `NSInMemoryStoreType`, which doesn't support SQLite-only features such as batch insert, update and delete requests

Related to check: ModelContainer and ModelConfiguration, Core Data, Persistent Container, Core Data Migration, Codable File Persistence, Swift Testing, test isolation, test fixtures, integration testing

---

Testing

Unit Testing

Q: What is unit testing? How do you write unit tests in Swift?

Short answer: A unit test checks one small piece of behavior (a function, a type, a view model) in isolation, quickly and deterministically, with no network, disk or UI. In Swift I write them with Swift Testing: a `struct` suite, `@Test` functions and `#expect` / `try #require`, following Arrange-Act-Assert.

Key points:

- Small, fast, isolated, deterministic; test behavior through the public API
- Swift Testing: `@Test func`, `#expect`, `try #require`, suites are plain types
- A fresh suite instance per test, so state doesn't leak between tests
- Inject dependencies (`any Protocol` or generics) and pass fakes

Related to check: Swift Testing, test isolation, given-when-then, testing error paths, view model testing, fake vs mock, TDD

Mocking and Stubbing

Q: How do you mock dependencies in tests?

Short answer: I put each external dependency (network, database, clock, analytics) behind a small protocol, inject it through `init`, and in tests pass a hand-written double instead of the real thing. A stub just returns canned values; a mock or spy also records how it was called so I can assert on the interaction.

Key points:

- Protocol at the boundary; the system under test depends on `any UserAPI` (or a generic), never on the concrete client
- Inject through `init`; never swap a global or a `shared` instance
- Assert outcomes first; assert interactions only when the call itself is the requirement

Related to check: test doubles (stub, spy, mock, fake), fake vs mock, test doubles best practices, dependency injection, URLProtocol, view model testing, Sendable

---

UI Testing

Q: What is UI testing? How does it differ from unit testing?

Short answer: UI tests drive the real, installed app from a separate test-runner process, finding elements through the accessibility layer and tapping, typing and asserting like a user would. They're written with XCTest's XCUITest APIs (`XCUIApplication`, `XCUIElement`), and that stays true in the Swift Testing era: Swift Testing has no UI automation, so unit and integration tests use Swift Testing while UI tests remain XCTest, often in the same project.

Key points:

- Black box: the test can't call app code or see app state, only what the accessibility tree exposes
- Queries: `app.buttons["login.submit"]`, matched by accessibility identifier (or label as a fallback)
- Synchronize with `waitForExistence(timeout:)`, never `sleep`
- Configure the app with `launchArguments` / `launchEnvironment` so it uses stub services and a clean state
- Page objects keep queries in one place so a UI change means one edit
- UI test methods are `@MainActor` in Swift 6, since `XCUIApplication` is main-actor isolated

Related to check: XCUITest, accessibility identifiers, UI test identifiers, page objects, test pyramid, flaky tests, test plans, CI testing, screenshot testing

---

Testing Resizable Layouts and iPhone Duo

Q: How do you test that your app adapts to resizable windows and iPhone Duo?

Short answer: I test sizes, not devices. The layout decisions (column counts, when to show a sidebar, which arrangement to use) live in small pure types that Swift Testing covers with parameterized tests over widths and size classes.

Key points:

- **Unit level:** extract layout policy (`columns(forWidth:)`, `showsSidebar(sizeClass:width:)`) and test it with `@Test(arguments:)` over widths, not device names
- **View controller level:** `traitOverrides.horizontalSizeClass`, set `view.frame`, `layoutIfNeeded()`, then assert on observable state; resize twice to prove it adapts both ways
- **Snapshot level:** a narrow and a wide width with the matching size class, plus large Dynamic Type and dark mode, rather than one hardcoded device
- **Previews:** `#Preview(traits: .fixedLayout(width:height:))` with size-class and Dynamic Type environment overrides for quick visual checks during development
- **Destinations:** a test plan or CI matrix over `iPhone 18 Pro`, `iPhone Duo` (Xcode 27.1) and an iPad, plus a nightly UI test pass
- **Manual checks:** Device Hub's resizable simulator for continuous resizing, the iPhone Duo simulator for folding, unfolding and vertical bars (iOS 27.1 SDK), and iPhone Mirroring at extremes

Related to check: Snapshot testing, Parameterized tests, View controller testing, UI testing, Test plans, CI testing, Screenshot testing, Adaptive layout for iPhone Duo, Resizable iPhone apps, iPhone Duo poses and vertical bars

---

XCTest and Async

Q: How do you test async code with XCTest?

Short answer: For `async` APIs I make the test itself `async throws` (`func testFetch() async throws`) and just `await` the call, then assert; XCTest waits for the test to finish. For callback- or delegate-based APIs I create an `XCTestExpectation`, fulfill it in the callback, and wait with `await fulfillment(of: [exp], timeout: 1)`; inside async tests `wait(for:)` is unavailable.

Key points:

- `func testX() async throws` + `await`; no expectation needed
- Callbacks: `expectation(description:)`, `exp.fulfill()`, `await fulfillment(of:timeout:)`
- `wait(for:timeout:)` only in synchronous tests; it's unavailable from async contexts
- `@MainActor` on the test or `XCTestCase` subclass when the code under test is main-actor isolated
- `expectedFulfillmentCount`, `isInverted` (assert something does *not* happen), `enforceOrder:`

Related to check: async testing patterns, testing async view models, Swift Testing (`confirmation`), @MainActor, XCTestExpectation

---

ViewModel Testing

Q: How do you test a ViewModel?

Short answer: Because the view model takes its dependencies through `init` behind protocols, I construct it in the test with a fake service, call its intent methods, and assert on its state. With Swift Testing that's a `@MainActor @Suite struct` (the view model is main-actor isolated), `@Test` functions and `#expect`.

Key points:

- Fake the dependency (a small struct conforming to the protocol), don't hit the network
- Arrange: build the view model with the fake; act: call `load()` / `increment()`; assert: `#expect` on state
- `@MainActor` on the suite so tests can touch main-actor state directly
- Test success, failure, and edge cases (empty list, retry)
- No UI; an `@Observable` view model is just a class with properties

Related to check: MVVM, view model design, testing async view models, Swift Testing, fake vs mock, dependency injection

---

Snapshot Testing

Q: What is snapshot testing? When would you use it?

Short answer: A snapshot test renders a view or view controller, or serializes a value, and compares the result with a reference file committed next to the tests. The first run records the reference; later runs fail on any difference and attach the diff, so unintended visual changes get caught in code review instead of by users.

Key points:

- `assertSnapshot(of: view, as: .image)`; the older `matching:` label is deprecated
- Strategies: `.image` (pixels), `.recursiveDescription` / `.dump` (text), `.json` for `Encodable` values
- Record modes: `.missing`, `.failed`, `.all`, `.never`, set with `withSnapshotTesting(record:)` or a suite trait
- References live in `__Snapshots__/` beside the test file and are reviewed in PRs like code
- Same simulator and OS for recording and verifying, or pixels differ

Related to check: Screenshot testing, golden file testing, UI testing, SwiftUI previews, Dynamic Type, size classes, test pyramid

---

TDD

Q: What is TDD? Describe the red-green-refactor cycle.

Short answer: TDD means writing a failing test before the production code, in short cycles. **Red**: write one small test for the next behavior and watch it fail (in Swift, "doesn't compile yet" counts).

Key points:

- Watching the test fail first proves it can fail (and that it tests the right thing)
- One behavior per cycle; green means *minimal*, even hard-coded, until another test forces generality
- Refactor is mandatory, not optional; it's where the design improves
- Swift Testing fits well: parameterized `@Test(arguments:)` makes adding a case one line

Related to check: unit testing, Swift Testing, parameterized tests, testability design, given-when-then, test naming, refactoring for testability

---

Accessibility Identifiers

Q: Why use accessibility identifiers for UI testing?

Short answer: UI tests find elements through the accessibility tree, and `accessibilityIdentifier` is the one property in it that exists purely for automation: VoiceOver never reads it and users never see it. That makes it the stable selector, unlike `accessibilityLabel`, which is user-facing text that changes with copy edits and localization.

Key points:

- `accessibilityIdentifier`: for tests, invisible, not localized
- `accessibilityLabel` / `Value` / `Hint`: user-facing, spoken by VoiceOver, localized
- Queries match by identifier (or label): `app.buttons["login.submit"]`
- Stable across languages, copy changes and A/B text variants
- Setting an identifier doesn't make an element accessible; it must already be in the tree

Related to check: UI test identifiers, UI testing, VoiceOver, accessibility labels, localization, Accessibility Inspector

---

Test Doubles: Mock, Stub, Spy, Fake

Q: What's the difference between mock, stub, spy, and fake?

Short answer: They're all test doubles, and they differ by what they're for. A **stub** feeds canned answers into the code under test; a **spy** records the calls it receives so the test can assert on them afterwards; a **mock** is a spy that also carries the expectations and verifies them; a **fake** is a real but simplified implementation, like an in-memory repository.

Key points:

- Stub: controls *indirect input* (what the dependency returns or throws)
- Spy: captures *indirect output* (which calls were made, with which arguments)
- Mock: spy + built-in expectations (`verify()`); assertions live in the double
- In Swift 6 the protocol is usually `Sendable`: stubs are structs, spies/fakes with state are actors, `@MainActor` classes or `Mutex`-backed

Related to check: mocking, fake vs mock, test doubles best practices, contract testing, dependency injection, Mutex, actors

---

Code Coverage

Q: What is code coverage? How do you use it?

Short answer: Code coverage measures which lines and functions ran while the tests ran. In Xcode you turn it on in the test plan (or with `-enableCodeCoverage YES`), and the report shows a percentage per target, file and function, with execution counts in the editor gutter.

Key points:

- Enable per test plan (choose which targets to gather) or `xcodebuild -enableCodeCoverage YES`
- Report navigator -> coverage; gutter shows how often each region ran
- CLI: `xcrun xccov view --report TestResults.xcresult`; packages: `swift test --enable-code-coverage`
- Coverage says *executed*, not *verified*; mutation testing measures the second
- Use it to guide: uncovered error paths and branches in critical code

Related to check: Mutation testing, test plans configuration, CI testing, parameterized tests, test pyramid, Swift Testing

---

Quick and Nimble

Q: What are Quick and Nimble? When would you use them?

Short answer: Quick is a third-party BDD test framework built on XCTest (`describe`, `context`, `it`, `beforeEach`), and Nimble is its matcher library (`expect(x).to(equal(5))`, `toEventually` for async polling). They were popular because XCTest's assertions and method names were hard to read as specifications.

Key points:

- Quick: `QuickSpec` subclass with `describe` / `context` / `it`, `beforeEach` / `afterEach`
- Nimble: matchers (`equal`, `beNil`, `contain`, `throwError`) and `toEventually` polling
- Swift Testing covers the BDD-ish readability: `@Suite("When logged in")`, `@Test("shows the profile")`

Related to check: Swift Testing, given-when-then, test naming, BDD, async testing patterns

---

URLProtocol for Mocking Network

Q: How do you use URLProtocol to mock network requests?

Short answer: `URLProtocol` sits inside the URL loading system, so a subclass can answer requests with canned responses while my real `URLSession`-based client, request building and decoding all run unchanged. I override `canInit(with:)`, `canonicalRequest(for:)`, `startLoading()` and `stopLoading()`, and in `startLoading()` report the response, data and finish (or an error) to `client`.

Key points:

- Override all four methods; call `didReceive` -> `didLoad` -> `urlProtocolDidFinishLoading`, or `didFailWithError`
- Inject the `URLSession` into the API client; tests build it from an ephemeral configuration with `protocolClasses`
- Parallel safety: the URL loading system creates protocol instances itself, so per-test data must be looked up from a static. Keep it in a `Mutex` keyed by something unique per test (here a UUID host)
- Good for testing the real networking stack; for logic above it, a stubbed `HTTPClient` protocol is simpler and faster

Related to check: URLSession, mocking, integration testing, contract testing, test isolation, parallel testing, Mutex, Sendable

---

ViewController Testing

Q: How do you test a UIViewController?

Short answer: I create the view controller with its dependencies injected through `init`, call `loadViewIfNeeded()` to run `viewDidLoad`, trigger behavior the way a user would (for example `sendActions(for: .touchUpInside)`), and assert on the visible outputs and on what the mocks recorded. These are fast in-process unit tests, so I write them with Swift Testing, marked `@MainActor` because UIKit is main-actor isolated.

Key points:

- Constructor injection, then `loadViewIfNeeded()` before touching outlets or subviews
- Simulate input with `sendActions(for:)`, or call delegate/data source methods directly
- If an action starts a `Task`, keep a handle to it and `await` it instead of sleeping
- `@MainActor` on the suite (Swift Testing) or the test method (XCTest)

Related to check: Swift Testing, view model testing, dependency injection, mocking, snapshot testing, UI testing, size classes, coordinators

---

SwiftUI Preview as Test

Q: Can you use SwiftUI Previews for testing?

Short answer: Not as automated tests. A `#Preview` is a development tool: it renders a view in Xcode's canvas so a person can look at it, but it doesn't assert anything and doesn't run as part of `xcodebuild test`.

Key points:

- Previews: manual, visual, no assertions, not run in CI
- One preview per interesting state; `@Previewable @State` for interactive previews
- Shared fixtures (`Profile.fixture`) feed previews, unit tests and snapshot tests
- Logic: Swift Testing on the model; appearance: snapshot tests; flows: UI tests; third-party tools such as ViewInspector inspect the view tree

Related to check: SwiftUI previews, snapshot testing, view model testing, test fixtures, Observation, dependency injection, ViewInspector

---

Parameterized Tests

Q: How do you run the same test with different inputs?

Short answer: In Swift Testing I write one `@Test(arguments: [...])` function that takes the input as a parameter, and the framework runs each argument as its own independent test case: it shows up separately in the results, runs in parallel, and can be re-run on its own. For input/expected pairs I pass `zip(inputs, expected)`, because two separate collections produce every combination (a cartesian product).

Key points:

- `@Test(arguments: collection)` with one parameter, or two collections for every combination
- `zip(a, b)` for paired input/expected values (avoids the cartesian product)
- Each argument is a separate test case: separate result, parallel, individually re-runnable
- `arguments: Theme.allCases` for enums; arguments must be `Sendable`

Related to check: Swift Testing, traits and tags, XCTContext.runActivity, test naming, unit testing

---

CI and Automated Testing

Q: How do you run tests in CI?

Short answer: CI runs the same `xcodebuild` commands you could run locally, on every pull request: pick a scheme and test plan, a simulator destination, and write results to an `.xcresult` bundle that the

CI system reports from. For speed I build once with `build-for-testing`, then run `test-without-building` (on one machine or sharded across several), with parallel testing on and the slow UI tests in their own test plan or job.

Key points:

- `xcodebuild test -scheme App -testPlan CI -destination 'platform=iOS Simulator,name=iPhone 18 Pro,OS=latest'`
- `build-for-testing` + `test-without-building` separates compiling from running
- `-parallel-testing-enabled YES` runs test classes across simulator clones; Swift Testing also runs tests in parallel in-process
- Narrow a run with `-only-testing:Target/Suite` / `-skip-testing:`, or by Swift Testing tags in the test plan
- `-resultBundlePath` + `xcrun xcresulttool` / `xcrun xccov` for reports and coverage
- Packages: `swift test --filter`, `--skip`, `--enable-code-coverage`, `--xunit-output`

Related to check: Test plans, test plans configuration, flaky tests, code coverage, xcodebuild, Xcode Cloud, fastlane, Swift Testing tags, UI testing

---

Test Naming Conventions

Q: What naming conventions do you use for tests?

Short answer: A test name should state the behavior being verified, so a failure in CI reads like a broken requirement: unit, scenario and expected result. In Swift Testing I keep the function name short and camelCase (`loadFailureShowsError`) and put the readable sentence in the display name, `@Test("Shows an error when the network fails")`, with the suite name giving the subject.

Key points:

- Name the behavior, not the method: unit + scenario + expected outcome
- Swift Testing: suite name = subject, display name = sentence, function name = short identifier
- XCTest: `test` prefix is required; underscores separate unit, scenario and result
- Parameterized tests: the argument shows in each case's name, so don't repeat inputs in the function name

Related to check: given-when-then, Swift Testing, test discovery, parameterized tests, unit testing

---

XCTest Assertions

Q: What XCTest assertions do you use? When?

Short answer: In XCTest I pick the most specific assertion, because it produces the most useful failure message: `XCTAssertEqual` prints both values, while `XCTAssertTrue(a == b)` only says "false". The core set is equality, booleans, nil checks, `XCTAssertThrowsError`/`XCTAssertNoThrow`, plus `XCTUnwrap` for optionals and `XCTFail` for unreachable branches.

Key points:

- `XCTAssertEqual` / `XCTAssertNotEqual` (with `accuracy:` for floating point)
- `XCTAssertTrue` / `XCTAssertFalse`, `XCTAssertNil` / `XCTAssertNotNil`
- `XCTAssertThrowsError(try f()) { error in ... }` and `XCTAssertNoThrow`
- `try XCTUnwrap(optional)`, `XCTFail("...")`; ordering with `XCTAssertGreaterThan` and friends

Related to check: Swift Testing, XCTUnwrap, testing error paths, unit testing, XCTExpectFailure

---

setUp and tearDown

Q: When do you use setUp and tearDown?

Short answer: Per-test setup creates a fresh system under test and fresh fakes, and teardown releases or cleans up anything that outlives the test (temp files, a `UserDefaults` suite, registered handlers). In Swift Testing there are no special methods: a new suite instance is created for every test, so setup is just `init()` (which can be `async throws`), properties can be `let` constants, and teardown is `deinit` on a `final class` or `actor` suite.

Key points:

- Swift Testing: `init()` = setUp, `deinit` = tearDown (class or actor suite only; structs have no `deinit`)
- `init() async throws` replaces `setUp() async throws`; a throwing `init` fails the test
- XCTest: `override func setUp()` / `tearDown()`, `setUpWithError()` / `tearDownWithError()`, `async throws` overloads, `addTeardownBlock { }`
- Class-level `override class func setUp()` runs once per test class; Swift Testing uses `static let` or a shared fixture for one-time work

Related to check: test isolation, Swift Testing suites, parameterized tests, testing singletons, dependency injection

---

Integration Testing

Q: What is integration testing? How does it differ from unit testing?

Short answer: A unit test checks one type in isolation with its dependencies faked; an integration test wires several real components together, say view model + repository + JSON decoding + persistence, and fakes only the outer edges like the network. That catches bugs unit tests can't: wrong wiring, decoding mismatches, mapping errors, actor-isolation hops between layers.

Key points:

- Real components in the middle, fakes at the boundary (HTTP client or `URLProtocol`, in-memory SwiftData)
- Fewer than unit tests, focused on critical flows (login, checkout, sync)
- Deterministic: no real network, no shared state, so they can still run in parallel
- `.tags(.integration)` to group and filter; slow ones can get `.timeLimit(.minutes(1))`
- UI tests (`XCUIApplication`) are a separate, end-to-end layer and stay in XCTest

Related to check: unit testing, test pyramid, URLProtocol, contract testing, mocking, test plans, UI testing, SwiftData

---

Testing Async ViewModels

Q: How do you test a ViewModel with async methods?

Short answer: If the view model's `load()` is `async`, the test just awaits it: `await viewModel.load()`, then `#expect` on the resulting state. The suite is `@MainActor` because the view model is, and the injected fake returns immediately (or throws) so there are no real delays and never any `sleep`.

Key points:

- Make the intent `async` so tests can await completion instead of polling
- Fakes return instantly or throw a known error
- `@MainActor @Suite struct` so the test reads main-actor state without hops
- `try #require(...)` unwraps or stops the test early with a clear failure

- `confirmation()` for callback/event APIs, e.g. an `onError` closure the view model fires

Related to check: view model testing, async testing patterns, Swift Testing, testing error paths, @MainActor, MVVM

Testing Error Paths

Q: How do you test error handling?

Short answer: I test error paths at two levels. For a throwing API I assert the exact error with Swift Testing's `#expect(throws: ValidationError.tooShort) { ... }`, or `#expect(throws: ValidationError.self)`, which returns the caught error so I can inspect its payload; typed throws (`throws(ValidationError)`) make that error type explicit in the signature.

Key points:

- `#expect(throws: SomeError.self) { ... }` for the type; `#expect(throws: SomeError.case) { ... }` for an `Equatable` value
- The macro returns the error (`E?`); `try #require(throws:)` returns it non-optionally and stops the test if nothing was thrown
- `#expect(throws: Never.self)` asserts nothing is thrown; async: `await #expect(throws: ...) { try await ... }`
- Typed throws (Swift 6.0): `func validate(_:) throws(ValidationError)`; callers and tests know the exact error type

Related to check: Swift Testing, typed throws, view model testing, testing async view models, XCTest assertions, parameterized tests

---

Designing for Testability

Q: How do you design code to be testable?

Short answer: I keep decisions in pure functions and value types, and push side effects (network, disk, time, randomness) to the edges behind small `Sendable` protocols injected through `init`. Then the core logic is tested with plain inputs and outputs, and the thin layer that coordinates it, usually a `@MainActor @Observable` view model, is tested with fakes.

Key points:

- Pure core, imperative shell: logic takes values, returns values
- Dependencies explicit in `init` (`any Protocol` or generics); no reaching for `.shared`
- Inject time and randomness: `now: @Sendable () -> Date`, a `Clock`, a random generator
- Observable state is plain properties, so tests just `await` an intent and read state

Related to check: refactoring for testability, testing singletons, test doubles best practices, view model testing, dependency injection, SOLID, Swift 6 concurrency

---

Flaky Tests

Q: What causes flaky tests? How do you fix them?

Short answer: A flaky test passes and fails on the same code. The usual causes are shared mutable state between tests, timing (fixed sleeps, real timers, animations), order dependence, and real external inputs such as the network, the clock, the locale or the file system.

Key points:

- Shared state and order dependence: singletons, `UserDefaults.standard`, static `URLProtocol` handlers, files, the keychain
- Timing: `sleep`, short timeouts, animations, `asyncAfter`, real `Task.sleep`
- External inputs: network, `Date()`, time zone, locale, random values
- Fixes: per-test instances created in the suite's `init()`, injected `Clock` and `Date` providers, stubbed network per `URLSession`, `await` instead of sleeping
- Stopgaps: `.serialized` on a suite, `withKnownIssue(isIntermittent: true)`, retry-on-failure in the test plan

Related to check: Test isolation, Swift Testing parallelism, mocking, testing singletons, testing UserDefaults, URLProtocol, async testing, test plans, CI testing

---

Testing Delegate Pattern

Q: How do you test code that uses delegates?

Short answer: I write a spy delegate that conforms to the protocol and records each call and its arguments, set it as the delegate, trigger the action, and assert on what was recorded. Delegates for UI components are usually `@MainActor`, so the spy is a `@MainActor final class` and the suite is `@MainActor @Suite struct`.

Key points:

- Spy delegate records calls (`selected: [Item]`), matching the protocol's isolation
- Synchronous action: trigger, then `#expect` on the spy
- Callback-style events: `await confirmation("didSelect", expectedCount: 1) { ... }`; the callback must happen before the closure returns
- Check the `weak` delegate contract: releasing the delegate leaves `nil`, no retain cycle

Related to check: delegate pattern, test doubles (spy), async testing patterns, confirmation, isolated conformances, memory management (weak references)

Performance Testing

Q: How do you add performance tests in XCTest?

Short answer: Performance tests are XCTest's `measure(metrics:options:block:)`: it runs a block several times, records one or more `XCTMetric`s, and compares the average with a baseline you set in Xcode, failing if the result is worse beyond the allowed deviation. They stay in XCTest even in a Swift Testing codebase, because Swift Testing has no `measure` equivalent, so performance tests usually live in their own XCTest class or target next to the Swift Testing unit tests.

Key points:

- `measure { }` records wall-clock time; `measure(metrics:)` picks metrics: `XCTClockMetric`, `XCTCPUMetric`, `XCTMemoryMetric`, `XCTStorageMetric`, `XCTApplicationLaunchMetric`, `XCTOSSignpostMetric`, `XCTHitchMetric(application:)` (iOS 26+)
- `XCTOSSignpostMetric.scrollingAndDecelerationMetric` and `XCTOSSignpostMetric(subsystem:category:name:)` for your own signposted intervals
- `XCTMeasureOptions`: `iterationCount`, `invocationOptions` (`.manuallyStart` / `.manuallyStop`)
- Baselines are stored per device in the project; set them from the test result in Xcode

Related to check: XCTest, XCTMetric, Instruments, os_signpost, MetricKit, app launch time, test plans, CI testing

---

Testing Combine Publishers

Q: How do you test Combine publishers?

Short answer: Combine is still common in existing code, and I test a pipeline by driving its inputs and collecting what it emits. For synchronous pipelines (subjects, `map`, `removeDuplicates`), I `sink` into an array, send inputs, and `#expect` on the array, with no waiting at all.

Key points:

- Synchronous pipeline: `sink` -> collect -> send inputs -> `#expect(received == [...])`
- Finite async publisher: `for try await value in publisher.values { ... }`
- Infinite publishers (subjects, `@Published`): add `.prefix(n)` or `.first()` so the loop ends
- Keep the `AnyCancellable` alive until you've asserted

Related to check: Combine, AsyncSequence, async testing patterns, Observation, Observations, view model testing, XCTestExpectation

---

Test Plans

Q: What are Xcode Test Plans? When would you use them?

Short answer: A test plan is an `.xctestplan` file, attached to a scheme, that says which tests run and how: which test targets, which tests or Swift Testing tags to include or skip, and one or more configurations that rerun the selection with different settings (language and region, environment variables, sanitizers, code coverage, repetition). A scheme can have several plans, so I typically keep a fast PR plan, a full nightly plan and a UI test plan, and CI selects one with `-testPlan`.

Key points:

- `.xctestplan` = test selection + configurations; a scheme can hold several, one default
- Selection by target, by suite or test, and by Swift Testing tags (include/exclude)
- Configurations multiply the run: e.g. English and German, or with and without Address Sanitizer
- Options: parallelization per target, randomized order, repetition (retry on failure, until failure), test timeouts, code coverage targets
- CLI: `-testPlan`, `-showTestPlans`, `-only-test-configuration`, `-skip-test-configuration`

Related to check: Test plans configuration, CI testing, Swift Testing tags, flaky tests, code coverage, schemes, localization testing

---

Testing Code That Uses Singletons

Q: How do you test code that uses singletons?

Short answer: I don't test *through* the singleton; I stop the code under test from reaching for it. I put what the code needs behind a protocol, have the singleton conform, and inject it through `init` with `.shared` as the default, so production code is unchanged and tests pass a stub.

Key points:

- Protocol for the capability the caller uses (`LocationProviding`), not the whole singleton
- Inject via `init`; default to `.shared` so call sites don't change

- Swift 6 rules for statics: `static let shared` is fine if the type is `Sendable` (or the type is `@MainActor`); `static var` is an error unless `@MainActor`, `Mutex`-protected, or `nonisolated(unsafe)` (last resort)
- Third-party singletons you can't change: wrap them in an adapter that conforms to your protocol

Related to check: refactoring for testability, testability design, dependency injection, singletons, Swift 6 global state rules, test isolation, parallel testing

---

Testing Code That Uses UserDefaults

Q: How do you test code that uses UserDefaults?

Short answer: I never let the code under test touch `UserDefaults.standard`; I inject a `UserDefaults` instance through `init`, and each test creates its own with `UserDefaults(suiteName: UUID().uuidString)`. A unique suite per test matters because Swift Testing runs tests in parallel by default, so a fixed name like `"Test"` would be shared and flaky.

Key points:

- Inject `UserDefaults` (or a small protocol) through `init`; default to `.standard` in production
- Per-test suite: `UserDefaults(suiteName: UUID().uuidString)`; parallel-safe, no ordering assumptions
- Swift Testing creates a new suite instance per test, so `init()` is per-test setup and `deinit` per-test teardown
- Clean up with `removePersistentDomain(forName:)`
- The SDK doesn't mark `UserDefaults` as `Sendable`, so keep the store and its test in one isolation domain (here the main actor)

Related to check: test isolation, setup and teardown, parallel testing, testing singletons, dependency injection, @AppStorage, Swift Testing suites

---

Given-When-Then

Q: What is the Given-When-Then pattern?

Short answer: Given-When-Then is a way to structure each test as a small specification: **Given** some starting state, **When** one action happens, **Then** a specific outcome is observable. It's the BDD phrasing of Arrange-Act-Assert, and it keeps tests readable because each one has exactly one trigger and a clear expectation.

Key points:

- Given = setup / fixtures, When = the single action, Then = assertions on observable outcome
- Same idea as Arrange-Act-Assert, phrased for behavior
- One "When" per test; several related `#expects` in "Then" are fine
- Swift Testing display names and nested suites carry the scenario wording without long function names

Related to check: test naming, unit testing, Swift Testing, XCTContext.runActivity, Quick and Nimble, test fixtures

---

Test Fixtures

Q: What are test fixtures? How do you use them?

Short answer: A fixture is the known state a test starts from: sample model values, a JSON payload, a pre-filled in-memory store. I build them with factory functions that have sensible defaults (`User.fixture(name: "Ada")`), so each test spells out only the values it cares about, and keep recorded JSON payloads as resource files to test decoding.

Key points:

- Factory methods with defaulted parameters; override only what the test is about
- Fixtures are values (`Sendable` structs), never shared mutable singletons
- Per-test setup: suite stored properties / `init()`; teardown in `deinit` of a `final class` or `actor` suite
- JSON fixtures as test-bundle resources (`Bundle.module` in a package) for decoding and contract tests
- Parameterized tests: pass several fixtures with `@Test(arguments:)`

Related to check: test isolation, setup and teardown, parameterized tests, given-when-then, fake vs mock, contract testing, golden-file testing

---

Golden File Testing

Q: What is golden file testing?

Short answer: A golden file test compares a function's output with a reviewed, committed "golden" file: generated JSON, a rendered report, a parsed AST, a migration script. When the output changes on purpose you regenerate the file and review the diff in the PR; when it changes by accident the test fails and shows you where.

Key points:

- Output vs committed file; any difference fails
- A record mode regenerates files on purpose (environment variable, or a library's record setting)
- Output must be deterministic: sorted keys, fixed dates, locale, time zone and IDs
- Text formats diff well in code review; prefer them over binary
- swift-snapshot-testing's `.json`, `.dump` and `.lines` strategies are ready-made golden files

Related to check: Snapshot testing, screenshot testing, test fixtures, contract testing, Codable, Swift Testing attachments

---

Mutation Testing

Q: What is mutation testing?

Short answer: Mutation testing checks how good your tests are by deliberately breaking the code. A tool makes small changes called mutants (>= to >, `true` to `false`, removing a call), runs the tests against each one, and reports which mutants "survived" because every test still passed.

Key points:

- Mutant = small code change; killed if a test fails, survived if all pass
- Mutation score = killed / total (excluding equivalent mutants)
- Finds weak assertions and missing boundary tests that coverage misses
- Tools: Muter for Swift; Mull for LLVM-based languages (mainly C/C++)
- Cost: one test run per mutant; scope it to a module or changed files

Related to check: Code coverage, parameterized tests, assertions, test pyramid, flaky tests, testing error paths

---

Contract Testing

Q: What is contract testing?

Short answer: Contract testing checks that a consumer (the iOS app) and a provider (the backend) agree on the API, the requests, response shapes and status codes, without running both together end to end. In consumer-driven contract testing (Pact is the best-known tool), the app's tests record the interactions they rely on into a contract file, and the backend's CI replays that file against the real service, so a breaking server change fails the provider build before release.

Key points:

- Consumer: the app states what it sends and what it needs back
- Provider: verifies it still satisfies every consumer's contract
- Faster and more targeted than E2E: each side is tested alone against the shared contract
- Swift-side checks: decode provider examples, encode requests and compare to the agreed shape, tag them `.contract`
- Schema-first alternative: an OpenAPI document plus generated client (e.g. swift-openapi-generator) keeps types in sync

Related to check: integration testing, URLProtocol, test fixtures, fake vs mock, Codable, API versioning, OpenAPI, test pyramid

---

Test Pyramid

Q: What is the test pyramid?

Short answer: The test pyramid is a guide to the mix of tests: many fast, isolated unit tests at the base, fewer integration tests in the middle that wire real components together, and a small number of slow end-to-end UI tests at the top. Lower layers are faster, more precise about what broke and less flaky, so most checks belong there, and the top layer only proves the critical journeys work end to end.

Key points:

- Unit (most): view models, domain logic, formatters; milliseconds each, Swift Testing
- Integration (some): real components together, fake only the edges (network via `URLProtocol`, an in-memory database)
- Snapshot (alongside): how components look, in-process and fast
- UI / E2E (few): XCTest + `XCUIApplication` for login, checkout, onboarding
- Inverted pyramid ("ice cream cone"): mostly UI tests; slow, flaky, vague failures

Related to check: Unit testing, integration testing, UI testing, snapshot testing, contract testing, test isolation, CI testing, testability design

---

Fake vs Mock

Q: What's the difference between a fake and a mock?

Short answer: A fake is a working, simplified implementation of a dependency, like an in-memory repository, and the test asserts on the resulting *state*. A mock records the calls it receives (and may verify expectations), and the test asserts on the *interaction*.

Key points:

- Fake: real behavior with a shortcut (memory instead of disk/network); state-based testing
- Mock: records calls and arguments; interaction-based testing
- Fakes are reusable across many tests; mocks are usually tailored per test
- In Swift 6, a stateful fake is typically an `actor`; a mock for a `@MainActor` system under test can be a `@MainActor final class`, so the test reads it without `await`

Related to check: test doubles (stub, spy, mock, fake), test doubles best practices, mocking, contract testing, test fixtures, actors

---

Test Isolation

Q: Why is test isolation important?

Short answer: Each test must pass or fail on its own, regardless of which other tests ran, in what order, or at the same time; otherwise failures become flaky and you can't trust or re-run a single test. That matters even more with Swift Testing, because tests run **in parallel by default**, in-process, so any shared mutable state (singletons, `UserDefaults.standard`, a static `URLProtocol` handler, files at a fixed path) turns into a race.

Key points:

- Independent, order-free and parallel-safe tests
- Swift Testing creates a new suite instance per test; setup goes in `init()`
- Inject per-test dependencies instead of reaching for singletons or globals
- Unique resources: defaults suites, temp directories, per-session `URLSessionConfiguration.protocolClasses`
- `.serialized` is a stopgap for legacy shared state, not the fix

Related to check: setup and teardown, testing singletons, URLProtocol stubbing, Swift Testing parallelism, fake vs mock, Swift 6 global state

---

Async Testing Patterns

Q: What patterns do you use to test async code?

Short answer: The default pattern is simple: make the code under test `async`, make the test `async`, `await` the call and assert on the result or state; with fakes that return immediately there's nothing to wait for. For in-flight state (like `isLoading`) I use a *gated* fake that signals when it's been called and suspends until the test lets it continue, so the test can assert mid-flight deterministically.

Key points:

- `await` the async API directly; no expectations, no sleeps
- `@MainActor` suite when the system under test is main-actor isolated
- Gated fakes (two `AsyncStreams`: "started" and "gate") for mid-flight assertions
- `AsyncSequence`: `for await value in stream.prefix(3)` so the test can't hang
- Callbacks/events: `confirmation(expectedCount:)`; legacy completion handlers: wrap in a continuation
- Safety net: `.timeLimit(.minutes(1))` on the suite or test

Related to check: XCTest and async, testing async view models, Swift Testing (`confirmation`), AsyncStream, task cancellation, fake vs mock

---

XCTUnwrap

Q: What is XCTUnwrap? When do you use it?

Short answer: `XCTUnwrap` is XCTest's throwing unwrap: `let user = try XCTUnwrap(optionalUser)` returns the value, or throws and records a failure with a clear message if it's `nil`. The test function is marked `throws`, so the rest of the test is skipped instead of crashing on a force unwrap or needing a `guard ... else { XCTFail(); return }`.

Key points:

- Signature: `try XCTUnwrap(expression, "message")`; the test must be `throws`
- On `nil`: records a failure and throws, ending the test cleanly
- Replaces force unwraps (which crash the whole test run) and `guard`/`XCTFail` boilerplate
- Swift Testing: `let user = try #require(users.first)`; `try #require(condition)` for a boolean precondition

Related to check: XCTest assertions, Swift Testing (`#require`), testing error paths, unit testing

---

XCTExpectFailure

Q: What is XCTExpectFailure?

Short answer: `XCTExpectFailure` marks a known issue in XCTest: failures inside it are reported as *expected failures*, so the test stays green while the bug is tracked. It's **strict by default**: if nothing fails, XCTest records a new failure, so you notice when the bug is fixed and remove the marker.

Key points:

- Closure form `XCTExpectFailure("reason") { ... }` scopes it to a block; the plain call `XCTExpectFailure("reason")` covers the rest of the test
- Strict by default: if no failure is recorded, XCTest records one for the unmatched expectation
- `strict: false` for flaky issues, `enabled:` to switch it on conditionally, `issueMatcher:` to match a specific issue
- Swift Testing: `withKnownIssue("reason", isIntermittent: true) { ... }`, plus `when:` and `matching:` closures

Related to check: Swift Testing (`withKnownIssue`), XCTest assertions, test isolation, test plans, XCTSkip vs `.disabled`

Test Discovery

Q: How does XCTest discover tests?

Short answer: XCTest discovers tests at runtime through the Objective-C runtime: it finds every subclass of `XCTestCase` in the test bundle and treats each instance method that starts with `test`, takes no arguments and returns `Void` (it may be `throws` and/or `async`) as a test. Anything else is a helper.

Key points:

- XCTest: `XCTestCase` subclass + `func test...()` with no parameters; found via the ObjC runtime
- Methods with parameters, non-`Void` returns or a different prefix are silently ignored
- Swift Testing: `@Test` anywhere (free function, struct, class, actor, enum); `@Suite` is optional
- Swift Testing tests can take arguments (`@Test(arguments:)`) and be `async`, `throws`, `@MainActor`
- Both are discovered in the same test target and listed together in Xcode's Test navigator

Related to check: Swift Testing, test naming, parameterized tests, test plans, XCTSkip and `.disabled`

XCTContext runActivity

Q: What is XCTContext.runActivity?

Short answer: `XCTContext.runActivity(named:block:)` groups part of an XCTest into a named step. In Xcode's test report the steps appear as a nested outline, any failure or attachment is shown under the step it happened in, and the block's return value is passed back so it can produce values.

Key points:

- `XCTContext.runActivity(named: "Step") { activity in ... }`, nestable, returns the block's result
- Failures and attachments are reported under the activity
- `activity.add(XCTAttachment(screenshot: app.screenshot()))` attaches evidence to the step
- The API is main-actor isolated in Swift, so call it from a `@MainActor` test
- Swift Testing: no activities; split into tests, use display names, comments and `Attachment.record`

Related to check: given-when-then, UI testing, XCTAttachment and Swift Testing attachments, parameterized tests, XCTUnwrap

---

Test Plans Configuration

Q: How do you configure test plans?

Short answer: A test plan has shared settings plus a list of configurations, and every selected test runs once per configuration, with each configuration overriding only what it needs: language and region, environment variables and launch arguments, sanitizers, repetition mode, timeouts and code coverage. Test selection sits beside that: test targets, suites or individual tests to skip, and Swift Testing tags to include or exclude.

Key points:

- Shared settings (`defaultOptions`) + per-configuration overrides (`configurations`)
- Language/region per configuration to catch localization and formatting bugs
- `environmentVariableEntries` / `commandLineArgumentEntries` for mocks and feature flags
- Selection: targets, `skippedTests` (Swift Testing `suites` and XCTest `xctestClasses`), and Swift Testing tags (include/exclude in the Tests tab)
- Per-target `parallelizable`; plan-level execution order, timeouts, repetition (`retryOnFailure`)
- CLI: `-testPlan`, `-only-test-configuration`, `-skip-test-configuration`, `-testLanguage`, `-testRegion`

Related to check: Test plans, CI testing, Swift Testing tags, flaky tests, localization, sanitizers, test isolation, code coverage

---

Swift Testing Framework

Q: What is the Swift Testing framework?

Short answer: Swift Testing is Apple's modern, macro-based test framework; it ships with Xcode 16+ and works in both Swift 5 and Swift 6 language modes. A test is any function marked `@Test`, assertions are just `#expect` and `try #require` (which show the actual values on failure), and suites are plain types with a fresh instance per test.

Key points:

- `@Test` functions can be `async`, `throws` and `@MainActor`; `@Test("Readable name")` sets a display name
- `#expect` records a failure and continues; `try #require` stops the test and unwraps optionals
- Suites: a `struct` is preferred; setup in `init()` (can be `async throws`), teardown in `deinit` of a `final class`/`actor` suite
- `@Test(arguments:)`: each argument is a separate, re-runnable test case
- Traits: `.tags`, `.timeLimit`, `.serialized`, `.disabled`, `.enabled(if:)`, `.bug`
- `confirmation()` for callback events, `withKnownIssue` for expected failures, `Attachment.record` (Swift 6.2) for attachments

Related to check: XCTest assertions, parameterized tests, setup and teardown, test isolation, testing error paths, async testing patterns, XCTExpectFailure vs withKnownIssue, test discovery

---

Swift Testing Traits and Tags

Q: How do traits and tags work in Swift Testing? When would you write a custom trait?

Short answer: A trait is a value you pass to `@Test` or `@Suite` that changes how a test is described, selected or run. The built-in ones cover conditions (`.enabled(if:)`, `.disabled`), metadata (`.bug`, `.tags`), scheduling (`.serialized`, `.timeLimit`) and issue handling.

Key points:

- **Conditions:** `.enabled(if:)` and `.disabled(if:)`, also with an `async` closure form; `.disabled("reason")` always skips. An `@available` attribute on a test function becomes a condition too, so the test is skipped on older OS versions
- **Metadata:** `.bug("https://...", id: 123)` links issues in results; `.tags(.networking, .slow)`
- **Scheduling:** `.timeLimit(.minutes(1))` (minutes only: second-based durations are unavailable); `.serialized` runs the tests and parameterized cases of that suite one at a time
- **Issue handling:** `.filterIssues { ... }` and `.compactMapIssues { ... }` rewrite or drop issues recorded inside the test
- **Custom traits:** conform to `TestTrait` (for `@Test`), `SuiteTrait` (for `@Suite`; `isRecursive` decides whether nested suites get it) or both, plus `TestScoping` to wrap execution in `provideScope(for:testCase:performing:)`

Related to check: Swift Testing, XCTest to Swift Testing migration, test plans, test plans configuration, CI testing, setup and teardown, parameterized tests, flaky tests, test isolation

---

Migrating XCTest to Swift Testing

Q: We have a few thousand XCTest tests. How would you migrate them to Swift Testing?

Short answer: Incrementally, and only where it pays off. Both frameworks run side by side in the same test target and the same `xcodebuild test` / `swift test` run, so there's no big-bang switch.

Key points:

- **Coexistence:** one target can hold both; never mix `@Test` functions into an `XCTestCase` subclass
- **Mapping:** `XCTestCase` class -> `struct` suite; `func testX()` -> `@Test func x()`; `setUp` -> `init()`; `tearDown` -> `deinit` in a `final class` or `actor` suite; `XCTAssertEqual(a, b)` -> `#expect(a == b)`; `XCTUnwrap` -> `try #require`; `XCTFail` -> `Issue.record`; `XCTExpectFailure` -> `withKnownIssue`; `XCTSkipIf` -> `.enabled(if:)` / `.disabled(if:)`

- **Loops over inputs** become `@Test(arguments:)`, so each input is a separately reported, re-runnable case
- **Async:** `await` the API directly. Callback-style events use `confirmation()`, which, unlike `XCTestExpectation`, doesn't wait: the event must happen before its closure returns, so wrap the callback API in an async function first
- **Stays in XCTest:** UI tests, `measure {}` / `XCTMetric` performance tests, and tests built on XCTest-only helpers you can't replace yet

Related to check: Swift Testing, Swift Testing traits and tags, parameterized tests, setup and teardown, XCTest async, async testing patterns, XCTExpectFailure vs withKnownIssue, test isolation, testing singletons

---

UI Test Identifiers

Q: How do you choose accessibility identifiers for UI tests?

Short answer: Pick identifiers that describe *what an element is*, not what it says: unique on the screen, stable across copy changes and localization, and namespaced by screen, like `login.email` or `checkout.payButton`. I keep them as constants in one file compiled into both the app and the UI test target, so a rename is a compile error rather than a failing test.

Key points:

- `accessibilityIdentifier` (UIKit) / `.accessibilityIdentifier(_:)` (SwiftUI); not shown or spoken
- Convention: `screen.element` or `screen.section.element`, consistent casing
- Shared constants between app and UI tests: one source of truth, no string typos
- Rows: `"feed.row.\(item.id)"` from a stable model ID
- Containers: identify the container too, so queries can be scoped (`app.otherElements["feed.list"].buttons[..`

Related to check: Accessibility identifiers, UI testing, page objects, localization, screenshot testing, flaky tests

---

Screenshot Testing

Q: What is screenshot testing? When do you use it?

Short answer: People use "screenshot testing" for two things. Often it means visual regression testing: render UI, compare with an approved image, fail on differences, which is the same idea as snapshot testing and is usually done in-process with swift-snapshot-testing.

Key points:

- Comparison: reference image + diff, fails automatically (snapshot tooling)
- Capture: `XCUIApplication.screenshot()` / `XCUIScreen.main.screenshot()` + `XCTAttachment`, reviewed by people
- `attachment.lifetime = .keepAlways`, or the screenshot is deleted when the test passes
- Swift Testing can attach rendered images too: `Attachment.record(image, named:)` (image attachments, Swift 6.3+)
- Export from the result bundle: `xcrun xcresulttool export attachments`

Related to check: Snapshot testing, golden file testing, UI testing, test plans configuration, localization, UI test identifiers, test pyramid

---

Test Doubles Best Practices

Q: What are best practices for test doubles?

Short answer: Keep doubles minimal and behind small protocols you own, inject them through `init`, and assert on outcomes rather than on the calls the code happened to make. I wrap third-party types like `URLSession` or `UserDefaults` in my own protocol instead of mocking them directly.

Key points:

- Narrow protocols (interface segregation): a double implements two methods, not twenty
- "Don't mock what you don't own": put an adapter around system and third-party APIs and double the adapter
- Outcome assertions first; interaction assertions only when the call is the behavior
- One double per test's need: stub for input, spy for output, fake for realistic behavior
- Data-race-safe doubles; no global swap points, which also keeps Swift Testing's parallel runs safe

Related to check: fake vs mock, test doubles (stub, spy, mock, fake), mocking, refactoring for testability, test fixtures, test isolation

---

Refactoring for Testability

Q: How do you refactor legacy code for testability?

Short answer: I work in small, safe steps: first pin down current behavior with characterization tests where I can, then find the hidden dependencies (`.shared` singletons, `UserDefaults.standard`, `Date()`, `URLSession.shared`) and turn each one into a seam. The usual move is to extract a small `Sendable` protocol the existing type already satisfies, inject it through `init`, and default the parameter to the production instance so call sites don't change.

Key points:

- Characterization tests first: record what the code does today, even if it looks wrong
- Extract protocol, conform the existing type with an empty extension, inject via `init`
- Default arguments keep production call sites compiling while tests pass doubles
- Inject time and randomness too (`now: @Sendable () -> Date`, a `Clock`)
- Swift 6 migration and testability go together: removing global mutable state fixes both

Related to check: testability design, testing singletons, test doubles best practices, fake vs mock, dependency injection, Swift 6 migration, test isolation

---

Testing App Intents

Q: How do you test App Intents?

Short answer: I test App Intents at three levels. Most of the logic lives in a use case the intent calls, so ordinary Swift Testing unit tests with injected fakes cover the rules, and `perform()` stays a thin adapter that barely needs testing.

Key points:

- **Unit level:** put the logic in a `Sendable` use case with protocol dependencies; test it with Swift Testing and an actor-based fake store
- **Queries:** `EntityQuery` types are plain structs; test `entities(for:)` and `entities(matching:)` against an in-memory store the same way

- **System level (iOS 27+):** `IntentDefinitions(bundleIdentifier:)` exposes `intents`, `entities`, `enums`, `transientEntities` and `valueQueries` by identifier. `makeIntent(...)` builds an intent with parameters and `run()` performs it; entity definitions offer `entities(matching:)`, `entities(identifiers:)`, `allEntities()`, `suggestedEntities()`, `spotlightQuery(_:)` and `viewAnnotations()`
- **Results:** `AnyAppEntity`, `AnyAppIntent` and `ResolvedIntentResult` use dynamic member lookup; reading a property with an explicit type (`let body: String = try note.body`) converts and throws on mismatch
- **Manual:** run the App Shortcut phrases with Siri, check the Shortcuts app's action list and parameter summaries, search in Spotlight, and try widgets and controls that use the intent

Related to check: App Intents architecture, Siri and app schemas, Swift Testing, test doubles, testing singletons, test isolation, UI testing

---

Testing Language-Model Features

Q: How do you test a feature built on a language model, when the output is different every time?

Short answer: I split the feature into a deterministic part and a probabilistic part and test them differently. Everything around the model (prompt building, availability handling, mapping `@Generable` output into domain types, tool `call` implementations, fallbacks) sits behind protocols and gets normal Swift Testing unit tests with a fake model that returns fixed output.

Key points:

- **Deterministic layer:** fakes for your own `Summarizing`-style protocol; assert exact state transitions and fallbacks
- **Golden set:** dozens to hundreds of real, anonymized inputs with expected outputs or criteria, versioned in the repo (`ModelSample`, loaded with `ArrayLoader` or `JSONLoader`)
- **Evaluators:** `Evaluator { sample, subject in ... }` returns a `Metric` (`.passing()`, `.failing(rationale:)`, `.scoring(_:)`); `ModelJudgeEvaluator` grades against `ScoreDimensions` with `.passFail`, numeric or custom scales; `ToolCallEvaluator` checks the tool-call trajectory against a `TrajectoryExpectation`
- **Aggregates:** `aggregateMetrics(using:)` computes the mean, median, minimum and so on; `EvaluationResult.aggregateValue(_:)` reads them, and `summary` / `detailed` are `DataFrames` for inspection
- **Thresholds, not equality:** assert `mean >= threshold`, and track the trend over time

Related to check: Foundation Models integration, Swift Testing traits and tags, test plans, test doubles, flaky tests, golden-file testing, parameterized tests

---

System Design

Image Caching System

Q: Design an image caching system for an iOS app.

Short answer: I'd build an image pipeline as an `actor`: a memory cache of *decoded, downsampled* images (`NSCache` with a byte-cost limit), a disk cache of the *encoded* bytes, and an in-flight map so ten cells asking for the same URL share one download. Decoding happens off the actor in a `@concurrent` function using ImageIO's thumbnail API, which decodes straight to the display size instead of inflating a 12-megapixel photo into ~48 MB of memory.

Key points:

- **Requirements:** fast repeat display, no duplicate downloads, bounded memory and disk, no main-thread decoding, cancellation on reuse, prefetching, works offline for already-seen images
- **Components:** `ImagePipeline` actor (L1 `NSCache<NSString, UIImage>`, in-flight `[String: Task<UIImage, Error>]`), L2 disk (a dedicated `URLSession` with a large `URLCache`, or files named by a SHA-256 of the URL in `Caches/`), a `@concurrent` downsampler, and a prefetcher
- **Data flow:** request(url, pixelSize) -> L1 hit -> in-flight join -> L2 / network bytes -> downsample -> store in L1 with cost -> return
- **Cache keys:** URL **plus** target size (and scale); a 60-pt avatar and a full-screen viewer need different decoded images
- **Trade-offs:** `URLCache` as the disk tier is free and honors HTTP headers, but its eviction isn't configurable and it may skip large responses; a custom disk cache gives control (LRU, size cap, expiry) at the cost of more code. Decoded memory is expensive (width x height x 4 bytes), encoded disk data is cheap
- **Failure modes:** memory pressure (`NSCache` evicts automatically; also clear on memory warnings), an image that fails to decode (cache a placeholder briefly, not forever), a URL whose content changes (versioned URLs from the server, or respect `ETag`), a waiting caller cancelled (the shared task keeps running for the others)
- **Platform notes:** `traitCollection.displayScale` or `@Environment(\.displayScale)` to compute pixel size, never `UIScreen.main`. In iOS 27, `AsyncImage` gains HTTP caching and `.asyncImageURLSession(_:)` (iOS 27+), which is enough for simple screens, but with an iOS 18 target and feed-scale needs (downsampling, prefetching) a custom pipeline is still the answer
- **Privacy/offline:** caches hold user content, so wipe on logout and apply file protection; only cache what can be re-downloaded
- **Testing:** a stub `URLProtocol` per session configuration (not a global static) to count requests; assert that concurrent requests for one URL make one request and that the decoded size respects `maxPixelSize`

Related to check: NSCache, Image Loading and Caching, Async Image Decoding, Lazy Loading (SwiftUI / `AsyncImage`), Caching Layer Design, Feed Loading, Memory Warnings

---

Scalable Chat App

Q: Design a scalable chat app. How would you architect it?

Short answer: A chat client is local-first: messages and conversations live in a SwiftData store that the UI observes with `@Query`, and the network only reconciles that store. Real time comes from a single WebSocket owned by an `actor`, which reconnects with backoff and resumes from the last server sequence number it applied; push notifications cover the app when it isn't running.

Key points:

- **Requirements:** 1:1 and group chats, instant send, real-time receive, history paging, read receipts and typing indicators, offline reading and queued sending, multi-device, media, end-to-end encryption for private messaging
- **Components:** `ChatConnection` (actor: WebSocket, reconnect, resume), `MessageStore` (`@ModelActor` over SwiftData: upsert/dedupe, `#Unique` on client ID and on server ID), `Outbox` (pending sends with retry), `ConversationModel` (`@MainActor @Observable`, or `@Query` in views), `MediaUploader` (background `URLSession`), and the push handling (Notification Service Extension)
- **Data flow (receive):** socket event -> decode -> `MessageStore.upsert` (dedupe by ID, place by `seq`) -> `@Query` updates the open conversation -> mark delivered/read in batches
- **Data flow (send):** compose -> insert `.pending` with `clientID` -> send over the socket (or REST) -> ack `{clientID, serverID, seq}` -> update to `.sent` -> on failure retry with backoff, then `.failed` with a Retry button

- **Ordering and gaps:** each conversation has a monotonically increasing `seq`; if the client sees `seq` 41 then 43, it fetches 42 over REST; on reconnect it sends `resume afterSeq` to catch up; display order uses `seq`, with pending messages at the end
- **History:** cursor pagination backwards (`before: oldestSeq`), loaded as the user scrolls up while keeping the scroll anchor; only recent messages are synced eagerly
- **Scaling (client view):** one socket per app, not per conversation; batch read receipts; throttle typing indicators; lazy lists with stable IDs; the store indexes `(conversationID, seq)` (`#Index`); media as thumbnails plus on-demand full size
- **Failure modes:** flaky networks (backoff with jitter, `NWPathMonitor` to reconnect promptly), duplicate delivery (dedupe by IDs), messages arriving while the app is suspended (push plus catch-up on foreground), the outbox stuck on one bad message (park it as failed)
- **Privacy/security:** end-to-end encryption (Signal-protocol style: per-device identity keys, a double ratchet per session) means the server stores ciphertext only; keys live in the Keychain or Secure Enclave; the Notification Service Extension decrypts the push payload; CryptoKit provides the primitives, including post-quantum ML-KEM (`MLKEM768`, iOS 26+) for hybrid key agreement
- **Adaptive UI:** a `NavigationSplitView` (conversations | thread) on iPad, iPhone Duo and wide windows, collapsing to a stack on narrow widths; drafts saved per conversation so resizing or relaunching never loses text
- **Testing:** Swift Testing on the store (dedupe, ordering, gap detection) with an in-memory container; a fake connection that replays scripted event streams including reconnects; UI tests for send/fail/retry

Related to check: WebSockets, WebSocket Reconnection, Offline Mode Design, Pagination Design, Push Notification System, Background Uploads (media), Data Synchronization, CryptoKit

---

Feed Loading (Instagram-style)

Q: How would you build Instagram-style feed loading?

Short answer: An Instagram-style feed is a ranked, media-heavy list, so the hard parts are *perceived speed* and *media*, not just paging. On launch I render the last cached page from SwiftData immediately, fetch the fresh ranked page in the background, and show a "New posts" pill instead of jumping the list.

Key points:

- **Requirements:** first content in under a second (from cache), smooth 120 Hz scrolling (8.3 ms per frame), mixed media (images, carousels, video), pull-to-refresh, new-content indicator, likes that feel instant, offline browsing of what was loaded
- **Components:** `FeedModel` (`@MainActor @Observable`: posts, paging state, playing post), `Feed–Service` (`Sendable`, cursor API), `ImagePipeline` (actor), a video player pool (reuse 2 - 3 `AVPlayers`), a SwiftData cache of the first pages, and an optimistic mutation queue for likes and comments
- **Data flow:** `.task` -> load cached page -> render -> fetch page 1 -> diff by post ID -> if the user has scrolled, buffer new posts behind the pill -> rows appearing near the end trigger the next page (cursor) -> rows appearing trigger image prefetch for the next N posts
- **Media:** the server sends several renditions (thumbnail, 1080 px, HLS for video) plus aspect ratio and a blurhash/dominant color, so the cell reserves its exact height before the image arrives (no layout jumps)
- **Trade-offs:** server ranking with cursors (stable pages, no duplicates) vs client sorting; prefetch distance vs data and battery (shorter on cellular / Low Data Mode); caching full pages vs only IDs with a separate media cache
- **Failure modes:** a slow network (skeletons, keep cached content), duplicate posts across pages after re-ranking (dedupe by ID), a failed like (roll back the optimistic state and show a toast), memory pressure from many decoded images (downsample, cost-limited `NSCache`)
- **Adaptive layout:** in a wide window (iPad, iPhone Duo, a resized iPhone app) keep a readable single column or switch to a grid based on *available width*, never device type; restore scroll position by post

ID, not offset
- **Privacy/offline:** the cache holds other people's content, so it lives in `Caches/` and is wiped on logout; impressions analytics go through the analytics pipeline with consent
- **Testing:** the model with a fake `FeedService` (cache-then-network, dedupe, pill behavior) in Swift Testing; the SwiftUI and Time Profiler / hitches instruments for scroll performance

Related to check: Infinite Scroll Feed, Pagination Design, Image Caching System, Caching Layer Design, Lazy Loading (SwiftUI), Scroll Performance, Offline Mode Design

---

Offline Mode Design

Q: How would you design offline mode for an app?

Short answer: Offline mode works best as **offline-first**: the local database is the source of truth the UI reads from, and the network only feeds it. With SwiftData, views use `@Query` and render immediately; every user edit is written locally *and* appended to an outbox of pending changes in the same save, and a `@ModelActor` sync engine pushes that outbox in order, then pulls server changes since a sync token.

Key points:

- **Requirements:** read everything previously loaded, create/edit/delete offline, sync automatically when back online, no lost or duplicated writes, clear sync status, conflict handling
- **Components:** SwiftData `ModelContainer` (entities + `PendingChange` outbox), repositories that write entity + outbox atomically, `SyncEngine` (`@ModelActor`), `SyncAPI` (`Sendable`), a connectivity watcher, and a small `@MainActor @Observable` sync-status model for the UI
- **Data flow (write):** user edit -> save entity + `PendingChange` in one `modelContext.save()` -> UI updates instantly via `@Query` -> when online, the engine sends changes oldest-first with the change ID as `Idempotency–Key` -> on success delete the outbox row and store the server version
- **Data flow (read):** pull with `since: syncToken` -> upsert by server ID (`#Unique`) -> handle tombstones for deletions -> save the new token
- **Conflicts:** last-writer-wins by server timestamp (simple, loses data), field-level merge, server wins with the local copy kept as a draft, or a user prompt for documents; version numbers (`updatedAt` / revision) detect conflicts on the server
- **Trade-offs:** a custom outbox + REST gives full control; `ModelConfiguration(cloudKitDatabase:)` or `CKSyncEngine` give Apple-managed sync for iCloud-only data; Core Data + `NSPersistentCloudKitContainer` is still common. Optimistic UI feels instant but must handle rejections
- **Failure modes:** a change rejected permanently (mark failed, show "needs attention", don't block the queue forever), ordering dependencies (create-before-update; stop at the first transient failure), the app killed mid-sync (idempotency makes replays safe), clock skew (trust server timestamps)
- **Background:** `BGAppRefreshTask` for opportunistic pulls; silent push to nudge a sync; `URLSession` with `waitsForConnectivity` for single requests
- **Privacy:** the store holds user data, so use file protection, wipe on logout, and keep it out of shared App Group containers unless widgets need it
- **Testing:** Swift Testing with an in-memory `ModelConfiguration(isStoredInMemoryOnly: true)` and a fake `SyncAPI`: edits offline produce outbox rows, sync drains them in order, a failure stops the drain, a replay doesn't duplicate; Network Link Conditioner for manual tests

Related to check: Offline-First, Data Synchronization, SwiftData (`@ModelActor`, `#Unique`, history), Core Data, `CKSyncEngine`, Network Reachability, Idempotency, Background Tasks

---

Push Notification System

Q: Design a push notification system for an iOS app.

Short answer: The pipeline is: the app gets an APNs device token and registers it with our backend; the backend stores tokens per user and device and sends JSON payloads to APNs over HTTP/2 with token-based (`.p8`) authentication; iOS delivers them as visible alerts, background (silent) pushes, or Live Activity updates. On the client I use the `async` `UNUserNotificationCenter` APIs, ask for permission in context (or start with **provisional** authorization so notifications arrive quietly without a prompt), pick the right **interruption level**, and route taps through the same deep-link router as universal links.

Key points:

- **Requirements:** reliable delivery of alerts, user control over categories, rich content, deep links on tap, background data refresh, multi-device users, no PII in transit, Live Activity updates for ongoing events
- **Client components:** `PushCoordinator` (authorization, token upload, `UNUserNotificationCenterDelegate`), notification categories and actions (`UNNotificationCategory`), a Notification Service Extension (`mutable-content: 1`), optionally a Notification Content Extension for custom UI, and the app router
- **Server components:** a device registry (user ID, token, environment, app version, locale, preferences), a fan-out worker, an APNs client (HTTP/2, JWT), handling of APNs responses (410 Unregistered -> delete the token), rate limiting and quiet hours
- **Payload/headers:** `apns-push-type` (`alert`, `background`, `liveactivity`...), `apns-priority` (10 immediate, 5 power-friendly; background pushes must use 5), `apns-collapse-id` to replace older notifications, `apns-expiration`, `thread-id` for grouping, `interruption-level` and `relevance-score` in `aps`
- **Interruption levels:** `passive` (no sound or wake), `active` (default), `time-sensitive` (breaks through Focus; needs the Time Sensitive capability, only for genuinely urgent items), `critical` (needs a special Apple entitlement)
- **Silent pushes:** `content-available: 1` wakes the app for ~30 seconds in `application(_:didReceiveRemoteNotification:)`; delivery is throttled and not guaranteed, and doesn't happen after the user force-quits the app, so it's a hint to sync, never the only path
- **Live Activities (ActivityKit):** request an activity with `pushType: .token` and upload each token from `activity.pushTokenUpdates`; **push-to-start** tokens (`Activity<...>.pushToStartTokenUpdates`) let the server start an activity remotely; **broadcast** pushes (`pushType: .channel(id)`) update every subscriber of an event (e.g. a match) with one request instead of one per device
- **Trade-offs:** sending content in the payload (simple; visible to APNs unless encrypted) vs sending an ID and letting the service extension fetch or decrypt (private, but must finish within the extension's ~30-second budget); provisional (no prompt, quieter) vs an explicit prompt (prominent, one chance)
- **Failure modes:** stale tokens (re-register every launch, delete on 410), development vs production APNs environment mismatch, a service extension that times out (iOS shows the original payload, so send a sensible fallback text), notification spam leading users to disable everything
- **Privacy:** no PII or message bodies in clear payloads for sensitive apps (end-to-end encrypt and decrypt in the extension), let users choose categories, and honor the Settings deep link `openNotificationSettingsURLString`
- **Testing:** `xcrun simctl push` with a `.apns` JSON file on the simulator, Apple's Push Notifications Console for real sends, unit tests for payload -> route parsing in Swift Testing

Related to check: APNs, Notification Service Extensions, Live Activities and WidgetKit, Deep Linking, Background Tasks, Scalable Chat App, Offline Mode Design

---

Pagination Design

Q: How would you implement pagination in an iOS app?

Short answer: I'd use **cursor-based pagination** for anything that changes while the user reads it: the server returns a page plus an opaque `nextCursor`, so inserts and deletes don't cause duplicates or gaps

the way offset/limit does. On the client, a `@MainActor @Observable` paginator owns the items, the cursor and an explicit load state (idle, loading, failed, exhausted); `loadNextPage()` is `async`, ignores calls unless it's idle, deduplicates by ID, and discards responses that arrive after a refresh.

Key points:

- **API contract:** `GET /items?cursor=...&limit=20` -> `{ items, nextCursor }`, where `nextCursor == nil` means the end; the cursor encodes the sort key plus a tie-breaker (e.g. `createdAt` + `id`), so ordering is stable
- **Cursor vs offset:** offset is fine for static, sortable data with "jump to page 7" needs (search results, admin tables); cursors are better for feeds, chats and anything live. Keyset/cursor queries are also cheaper for the database at depth
- **Client state machine:** `idle -> loading -> idle | exhausted | failed`; `failed -> idle` only through an explicit retry; `refresh()` resets everything and bumps a generation counter so a late response from the old list is dropped
- **Data flow:** trigger (last rows appear / prefetch) -> `loadNextPage()` -> `api.items(after: cursor)` -> filter out IDs already seen -> append -> update cursor and state -> views re-render through Observation
- **Trade-offs:** page size (bigger = fewer requests but slower first paint; 20 - 50 typical); prefetch distance; storing pages in memory only vs persisting the first page(s) in SwiftData for instant launch and offline
- **Failure modes:** duplicate triggers (state guard), items shifting between pages (ID dedupe), errors (a `.failed` state with Retry instead of a stuck spinner), cancellation (a cancelled load returns to `.idle` so it can be retried), an expired cursor (restart from the first page)
- **Offline:** cache the first page(s) and the cursor; when offline show cached items plus an inline "can't load more" footer
- **Testing:** Swift Testing with a fake `ItemsAPI` (an `actor` that returns scripted pages): concurrent `loadNextPage()` calls produce one request, a duplicate item is filtered, an error sets `.failed` and `retry()` recovers, `refresh()` during a load ignores the stale result

Related to check: Infinite Scroll Feed, Feed Loading, Networking Pagination, Diffable Data Source, Lazy Loading (SwiftUI), Offline Mode Design

---

Modular App Architecture

Q: How would you architect a modular iOS app?

Short answer: A modular app is a set of Swift packages where each feature exposes a **small public surface** (an entry-point view or factory and its route types) and hides everything else, and features depend on each other only through lightweight **interface modules** of protocols. The app target is the composition root: it creates the live services, builds each feature with its dependencies, and injects them through `init` or the SwiftUI environment.

Key points:

- **Requirements:** features buildable and testable alone, no feature-to-feature coupling, cross-feature navigation, shared design system and core services, Swift 6 concurrency safety across module boundaries
- **Module types:** *Core* (networking, persistence, logging: `Sendable` services), *DesignSystem* (components, Liquid Glass-aware styles), *Interface* modules (protocols, route enums, DTOs; no implementation), *Feature* modules (views, `@MainActor @Observable` models, internal services), *App* (composition)
- **Data flow / navigation:** a feature asks for another feature's screen through its interface (e.g. `ProfileRouting.profileView(userID:)`) or emits a route value (`AppRoute.profile(id)`) that the

app's `@Observable` router turns into a `NavigationPath` push. Deep links parse into the same route values
- **Dependency injection:** protocols for services (`any ProfileClient`), passed through `init`; environment values (`@Entry`) for things many views need; no global singletons (Swift 6 rejects unprotected global mutable state)
- **API design:** `public` only for the entry points; `package` access for cross-target helpers inside one package; `@MainActor` on UI-facing protocols; `Sendable` on service protocols so they can be shared across actors
- **Trade-offs:** returning `AnyView` at the module boundary (simple, slight diffing cost) vs generic/associated-type APIs (faster, but leak types); interface modules add targets but break compile-time coupling; static linking (SPM default, faster launch) vs dynamic frameworks (shared between app and extensions, slower launch unless merged with mergeable libraries)
- **Failure modes:** an interface module that slowly fills with implementation, two features both "owning" a model, environment values that silently fall back to defaults (make missing dependencies fail loudly in debug builds)
- **Testing:** each feature's tests inject fakes for its interfaces; previews use `#Preview` with fake dependencies; the app has a few integration tests of the real wiring

Related to check: Large Codebase Structure, Modular Architecture, Dependency Injection, Navigation Architecture, Deep Linking, Clean Architecture, Access Control (`package`)

Analytics Implementation

Q: How would you implement analytics in an iOS app?

Short answer: I'd put a small, typed `track` API in front of an `actor` that buffers events, persists them, and uploads them in batches, so the UI never waits on analytics and nothing is lost to a crash or a dead network. Vendors (Firebase, Amplitude, our own endpoint) are pluggable `Sendable` transports behind one protocol.

Key points:

- **Requirements:** a typed event schema (name, typed properties, timestamp, UUID), non-blocking calls from any actor, ordered and durable delivery, batching to save battery, consent and opt-out, several destinations
- **Components:** an `Analytics` actor (buffer, consent, flush), a disk-backed queue (append-only file or a small SwiftData store), `AnalyticsTransport` implementations per vendor, and screen/action helpers in the UI layer
- **Data flow:** `track` -> `AsyncStream` (keeps call order) -> actor buffer + disk -> batch of ~50, or on a timer, or when the scene enters the background -> transport -> delete only after a 2xx
- **Trade-offs:** batching is cheaper on the radio but adds latency; client-side vendor SDKs are easy but each adds binary size, launch cost and its own privacy manifest, so many teams send one stream to their backend and fan out server-side
- **Failure modes:** offline (keep the queue, cap its size, drop oldest), crash (the queue is on disk), duplicate sends after a timeout (the event UUID lets the server deduplicate), clock skew (send device time *and* upload time)
- **Privacy:** declare collected data types in `PrivacyInfo.xcprivacy` (`NSPrivacyCollectedDataTypes`, `NSPrivacyTracking`, `NSPrivacyTrackingDomains`). Ask `ATTrackingManager.requestTrackingAuthorizati` only if you really link data with other companies' data for tracking; when the user hasn't allowed tracking, iOS blocks connections to declared tracking domains. First-party product analytics without tracking doesn't need ATT, but still needs disclosure and an opt-out
- **Testing:** inject a fake transport (an `actor` that records batches) and assert on the event names and properties a feature emits

Related to check: Analytics Architecture, Analytics Events, Logging (`os.Logger`), Privacy Manifests, App Tracking Transparency, Offline Mode Design, Remote Configuration (sampling switches)

Infinite Scroll Feed

Q: How would you build an infinite scroll feed?

Short answer: Infinite scroll is a paginated model plus a trigger: when the user gets within *N* items of the end, ask the model for the next page; the model makes sure only one request is in flight, deduplicates items, and tracks idle / loading / failed / exhausted states. In UIKit the trigger is `UICollectionViewDataSourcePrefetching`, which also gives me image prefetch and cancel hooks, and the UI is a diffable data source keyed by item ID.

Key points:

- **Requirements:** seamless loading before the user hits the end, no duplicates or gaps, retry on error, pull-to-refresh, position kept across rotation/resize and relaunch, stable memory after thousands of rows
- **Components:** `PaginatedList` (`@MainActor @Observable`; see Pagination Design for the paging logic), the view controller or view, a diffable data source / `ForEach` keyed by stable IDs, an image prefetcher, and a footer cell for loading/error/end
- **Data flow:** prefetch callback or row `.task` -> `model.loadNextPage()` (no-op unless idle) -> API with cursor -> model appends deduped items -> UIKit: `updateProperties()` re-runs and applies a snapshot (iOS 26+) / SwiftUI: `body` re-renders the rows that changed
- **Trigger distance:** based on items (10 - 20 ahead) or points (one to two screen heights), larger on fast scrolls; prefetching gives direction and velocity for free
- **Trade-offs:** infinite scroll vs a "Load more" button (scroll is engaging; a button gives users a reachable footer and fewer accidental loads); keeping everything in memory vs windowing (drop pages far above, keep IDs, reload on scroll back)
- **Failure modes:** duplicate trigger calls (model guard), duplicate IDs (filter, or `apply` crashes in UIKit / identity breaks in SwiftUI), errors (footer with Retry; no automatic retry loop), the end of the feed (stop triggering, show "You're all caught up")
- **Adaptive layout:** on iPad, iPhone Duo and resizable windows the number of visible rows changes, so compute trigger distance from the visible range, not a fixed index; in a compositional layout pick columns from the container's effective width
- **Testing:** Swift Testing on the model (no duplicate requests, dedupe, retry, exhausted state); UI test that scrolls to the bottom with a stubbed server

Related to check: Pagination Design, Feed Loading, Diffable Data Source, Lazy Loading (SwiftUI), Image Caching System, App State Restoration

---

Caching Layer Design

Q: Design a caching layer for an iOS app.

Short answer: I'd build a two-level, cache-aside layer for API responses: an in-memory L1 and an on-disk L2 in `Caches/`, both behind one `actor` so concurrent access is safe and concurrent misses for the same key share **one** network request (single-flight). Every entry carries a timestamp, so the caller can choose a freshness policy: fresh-only, or stale-while-revalidate, which shows cached data now and refreshes in the background.

Key points:

- **Requirements:** fast repeat reads, fewer requests, data on cold start or offline, bounded memory and disk, correct invalidation, per-user isolation
- **Components:** a `ResponseCache` actor (L1 dictionary or `NSCache`, in-flight `Task` map, TTL), a `DiskStore` (one file per hashed key, or SQLite), the repository that decides policy per endpoint, and `URLCache` for responses whose HTTP caching headers you trust
- **Data flow:** repository -> `cache.value(for: key) { try await api.fetch() }` -> L1 hit? -> L2 hit and fresh? -> join an in-flight task? -> otherwise load, store in L1 and L2, return
- **Strategies:** cache-aside (the default on clients), write-through (update the cache when a mutation succeeds), stale-while-revalidate (best perceived speed), HTTP caching (`ETag` / `Cache-Control` with `URLCache`) when the backend sets headers correctly
- **Trade-offs:** an actor serializes access (simple, async) whereas a `Mutex` around a dictionary gives synchronous reads; `NSCache` evicts automatically under memory pressure but isn't LRU and doesn't enumerate; a custom LRU is predictable but you handle memory warnings yourself
- **Failure modes:** cache stampede (single-flight solves it), unbounded disk growth (size cap plus LRU sweep at launch or in a `BGProcessingTask`), corrupt or old-format entries (treat a decode failure as a miss), serving another user's data after logout (namespace keys by user and wipe on sign-out)
- **Privacy/offline:** `Caches/` can be purged by the system and isn't backed up, so only cache re-fetchable data; apply file protection; never cache tokens; offline mode reads stale entries deliberately and labels them as such
- **Testing:** inject a clock and a fake loader; assert that two concurrent `value(for:)` calls trigger one load, that TTL expiry causes a reload, and that invalidation removes entries

Related to check: NSCache, Caching Strategy, Response Caching / `URLCache`, Request Deduplication, Image Caching System, Offline Mode Design, Actors and reentrancy

---

App State Restoration

Q: How would you handle app state restoration?

Short answer: State restoration makes a relaunch look like the user never left: the same tab, navigation stack, scroll position and half-typed draft, restored **per scene**, because on iPad, Mac and resizable iPhone windows each window has its own state. In SwiftUI I keep navigation in an `@Observable` router whose `NavigationPath` is `Codable`, and save it plus small UI values with `@SceneStorage`.

Key points:

- **Requirements:** restore tab, navigation stack, selection, scroll position and drafts after the system terminates the app in the background; per window; fast; never restore secrets
- **Components:** a `Router` (`@MainActor @Observable`, owns `NavigationPath`), `@SceneStorage` for per-scene values, the persistence layer (SwiftData/files) for real data such as drafts, and the deep-link handler, which shares the same "route" vocabulary
- **Data flow:** a state change -> encode `path.codable` -> `@SceneStorage` -> the system saves it with the scene -> on relaunch, decode -> set `router.path` -> destination views load by ID
- **UIKit:** `stateRestorationActivity(for:)` returns an `NSUserActivity` whose `userInfo` holds IDs; `scene(_:willConnectTo:options:)` reads `session.stateRestorationActivity`; `scene(_:restoreInteractionStateWith:)` is called after connection. The older view-controller API (`restorationIdentifier`, `encodeRestorableState(with:)`) still exists but isn't the scene-based path
- **Trade-offs:** `@SceneStorage` is small, per scene, and discarded when the user deletes the scene (closes the window in the switcher); durable data like a long draft belongs in SwiftData. `NavigationPath.codable` is `nil` if any element isn't `Codable`, so route values must be `Codable` IDs, not model objects
- **Failure modes:** the item was deleted or the user signed out (fall back to the root), an app update changed the route schema (version it or ignore what fails to decode), restoring into a screen behind

a lock (re-authenticate first)
- **Privacy:** never put tokens, message bodies or health data in `@SceneStorage` or `NSUserActivity`; scene snapshots are written to disk, so hide sensitive screens when the scene goes inactive
- **Testing:** in Xcode, background the app, then stop it from Xcode (the system-termination path) and relaunch; swiping it away in the app switcher deliberately discards the state. Unit-test the router's encode -> decode round trip with Swift Testing

Related to check: NavigationStack and Navigation, Navigation Architecture, Deep Linking, Scenes and the UIScene life cycle, `@SceneStorage` vs `@AppStorage`, Infinite Scroll Feed (scroll position)

Background Uploads

Q: How would you handle background uploads?

Short answer: Uploads that must survive the app being suspended or terminated by the system go through a **background `URLSession`**: the transfer runs in a system daemon, and iOS relaunches the app in the background to deliver the results. Every upload is a persisted record (state, file path, server ID) written to disk *before* the transfer starts, so the app can rebuild progress and retry after any relaunch.

Key points:

- **Requirements:** large files (photos, video), survive suspension and system termination, show progress, retry, no duplicates on the server, respect cellular data and Low Power Mode
- **Components:** an upload store (SwiftData or a file), an `Uploader` that owns the one background session, a `Sendable` session delegate, the app delegate's `handleEventsForBackgroundURLSession` hook, and an `@Observable` model that shows progress
- **Data flow:** the user picks media -> compress/export to a file in the app container -> insert an `UploadRecord(.pending)` -> `uploadTask(with:fromFile:)` with `taskDescription = record.id` -> the daemon transfers -> delegate `didCompleteWithError` -> update the record -> `urlSessionDidFinishEvents` -> call the saved system completion handler
- **Background session rules:** upload tasks **from a file** and download tasks only (no data tasks, no `upload(for:from: Data)`); use delegates, not async convenience methods or completion handlers; one session per identifier, recreated at every launch with the same identifier so pending events can be delivered; `sessionSendsLaunchEvents = true`
- **Trade-offs:** `isDiscretionary = true` lets iOS wait for Wi-Fi and power (good for backups, bad for "send this now"). Big files: chunked or resumable uploads (`URLSessionUploadTask` resume data, `cancel(byProducingResumeData:)`, `uploadTask(withResumeData:)`), if the server supports the resumable-upload protocol, versus one request per file
- **Failure modes:** the user force-quits from the app switcher (iOS cancels the background transfers; resume on next launch from the store), the server rejects the file (mark failed and surface a retry), duplicate completion after a relaunch (an `Idempotency-Key` header and server-side dedup), the file was deleted from a temporary directory (copy it into Application Support first)
- **Privacy/offline:** background sessions wait for connectivity on their own; respect `allowsExpensiveNetworkAccess` / `allowsConstrainedNetworkAccess` for cellular and Low Data Mode; use file protection that still allows reading while locked (`.completeUntilFirstUserAuthentication`)
- **Testing:** a stubbed transport for the record state machine (Swift Testing); on device, start an upload, background the app, and stop it from Xcode to exercise the relaunch path

Related to check: URLSession, Background Downloads, Download Resume, Multipart Upload, Upload Progress, Idempotency, Background Tasks (`BGTaskScheduler`), Offline Mode Design

---

Feature Flags Design

Q: How would you design a feature flag system?

Short answer: Feature flags are typed, short-lived on/off switches for code paths: progressive rollouts, kill switches, and A/B experiments. On the client I'd keep a `@MainActor @Observable` flag store that resolves each flag as *local override -> last fetched value -> compiled-in default*, loads the cached values synchronously at launch, and refreshes in the background, so no screen ever waits on the network.

Key points:

- **Requirements:** instant reads, safe defaults offline, remote kill switch, gradual rollout by percentage/segment/app version, overrides for QA and UI tests, exposure logging for experiments
- **Components:** a `Flag` enum (key, default, owner), the `FeatureFlags` store (`@MainActor @Observable`), a `FlagClient` (`Sendable`: your API, Firebase Remote Config or LaunchDarkly behind a protocol), a disk cache, and a debug menu
- **Data flow:** launch -> load cache -> UI reads `flags.isEnabled(.newCheckout)` -> background `refresh()` -> store updates -> SwiftUI views that read the flag re-render (per-property Observation tracking)
- **Consistency:** decide when a change applies. Kill switches apply immediately; UI-shaping flags are usually "sticky" for the session (snapshot at launch, apply new values next launch) so a screen doesn't change under the user's finger
- **Trade-offs:** server-side evaluation keeps targeting rules and user attributes off the device and allows instant changes, but needs a request; client-side evaluation works offline but ships rules to the client. Vendor SDKs vs in-house: speed of setup vs binary size, privacy manifest and cost
- **Failure modes:** fetch fails (keep last known good), a malformed payload (validate, ignore unknown keys), a flag removed from the server while old app versions still read it (the default must be safe), flag combinations nobody tested
- **Privacy:** send only the attributes targeting needs (an opaque user ID, app version, locale); list the vendor SDK's data collection in the privacy manifest
- **Testing:** inject overrides; parameterized Swift Testing over both states (`@Test(arguments: [true, false])`); UI tests pass overrides through launch arguments

Related to check: Feature Flags, Feature Toggles, Remote Configuration, A/B Testing Architecture, Analytics Implementation, Dependency Injection

---

Remote Configuration

Q: How would you design remote configuration?

Short answer: Remote configuration is a typed, versioned document of tunable values (timeouts, limits, copy, URLs for non-critical content, minimum supported app version) that the server can change without an App Store release. I'd model it as a `Codable` struct with a bundled default, keep the *last known good* copy on disk, refresh it in the background with conditional requests (`ETag`), and **validate before applying**, so a bad push can't break every client at once.

Key points:

- **Requirements:** never block launch, work offline, typed values with safe defaults, segment by app version / platform / locale, fast rollback, no secrets, auditability
- **Components:** `AppConfig` (`Codable`, `Sendable`, schema version), `RemoteConfigStore` (actor: current value, ETag, persistence), the config endpoint or vendor (Firebase Remote Config etc. behind a protocol), and consumers that read a snapshot
- **Data flow:** launch -> load from disk (or bundled defaults) synchronously -> app starts -> background `refresh()` (`If-None-Match`) -> 304: done / 200: decode -> validate -> persist atomically -> publish the new value

- **Precedence:** bundled defaults < last known good from disk < freshly fetched (validated) < debug overrides
- **When to apply:** values read per request (timeouts, limits) can change live; values that shape UI or app structure apply on next launch or at a safe point; a "minimum version" value triggers a soft or hard update prompt
- **Trade-offs:** a single document (atomic, easy to version) vs per-key fetches; vendor SDK (targeting UI, experiments) vs in-house (control, privacy, no SDK weight); polling on foreground vs silent push to invalidate (push is a hint, not guaranteed)
- **Failure modes:** malformed or out-of-range values (validation rejects the whole document and keeps the last known good), a new schema an old app doesn't understand (`schemaVersion` check; ignore unknown keys), a fetch storm after a push to all users (jitter), the config server down (cached copy plus defaults)
- **Security/privacy:** never put API keys or secrets in config (it's readable by anyone with the app); sign the payload or use pinned TLS if config can change behavior that matters for security; send only coarse targeting attributes
- **Testing:** Swift Testing with a stub `URLProtocol` per session: 304 keeps the current value, invalid values are rejected, decode failures fall back, persistence round-trips; parameterized tests over representative config documents

Related to check: Feature Flags Design, Config Management, A/B Testing Architecture, Caching Layer Design, Secrets Management, Backward Compatibility

---

Large Codebase Structure

Q: How would you structure a large iOS codebase?

Short answer: For a large app with several teams I organize **by feature first, layered inside**, and make the boundaries real with Swift Package Manager targets instead of folders, so the compiler enforces the dependency direction *features -> shared interfaces -> core*. A thin app target is the composition root that wires implementations together; everything else lives in local packages that build and test on their own.

Key points:

- **Requirements:** many engineers working in parallel without merge pain, fast incremental builds and tests, clear ownership, safe refactors, gradual adoption of new tech (Swift 6, SwiftData, new UI)
- **Layout:** `App/` (composition root, scenes, entitlements), `Packages/Modules` with **Core** (models, networking, persistence, logging), **DesignSystem**, **Interfaces** (protocols and route types), **Features** (Feed, Profile, Checkout…), and **TestSupport** (fakes)
- **Dependency rules:** features never import other features; they talk through interface targets or the router. Core never imports UI. No cycles (SPM rejects them)
- **Access control:** `public` only for the module's API; `package` for things shared between targets of the same package but hidden from the app; `internal` for everything else
- **Build and tooling:** SPM targets build in parallel; Xcode's explicitly built modules make module dependencies explicit and cacheable; package traits (Swift 6.1+) for optional features; mergeable libraries if you ship many dynamic frameworks; SwiftLint/SwiftFormat and a dependency-rule check in CI
- **Concurrency migration:** Swift 6 language mode is per target. New packages start in Swift 6; legacy targets stay in Swift 5 mode with complete checking until migrated. App targets created in Xcode 26+ default to `MainActor` isolation; packages don't, so be explicit
- **Trade-offs:** feature-first (team ownership, deletable features) vs layer-first (simple at small scale, but every change touches every layer); a few large modules (simpler graph) vs many small ones (better isolation and caching, more boilerplate); monorepo (atomic changes) vs a repo per module (versioning overhead)
- **Failure modes:** a "Common" or "Utils" grab-bag module that everything depends on (rebuild storms), circular dependencies worked around with singletons, a god `AppDelegate`/`AppState`

- **Testing:** each feature package has its own Swift Testing target that runs without the app; snapshot and UI tests live at the app level; CI runs only the affected modules' tests

Related to check: Modular App Architecture, Modular Architecture, Clean Architecture, Code Organization, Dependency Injection, Dependency Graph, Access Control (`package`)

News Reader App

Q: Design a news reader app: a home feed of sections, article pages, offline reading and breaking-news alerts.

Short answer: A news app is read-heavy and freshness-sensitive, so I design for an instant first screen and cheap refreshes. The home feed is cached in SwiftData and rendered immediately, then refreshed with conditional requests (`ETag`) so an unchanged feed costs a 304; `BGAppRefreshTask` warms it before the user opens the app.

Key points:

- **Requirements:** fast cold start, sections and a personalized "For You" feed, readable articles with Dynamic Type, offline reading of saved and recently read articles, breaking-news alerts, widgets, paywall/metering for subscriptions, low data use
- **Components:** `FeedRepository` (SwiftData cache + API, stale-while-revalidate), `ArticleRenderer` (blocks -> SwiftUI views), `OfflineLibrary` (`@ModelActor`), the shared image pipeline, `PushCoordinator` for alerts, a widget extension reading an App Group snapshot, and an `@Observable` router for deep links
- **Data flow (feed):** launch -> `@Query` shows cached sections -> `GET /feed` with `If-None-Match` -> 304 (nothing to do) or 200 (upsert by article ID, keep scroll position, show a "New stories" pill instead of jumping)
- **Data flow (article):** tap -> render cached blocks if present -> fetch the latest revision -> re-render only if the revision changed (stories get updated after publishing)
- **Why blocks, not HTML:** native text scales with Dynamic Type and VoiceOver, images go through the shared pipeline at the right size, offline storage is just data, and the same blocks feed widgets and summaries. HTML in a web view is quicker to ship but harder to style, slower to show and awkward offline
- **Trade-offs:** server-ranked personalization (consistent across devices, needs reading history on the server) vs on-device ranking (private, limited signals); aggressive prefetching (instant reads, data and battery cost) vs on demand; Low Data Mode (`URLSessionConfiguration.allowsConstrainedNetworkAccess = false` for prefetches)
- **Freshness and scale:** CDN-cached feed responses with short TTLs and `ETags`; versioned article revisions; push for breaking news instead of polling; `BGAppRefreshTask` is opportunistic, so never rely on it for correctness
- **Failure modes:** offline at launch (show the cache with an "Updated 2h ago" label), a partially downloaded saved article (save atomically: images first, then the record), a retracted story (the server's tombstone removes it from caches and saved lists), a push storm on a big event (collapse IDs and `relevance-score`)
- **Privacy:** reading history is sensitive; keep it on device unless the user opts into personalization, declare it in the privacy manifest, and avoid third-party trackers in article views
- **Adaptive UI:** `NavigationSplitView` (sections | stories | article) on iPad, iPhone Duo and wide windows; reading width capped for line length, never derived from the device; article text uses `.font(.body)` styles so it scales
- **Testing:** Swift Testing for block decoding (golden JSON fixtures, unknown block types are skipped, not fatal), the 304 path with a stub `URLProtocol` per session, and the offline library's all-or-nothing save; snapshot tests of the renderer at several Dynamic Type sizes

Related to check: Feed Loading, Infinite Scroll Feed, Caching Layer Design, Image Caching System,

Push Notification System, Background Tasks (`BGAppRefreshTask`), WebView, WidgetKit Integration, Live Activities Delivery Tracking, On-Device AI Feature

---

Photo Upload Pipeline

Q: Design the photo upload pipeline for a social app. The user picks up to 20 photos, writes a caption and taps Post.

Short answer: I'd split posting into two phases: **upload the photos as blobs**, then **create the post** that references their server IDs. Tapping Post saves a `PostDraft` with one record per photo in SwiftData and returns the user to the feed with an optimistic "Posting…" cell.

Key points:

- **Requirements:** pick many photos without a permission prompt, instant return to the feed, progress, survives backgrounding and relaunch, bounded memory, no duplicate posts, no location leaked unless the user opts in, works on cellular and Low Data Mode
- **Components:** `PhotosPicker` (out-of-process, so no photo-library permission), a `PostDraft` + `PhotoUpload` SwiftData model (state, local file, server ID), a `@concurrent PhotoPreparer`, the background `Uploader` (see Background Uploads), a `PostPublisher` that creates the post, and a `@MainActor @Observable` model for the "Posting…" cell
- **Data flow:** pick -> load each `PhotosPickerItem` with `loadTransferable` -> write the draft (`.picked`) -> prepare to a file in Application Support (`.prepared`) -> request presigned upload URLs -> enqueue background uploads with the photo ID as `Idempotency-Key` (`.uploading`) -> completions store server IDs (`.uploaded`) -> when all are uploaded, `POST /posts` with the draft ID as idempotency key -> delete local files
- **State machine per photo:** `picked -> prepared -> uploading -> uploaded`, with `failed(retryable:)`; the post has `draft -> publishing -> published | needsAttention`. The UI is derived from these records, never from live task objects
- **Trade-offs:** client-side resizing saves bandwidth and server CPU but loses originals (offer "upload full quality" on Wi-Fi only); presigned URLs to object storage keep app servers out of the data path, at the cost of a second call to finalize; one request per photo parallelizes and retries individually, while a single multipart request is simpler but all-or-nothing
- **Scaling and limits:** decode at most 2 - 3 photos at once (a 48-MP HEIC decodes to ~190 MB at full size, which is why preparation downsamples with ImageIO instead of `UIImage(data:)`); the background session schedules transfers itself; server-side, generate thumbnails and variants asynchronously after upload
- **Failure modes:** the app is killed mid-preparation (resume from the last persisted state at launch), a temporary file disappears (prepare into Application Support, not `tmp/`), a photo is rejected (mark only that photo failed and let the user remove it), the post call times out after the server created it (the idempotency key returns the existing post), the user force-quits (iOS cancels background transfers; re-enqueue at next launch)
- **Privacy:** `PhotosPicker` needs no library permission and returns only what the user chose; re-encoding drops EXIF GPS and device data by default; show a "Include location" toggle if the product wants it; the privacy manifest declares photos as collected user content
- **Adaptive UI:** the composer is a sheet that works at any width (iPad, iPhone Duo, resizable windows); keep the draft in SwiftData so resizing or relaunching never loses the caption
- **Testing:** Swift Testing for the state machine with a fake uploader (every state transition, replay after "relaunch", one photo failing); a fixture image with GPS EXIF to assert the prepared file has none; on device, background the app mid-upload and stop it from Xcode

Related to check: Background Uploads, Multipart Upload, Upload Progress, Image Downsampling, Image Caching System, Idempotency, Offline Mode Design, Stories Camera Feature, Background Tasks (`BGContinuedProcessingTask`)

Search Typeahead

Q: Design search-as-you-type suggestions for a shopping app's search bar.

Short answer: Typeahead is a latency and request-volume problem. I debounce keystrokes with **task cancellation**: a `.task(id: query)` that sleeps about 250 ms, so each keystroke cancels the previous task and only a pause in typing sends a request.

Key points:

- **Requirements:** suggestions within ~100 - 200 ms of a pause, no flicker, correct results when responses arrive out of order, low request volume, works with poor connectivity, recent searches, keyboard and VoiceOver friendly
- **Components:** `.searchable` + `.searchSuggestions` (or `UISearchController`), a `@MainActor @Observable` search model or view state, the `SuggestionEngine` actor (normalization, prefix cache, single-flight), a local index (recent searches in SwiftData, popular terms bundled or synced), and a `SuggestAPI` behind a `Sendable` protocol
- **Data flow:** keystroke -> `query` changes -> previous task cancelled -> sleep 250 ms -> normalize (trim, lowercase, fold diacritics) -> cache -> parent-prefix filter -> network -> check cancellation -> publish results
- **Debounce vs throttle:** debounce sends after the user pauses (fewer requests); a throttle sends at most every N ms while typing (fresher results on slow typists). Many apps combine both, or skip the delay entirely for the first two characters served from local data
- **Ordering:** cancellation handles most stale responses; the check after the `await` handles a response that completes just as a new keystroke arrives. A sequence number per request is the equivalent in UIKit or Combine code
- **Server contract:** `GET /suggest?q=swi&limit=10` returns items plus `complete: true` when fewer than `limit` matches exist, which is what makes local refinement safe; responses are CDN-cacheable for popular prefixes; rate-limit per device and honor `429` + `Retry-After`
- **Trade-offs:** bigger local index (instant, offline, larger download) vs server-only (always fresh, every prefix costs a round trip); caching by normalized prefix (high hit rate) vs exact text (simpler); showing local results first (fast) vs waiting to merge (no reordering flicker; merge by stable ID and keep positions of items already shown)
- **Failure modes:** network errors (keep the current list, don't flash an error per keystroke), slow responses (show local results and a subtle spinner), a burst of 429s (back off and fall back to local), memory growth (bound the cache with LRU)
- **Privacy:** queries are personal data; log them only with consent, never with PII in URLs beyond the query itself, let users clear recent searches, and declare search history in the privacy manifest if it leaves the device
- **Testing:** Swift Testing for normalization and parent-prefix refinement (parameterized), a fake `SuggestAPI` that counts calls and returns responses in a controlled order, and a test that a slow first response never replaces a faster second one

Related to check: Request Cancellation, API Rate Limiting, Request Deduplication, Caching Layer Design, Task Modifier (`.task(id:)`), AsyncSequence, AttributedString, Accessibility

---

Video Streaming App

Q: Design a video streaming app like a subscription TV service: browsing, playback, offline downloads and Picture in Picture.

Short answer: Playback is **HLS through `AVPlayer`**: the server encodes each title into a ladder of bitrates and resolutions described by a multivariant playlist, and AVFoundation's adaptive bitrate logic switches

variants as bandwidth changes, so the app doesn't pick bitrates itself. I'd use `AVPlayerViewController` (or `VideoPlayer` in simple SwiftUI screens) to get system controls, AirPlay, captions and Picture in Picture for free, and an `@Observable` playback model for resume position, analytics and Up Next.

Key points:

- **Requirements:** fast start (under ~2 s), smooth adaptive playback, resume where you left off on any device, captions and audio tracks, PiP and AirPlay, background audio for audio-first content, offline downloads with expiry, DRM, parental controls
- **Components:** catalog/browse (feed + image pipeline), `PlaybackModel` (`@MainActor @Observable`, owns the `AVPlayer`), `AVPlayerViewController` wrapper, `ProgressReporter` (resume points to the server), `OfflineVideos` (asset download session + SwiftData records), `ContentKeyManager` (`AVContentKeySession`), and Now Playing (`MPNowPlayingInfoCenter`, remote commands)
- **Data flow (play):** tap -> fetch a signed playlist URL and resume position -> `AVPlayerItem` with `preferredForwardBufferDuration` left to the system -> seek to resume point -> play -> report progress every ~10 s and on pause/background -> mark watched near the end
- **Data flow (download):** choose quality -> `AVAssetDownloadConfiguration` with a variant qualifier (e.g. <= 1080p) -> background download -> persist the location from `willDownloadTo` -> request persistable FairPlay keys -> play offline from an `AVURLAsset` on the local URL
- **Adaptive bitrate:** let AVFoundation choose; constrain with `preferredPeakBitRate` / `preferredMaximumResolution`, and the `...ForExpensiveNetworks` variants (cellular) for data saver settings; startup quality comes from the server's variant ordering
- **PiP:** `AVPlayerViewController` supports it automatically with the audio session category `.playback` and the "Audio, AirPlay, and Picture in Picture" background mode; `canStartPictureInPictureAutomaticallyFromInline` for inline players; restore the UI in the PiP restore delegate callback
- **Trade-offs:** system player UI (accessible, consistent, less branding) vs custom controls over `AVPlayerLayer` (branding, more work, must rebuild accessibility and PiP); download quality vs storage; DRM (studio requirement) vs complexity
- **Failure modes:** stalls (observe `timeControlStatus` and `isPlaybackLikelyToKeepUp` for UI, not for bitrate decisions), expired signed URLs (refresh and replace the item at the current time), offline keys expiring (re-acquire when online, block playback with a clear message), storage full (check before downloading; let the user manage downloads)
- **Scale:** CDN delivery with signed, short-lived URLs; segment caching at the edge; concurrent-stream limits enforced by the license server
- **Privacy:** watch history is personal data; sync it for resume but let users clear it; declare it in the privacy manifest; parental profiles stored server-side
- **Adaptive UI:** the player fills its container, not the screen; on iPhone Duo and resizable windows, use the available size and don't lock orientation globally; a `NavigationSplitView` catalog on wide layouts
- **Testing:** Swift Testing for the playback model state machine with a fake player protocol; the download record lifecycle with a fake store; manual tests with Network Link Conditioner and HLS test streams

Related to check: Feed Loading (autoplay in feeds), Podcast Player, Background Downloads, Download Manager, Image Caching System, Background Tasks, Adaptive Layout (iPhone Duo), Offline Mode Design

---

Podcast Player

Q: Design a podcast app: streaming and downloads, background playback, lock-screen controls, CarPlay, and synced listening progress.

Short answer: The core is a single **playback engine** that owns one `AVPlayer` (or `AVQueuePlayer`), configured with an `AVAudioSession` in the `.playback` category and the `audio` background mode, so audio continues when the app is backgrounded. The engine is the source of truth for "what's playing", and three surfaces read from it: the app UI, the system's Now Playing card and remote commands (lock screen,

Control Center, AirPods, CarPlay), and CarPlay's own templates.

Key points:

- **Requirements:** stream or play downloaded episodes, continue in the background, lock-screen/AirPods/CarPlay controls, variable speed, sleep timer, auto-download new episodes, resume position on every device, offline listening
- **Components:** `PlaybackEngine` (`@MainActor @Observable`, owns `AVPlayer`, queue, rate, sleep timer), `NowPlayingBridge` (MediaPlayer or NowPlaying), `DownloadManager` (background `URLSession`, an actor tracking tasks), `Library` (SwiftData: podcasts, episodes, download state, progress), `FeedRefresher` (`BGAppRefreshTask` + push), and a CarPlay scene delegate
- **Data flow (play):** user taps episode -> engine picks a local file if downloaded, else the remote URL -> `AVPlayerItem` -> periodic time observer saves progress every ~5 s and on pause -> Now Playing is updated with elapsed time and rate, not every tick
- **Data flow (download):** auto-download policy (Wi-Fi only, keep the latest N) -> background download task -> `urlSession(_:downloadTask:didFinishDownloadingTo:)` moves the file into Application Support with `isExcludedFromBackup` -> the episode is marked downloaded -> the app may be relaunched in the background to handle completion
- **Interruptions and routes:** observe `AVAudioSession.interruptionNotification` (pause on calls, resume only if `.shouldResume`), and `routeChangeNotification` (pause when headphones are unplugged)
- **Progress sync:** store `(episodeID, position, updatedAt, deviceID)`; on conflict keep the most recent update, but treat "finished" as sticky; sync on pause, background and app launch
- **CarPlay:** the audio app entitlement plus a `CPTemplateApplicationSceneDelegate` with list templates (Up Next, Downloads, Shows); CarPlay's Now Playing screen uses the same Now Playing info and commands, so there's no separate player
- **Trade-offs:** streaming saves storage but fails in tunnels; downloads need storage management (auto-delete played episodes). `AVQueuePlayer` gives gapless next-episode transitions, while a single `AVPlayer` with manual item replacement is simpler to reason about
- **Failure modes:** the session not active when backgrounding (no audio), Now Playing info lagging the player (elapsed time drift: set elapsed time and rate together), downloads killed by the system (resume data), huge feeds (parse incrementally, page episodes)
- **Privacy/offline:** listening history is personal data; sync it under the user's account only, and declare it in the privacy manifest; downloads and progress work fully offline, and sync catches up later
- **Testing:** Swift Testing on the engine with a protocol-wrapped player (seek, rate, progress saving, sleep timer); a fake download session; interruption handling by posting the notifications; manual CarPlay checks in the CarPlay simulator

Related to check: Background Downloads, Background Uploads, Download Resume, Background Fetch, Offline Mode Design, App Intents, Observable Macro, Battery Optimization

---

Maps and Places Search

Q: Design a places search feature: a map, search as you type, place details with Look Around, and saved places that work offline.

Short answer: I'd build it on **MapKit for SwiftUI**: a `Map` bound to a `MapCameraPosition` and a selection, with `Markers` for results. Search runs from `.task(id: query)` so each keystroke cancels the previous request, with a short debounce before calling `MKLocalSearch` biased to the visible region; `MKLocalSearchCompleter` drives the suggestion list.

Key points:

- **Requirements:** search as you type with suggestions, results on the map and in a list, details with photos/Look Around, save places, offline access to saved places, location-aware results without

demanding location permission

- **Components:** `PlaceSearchModel` (`@MainActor @Observable`: query, suggestions, results, state), a `SearchService` wrapping `MKLocalSearchCompleter` and `MKLocalSearch` behind a protocol, `SavedPlace` (`@Model`: identifier, name, coordinate, note, savedAt), a `LocationProvider` around `CLLocationUpdate.liveUpdates()`, and the map/list views
- **Data flow (search):** keystroke -> completer updates suggestions (cheap, fragment-based) -> debounce 300 ms -> `MKLocalSearch` with `naturalLanguageQuery`, `resultTypes`, and the visible region from `onMapCameraChange` -> results become markers and list rows -> selection drives the details sheet (`mapItemDetailSheet(item:)` or a custom sheet)
- **Data flow (save):** store `mapItem.identifier?.rawValue` (an `MKMapItem.Identifier`) plus a snapshot of name and coordinate -> render saved pins from SwiftData immediately -> when online, refresh details with `MKMapItemRequest(mapItemIdentifier:)`
- **Location permissions:** request when-in-use only from a user action; `CLServiceSession` declares the authorization the feature needs while it's active; handle reduced (approximate) accuracy by widening the search region; never block search on location, since a region from the map works without it
- **Trade-offs:** MapKit is free with Apple's data and needs no API key, but third-party apps can't download map tiles for offline use; an SDK with offline vector tiles adds size and licensing cost. Completer suggestions are fast, full searches are richer and rate-limited
- **Scaling/failure modes:** throttling (`MKError.loadingThrottled`): debounce, cancel stale tasks, cache recent query results; many results: cluster markers or cap the count; flaky network: keep the last results visible with an offline banner; place data changing (closed businesses): refresh saved places opportunistically, not on every launch
- **Adaptive layout:** on wide windows (iPad, iPhone Duo inner display) show the list and map side by side, and on narrow ones use a bottom sheet with detents; decide from available width, not device type
- **Privacy/offline:** location is used in-process only, and never sent to your server unless the feature needs it (then coarsen it); purpose strings explain the benefit; saved places are user data, so wipe them on logout
- **Testing:** Swift Testing on `PlaceSearchModel` with a fake `SearchService` (debounce cancels earlier queries, empty and error states, offline fallback); simulated locations in the scheme; UI tests for search -> select -> save

Related to check: MapKit, Core Location, SwiftData (SwiftUI), Offline Mode Design, Task Modifier, Adaptive Layout (iPhone Duo), Request Cancellation, Caching Layer Design

---

Ride-Hailing Live Tracking

Q: Design live tracking for a ride-hailing app: the driver's app shares its location, and the rider sees the car move on a map with an ETA, even from the Lock Screen.

Short answer: There are two clients. The **driver app** streams location with `CLLocationUpdate.liveUpdates(.automotiveNavigation)`, keeps it flowing in the background with a `CLBackgroundActivitySession` (plus the location background mode), filters inaccurate fixes and sends a ping every couple of seconds over a WebSocket.

Key points:

- **Requirements:** driver position visible to the rider within a few seconds, smooth movement, ETA, works with the driver app in the background, survives network drops, battery-conscious, location shared only during an active trip
- **Driver components:** `LocationStreamer` (live updates + background activity session + throttling), `TripChannel` (WebSocket actor with reconnect and a small buffer of the latest fix only), trip state machine (`enRoute`, `arrived`, `onTrip`, `completed`)
- **Rider components:** `TripModel` (`@MainActor @Observable`: driver coordinate, heading, ETA,

state), a `Map` with an `Annotation` for the car and a route polyline, `ActivityKit` integration for the Lock Screen
- **Server components:** trip service (state, assignment), a location ingest endpoint behind the socket gateway, map-matching and ETA service, a pub/sub fan-out per trip, and an APNs sender for Live Activity updates
- **Data flow:** driver fix -> filter (accuracy < 50 m, not stationary) -> throttle to one ping per ~2 s -> WebSocket -> server map-matches to the road, updates ETA -> fan-out to the rider's socket and APNs Live Activity updates -> rider animates the annotation over the interval to the new position
- **Smoothing:** animate linearly over the expected ping interval with the reported course; snap if the jump is implausibly large; the server's map-matching keeps the car on roads
- **Trade-offs:** WebSocket while the rider app is open vs push-only when it isn't (Live Activity updates via APNs); higher ping rates (smoother, more battery and data) vs lower (cheaper; rely on interpolation); computing the ETA on the server (consistent for both sides) vs on device with MapKit directions
- **Scaling:** every active driver sends a ping every couple of seconds, so ingest is write-heavy: keep current positions in an in-memory geo store, persist sampled points only, and fan out per trip, not per city
- **Background rules:** start the `CLBackgroundActivitySession` while the app is in the foreground; the background mode must be declared; iOS shows the location indicator, which is honest UX for a driver app; stop everything when the trip ends
- **Failure modes:** GPS drift in urban canyons (accuracy filter + map-matching), tunnels (keep the last known position with a "last updated" hint; extrapolate briefly at most), socket drop (reconnect with backoff; send only the latest fix, never a backlog), authorization changed to denied mid-trip (`authorizationDenied` on the update; tell the driver), stale Live Activity (`staleDate` so the system can mark it outdated)
- **Privacy:** riders see the driver only during the trip; the driver app requests "When In Use" and relies on the background activity session rather than "Always"; positions are retained server-side only as long as policy requires; exact rider pickup is visible to the assigned driver only; purpose strings explain why
- **Adaptive UI:** the map fills the available space; on iPhone Duo or iPad the trip details sit beside the map (`ArrangementView` (iOS 27.1+) or a split layout) instead of over it
- **Testing:** Swift Testing for the filter/throttle logic with a scripted `AsyncStream` of fixes (inject the update source behind a protocol); the simulator's location simulation and GPX files for routes; field tests in the car with the Power Profiler

Related to check: Live Activities Delivery Tracking, WebSockets, WebSocket Reconnection, Scalable Chat App (connection actor), Push Notification System, Maps Places Search, Onboarding and Permissions Flow, Battery Optimization, ArrangementView, AsyncSequence

---

Booking and Calendar App

Q: Design an appointment booking app: browse availability, book a slot, add it to the user's calendar, and send reminders.

Short answer: The server owns availability and is the only place that can prevent double-booking, so the client's job is to **show slots honestly, book with a hold-then-confirm flow, and treat time correctly**. Every time crosses the wire as an absolute instant plus the venue's IANA time zone ID, never a local wall-clock string, and the UI shows venue time (and the user's time when they differ).

Key points:

- **Requirements:** browse slots by provider/date, book in a few taps, no double-booking, correct times across zones and DST, add to calendar, reminders, reschedule/cancel, offline access to upcoming bookings
- **Components:** `AvailabilityService` (paged by day, short cache TTL), `BookingService` (actor:

hold, confirm, cancel; idempotency keys), `Booking` store (SwiftData; upcoming bookings available offline), `CalendarExporter` (EventKit), `ReminderScheduler` (`UNUserNotificationCenter`), and a `@MainActor @Observable` booking flow model
- **Data flow (book):** select slot -> `POST /holds` (slot ID, idempotency key) -> server holds it for ~5 minutes and returns an expiry -> user completes details/payment while a countdown shows -> `POST /bookings` with the hold ID -> confirmed booking saved locally -> calendar and reminders offered
- **Double-booking:** prevented server-side with a unique constraint or transactional check per (resource, slot); the client never assumes availability from a cached list. Conflict (HTTP 409) -> explain, refresh that day, keep the user's other inputs
- **Time zones:** store `Date` (UTC instant) + `venueTimeZoneID`; generate slots on the server in the venue's zone; format with a `Date.FormatStyle` whose `timeZone` is the venue's; recurring events and DST shifts are calculated with a `Calendar` in the venue zone, never by adding 86,400 seconds
- **Calendar permissions:** `requestWriteOnlyAccessToEvents()` to add events; `EKEventEditViewController` (EventKitUI) can add an event with no permission prompt at all, since the user confirms in system UI; full access (`requestFullAccessToEvents()`) only if the app shows the user's existing events to avoid clashes, with a purpose string that says why. Info.plist keys: `NSCalendarsWriteOnlyAccessUsageDescription` / `NSCalendarsFullAccessUsageDescription`
- **Reminders:** a local notification 24 h and 1 h before (`UNCalendarNotificationTrigger` built from date components in the venue zone, or a time-interval trigger from the instant); reschedule when the booking changes; server push for cancellations by the provider; time-sensitive interruption level only for imminent appointments
- **Trade-offs:** holds add a step and server state but stop people losing a slot at the payment screen; optimistic "booked!" without a hold feels faster but fails at the worst moment. Exporting to the calendar duplicates data the app can't update later (write-only can't read it back), so store the `eventIdentifier` and include a deep link in the event
- **Failure modes:** hold expiry during payment (refresh or re-hold), retries creating duplicate bookings (idempotency key per attempt), the device clock being wrong (show server-provided expiry as a duration), the user traveling (always show the zone)
- **Testing:** Swift Testing with parameterized time-zone cases (DST boundaries, half-hour zones such as Asia/Kolkata, the date line), a fake booking API that returns 409/expired holds, and an in-memory SwiftData store

Related to check: EventKit, Date and Time Zones, Idempotency, Push Notification System, Offline Mode Design, Parameterized Tests, Deep Linking, Error Handling Architecture

---

E-commerce Cart and Checkout

Q: Design the cart and checkout for a shopping app: a cart that syncs across devices, Apple Pay, and orders that are never charged twice.

Short answer: The cart is **server-owned and versioned**: the app keeps a local copy for instant UI and offline edits, sends changes as small operations (set quantity of SKU), and reconciles with the server's version, which also re-prices items and checks stock. Checkout is where correctness matters most, so the server re-validates the cart at a specific **version**, the client sends one **idempotency key per checkout attempt** and reuses it on every retry, and the payment is authorized only inside that idempotent order request.

Key points:

- **Requirements:** add/remove instantly (also offline), cart shared across devices and web, accurate prices/taxes/shipping at checkout, Apple Pay and card payments, no duplicate charges or orders, clear handling of out-of-stock and price changes, order status afterwards
- **Components:** `CartStore` (local SwiftData copy + pending operations), `CartSync` (actor: pushes operations, pulls the server cart with its `version`), `CheckoutService` (actor: quote, place order,

idempotency keys), `ApplePayCoordinator` (`PKPaymentRequest`, the authorization controller and its delegate), `OrderStatusModel` (polling or push), and the `@MainActor @Observable` checkout flow model
- **Data flow (cart):** tap "Add" -> local insert + pending op -> UI updates -> `CartSync` sends ops with the base version -> server applies, re-prices, and returns the new cart + version -> local copy replaced; on conflict, the server's cart wins and changes are shown ("1 item is no longer available")
- **Data flow (checkout):** "Checkout" -> `POST /checkout/quote` locks a cart version with totals -> Apple Pay sheet shows the quoted summary items -> the user authorizes -> `placeOrder(cartVersion, paymentToken)` with the attempt's `Idempotency-Key` -> server reserves stock, charges the token, creates the order atomically -> the delegate completes the Apple Pay sheet with success or with errors (e.g. a shipping address the store can't deliver to)
- **Idempotency:** the key identifies the *attempt*, not the request: retries after a timeout reuse it, so the server returns the original result instead of charging again; a new attempt (after the user changes the cart) gets a new key. The server stores key -> result for at least 24 h
- **Inventory races:** a reservation at order time (or a short hold at quote time) with expiry; 409 when the cart version or stock no longer matches -> show what changed and requote; never decrement stock on "add to cart"
- **Trade-offs:** a server-owned cart syncs across devices and prevents price tampering, but needs offline queuing; a local-only cart is simpler but can't follow the user. Holding stock at quote time is fairer for scarce items but can lock out other buyers, so use short holds only for limited drops
- **Failure modes:** timeout after the charge succeeded (retry with the same key returns the order), the app killed mid-checkout (persist the attempt key with the cart version; on relaunch, query by key), double taps (disable the button and let the actor serialize), Apple Pay unavailable (`PKPaymentAuthorizationController.canMakePayments(usingNetworks:)` decides whether to show the button)
- **Privacy/security:** prices and totals are always computed server-side; the Apple Pay token is opaque and goes only to your server/processor; no card data in logs or analytics; addresses are personal data (privacy manifest)
- **Testing:** Swift Testing on `CheckoutService` with a fake HTTP client that times out then succeeds (exactly one order), returns 409 (new key on the next attempt), and 402 (declined); cart merge tests with conflicting versions; Apple Pay sandbox testers for the full flow

Related to check: Idempotency, Retry Logic, Saga Pattern, Offline Mode Design, Data Synchronization, Error Handling Architecture, Request Deduplication, Banking App Security

---

StoreKit 2 Subscriptions

Q: Design subscriptions for an app with monthly and yearly plans, a free trial, offer codes and a web account. How do you know, on every device, whether the user is entitled to Pro?

Short answer: With StoreKit 2 the device can answer "is this user entitled?" by itself: `Transaction.currentEntitlements` yields the signed, verified transactions that are active right now, so I derive access from that instead of storing an `isPro` flag. At launch I start a listener on `Transaction.updates`, because renewals, refunds, Ask to Buy approvals and purchases on other devices arrive there; every verified transaction is applied and then `finish()`ed.

Key points:

- **Requirements:** show plans with localized prices, trials and offers; purchase, upgrade/downgrade/crossgrade within a subscription group; restore; refunds and revocations remove access; access on all the user's devices and on the web; offer codes, win-back and promotional offers; works offline for an active subscriber
- **Components:** product configuration (App Store Connect + a `.storekit` file for local testing), `Entitlements` (`@MainActor @Observable`), a paywall (`SubscriptionStoreView` or custom with

`Product.products(for:)`), the `Transaction.updates` listener started at launch, and a backend with a notifications endpoint and the App Store Server API client
- **Data flow (purchase):** paywall -> `product.purchase(options: [.appAccountToken(userID)])` -> `.success(.verified(transaction))` -> unlock -> send `transaction.jsonRepresentation` (or the transaction ID) to the server -> `finish()` -> the server links the original transaction ID to the account
- **Data flow (lifecycle):** renewal, billing retry, grace period, expiry, refund -> App Store Server Notification to the backend -> backend updates the account; the device sees the same changes through `Transaction.updates` and `Product.SubscriptionInfo.Status`
- **Entitlement on device:** iterate `Transaction.currentEntitlements`, accept `.verified` only, ignore revoked transactions; `Product.SubscriptionInfo.status(for: groupID)` gives the renewal state (`.subscribed`, `.inGracePeriod`, `.inBillingRetryPeriod`, `.expired`, `.revoked`) for UI like "Payment problem: update your card"
- **Offers:** introductory offers (eligibility checked by StoreKit), **offer codes** redeemed through `.offerCodeRedemption(isPresented:)` (the iOS 27 SDK adds `offerCodeRedemption(options:isPresented:onComp` (iOS 27+), which returns the verified transaction, and deprecates the old form), **promotional offers** signed by the server (`.promotionalOffer(_:compactJWS:)` replaces the signature-based option deprecated in iOS 26), and **win-back offers** (`.winBackOffer(_:)` purchase option) for lapsed subscribers
- **Trade-offs:** device-only entitlement (no server, works offline, Apple platforms only) vs server-backed (cross-platform, web access, fraud and family logic, more work); `SubscriptionStoreView` (fast, App Review-friendly, less custom) vs a custom paywall
- **Failure modes:** never `finish()`ed transactions (they are redelivered, which is the safety net for crashes mid-unlock), unverified transactions (don't unlock), the purchase returns `.pending` (Ask to Buy or extra authentication; the result arrives later via updates), clock or offline issues (cached signed transactions still verify offline), Family Sharing (`ownershipType == .familyShared`)
- **App Review and policy:** a "Restore Purchases" button (`AppStore.sync()`), a link to manage subscriptions (`.manageSubscriptionsSheet` or `AppStore.showManageSubscriptions(in:)`), clear pricing and trial terms on the paywall
- **Privacy:** `appAccountToken` is an opaque UUID, not an email; the server stores transaction IDs, not payment details
- **Testing:** StoreKit configuration files and `StoreKitTest` (`SKTestSession`) for renewals, refunds, Ask to Buy and billing failures in fast time; Swift Testing on the entitlement model; sandbox and TestFlight before release; the server's notification handler tested with Apple's test notification endpoint

Related to check: Authentication System (account linking), Remote Configuration (paywall experiments), Feature Flags Design, A/B Testing Architecture, Analytics Implementation, Idempotency

---

Fitness Tracker with HealthKit

Q: Design a fitness tracking app that records workouts, reads the user's health data, and keeps a dashboard up to date, even while the app isn't running.

Short answer: HealthKit is the **shared, user-controlled store**; my app reads from it and writes to it, and keeps its own local cache only for fast dashboards and app-specific data. Authorization is per data type and asymmetric: the app learns whether the user allowed it to *write* a type, but never whether it can *read* one (a denied read just looks like no data).

Key points:

- **Requirements:** record workouts with live metrics, show history and trends (steps, active energy, heart rate), stay in sync with data from other apps and the Watch, update while backgrounded, respect the user's choices and Health privacy rules
- **Components:** `HealthAuthorization` (asks for the minimum types at the moment they're

needed), `WorkoutRecorder` (session + live builder), `WorkoutImporter` (actor; anchored queries, anchors persisted per type), `MetricsStore` (SwiftData cache for dashboards), `TrendsService` (`HKStatisticsCollectionQuery` for daily buckets), and the dashboard models
- **Data flow (record):** start -> `HKWorkoutSession` (iOS 26+ on iPhone) with an `HKWorkoutConfiguration` -> the live builder collects samples -> pause/resume events -> end -> `finishWorkout()` saves to HealthKit -> the importer picks it up like any other workout, so there's one code path
- **Data flow (sync):** at launch, enable background delivery for workouts/steps -> register `HKObserverQuerys` -> when HealthKit signals a change, run `HKAnchoredObjectQueryDescriptor` from the saved anchor -> upsert by sample UUID, delete by `HKDeletedObject.uuid` -> save the new anchor -> call the observer's completion handler
- **Authorization:** request in context ("Connect Health" in onboarding or the first workout), with clear purpose strings (`NSHealthShareUsageDescription`, `NSHealthUpdateUsageDescription`); read requests silently return less data if denied, so design empty states that say "No data, or Health access is off" and link to Settings; `statusForAuthorizationRequest(toShare:read:)` tells you whether the sheet would be shown
- **Background delivery:** the HealthKit background-delivery entitlement; frequencies are a request, and some types (steps) are delivered at most hourly; observer queries must be registered on **every** launch (in `didFinishLaunching`), and the completion handler must be called or HealthKit backs off
- **Trade-offs:** reading directly from HealthKit keeps one source of truth but is slower for complex dashboards; a local cache is fast but must handle deletions (hence anchored queries). Statistics queries do the aggregation and de-duplication across sources (Watch + iPhone steps) for you, so don't sum raw samples
- **Failure modes:** a locked device (Health data is protected; queries fail while locked, so retry on unlock), duplicate imports (upsert by UUID), anchor saved before data (save the anchor last), time-zone changes in daily buckets (anchor bucket boundaries in `Calendar.current`)
- **Privacy:** Health data can't be used for advertising or sold, and anything uploaded needs explicit consent and a privacy policy; keep it out of analytics events and logs; local caches use complete file protection
- **Testing:** wrap `HKHealthStore` behind a protocol and test the importer with Swift Testing (added, deleted, replayed changes; anchor saved only after success); the simulator supports HealthKit with manually entered data in the Health app

Related to check: HealthKit, Background Fetch, Data Synchronization, Offline Mode Design, SwiftData (SwiftUI), Securing Sensitive Data, Actors, Battery Optimization

---

Stories Camera Feature

Q: Design an Instagram-style Stories camera: live preview, photo and video capture, edits, and reliable uploads.

Short answer: The camera is a **capture pipeline owned by one actor whose executor is a dedicated serial queue**: `AVCaptureSession` configuration, start/stop and delegate callbacks all run on that queue, so nothing blocks the main thread and nothing races. The UI is a SwiftUI screen showing an `AVCaptureVideoPreviewLayer` through a representable, with a shutter that supports tap for photo and hold for video, the hardware Camera Control (iPhone 16 and later) through `AVCaptureControls` and capture events (`onCameraCaptureEvent`), and the volume buttons.

Key points:

- **Requirements:** camera ready in under a second, photo and up to 60 s video, front/back switching, zoom and flash, overlays (text, stickers), post reliably on bad networks, never lose a capture, battery- and thermal-aware
- **Components:** `CaptureService` (actor on a `DispatchSerialQueue`: session, inputs, `AVCapturePhotoOutput`, `AVCaptureMovieFileOutput`, controls), `CameraModel` (`@MainActor`

`@Observable`: state, permissions, errors), `PreviewView` (`UIViewRepresentable` hosting the preview layer), `Editor` (overlay model + Core Image/AVFoundation composition), `Exporter` (`@concurrent`, writes H.264/HEVC + JPEG/HEIC), `UploadQueue` (SwiftData records + background session)

- **Data flow:** open camera -> check/request camera (and microphone for video) permission -> `configure()` then `start()` on the capture queue -> capture -> file in a temporary directory -> editor -> export to a file -> `UploadQueue.enqueue(file, metadata)` -> background upload -> server transcodes and publishes -> local record marked posted
- **Camera Control:** if `session.supportsControls`, add `AVCaptureSystemZoomSlider` and `AVCaptureSystemExposureBiasSlider`, and set a controls delegate (controls stay inactive without one) to hide on-screen UI when the control's overlay is expanded. Map the hardware press to capture with `onCameraCaptureEvent` (SwiftUI) or `AVCaptureEventInteraction` (UIKit)
- **Uploads:** upload *from a file* in a background `URLSession` configuration (data uploads aren't allowed in background sessions); large videos use resumable or chunked upload with the server acknowledging offsets; idempotent create by client-generated story ID; retry with backoff
- **Trade-offs:** `AVCaptureMovieFileOutput` is simple and efficient; `AVAssetWriter` with video data output is needed for live filters or custom encoding at the cost of complexity. Uploading the original and transcoding on the server is simpler; compressing on device saves bandwidth but costs battery and time
- **Failure modes:** session interruptions (phone call, another app using the camera, multitasking on iPad: observe `AVCaptureSession.wasInterruptedNotification` and show a state), thermal pressure (lower frame rate or resolution when `ProcessInfo.thermalState` is serious), runtime errors (restart the session), low storage (check before recording), the app killed mid-upload (the background session finishes, and the app reconciles on relaunch)
- **Adaptive layout:** the preview fills available space using the view's bounds and rotation angles from `AVCaptureDevice.RotationCoordinator`, never `UIDevice` orientation or screen size, which matters on iPad, iPhone Duo and resizable windows
- **Privacy:** purpose strings for camera, microphone and photo library add-only access; strip location metadata unless the user opts in; the green/orange indicators make misuse visible, so stop the session when the screen disappears
- **Testing:** the simulator has no camera, so put `CaptureService` behind a protocol and test `CameraModel` and the upload queue with fakes in Swift Testing; device tests for capture; the background upload path with a local test server

Related to check: Background Uploads, Multipart Upload, Upload Progress, Actors, Global Actor Isolation, UIViewRepresentable, Battery Optimization, Adaptive Layout (iPhone Duo), Offline Mode Design

---

Barcode Scanner Feature

Q: Design a barcode-scanning feature for a retail app: point the camera at a product, look it up, add it to the cart. Which frameworks would you use, and how do you make it fast and reliable?

Short answer: For a live scanner I'd use VisionKit's `DataScannerViewController`: it wraps the camera session, Vision's barcode detection, guidance text, highlighting and pinch-to-zoom in one view controller, so the feature is mostly product logic. I restrict it to the symbologies we actually sell (EAN-13, EAN-8, UPC-E) for speed and fewer false reads, deduplicate payloads, and hand each new code to a lookup service with a cache.

Key points:

- **Requirements:** scan in under a second in normal light, one product per scan (no double adds), works one-handed, clear feedback (haptic, highlight, product card), fallbacks for no camera / denied permission / damaged codes, accessible to VoiceOver users

- **Components:** `BarcodeScanner` (`UIViewControllerRepresentable` around `DataScannerViewController`), a `ScanSession` model (`@MainActor @Observable`: state, last payload, cooldown), `ProductLookup` (`Sendable` service with an in-memory + disk cache keyed by GTIN), a manual-entry sheet, and a photo fallback using Vision
- **Data flow:** camera frame -> VisionKit recognizes a barcode -> delegate `didAdd` -> dedupe by payload -> validate the check digit locally -> lookup (cache first, then API) -> product card with "Add to cart" -> haptic via `.sensoryFeedback`
- **Availability and permission:** `DataScannerViewController.isSupported` (hardware: A12 Bionic or later) decides whether the feature exists at all; `isAvailable` is `false` when camera access is denied or restricted, so show an explanation and a Settings link instead of a black view; add `NSCameraUsageDescription` that explains the benefit; ask on first tap of "Scan", never at launch
- **Performance:** few symbologies, `recognizesMultipleItems: false` for a single-product flow, `qualityLevel: .balanced` (`.accurate` for small or dense codes like Data Matrix), `regionOfInterest` matching the on-screen reticle, turn off high-frame-rate tracking when you don't draw custom highlights, and `stopScanning()` as soon as the sheet goes away so the camera and Neural Engine power down
- **Trade-offs:** `DataScannerViewController` (minimal code, system UI, iOS-only, limited customization) vs `AVCaptureSession` + `AVCaptureMetadataOutput` (full control of the camera, older and lower-level, more code) vs `AVCaptureSession` + Vision requests on frames (custom pipelines, most work). Text recognition (prices, serial numbers) comes almost free with VisionKit via `.text(textContentType:)`
- **Failure modes:** the same code reported again as it re-enters the frame (dedupe by payload plus a short cooldown), unknown product (offer manual search; log the GTIN), glare and blur (guidance text, a torch toggle, zoom), lookup offline (queue it, show the code), `becameUnavailableWithError` mid-session (camera taken by another app or restricted)
- **Adaptive UI:** the scanner is presented full-screen on iPhone and as a sheet or side panel next to the cart on iPad and an unfolded iPhone Duo; size the reticle and `regionOfInterest` from the view's bounds, never from the screen size
- **Privacy/offline:** camera frames never leave the device; only the payload is sent; cache recent lookups so re-scans work offline; the privacy manifest declares any product-search data you collect
- **Testing:** the scanner itself can't run in the simulator (no camera), so put logic behind the payload: Swift Testing for check-digit validation, dedupe/cooldown and lookup caching with a fake service; Vision's `DetectBarcodesRequest` over fixture images in tests; device testing in poor light

Related to check: UIViewRepresentable, Onboarding and Permissions Flow, Caching Layer Design, Sensory Feedback, Accessibility, Battery Optimization, Request Deduplication

---

Image Editor with Undo

Q: Design a photo editor with adjustments, crop and filters, unlimited undo and redo, and a live preview that never stutters. How do you model the edits and keep memory under control?

Short answer: I'd make editing non-destructive: the original image is never modified, and the document is a small, ordered recipe of edit steps (`exposure(+0.3)`, `crop(rect)`, ...) that a Core Image pipeline replays to produce pixels. Because the recipe is a few bytes per step, undo and redo are just snapshots of the recipe on two stacks (or the command pattern with inverses when steps are large), so unlimited undo costs almost no memory.

Key points:

- **Requirements:** live preview while dragging sliders at 60 - 120 fps, unlimited undo/redo with named actions, re-editable later (edits saved, not baked), full-resolution export, bounded memory for 48-megapixel photos, works offline

- **Components:** `Edit` (`Sendable` value enum, `Codable` for saving), `EditorModel` (`@MainActor @Observable`: recipe, undo/redo stacks, preview), a renderer (`@concurrent` function, shared `CIContext`), a proxy source (downsampled `CIImage` for preview) and the original for export, an exporter that writes HEIC/JPEG plus the recipe sidecar
- **Data flow:** slider moves -> the model updates the *last* step in place while dragging (no undo entry yet) -> render the preview at view pixel size -> on drag end commit one undo step -> on save render full resolution in the background and write
- **Undo design:** memento (snapshot the whole recipe; trivial when the recipe is small and immutable) vs command pattern (each command knows `apply` and `revert`; needed when steps carry heavy data like brush strokes or masks). Coalesce slider drags into one step; name actions; bridge to `UndoManager` so ⌘Z, shake-to-undo and the Edit menu work
- **Core Image pipeline:** a `CIImage` is a lazy recipe too; chaining filters builds a graph and nothing renders until `createCGImage` or drawing into a Metal-backed view; Core Image fuses filters and works in an extended linear sRGB working space by default
- **Memory:** never hold full-resolution decoded bitmaps per undo step; preview from a downsampled proxy (`CGImageSourceCreateThumbnailAtIndex`, or a Lanczos scale in Core Image); one `CIContext` per app (contexts are expensive and cache intermediates); release the export bitmap right after writing
- **Trade-offs:** CPU snapshots of rendered images per step (instant undo, huge memory) vs recipe replay (tiny memory, re-render on undo, which is fine because rendering is fast); a `CGImage` preview in SwiftUI (simple) vs an `MTKView` with `CIRenderDestination` (lowest latency for full-screen live preview)
- **Scaling / failure modes:** very long recipes (collapse consecutive steps of the same kind; cap or checkpoint with a cached intermediate), slow devices (lower preview resolution while dragging, full preview on release), memory warnings (drop caches and the proxy, keep the recipe), export interrupted by backgrounding (`BGContinuedProcessingTask`, iOS 26+, for a user-initiated long export)
- **Adaptive UI:** canvas plus tool palette; tools in a bottom bar on a narrow iPhone, a sidebar or inspector on iPad and an unfolded iPhone Duo; preview size comes from the canvas's own geometry (`onGeometryChange`), which also sets the proxy resolution
- **Privacy/offline:** fully on-device; Photos access with the limited library picker (`PhotosPicker` needs no permission); save edits back with `PHContentEditingOutput` so other apps see them as revertible adjustments
- **Testing:** Swift Testing on `EditorModel` (perform, undo, redo, coalescing) with a stub renderer; golden-image tests of the pipeline at small sizes with a tolerance; Allocations and the Hangs instrument while scrubbing sliders

Related to check: Document Editor App, SwiftData Undo, Image Caching System, Image Downsampling, Memory Warning Handling, Instruments Allocations, Async Image Decoding, Metal GPU, Undo Management (`UndoManager`), Swift Concurrency (`@concurrent`)

Document Editor App

Q: Design a document-based editor, say a whiteboard or a structured notes app, with autosave, undo, iCloud Drive and multiple windows. What does the architecture look like?

Short answer: I'd lean on the system document infrastructure rather than inventing my own file handling: a `DocumentGroup` scene gives the document browser, file coordination, autosave, iCloud Drive and version-conflict handling. On iOS 27 the document is an `@Observable` class conforming to `Document` (iOS 27+), whose file format is a plain `Codable` snapshot read and written off the main actor; with an iOS 18 target the same design uses `ReferenceFileDocument` (or `UIDocument` in UIKit).

Key points:

- **Requirements:** create/open/rename from the Files-style browser, edits never lost (autosave, crash

safety), unlimited undo/redo per document, iCloud Drive with the same document open on two devices, several documents in separate windows on iPad, Mac and resized iPhone, large documents open quickly

- **Components:** `BoardDocument` (`@MainActor @Observable`, `Document` iOS 27+), a `Codable` snapshot type that *is* the file format (with a version field), editor views using `@Bindable`, an `UndoManager` from `@Environment(\.undoManager)`, optional `DocumentGroupLaunchScene` + `NewDocumentButton(_:source:)` (iOS 27+) for templates
- **Data flow (open):** SwiftUI coordinates the read -> `reader(configuration:)` decodes off the main actor -> `apply(snapshot:previous:)` updates the model on the main actor -> the editor renders
- **Data flow (edit -> save):** gesture -> model method mutates state and registers undo -> the document is dirty -> SwiftUI autosaves later: `snapshot(contentType:)` captures a value on the main actor, the writer encodes and writes it in the background with coordinated access
- **Undo design:** one undo step per *user intent* (a drag ends, a text field commits), not per keystroke or per frame; name actions (`setActionName`) for the Edit menu; group compound changes; clear nothing manually, SwiftUI owns the stack per document
- **iCloud and conflicts:** documents live in the app's iCloud Drive container; the document infrastructure handles file coordination and version conflicts; while a conflict is unresolved or the file is downloading, `@Environment(\.documentConfiguration)?.isEditable` is `false`, so the UI must go read-only. In UIKit, `UIDocument.State.inConflict` plus `NSFileVersion.unresolvedConflictVersionsOfItem(at:)` is the manual path: pick or merge, then mark versions resolved
- **Trade-offs:** a single JSON file is simple and diff-friendly but rewrites everything on save; a package (directory `FileWrapper`) lets the writer reuse the `previous` wrapper and rewrite only changed parts (large images stay untouched). `FileDocument` (value type) gets undo for free but copies the whole document on every change
- **Scaling / failure modes:** huge documents (lazy-load assets from the package, thumbnail first), schema evolution (version field + migration in the reader; never crash on an unknown future version, open read-only), save failures (surface them; keep the in-memory state), edits arriving from another device mid-session (on a re-read, `apply(snapshot:previous:)` also receives the previously loaded snapshot, so it can apply just the difference)
- **Adaptive UI:** each window owns its own document and undo stack; on iPhone Duo and iPad the inspector becomes a sidebar or a sheet based on width (`.inspector` adapts), never on device type
- **Privacy/offline:** documents are user content in their iCloud Drive; files are available offline once downloaded; declare the exported `UTType` in Info.plist; no document content in analytics
- **Testing:** Swift Testing for the snapshot codec (round-trip, old versions decode) and for model methods with a real `UndoManager` (`undo()` restores, `redo()` reapplies); UI tests for open -> edit -> relaunch -> content persisted

Related to check: DocumentGroup, Document-Based Persistence, iCloud Documents, @Observable macro, Undo Management (`UndoManager`), Image Editor with Undo, Sync Across Devices, Scenes and Multiple Windows, Swift Concurrency (`@concurrent`, `sending`), Codable

---

Download Manager

Q: Design a download manager for an app that lets users download large files (courses, maps, podcasts) for offline use: a queue, progress, pause and resume, and downloads that finish in the background.

Short answer: The manager has two sources of truth that must agree: a **persisted queue** (SwiftData records with state, remote URL, destination and resume data) and **one background `URLSession`** whose transfers run in a system daemon and survive suspension and termination. The UI only reads the records.

Key points:

- **Requirements:** queue many files, a per-item and overall progress UI, pause/resume/cancel, continue

in the background and after the app is terminated by the system, resume after network loss without starting over, Wi-Fi-only option, storage management, no duplicate downloads

- **Components:** `DownloadRecord` (SwiftData: id, URL, state, bytes, resume data, local path), `DownloadManager` (actor: queue policy, concurrency limit, reconciliation), the session delegate (`Sendable`, writes state changes to the store), the app delegate's `handleEventsForBackgroundURLSession` hook (see Background Downloads), and an `@Observable` progress model fed by `@Query`
- **Data flow:** enqueue -> record `.queued` -> manager starts up to N tasks with `taskDescription = record.id` -> `didWriteData` updates progress (throttled writes) -> `didFinishDownloadingTo` moves the file synchronously into Application Support -> `didCompleteWithError` marks `.done`, or saves resume data and marks `.paused`/`.failed` -> start the next queued item
- **Pause and resume:** `cancel(byProducingResumeData:)` and `downloadTask(withResumeData:)`; resume only works if the server supports range requests with a stable `ETag`/`Last-Modified` (see Download Resume). Resume data also arrives in the error's `userInfo` when a transfer fails
- **Queue policy:** a concurrency cap (2 - 3) so one file finishes before ten crawl, user-reordered priority, `allowsCellularAccess` / `allowsExpensiveNetworkAccess` from the user's Wi-Fi-only setting, `isDiscretionary` for non-urgent prefetches
- **Trade-offs:** background sessions survive termination but are delegate-only and scheduled by the system (slower to start, especially when the app is in the background); a foreground session with async `download(for:)` is simpler but dies with the app. Background Assets downloads before launch (install/update) but suits app content, not user-chosen files
- **Failure modes:** force-quit from the app switcher (iOS cancels background transfers; the next launch reconciles and resumes), a reboot, disk full (check `volumeAvailableCapacityForImportantUsage` before starting and handle `NSURLErrorCannotWriteToFile`), a server file changed between pause and resume (the resume fails; restart from zero), the temporary file read after the delegate returns (it's already deleted)
- **Storage:** keep downloads in Application Support with `isExcludedFromBackup`, show sizes, and let users delete; anything re-downloadable can live in Caches if losing it is acceptable
- **Privacy:** file protection that still allows background writes (`.completeUntilFirstUserAuthentication`); disk-space checks are a required-reason API in the privacy manifest
- **Testing:** Swift Testing for the reconciliation and queue policy with a fake session protocol and in-memory store (vanished task with/without resume data, concurrency cap respected); on device, start downloads, background the app, kill it from Xcode, relaunch

Related to check: Background Downloads, Download Resume, Background Uploads, Video Streaming App (HLS downloads), Podcast Player, Background Tasks, File Storage, File Protection Classes, Actor Reentrancy

---

Offline-First Notes App

Q: Design an offline-first notes app that syncs across a user's iPhone, iPad and Mac through iCloud. How do you store notes, sync them and handle conflicts?

Short answer: Notes live in a local SwiftData store that the UI reads through `@Query`, so the app is fully usable offline and launches instantly; iCloud sync is a background process that reconciles that store. I'd start with SwiftData's built-in mirroring, `ModelConfiguration(cloudKitDatabase: .private("iCloud.com.example.notes"))`, because it costs almost nothing.

Key points:

- **Requirements:** create, edit, delete and search offline; sync across a user's devices within seconds when online; never lose an edit; deletes propagate; sign-out or account switch removes the previous user's data; attachments (images, PDFs); optional sharing with other people
- **Components:** SwiftData `@Model Note` (`id`, `title`, `body`, `modifiedAt`, `lastSyncedBody`, stored

CloudKit system fields), a `@ModelActor NoteStore`, a `CKSyncEngine` with a `Sendable` delegate, a small persisted sync state, and `@MainActor @Observable` editor models

- **Data flow (local edit):** the editor writes to the store (debounced while typing) -> the store records `.saveRecord(id)` in `syncEngine.state` -> the engine batches sends when the network allows -> `nextRecordZoneChangeBatch` builds `CKRecords` from local notes -> on success, store the new system fields
- **Data flow (remote change):** CloudKit push -> the engine fetches -> `.fetchedRecordZoneChanges` -> upsert or delete locally -> `@Query` updates every open view
- **Mirroring vs `CKSyncEngine`:** mirroring is automatic but last-writer-wins per record, private database only, and constrains the schema (attributes need defaults or must be optional, no `#Unique`, relationships optional); `CKSyncEngine` (iOS 17) means more code but you own the merge, the schema mapping and sharing. `NSPersistentCloudKitContainer` is the Core Data equivalent of mirroring and also supports shared databases
- **Conflicts:** CloudKit rejects a stale save with `CKError.serverRecordChanged` and includes the `serverRecord`. For a note body I do a three-way merge (common ancestor = `lastSyncedBody`, plus local and server versions); if the same lines changed on both sides I keep both and mark the note "conflicted" rather than drop text. Title and metadata can be last-writer-wins
- **Failure modes:** the sync state isn't persisted (every launch re-downloads everything), quota exceeded (surface it, keep working locally), the zone was deleted on another device (re-create or wipe locally), a large attachment (use `CKAsset` and keep the file on disk, not in the model), the user isn't signed in to iCloud (keep working locally and sync later)
- **Account changes:** on `.accountChange` with sign-out or switch, erase the local store and engine state; that's both privacy and correctness
- **Privacy:** the private database is only readable by the user; `encryptedValues` on `CKRecord` adds end-to-end encryption for sensitive fields; file protection on the store; nothing in `UserDefaults` but preferences
- **Adaptive UI:** a three-column `NavigationSplitView` (folders | notes | editor) collapsing to a stack on narrow widths, iPhone Duo and resizable windows included; the selection lives in the router so resizing keeps the open note
- **Testing:** Swift Testing on the merge function (parameterized cases: both edited, one deleted, identical edits); the store with an in-memory `ModelConfiguration`; the delegate with scripted events; real sync with two simulators signed in to a test iCloud account

Related to check: Data Synchronization, Sync Across Devices, Offline Mode Design, Offline-First, SwiftData with SwiftUI, Core Data, Realtime Collaboration, Encryption at Rest, Push Notification System

---

Sync Across Devices

Q: Design sync for a journaling app so a user's entries stay in step across their iPhone, iPad and Mac. Would you use CloudKit or your own backend, and how do you handle conflicts and account changes?

Short answer: The first decision is who the data belongs to: if it's one user's private data on Apple devices only, CloudKit's private database is the strongest answer (no servers to run, no login, data counted against the user's iCloud storage, end-to-end encryption available), and `CKSyncEngine` lets me keep my own SwiftData store while CloudKit handles scheduling, change tokens, push and retries. If the data must reach Android or the web, be shared with other people at scale, or be processed server-side, I'd build the same local-first loop against my own backend.

Key points:

- **Requirements:** edits on any device appear on the others within seconds when online, offline editing everywhere, no lost edits when two devices change the same entry, deletes that propagate, correct behavior when the user signs out of iCloud or switches accounts, small battery and data cost

- **Decision matrix:** SwiftData `ModelConfiguration(cloudKitDatabase:)` or `NSPersistentCloudKitContainer` (least code, automatic mirroring, limited control: schema rules, no custom conflict handling) -> `CKSyncEngine` (own store, own conflict policy, own zones; iOS 17+, so available at an iOS 18 target) -> custom backend (any platform, server logic, your auth and ops cost)
- **Components:** SwiftData store, a `SyncCoordinator` actor owning the `CKSyncEngine` and its persisted `State.Serialization`, a record mapper (`Entry` ↔ `CKRecord` with a stable record name = entry UUID), a conflict resolver per entity, and a small `@MainActor @Observable` sync status for the UI
- **Data flow (local edit):** save to SwiftData -> tell the engine `.saveRecord(id)` -> the engine batches and sends when appropriate -> success clears pending state; `serverRecordChanged` triggers the conflict policy
- **Data flow (remote edit):** silent push from CloudKit -> the engine fetches zone changes -> `fetchedRecordZoneChanges` -> upsert by record name, delete by ID -> `@Query` views update
- **Conflicts:** per entity: entry text uses last-writer-wins on the server's modification time *or* keeps both versions as a "conflicted copy" for long text; tags and checklists merge as sets; counters use operations, not totals. Keep the server record's change tag (system fields) so the next save is a proper update, not a blind overwrite
- **Account changes:** sign-in uploads existing local data; sign-out and account switch wipe local synced data and the engine state, because it belongs to the previous Apple Account; a zone deleted elsewhere (or "Delete data from iCloud" in Settings) removes local data; an encrypted-data reset means the server copy is gone, so re-upload
- **Trade-offs:** CloudKit costs nothing to run but is Apple-only and offers no server-side code; a custom backend gives cross-platform reach and server features but needs auth, storage, push and conflict logic you own. Custom zones in CloudKit enable atomic batches and change tokens; the default zone does not support them
- **Scaling / failure modes:** first sync of thousands of records (the engine pages; show progress), quota exceeded on the user's iCloud (surface it, keep data local), rate limiting (`CKSyncEngine` honors retry-after), schema evolution (add fields only; old clients ignore unknown keys; deploy the schema to production before release)
- **Adaptive UI:** the same entry may be open on an iPad and a Mac at once; editors should observe the model, and a remote change to the open entry shows non-destructively (merge, or "updated on another device") instead of replacing text under the cursor
- **Privacy:** private database only; enable encrypted fields (`encryptedValues`) for journal text; no analytics on content; account switches must never leave the previous user's entries visible
- **Testing:** Swift Testing on the mapper and the three-way merge with fixture records (one side changed, both changed, set additions and removals); a fake engine-facing protocol for account-change handling; two simulators signed into a test iCloud account for end-to-end checks; Network Link Conditioner for flaky networks

Related to check: Data Synchronization, CKSyncEngine, Offline Mode Design, Offline-First, SwiftData and SwiftUI, Core Data, Background Tasks, Push Notification System (silent pushes), Multiplatform App, Document Editor App

---

Realtime Collaboration

Q: Design real-time collaborative editing, like a shared whiteboard or document that several people edit at once, including edits made offline.

Short answer: Every client applies edits **locally first** and exchanges small operations over a WebSocket, so typing never waits for the network. The core decision is how concurrent edits converge.

Key points:

- **Requirements:** sub-second propagation, no lost edits, offline editing that merges cleanly later, pres-

ence (avatars, cursors, selections), undo that only undoes *my* changes, history/versions, permissions (view, comment, edit)
- **Components:** `DocumentReplica` (CRDT state, applies local and remote ops), `CollabConnection` (WebSocket actor: join, resume from a version vector, reconnect with backoff), `OpOutbox` (SwiftData: unacknowledged ops), `PresenceChannel` (ephemeral, throttled), `@MainActor @Observable` document view model, and the server (relay, persistence, snapshots, auth)
- **Data flow (local edit):** gesture -> op created with a fresh stamp -> applied to the local replica -> UI updates immediately -> appended to the outbox -> sent -> server acknowledges and broadcasts -> removed from the outbox
- **Data flow (remote edit):** socket message -> `apply(op)` (idempotent, so duplicates are harmless) -> view model publishes the changed elements only
- **OT vs CRDT:** OT keeps documents small and has a long track record (Google Docs) but needs a central server and correct transform functions for every pair of operation types; CRDTs merge without coordination and suit offline and peer-to-peer, at the cost of metadata (tombstones, IDs per character for text) that needs compaction. Text CRDTs (RGA, Yjs/Automerge-style sequences) are much harder than map/register CRDTs; use a proven library (Automerge has a Swift package; Yjs-compatible ports exist) rather than inventing one
- **Granularity:** a whiteboard is a map of shapes where each property is a last-writer-wins register (the code below); rich text needs a sequence CRDT. Choosing the model per data type keeps metadata small
- **Offline merges:** ops queue in the outbox with their stamps; on reconnect, send the outbox and request ops since the last known version; the CRDT merge is order-independent, so no special conflict UI is needed except where intent truly conflicts (two people move the same shape: one wins deterministically)
- **Presence:** cursor and selection updates are ephemeral, throttled to ~10/s, never persisted, and dropped on disconnect after a timeout
- **Scaling:** one room per document on the server, sticky routing, periodic snapshots so joins don't replay the whole history, compaction of tombstones once all replicas have seen them
- **Failure modes:** a client offline for weeks (its ops still merge; the server may require a snapshot first), clock skew (logical clocks, never wall clocks, for ordering), a malformed op (validate on the server, reject without breaking the room), permission revoked mid-session (server closes the socket; the client keeps a read-only copy)
- **Privacy/security:** authorize every op on the server against the document's ACL; end-to-end encryption is possible with CRDTs because the server only relays opaque ops, at the cost of server-side search and snapshots
- **Testing:** Swift Testing property-style tests: apply the same ops in random orders and with duplicates to several replicas and `#expect` identical states; fuzz offline/online sequences; a fake connection that delays and reorders messages

Related to check: Scalable Chat App, WebSockets, WebSocket Reconnection, Offline Mode Design, Offline-First Notes App, Document Editor App, Data Synchronization, Event Sourcing, Actors

---

End-to-End Encrypted Messaging

Q: Design end-to-end encryption for a messaging app, including how push notifications show message previews.

Short answer: The rule that drives everything: **the server routes ciphertext and public keys, and never sees a private key or plaintext**. Each *device* (not each user) has its own long-term identity key pair, generated on the device and kept in the Keychain or Secure Enclave; the server publishes the public halves in a key directory.

Key points:

- **Requirements:** only participants can read messages, multi-device per user, forward secrecy, verify who you're talking to, previews in notifications, history on a new device (opt-in), works offline with queued sends
- **Components:** `KeyManager` (generate, store and rotate identity and pre-keys; Keychain with an access group shared with the extension), `SessionStore` (ratchet state per peer device, in the App Group container with file protection), `Envelope` crypto (seal/open), the chat transport from Scalable Chat App, a Notification Service Extension, and a safety-number/QR verification screen
- **Data flow (send):** fetch the recipient's device list and public keys (cached, signed) -> for each device, advance that session's ratchet -> seal the payload with the header as authenticated data -> upload one envelope per device -> server fans out
- **Data flow (receive/push):** APNs delivers `{ciphertext, senderDevice, messageID}` with `mutable-content: 1` -> the NSE loads keys and session state -> decrypts -> replaces the notification's title and body (or falls back to "New message" on failure) -> writes the plaintext to the shared store so the app doesn't decrypt twice
- **Key management:** identity signing keys sign each device's KEM public keys, so the directory can't swap them silently; Secure Enclave keys (`SecureEnclave.P256`, and `SecureEnclave.MLKEM768` / `MLDSA65` on iOS 26+) are non-exportable, which is great for identity but means a new device enrolls new keys and existing devices re-encrypt or the user starts fresh. Rotate pre-keys regularly; revoke a device by removing it from the list and re-keying groups
- **Trust:** safety numbers or QR codes compare identity keys out of band; key-change warnings in the conversation; key transparency (an append-only log of directory changes) at large scale
- **Trade-offs:** per-device fan-out costs bandwidth for big groups, so groups use sender keys (encrypt once with a group key, distribute that key pairwise); building a double ratchet yourself is risky, so prefer an audited library (libsignal) and use CryptoKit for primitives and HPKE; backups are either off, or encrypted with a user-held key
- **Failure modes:** the NSE runs with a ~24 MB memory limit and a short time budget, so keep decryption lean; the device may be locked: Keychain items the NSE needs must use `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`, and its files need `.completeUntilFirstUserAuthentication`; ratchet state updated by the NSE and the app at the same time (coordinate with a file lock or let only one side advance); messages arriving out of order (keep skipped message keys for a bounded window)
- **Privacy:** metadata still leaks (who talks to whom, when); minimize server logs, avoid putting sender names in the push payload, and make previews a user setting
- **Testing:** Swift Testing round-trips for seal/open, tamper tests (flip a byte, change the header: open must throw), out-of-order and replay cases, and an NSE test harness that feeds recorded push payloads

Related to check: CryptoKit, Scalable Chat App, Push Notification System, Keychain, Securing Sensitive Data, File Protection Classes, Encryption at Rest, Banking App Security, Secrets Management

---

Live Activities for Delivery Tracking

Q: Design delivery tracking with Live Activities: the order's progress and ETA on the Lock Screen and in the Dynamic Island, updated in real time, even when the app isn't running.

Short answer: A Live Activity is a widget-extension view driven by an `ActivityAttributes` type: static data (the order ID) plus a `ContentState` (stage, ETA, courier). The app starts it with `Activity.request(attributes:content:pushType: .token)` when the order is placed, uploads every token from `activity.pushTokenUpdates` to the server, and from then on the **server updates it through APNs** (`apns-push-type: liveactivity`), so updates arrive while the app is suspended or terminated.

Key points:

- **Requirements:** start when an order is placed (in the app or elsewhere), stage and ETA updates within

seconds, an alert for important stages (“Courier is nearby”), end and dismiss after delivery, respect the user’s Live Activity settings, low battery cost

- **Components:** `DeliveryAttributes` (shared by app and widget extension), the widget extension’s `ActivityConfiguration` (Lock Screen view + Dynamic Island compact/minimal/expanded), the app’s `ActivityController` (start, token upload, observe state), the backend’s token registry and APNs sender
- **Data flow (start in app):** order placed -> `areActivitiesEnabled` -> `Activity.request(..., pushType: .token)` -> upload each push token with the order ID -> backend stores it
- **Data flow (update):** order service event -> backend builds `{"aps": {"timestamp", "event": "update", "content-state": {...}, "stale-date", "alert"?}}` -> APNs with the activity’s token, topic `<bundle id>.push-type.liveactivity` -> system updates the UI without waking the app
- **Data flow (end):** `"event": "end"` with a final content state and `dismissal-date`, or `activity.end(_:dismissalPolicy:)` from the app
- **Push-to-start (iOS 17.2):** the app uploads the token from `Activity<DeliveryAttributes>.pushToStartTokenU` at launch; the server sends `"event": "start"` with `attributes-type`, `attributes` and `content-state`; the system then gives the app background time to receive the new activity through `Activity.activityUpdates`, where it observes `pushTokenUpdates` and uploads the update token, so the app must observe both sequences from launch
- **Broadcast (iOS 18):** a channel per restaurant or event, created on the server; `pushType: .channel(channelID)`; one APNs request updates every subscriber, ideal for shared events, not for per-user deliveries
- **Content-state rules:** keep it small (the payload limit is about 4 KB), use `Date`s with `Text(timerInterval:)` / `Text(_:style:)` so the countdown animates without updates, set `staleDate` so the UI can show “Updating…” if the server goes quiet, and send `relevance-score` when there are several activities
- **Trade-offs:** server push (works while the app is closed, needs backend work) vs local updates from the app (only while it runs); frequent updates (`NSSupportsLiveActivitiesFrequentUpdates`, still budgeted; `frequentPushesEnabled` tells you if the user allowed it) vs a few meaningful stage changes; priority 10 for alerts vs 5 for silent ETA nudges (cheaper on the budget)
- **Failure modes:** tokens rotate (upload every value from the sequence), the user disables Live Activities (`activityEnablementUpdates`; fall back to notifications), the activity ends by system limits (activities have a maximum duration; observe `activityStateUpdates`), APNs throttling under heavy updates (coalesce on the server), a pushed state that doesn’t decode (keep `ContentState` backward compatible)
- **Privacy:** Lock Screen content is visible to anyone holding the phone; show first names and approximate info only, and nothing sensitive in `alert` text
- **Testing:** Swift Testing for `ContentState` encoding against the server’s JSON fixtures; Xcode previews of every presentation (Lock Screen, compact, minimal, expanded) and stage; `xcrun simctl push` or the Push Notifications Console with a `liveactivity` payload on device

Related to check: Push Notification System, WidgetKit Integration, Ride-Hailing Live Tracking, App Intents, Upload Progress, Background Tasks

---

Habit Tracker with Widgets and App Intents

Q: Design a habit tracker where most interactions happen outside the app: checking off habits from a widget, Control Center, Siri and Shortcuts. How do the app, the widget and the intents share data?

Short answer: The center of the design is one shared store in an App Group container that the app, the widget extension and the intents all open, plus a set of App Intents that are the only way state changes, whether the tap comes from the app, an interactive widget, a control or Siri. A widget button runs `CompleteHabitIntent` in the extension’s process without launching the app; the intent writes to the shared SwiftData store and asks WidgetKit to reload, and the app picks up the change through `@Query` when it

comes to the foreground.

Key points:

- **Requirements:** mark a habit done from the Home Screen, Lock Screen, StandBy, Control Center and Siri; streaks that stay correct across midnight and time zones; widgets that never show stale "not done" after a tap; optional sync across the user's devices; optional Live Activity for a timed habit (a 20-minute workout)
- **Components:** `HabitKit` framework or package shared by all targets (SwiftData models, `SharedStore`, intents, entities), the app, a widget extension (`AppIntentConfiguration` widgets + `ControlWidgets`), `HabitEntity: AppEntity` with an `EntityQuery`, `CompleteHabitIntent`, `HabitShortcuts: AppShortcutsProvider` for zero-setup Siri phrases
- **Data flow (widget tap):** `Button(intent: CompleteHabitIntent(habitID:))` -> the system runs `perform()` in the widget extension -> write to the App Group store -> the system reloads the tapped widget after `perform()`; the intent reloads *other* kinds that show the same data (`WidgetCenter.shared.reloadTimelines(ofKind:)`, `ControlCenter.shared.reloadControls(ofKind:)`) -> timeline providers read the store -> new snapshots rendered
- **Data flow (app):** views use `@Query`; on `scenePhase == .active` the context sees changes the extension saved; SwiftData history (`fetchHistory`) can tell the app exactly what changed while it was away
- **Storage:** `ModelConfiguration(groupContainer: .identifier("group...."))` so every process opens the same file; small settings in `UserDefaults(suiteName:)`; keep widget reads cheap (fetch today's habits only, not full history)
- **Controls (iOS 18):** a `ControlWidgetToggle` "Meditated today" or a `ControlWidgetButton` that runs the same intent; controls also appear on the Lock Screen and the Action button
- **Live Activities (optional):** ActivityKit for a running habit timer, started from an intent; updates are local while the app runs, or push-based (see Push Notification System)
- **Trade-offs:** intents running in the extension (instant, no app launch, tight memory and time limits) vs `openAppWhenRun` (full app, slower); precomputed timelines (budgeted, predictable) vs reloading on every write (fresh, but reload only the affected kinds, never `reloadAllTimelines()` on each change); SwiftData in the App Group (simple) vs a tiny JSON snapshot for the widget (fastest reads, two sources of truth)
- **Failure modes:** two processes writing at once (keep writes small, save immediately, re-fetch before mutating), the widget showing yesterday after midnight (timeline entries at the next start-of-day and a `.after(midnight)` reload policy), deleted habits referenced by a configured widget (the entity query returns nothing; show a placeholder), schema migration while the extension has an old copy open (ship migrations in the shared package)
- **Adaptive UI:** widget families from `.accessoryCircular` to `.systemLarge`; the app's list/detail is a `NavigationSplitView` that becomes two columns on iPad and an unfolded iPhone Duo
- **Privacy/offline:** everything works offline; the App Group container holds personal health-adjacent data, so use file protection and exclude it from analytics; widgets on the Lock Screen may be visible to others, so offer a redacted style (`.privacySensitive()`)
- **Testing:** Swift Testing on `perform()` and streak logic against an in-memory container; App Intents can be unit-tested by calling `perform()` directly; timeline providers tested with fixed dates; widget previews in Xcode for every family

Related to check: WidgetKit Integration, App Groups Shared Container, App Intents Architecture, SwiftData and SwiftUI, Offline Mode Design, Push Notification System (Live Activities), Sync Across Devices, Feature Flags Design

---

Notification Preferences Center

Q: Design an in-app notification preferences screen: users choose which kinds of notifications they get, across devices, and it has to stay consistent with the iOS Settings switch. How would you build it?

Short answer: There are two layers of truth and the design has to show both honestly: iOS owns *whether* the app may notify at all (authorization, alert style, Focus and summary settings), and our server owns *what* we send (per-topic opt-ins, quiet hours, channels). The preferences screen reads the system state with `notificationSettings()` every time it appears, reads and writes topic preferences through an API keyed by user (so they follow the user to every device), and the server filters before it ever calls APNs.

Key points:

- **Requirements:** per-topic toggles (order updates, messages, price drops, newsletter), per-channel where relevant (push, email), quiet hours, preferences shared across the user's devices, a clear message when iOS has notifications off, legal opt-in for marketing, no dark patterns
- **Components:** `NotificationPreferences` (`@MainActor @Observable`: system status + topic set), `PreferencesAPI` (`Sendable`, `GET/PUT /me/notification-preferences`), `PushCoordinator` (authorization, token, delegate; see Push Notification System), a server preference store consulted by the send pipeline
- **Data flow (toggle):** user flips "Price drops" -> optimistic UI -> PUT with the new value -> on failure roll back and explain -> server stores per user; every later send checks it before fan-out
- **Data flow (system state):** on appear and on `scenePhase == .active` -> `UNUserNotificationCenter.current().notificationSettings()` -> if `.denied`, show a banner with a button to `UIApplication.openNotificationSettingsURLString`; if `.provisional`, explain that notifications arrive quietly in Notification Center and, once the user has engaged, offer the full prompt
- **Topic -> system mapping:** each topic gets a `UNNotificationCategory` (actions, `thread-id` grouping), an interruption level (`passive` for digests, `active` for messages, `timeSensitive` only for things like a delivery at the door), and a `relevance-score` for summaries; transactional topics may be on by default, marketing must be opt-in
- **Settings round trip:** request authorization with `.providesAppNotificationSettings`; iOS then shows " Notification Settings" in Settings, and tapping it calls `userNotificationCenter(_:openSettingsFor:)`, where the router opens this screen
- **Trade-offs:** server-side preferences (cross-device, enforceable, also cover email) vs local-only (simple, but other devices keep getting pushes); filtering on the server (saves pushes and battery) vs filtering in a Notification Service Extension (can't fully suppress a visible push, so only for enrichment); provisional (no prompt, quiet delivery) vs an explicit prompt (prominent, one chance)
- **Failure modes:** offline toggle (queue the change, show "will sync"), conflicting edits on two devices (last write wins per topic with a server timestamp is fine here), the user disables a topic but a push is already in flight (accept it; send `apns-expiration` for timely content), the system switch off while the in-app toggles look on (always show the system banner first)
- **Adaptive UI:** a `Form` with sections per topic group; on iPad and an unfolded iPhone Duo it lives in the settings column of a `NavigationSplitView`; Dynamic Type and VoiceOver labels that read the topic and its state
- **Privacy:** preferences are personal data stored server-side; marketing consent needs an audit trail (when, which wording); no PII in payloads; honor "Allow Notifications" off as absolute
- **Testing:** Swift Testing on the model with a fake API (optimistic toggle, rollback on failure, refresh merges); the system state behind a small protocol so tests can simulate `.denied` and `.provisional`; server tests that the send pipeline drops opted-out topics

Related to check: Push Notification System, Onboarding and Permissions Flow, Deep Linking, Forms and Controls, Remote Configuration, Idempotency

---

Onboarding and Permissions Flow

Q: Design the onboarding for a new app that needs notifications, location and possibly tracking permission. How do you maximize opt-in without annoying users or getting rejected in App Review?

Short answer: Each system permission alert can be shown once, so I treat it as a scarce resource: never

ask at launch, ask at the moment the feature needs it, and precede it with a short priming screen in our own UI that explains the benefit and offers "Not now", which costs nothing and lets us ask again later in context. The flow is a small state machine driven by what the user chose to do and what iOS has already decided, so it skips anything already granted or denied and works identically on first launch and on the tenth.

Key points:

- **Requirements:** value before asks (users see the app before any alert), permissions requested only for features the user picked, graceful paths for "denied", resumable if the app is killed mid-onboarding, accessible, measurable, App Review compliant
- **Components:** `OnboardingFlow` (`@MainActor @Observable` state machine), a `PermissionGateway` protocol with live implementations per permission, priming screens, an experiment client for copy and ordering variants, analytics events, and an `@AppStorage` flag for "onboarding completed" plus per-permission "deferred at" dates
- **Data flow:** intro -> user picks goals (e.g. "remind me", "nearby offers") -> the flow builds a queue of permissions those goals need -> filters out anything not `notDetermined` -> for each: priming screen -> "Allow" shows the system alert / "Not now" defers -> finished -> first real screen
- **Per permission:** notifications via `requestAuthorization(options:)`, or **provisional** authorization with no prompt at all, upgraded later when the user engages; location as *when in use* first (`CLServiceSession(authorization: .whenInUse)`, iOS 18, or `requestWhenInUseAuthorization()`), *always* only for a feature that truly needs it and later; App Tracking Transparency (`ATTrackingManager.requestTrackingAuthorization()`) only if we actually track across apps, never before the app is active, and with the purpose string in Info.plist
- **Deferring:** "Not now" stores the date and the context; the next ask happens in context (saving the first favorite store -> location priming), with a cool-off so users aren't nagged
- **Denied paths:** explain what still works, offer a one-tap link to Settings (`UIApplication.openSettingsURLString`), and re-check status on every return to the foreground
- **A/B testing:** variants change copy, illustration and timing, not the system alert; assign once per install, log exposure when the priming screen appears, and judge on retention and feature use, not just the opt-in rate
- **Trade-offs:** a single upfront "permissions page" (simple, low opt-in, high denial) vs contextual asks (higher opt-in, more engineering); priming (higher grant rate, one more tap) vs direct alerts; provisional notifications (no friction, quiet delivery) vs the full prompt
- **Failure modes:** the app killed between the priming screen and the alert (resume from persisted state; status is re-read anyway), a permission revoked in Settings later (features re-check at use), MDM or Screen Time restrictions (`.restricted` must not loop), ATT unavailable because the user disabled "Allow Apps to Request to Track" (status is `.denied` without a prompt)
- **Adaptive UI and accessibility:** onboarding pages as a vertically scrolling layout that survives the largest Dynamic Type size; two-column illustration + text on iPad and an unfolded iPhone Duo based on width; VoiceOver order of title -> benefit -> buttons; respect Reduce Motion in animations
- **Privacy:** purpose strings that are specific and honest; privacy manifest and nutrition labels consistent with what we request; no fingerprinting as a "fallback" when ATT is denied (prohibited)
- **Testing:** Swift Testing on `OnboardingFlow` with a fake gateway (skips decided permissions, defers, finishes); UI tests that call `app.resetAuthorizationStatus(for: .location)` (it covers location, camera, photos and similar, not notifications or tracking, which need a freshly erased simulator); experiment variants forced via launch arguments

Related to check: Notification Preferences Center, Push Notification System, A/B Testing Architecture, Feature Flags Design, Analytics Implementation, Accessibility at Scale, App Storage, Barcode Scanner Feature (camera permission)

Authentication System

Q: Design the authentication system for a new consumer app: sign-up, sign-in, staying signed in, and account deletion.

Short answer: I'd make **passkeys** the primary credential and offer **Sign in with Apple** next to them, both through one `ASAuthorizationController` sheet, so there are no passwords to phish, leak or reset. The server verifies the passkey assertion (WebAuthn) or the Apple identity token, then issues a short-lived access token and a rotating refresh token, which the app stores in the **Keychain**.

Key points:

- **Requirements:** fast sign-up with no password, sign-in on new devices, stay signed in across launches and background refresh, multiple devices, sign out everywhere, account deletion in the app, phishing resistance, recovery when a device is lost
- **Components:** `SignInCoordinator` (`ASAuthorizationController`, passkey + Apple ID requests), `AuthAPI` (challenge, verify, refresh, revoke, delete), `TokenProvider` actor (see Authentication), Keychain store, `SessionModel` (`@MainActor @Observable`: `signedOut`, `signedIn(User)`, `expired`), and the app router that swaps root flows
- **Sign-up:** the server issues a registration challenge -> `createCredentialRegistrationRequest(challenge:name:userID:)` -> the server stores the public key; iOS 26 adds an account-creation request (`ASAuthorizationAccountCreationProvider` (iOS 26+)) that creates the account and passkey in one system sheet with prefilled contact details
- **Sign-in:** challenge -> assertion (passkey) or identity token (Apple) -> server verification -> tokens -> Keychain -> `SessionModel.signedIn`; AutoFill-assisted requests let the passkey appear in the QuickType bar of the username field
- **Staying signed in:** access token ~15 min, refresh token rotated on each use and bound to the device; refresh proactively near expiry and reactively once on 401; Keychain item `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` so background work can refresh and tokens never restore onto another device
- **Sign in with Apple specifics:** the email (possibly a private relay address) and name are delivered only on the first authorization, so the server must store them then; check `ASAuthorizationAppleIDProvider().credentialState(forUserID:)` at launch and sign out on `.revoked`
- **Account deletion:** a visible in-app path -> re-authenticate -> the server deletes or schedules deletion, revokes refresh tokens and calls Apple's token revocation endpoint for Sign in with Apple users -> the app clears Keychain, SwiftData, caches and App Group data; passkey providers can be told the credential is gone (`ASCredentialDataManager.reportUnknownPublicKeyCredential` (iOS 26.2+))
- **Trade-offs:** passkeys + Apple only (best security, users without an Apple-synced device need another path) vs adding email magic links or passwords (wider reach, weaker); token-based vs cookie sessions (`HTTPCookieStorage`) for apps sharing a web backend
- **Failure modes:** refresh token reuse detected (server revokes the whole token family; the app signs out), clock skew (server-side expiry decisions, client refreshes a little early), Keychain unavailable before first unlock (defer, don't sign out), an interrupted sign-up (idempotent account creation keyed by the passkey user ID)
- **Security extras:** App Attest assertions on sensitive endpoints, rate limiting on the server, no tokens in logs (`os.Logger` privacy), never a client secret in the app
- **Testing:** Swift Testing for `TokenProvider` single-flight (ten concurrent callers, one refresh), the session state machine, and deletion wiping every store; a fake `AuthAPI`; the passkey flow on device with an associated domain in developer mode

Related to check: Authentication, OAuth 2.0 Flow, Keychain, Secrets Management, Securing Sensitive Data, Banking App Security, Request Interceptors, StoreKit 2 Subscriptions (account linking), Deep Linking

---

Banking App Security

Q: Design the security architecture for a mobile banking app. What would you protect on the device, and what would you leave to the server?

Short answer: I start from the assumption that the device and the binary are hostile: anything shipped in the app can be read, patched or replayed, so every real decision (can this user move this money?) is made on the server, and the client's job is to make attacks expensive and give the server good signals. On the device that means passkeys or a strong login plus a biometric-gated Secure Enclave key for step-up, tokens in the Keychain with `...ThisDeviceOnly` classes, TLS with pinning, App Attest so the server knows requests come from our genuine app, and hygiene around screenshots, the app switcher and idle sessions.

Key points:

- **Requirements:** strong login and fast re-login, step-up for high-risk actions (payments, new payees, limit changes), no secrets recoverable from a stolen or backed-up device, resistance to man-in-the-middle and to tampered or automated clients, nothing sensitive on screen when it shouldn't be, regulator-friendly audit trail
- **Components:** `AuthService` (passkeys via `ASAuthorizationPlatformPublicKeyCredentialProvider`, or OAuth code + PKCE via `ASWebAuthenticationSession`), `TokenStore` actor over the Keychain, `StepUpSigner` (Secure Enclave P-256 key with `.biometryCurrentSet` access control), `AppIntegrity` actor (App Attest), a pinned `URLSession`, a `SessionGuard` (`@MainActor @Observable`) for idle timeout and privacy cover
- **Data flow (payment):** user confirms -> Face ID unlocks the Secure Enclave key -> client signs the canonical payment payload -> App Attest assertion over the same bytes -> request with access token, signature and assertion -> server checks token, device-key signature, attestation counter and risk score, then executes or asks for more
- **Keys and storage:** refresh token in the Keychain with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly` (never restored to another device), short-lived access token in memory only; the step-up key is generated inside the Secure Enclave and is non-exportable, so a stolen token alone can't authorize a transfer; `.biometryCurrentSet` invalidates the key when someone enrolls a new face or finger
- **Transport:** ATS on, `NSPinnedDomains` with SPKI hashes for the API host (plus a backup pin and a rotation plan), no `NSAllowsArbitraryLoads`
- **Integrity signals:** App Attest (`DCAppAttestService`) proves a genuine app on genuine Apple hardware; jailbreak heuristics (writable system paths, suspicious dylibs) are trivially hooked, so they only feed the server's risk score; keep debugging and logging of PII out of release builds
- **Screen privacy:** cover sensitive views as soon as the scene goes inactive (the app switcher already shows the app then), `.privacySensitive()` on balances, react to `sceneCaptureState` (screen recording or mirroring) by hiding card numbers, and treat `userDidTakeScreenshotNotification` as after-the-fact only (you can't block screenshots)
- **Sessions:** idle timeout (e.g. 5 minutes, enforced by the server too), re-auth with Face ID on return from background after N minutes, hard expiry of refresh tokens, server-side revocation on "log out all devices"
- **Scaling / failure modes:** App Attest unsupported or failing (older devices, simulator, Apple outage): the server falls back to stricter limits rather than locking customers out; attestation counter replay is rejected; pin rotation without bricking old app versions (ship the next key's pin early); Face ID changed -> re-enroll the step-up key after a full login
- **Privacy/compliance:** privacy manifest (collected data types, required-reason APIs, SDK manifests), no third-party analytics SDK reading screens with account data, `os.Logger` with `.private` for anything identifying, data protection `.complete` for cached statements
- **Adaptive UI:** on iPad and iPhone Duo two scenes may be open; the privacy cover and idle timer must be per app session, not per window, so one active window can't keep another unlocked forever
- **Testing:** Swift Testing with fake `AttestationAPI` and Keychain stores; server contract tests for replayed and modified assertions; manual pen tests with a proxy (pinning must fail closed)

Related to check: App Attest and DeviceCheck, Secure Enclave and CryptoKit, Passkeys, Securing Sensitive Data, Keychain, Certificate Pinning, SSL Pinning, OAuth2 Flow, Authentication, Secrets Management, App Transport Security, CryptoKit, Push Notification System (no PII in payloads)

Adaptive App for iPhone, iPhone Duo and iPad

Q: Design an app that works great on iPhone, iPhone Duo and iPad. Walk me through the layout, navigation and state.

Short answer: I'd design for a continuous range of window sizes rather than three devices, because that's what the app actually gets in iOS 27: a phone-sized window, an iPhone Duo going from its outer display to its 7.6" inner display, resizable iPhone windows in iPhone Mirroring and on iPad, and freely resizable iPadOS 26 windows. The navigation model is selection-driven and lives in `@Observable` models and `@SceneStorage`, so the same state renders as a stack when narrow and as a split view when wide, and folding or resizing never loses the user's place.

Key points:

- **Requirements:** one codebase and binary; usable at every size from a narrow window to a full iPad; no lost context on fold/unfold, rotation or resize; per-window state on iPad; accessible at large Dynamic Type; iOS 18 deployment target with iOS 26/27 features behind availability checks
- **Layout architecture:** structure from size classes (compact -> stack and tab bar, regular -> sidebar and split view), detail from measured width (`onGeometryChange`, compositional layout's `effectiveContentSize`), two-pane content in `ArrangementView` (iOS 27.1+) with a `ViewThatFits` fallback; reading widths capped (`.frame(maxWidth:)`, `readableContentGuide`)
- **Navigation model:** selection is the source of truth (`selectedTripID`, then a `NavigationPath` for deeper pushes). `NavigationSplitView` turns selection into a pushed detail when collapsed and a side-by-side detail when expanded, so one model serves every size; deep links set the same selection
- **State preservation:** app data in a shared `@MainActor @Observable` store (SwiftData underneath); per-window UI state (selection, tab, path) in `@SceneStorage` or a per-scene router; scroll position restored by item ID with `ScrollPosition`, not by offset; drafts saved to the store as the user types, never kept only in view `@State`
- **Trade-offs:** system containers give folding, resizing and vertical bars for free but constrain custom chrome; a custom `Layout` is more flexible but has to re-implement collapse behavior. Fewer breakpoints (two or three) are easier to test than per-device tuning
- **Failure modes:** selection lost when the split view collapses; a sheet presented from a column that disappears on fold; content under the vertical bar; expensive re-layout on every frame of a live resize; a hardcoded `UIScreen.main.bounds` path still lurking in an old UIKit screen
- **Testing:** parameterized layout-policy tests over widths and size classes, snapshot matrix (narrow/wide x Dynamic Type x dark), UI tests on iPhone 18 Pro, iPhone Duo and iPad destinations; manual passes folding mid-task, on the resizable simulator and in iPhone Mirroring at extremes
- **Rollout by SDK tier:** (1) audit and fix anti-patterns (`UIScreen.main`, idiom and orientation checks, fixed heights) and adopt the UIScene life cycle, which apps built with the iOS 27 SDK require; (2) ship on the iOS 27 SDK so the app resizes (remove `UIRequiresFullScreen`, which now only makes resizing discrete); (3) ship on the iOS 27.1 SDK for edge-to-edge iPhone Duo with vertical bars, `ArrangementView` and `axisBehavior` tuning; (4) add iPhone Duo App Store screenshots for both displays

Related to check: Adaptive layout for iPhone Duo (SwiftUI), ArrangementView, Resizable iPhone apps, iPhone Duo poses and vertical bars, Discrete resizing and `UIRequiresFullScreen`, Scene life-cycle migration, App state restoration, Testing resizable layouts, Multiplatform app, Navigation architecture

---

Multiplatform App

Q: Design an app that ships on iPhone, iPad, Mac and Apple Vision Pro from one codebase. How do you structure it, and where do you let the platforms differ?

Short answer: I'd make one SwiftUI multiplatform app target backed by Swift packages: a platform-neutral core (models, persistence, networking, business logic) that compiles everywhere, a shared UI package of adaptive views, and thin per-platform layers only where the platforms genuinely differ, such as Mac settings and menus, visionOS volumes, and iPhone widgets. The layout is driven by available space, not by device: `NavigationSplitView`, size classes and `onGeometryChange` give one interface that works on a narrow iPhone, a resized iPhone Duo window, an iPad, a Mac window and a visionOS window.

Key points:

- **Requirements:** feature parity where it makes sense, platform-appropriate input (touch, pointer, keyboard, gaze and pinch), menus and shortcuts on Mac and iPad, multiple windows, shared data across devices, one release train
- **Components:** `AppCore` package (models, SwiftData store, API client, `Sendable` services), `AppUI` package (adaptive views, design system), the app target with `#if os(...)` scene extras, extension targets (widgets, share extension), and a `PlatformServices` protocol for the rare API that only exists on one platform
- **Targets:** one multiplatform target ("Supported Destinations": iPhone, iPad, Mac, Apple Vision) instead of four apps; shared packages declare all platforms in `Package.swift`; the App Store can sell it as a universal purchase
- **Mac choices:** native SwiftUI on macOS (best Mac feel: real menus, `Settings` scene, `.defaultSize`, tables, inspectors), Mac Catalyst ("Optimize for Mac") to bring a UIKit iPad app across with modest work, or "Designed for iPad" on Apple silicon Macs for zero work and a clearly non-native feel. visionOS has the same split: a native visionOS app or the iPad app running in compatibility mode
- **Platform differences, smallest tool first:** adaptive layout (`NavigationSplitView`, `ViewThatFits`, size classes) -> modifiers that exist everywhere but behave per platform (`.toolbar`, `.commands`, `.keyboardShortcut`, `.contextMenu`) -> `#if os(macOS)` for whole scenes or views -> protocol-injected services for platform-only frameworks
- **Data flow:** the core package is the only place that touches storage and network; views read `@MainActor @Observable` models; sync across a user's devices goes through CloudKit or the backend (see Sync Across Devices)
- **Trade-offs:** maximum sharing makes everything look like the lowest common denominator; maximum divergence multiplies work and bugs. Share models and logic 100%, views ~80%, and give each platform its signature interactions (Mac menu bar and keyboard, iPad pointer and Stage Manager, visionOS ornaments and volumes)
- **Scaling / failure modes:** `#if` sprawl (move it into one file per platform or behind protocols), APIs that compile only on some SDKs (CI builds every destination on every PR), UIKit-only dependencies blocking a native Mac build, assumptions about one window or one screen size
- **Adaptive layout:** never branch on `userInterfaceIdiom` or screen bounds to "detect iPad"; branch on available width. Resizable iPhone apps in iOS 27 (iPad, iPhone Mirroring, iPhone Duo) make this mandatory even for "iPhone-only" code paths (see Adaptive App for iPhone Duo and iPad)
- **Privacy/offline:** one privacy manifest per target and per SDK; the core store works offline on every platform; Mac sandbox entitlements (file access, network client) are separate from iOS capabilities
- **Testing:** Swift Testing in the core package runs on macOS in seconds (`swift test`); CI builds and tests each destination; UI tests per platform for platform-specific flows only

Related to check: Modular App Architecture, Large Codebase Structure, Adaptive App for iPhone Duo and iPad, Sync Across Devices, Scenes and Multiple Windows (UIScene), Size Classes, Toolbar, Design Systems

---

On-Device AI Feature

Q: Design an AI feature, say summarizing a product's customer reviews, using Apple's on-device models. How do you handle devices that can't run it, bad outputs, privacy and quality?

Short answer: I'd build it on the **Foundation Models** framework (iOS 26+): a `LanguageModelSession` on `SystemLanguageModel.default`, with **guided generation**, so the model fills a `@Generable` Swift type instead of returning free text I have to parse. The feature is optional by design: I check `model.availability` first and simply hide the UI when the device isn't eligible, Apple Intelligence is off, or the model is still downloading, and every failure (guardrail, refusal, context too long) degrades to "See all reviews" rather than an error.

Key points:

- **Requirements:** a short neutral summary plus up to three themes, under ~2 s after the section appears, no hallucinated claims, private, works offline, invisible on devices without Apple Intelligence, cheap on battery, accessible and localized
- **Components:** `ReviewSummarizer` (session setup, prompt, typed output, error mapping), a `@MainActor @Observable` section model (idle -> loading -> summary | hidden | fallback), a cache of summaries keyed by product ID + review-set hash, and an evaluation harness in the test target
- **Data flow:** section appears -> check availability -> cache hit? -> `session.prewarm()` when the user is likely to need it -> `respond(to:generating:)` with a bounded slice of reviews -> validate (non-empty, length limits) -> cache -> render; the source reviews stay one tap away
- **Guided generation:** `@Generable` structs and enums with `@Guide` descriptions and constraints (`.maximumCount(3)`, ranges, `anyOf`) produce well-formed output by construction; `streamResponse` gives `PartiallyGenerated` values for progressive UI
- **Context limits:** the on-device model has a small context window (`contextSize`; `tokenCount(for:)` (iOS 26.4+) to measure a prompt), so select and truncate input: recent and helpful reviews first, a fixed cap, or summarize in chunks and then summarize the summaries
- **Tools:** a `Tool` lets the model call back into the app (e.g. fetch product specs) instead of stuffing everything into the prompt; keep tools few and deterministic
- **Fallbacks:** unavailable -> hide; content blocked or refused -> fallback text; too long -> shrink input and retry once; otherwise -> fallback and log a category (never the content). In iOS 27, a session can also target other `LanguageModel` providers, including `PrivateCloudComputeLanguageModel` (iOS 27+) with a quota you must check, which is a deliberate privacy and product decision, not a silent fallback
- **iOS 27 changes:** errors move to `LanguageModelError` (iOS 27+), and `LanguageModelSession.GenerationError` is deprecated, so code supporting both checks for both (below); custom adapters (`SystemLanguageModel.Adapter`) were deprecated in iOS 26.4 and are unavailable in the iOS 27 SDK; dynamic instructions and profiles and multimodal prompts are new
- **Own models:** for a task-specific model of your own (classification, embeddings, a fine-tuned network), Core ML is the established runtime, and Apple introduced a new **Core AI** framework in iOS 27 for running your own models on device; I'd check Apple's current guidance on Core AI vs Core ML before committing, because that's choosing a model runtime, not writing a prompt
- **Trade-offs:** on-device (private, free, offline, smaller model) vs server LLMs (stronger, cost per call, data leaves the device, needs consent and a privacy policy update); precomputing summaries on the server for popular products vs per-device generation
- **Privacy:** no prompt or output logging; analytics record outcome categories and thumbs-up/down only; label the content as AI-generated; on-device inference sends nothing anywhere, but switching to a server model means updating the privacy manifest, the privacy policy and the user-facing explanation
- **Evaluation and testing:** a curated set of review bundles with expected properties (mentions the top complaint, no invented facts, length limits), run against every prompt change and new OS. The **Evaluations** framework (iOS 27+, used from the test target with Swift Testing) runs such sets with custom `Evaluators`, `ModelJudgeEvaluator` for model-graded criteria like tone, and `ToolCallEvaluator` for tool use, and exports results to compare runs. Unit tests inject a fake summarizer protocol so UI

states are tested without the model

Related to check: Siri AI Integration, App Intents, Analytics Implementation, Feature Flags Design (gradual rollout), Remote Configuration (prompt versions), Privacy Manifests, Caching Layer Design, News Reader App

---

Siri and Apple Intelligence Integration

Q: Design how a recipe app exposes its content and actions to Siri, Spotlight and Apple Intelligence.

Short answer: I'd treat App Intents as a second public API for the app: **entities** (the nouns: recipes, collections) and **intents** (the verbs: open, add to list, start cooking), backed by the same repository the UI uses. Entities conform to `IndexedEntity` so they go into Spotlight's semantic index with `CSSearchableIndex.indexAppEntities(_:)`.

Key points:

- **Requirements:** find recipes from Spotlight and Siri, run actions by voice or Shortcuts without opening the app, "this"/"that" references to on-screen content, results that stay correct after edits and deletes, no extra privacy exposure
- **Components:** an `AppIntents` target or package holding entities, queries and intents (shared with widgets and extensions); a `Sendable` repository the queries call; a Spotlight indexer; an `AppShortcutsProvider` for zero-setup phrases; the app's router for intents that open UI
- **Data flow (indexing):** repository save -> indexer maps models to `RecipeEntity` -> `indexAppEntities` -> on delete, `deleteAppEntities(identifiedBy:ofType:)`; a full re-index on schema version change or first launch
- **Data flow (invocation):** Siri or Spotlight picks an intent -> the system resolves parameters through `EntityQuery.entities(for:)` by ID -> `perform()` runs in the app process (or an extension) -> returns a dialog/snippet, or an `OpenIntent` hands the entity to the router
- **Schemas:** Apple's schema domains (`.mail`, `.photos`, `.journal`, `.notes`, `.reader`, `.browser` and others) describe shapes Apple Intelligence already understands. Use one where it matches, and plain `AppEntity`/`AppIntent` otherwise. The macros were named `@AssistantEntity` / `@AssistantIntent` before the Xcode 27 SDK; the new names still deploy to iOS 18. In iOS 27 the per-domain `search` intents are deprecated in favour of `.system.searchInApp` (iOS 27+)
- **On-screen awareness ("view annotations"):** SwiftUI `.appEntityIdentifier(_:)` and `.appEntityUIElements(_:)`, UIKit `appIntentsDataSource` on table/collection views, and `NSUserActivity.appEntityIdentifier` (iOS 18.2+). `RelevantEntities.shared.updateEntities(_:for:)` (iOS 27+) tells the system which entities matter in a context such as now-playing audio
- **Trade-offs:** in-app intents are simplest and share state; an App Intents extension launches faster and avoids loading the UI. Indexing everything gives better recall but costs battery and leaks more into the index; index what users actually search for
- **Failure modes:** a stale index after a server-side delete (queries must tolerate missing IDs and return fewer entities), slow `entities(for:)` (Siri times out: keep it a local lookup), intents run while the app is locked (data protection: decide which intents need `.completeFileProtection` data and require unlock), schema changes between versions (re-index on upgrade)
- **Privacy/offline:** queries read the local store, so they work offline; don't index private content (health notes, payment data) and use `hideInSpotlight` where needed; nothing leaves the device unless the user's request does
- **Testing:** Swift Testing on queries and `perform()` with an injected fake repository; the AppIntentsTesting framework (iOS 27) for running intents the way the system does; manual checks in the Shortcuts app and Spotlight

Related to check: App Intents, Spotlight (Core Spotlight), Deep Linking, Navigation Architecture, Foundation Models, WidgetKit Integration, Repository Pattern, Modular App Architecture

---

Swift 6 Migration Plan

Q: Your app has 50 SPM modules, all in Swift 5 mode. Design the plan to move the whole codebase to Swift 6 language mode.

Short answer: I'd run it as a **bottom-up, module-by-module migration** over the dependency graph, because Swift 6 language mode is set per module and a module can only be fully checked once the modules it imports expose correct `Sendable` and isolation annotations. Each module goes through the same ramp: Swift 5 mode with complete concurrency checking (`StrictConcurrency`) as warnings, fix them, then flip to `.swiftLanguageMode(.v6)`, where they become errors and can't come back.

Key points:

- **Requirements:** keep shipping weekly during the migration, no behavior changes hidden in "concurrency fixes", every module in Swift 6 mode at the end, no new `@unchecked Sendable` without review, measurable progress
- **Inventory:** generate the module graph (`swift package show-dependencies`), tag each module as leaf/core, service, UI feature or legacy; count warnings per module with complete checking on; list third-party dependencies and whether they ship Sendable annotations
- **Policy decisions (once, for the org):** UI feature modules use default `MainActor` isolation (`.defaultIsolation(MainActor.self)`), core/service packages stay nonisolated by default; enable `NonisolatedNonsendingByDefault` everywhere so `nonisolated async` functions stay on the caller's actor, and use `@concurrent` for deliberate background work; `InferIsolatedConformances` for main-actor types conforming to delegate protocols; services are `Sendable` protocols, shared mutable state goes in actors or `Mutex`
- **Waves:** wave 1 is leaf modules (models, utilities: mostly value types, quick wins); wave 2 is networking/persistence services (actors, `Sendable` protocols); wave 3 is UI features (mostly `@MainActor` fixes, fast once services are done); wave 4 is legacy code, wrapped behind `Sendable` protocols if it can't be fixed in time
- **Third-party and Apple SDKs:** `@preconcurrency import` for modules that aren't annotated yet, recorded in a tracking list and removed when the dependency updates; prefer upgrading dependencies first
- **Trade-offs:** module-by-module keeps every PR small and shippable but means mixed modes for months; a "big bang" branch finishes faster on paper but conflicts with everyone. `nonisolated(unsafe)` and `@unchecked Sendable` unblock a wave but are debt: require a comment and a ticket
- **Scaling the people side:** a small enablement group writes the playbook (common fixes with before/after), pairs with each feature team for their first module, and owns the CI checks; feature teams migrate their own modules
- **Failure modes:** fixing warnings by sprinkling `@MainActor` on everything (moves work onto the main thread; watch the Hangs instrument), `Task.detached` and `DispatchQueue` workarounds instead of fixing ownership, behavior changes from `nonisolated(nonsending)` (code that relied on hopping off the actor now needs `@concurrent`), and runtime isolation assertions from legacy callbacks arriving on background queues
- **CI guardrails:** a module's language mode can only move forward (a script diffs `Package.swift`); warnings-as-errors for concurrency diagnostics in migrated modules; the Thread Sanitizer on the test suite for code that still uses locks or GCD
- **Testing:** existing unit tests must stay green at each flip; add Swift Testing tests around actors that replace queues; run the Swift Concurrency instrument (Xcode 27) on hot paths to catch excessive actor hops

Related to check: Swift 6 Migration, Sendable, Actor Isolation Rules, Preconcurrency, Main Actor, Global Actor Isolation, Modular App Architecture, Large Codebase Structure, Migration Strategy, Thread Sanitizer

App Performance Program

Q: You're the tech lead for a large app whose launch time and hangs have slowly got worse. Design a performance program for the team: budgets, measurement, CI and how regressions get caught and fixed.

Short answer: I'd treat performance like any other product requirement: a few user-facing metrics with explicit budgets, field data as the source of truth, lab tests in CI to stop regressions before they ship, and clear ownership when a budget is broken. The field side comes from MetricKit (launch time, hang rate, hitches, memory, terminations), aggregated per app version and device class on a dashboard; the lab side is a small set of XCTest performance tests on fixed devices with baselines; and between them sit signposts in our own code so both views speak the same names.

Key points:

- **Requirements:** budgets the whole team understands, detection within one build (lab) and one release (field), attribution to a change or a team, no flaky CI gates, low overhead in production
- **Budgets (examples, set per app):** cold launch to first useful frame p50 < 400 ms / p90 < 1 s on the oldest supported device class; hang rate (MetricKit hang time per hour) under a threshold; scroll hitch ratio under ~5 ms/s on the main feed; peak memory below a device-class limit; app download size and install size per release; zero main-thread disk or network I/O in launch
- **Components:** a `Performance` package (signpost helpers built on MetricKit's `mxSignpost` log handles, launch phase markers), MetricKit ingestion (`MXMetricManager` subscriber, or `MetricManager` async sequences on iOS 27+) forwarding JSON to the backend, a dashboard (per version, per device class, p50/p90), XCTest performance tests in a dedicated test plan, a size report from the archive, and a weekly perf review rotation
- **Data flow (field):** device collects metrics -> payload roughly daily, diagnostics with call stacks soon after the event -> app uploads JSON -> backend symbolicates and aggregates -> dashboard compares the new version against the last one -> alert when a budget is crossed for a statistically meaningful population
- **Data flow (lab):** PR merges -> nightly perf job on real devices (not shared simulators) -> launch, scroll and memory tests against baselines -> regression beyond tolerance opens an issue with the commit range; Instruments run comparison (Xcode 27) to diff traces of good and bad builds
- **Launch work:** defer SDK initialization off the critical path, avoid static initializers, use mergeable libraries / fewer dynamic frameworks, render the first screen from cache, measure with `applicationLaunchMetrics` in the field and `XCTApplicationLaunchMetric` in the lab
- **Trade-offs:** lab tests are repeatable but unrepresentative (one device, clean state); field data is truthful but late and noisy. Hard CI gates catch regressions but flake on noisy metrics, so gate on launch and size (stable), and alert on hitches and memory (noisier). Budgets per device class beat one global number
- **Failure modes:** baselines drifting upward unnoticed (review them quarterly, never auto-accept), noisy devices (fixed device pool, thermals settled, multiple iterations, median), a third-party SDK update adding 200 ms (SDKs get budgets too), metrics without attribution (signpost each subsystem)
- **Adaptive UI and devices:** measure resizing too: live resize on iPad and iPhone Duo must not relayout expensive views every frame; include the slowest supported iPhone in the device pool
- **Privacy:** MetricKit data is aggregated and anonymous, but custom diagnostics can capture user content in logs; strip PII and declare collection in the privacy manifest
- **Testing the program itself:** inject a deliberate 300 ms sleep in a branch and confirm the pipeline catches it within one run

Related to check: Observability Pipeline, MetricKit, App Launch, Startup Optimization, Instruments, Time Profiler, Reduce Binary Size, Scroll Performance, Performance Testing, CI Testing, Test Plans

Observability Pipeline

Q: Design the observability pipeline for a large iOS app: logs, performance signals, crashes and hangs.

Short answer: I'd split observability into three channels with different costs and audiences. **Logs** go through `os.Logger` with privacy annotations: cheap, on-device, readable in Console and collected on demand.

Key points:

- **Requirements:** find and prioritize crashes within hours, track launch/hang/scroll regressions per release, debug a single user's issue with consent, near-zero overhead, no PII leaves the device, works with intermittent connectivity
- **Components:** a `Log` namespace of `Loggers` per subsystem/category; a shared `OSSignposter` per area; a `DiagnosticsCollector` that consumes MetricKit; a crash reporter SDK; a `Telemetry–Uploader` (`Sendable`, batched, background `URLSession`); a backend that symbolicates with uploaded dSYMs and groups by call-stack signature
- **Data flow (field):** MetricKit delivers metric reports roughly daily and diagnostic reports (crash, hang, CPU and disk-write exceptions, app launch, memory) as they're available -> collector encodes them (`MetricReport` and `DiagnosticReport` are `Codable` on iOS 27; `jsonRepresentation()` before) -> upload queue on disk -> backend symbolicates and dashboards by app version
- **Data flow (support):** user taps "Send diagnostics" -> app reads its own recent entries with `OSLog–Store(scope: .currentProcessIdentifier)` -> filters by subsystem -> attaches to a ticket with explicit consent
- **Custom metrics:** signpost intervals created with the MetricKit log handle (`MXMetricManager.makeLogHandle(cate` superseded by `MetricManager.logHandle(category:)` on iOS 27) are aggregated into the daily report: e.g. "checkout submit" duration histograms without shipping raw events
- **Trade-offs:** MetricKit is free, private and covers terminations a crash SDK can't catch (watchdog kills, memory), but it is delayed and aggregated; a crash SDK gives minute-level alerting and breadcrumbs but adds binary size, startup cost and a privacy-manifest entry. Most teams run both
- **Scaling:** sample verbose telemetry (e.g. 1% of sessions) behind remote config; cap the on-disk queue; upload on Wi-Fi/charging via a background session; version the event schema; dashboards keyed by build number so a regression is attributable to a release
- **Failure modes:** log spam hurting performance (`.debug` is cheap but not free; never log in tight loops), missing dSYMs (unsymbolicated crashes: upload them in CI), crash reporters that conflict with each other's signal handlers (run one), telemetry that itself causes hangs (never upload synchronously)
- **Privacy:** interpolated values in `Logger` are private by default in release logs; mark only safe values `.public` and hash identifiers with `.private(mask: .hash)`. Declare collected data types and any tracking domains in `PrivacyInfo.xcprivacy`, including what third-party SDKs collect
- **Testing:** unit-test the collector and uploader with Swift Testing (encode a fixture report, assert the upload); Xcode's simulated MetricKit payloads for end-to-end checks; `XCTOSSignpostMetric` / `XCTClockMetric` performance tests in XCTest; Instruments run comparison (Xcode 27) to compare traces before and after a change

Related to check: MetricKit, Logging Architecture, Logging Levels, Crash Reporting, Symbolication, Instruments, Analytics Implementation, Remote Configuration, Privacy Manifests

---

Accessibility at Scale

Q: Design an approach that keeps a 40-engineer iOS app accessible release after release.

Short answer: At scale, accessibility is a property of the **design system and the pipeline**, not of individual screens. The shared components (buttons, cards, rows, form fields) ship with correct labels, traits, Dynamic Type behavior, contrast-safe colors and Reduce Transparency fallbacks built in, so feature teams get them for free.

Key points:

- **Requirements:** WCAG-aligned contrast and target sizes, every screen usable with VoiceOver, Voice Control and Switch Control, layouts that work up to the largest accessibility text sizes, respect for Reduce Motion / Reduce Transparency / Increase Contrast / Bold Text, regressions caught before release
- **Components (design system):** semantic color tokens with high-contrast variants in the asset catalog; text styles only (`.font(.body)`, `UIFont.preferredFont(forTextStyle:)`), no fixed point sizes; spacing via `@ScaledMetric` / `UIFontMetrics`; components that switch from horizontal to vertical layout at accessibility sizes (`dynamicTypeSize.isAccessibilitySize`, `ViewThatFits`); a minimum 44x44 pt hit target; motion that checks Reduce Motion
- **Liquid Glass (iOS 26+):** standard bars, tab bars and `.glassEffect` adapt to Reduce Transparency and Increase Contrast automatically. Custom materials must check `accessibilityReduceTransparency` themselves. iOS 27 adds a user transparency setting, so never depend on a particular translucency to reach contrast: keep text on solid or sufficiently opaque backgrounds
- **Components (semantics):** labels for icon-only buttons; `.accessibilityElement(children: .combine)` on rows so VoiceOver reads one sentence; custom actions (`.accessibilityAction(named:)`) instead of hidden swipe gestures; headers marked with `.isHeader` for rotor navigation; `accessibilityIdentifier` for tests only, never as a label
- **Pipeline:** UI tests run `performAccessibilityAudit` (contrast, Dynamic Type, clipped text, hit regions, missing descriptions) on each critical flow; snapshot tests render components at default and accessibility XXXL sizes, light/dark, and RTL; a custom lint rule flags `.font(.system(size:))` and icon buttons without labels
- **Process:** an accessibility section in the PR template; per-feature acceptance criteria; a VoiceOver/Voice Control walkthrough as part of release QA; a known-issues list with tickets (suppressed in audits only with a link); Accessibility Nutrition Labels on the App Store kept truthful
- **Trade-offs:** strict audits on every screen catch more but produce false positives (decorative images, custom controls) that need an allow-list; snapshot tests at many sizes are slower, so run the full matrix nightly and a subset per PR
- **Scaling/failure modes:** teams bypassing the design system (track usage of raw `Text`/`Image` in feature modules), components drifting between SwiftUI and UIKit implementations, audits that only cover the happy path (add error and empty states), large text breaking fixed-height cells (self-sizing cells only)
- **Adaptive layout:** accessibility sizes and narrow widths (the iPhone Duo outer display, resizable windows on iPad and in iPhone Mirroring) stress the same layouts; size decisions use available width, never device checks
- **Testing:** Accessibility Inspector during development, the audit in CI, the Environment Overrides panel in Xcode for quick size/contrast checks, and real assistive-technology users in research rounds

Related to check: Accessibility (SwiftUI), Accessibility (UIKit), Dynamic Type, Scaled Metric, Design Systems, Liquid Glass, UI Testing, Accessibility Identifiers, Snapshot Testing, Adaptive Layout (iPhone Duo)

Localization at Scale

Q: Your app ships in 30 languages, with a dozen teams adding strings every sprint. Design the localization pipeline so releases never ship untranslated, truncated or mirrored-wrong UI.

Short answer: At this size localization is a pipeline, not a coding convention: String Catalogs in every module are the single source, Xcode extracts strings at build time, a translation management system (TMS) receives exported `.xcloc` bundles and returns translations, and CI refuses a release branch when a shipping language has untranslated or stale strings. Quality comes from three checks that run without translators: pseudolocales and right-to-left runs in UI tests, per-locale screenshots reviewed by language owners, and lint rules against concatenation and hard-coded text.

Key points:

- **Requirements:** every user-visible string localizable, plurals and grammar right in every language, release-blocking completeness for shipping languages, translator context (comments, screenshots), fast turnaround so teams aren't blocked, correct RTL layout, no translation-induced crashes (bad format specifiers)
- **Components:** a `Localizable.xcstrings` per package (`bundle: .module`) and per extension, a shared `L10n` package for common strings (OK, Cancel, errors), the TMS (vendor or in-house) with a sync job, a CI completeness test, UI-test screenshot jobs per locale, and an in-app language override for QA (per-app language in Settings)
- **Data flow:** developer writes `Text("Saved \(count) photos")` with a `comment:` -> build extracts it into the catalog as *new* -> nightly job runs `xcodebuild -exportLocalizations` -> `.xcloc` bundles to the TMS -> translators work with comments and screenshots -> `xcodebuild -importLocalizations` -> catalog states become *translated* -> CI gate passes
- **String freeze:** feature strings land at least one sprint before the release branch; late strings ship behind a feature flag in shipping languages only after translation, or the feature waits
- **Pluralization and grammar:** plural variants per language in the catalog (Arabic has six categories, Japanese one); device variants (iPhone/iPad/Mac) only when the text truly differs; automatic grammar agreement (`inflect: true`) where Foundation supports it
- **Pseudolocales:** scheme options *Double-Length*, *Accented*, *Bounded String* and *Right-to-Left* Pseudolanguage expose truncation, unlocalized text (no brackets or accents) and mirroring bugs before any translator is involved; run the UI test suite once per pseudolocale
- **RTL:** leading/trailing everywhere, SF Symbols that mirror automatically, `.flipsForRightToLeftLayoutDirection(true)` only for custom directional images, and never mirror media controls, charts with time axes or phone numbers
- **Trade-offs:** source-text keys (readable, Xcode default, but an English copy change invalidates every translation) vs semantic keys with `defaultValue:` (stable across copy edits, more ceremony); a central catalog (one place for translators, merge conflicts for everyone) vs per-module catalogs (ownership, more files to sync); machine or agent drafts (fast, needs review) vs human-only (slow, consistent voice)
- **Failure modes:** format-specifier mismatches (`%@` vs `%lld`) crash or garble at runtime, so validate placeholders on import; stale strings after a copy change; server-provided text in the wrong language (send `Accept-Language` from `Locale.preferredLanguages`); strings resolved at app launch then the user switches language (resolve at display with `LocalizedStringResource`)
- **Adaptive UI:** German and Finnish at the largest Dynamic Type size in the narrowest window you support (iPhone Duo's outer display, a narrow iPad window) is the worst case; screenshot it
- **Privacy:** the TMS sees UI text, never user data; screenshots for translators must use fixture accounts
- **Testing:** a Swift Testing test that parses each catalog and fails per language with the untranslated keys; pseudolocale UI-test runs; per-locale snapshot tests for key screens

Related to check: Localization Architecture, Modular App Architecture, Feature Flags Design, UI Testing, Snapshot Testing, Parameterized Tests, Dynamic Type, Accessibility at Scale, CI Testing

---